

SnmpAgentV2C

Fields

name	type	nullable	description
enable	Bool	false	--
community	String	false	--

Used by an

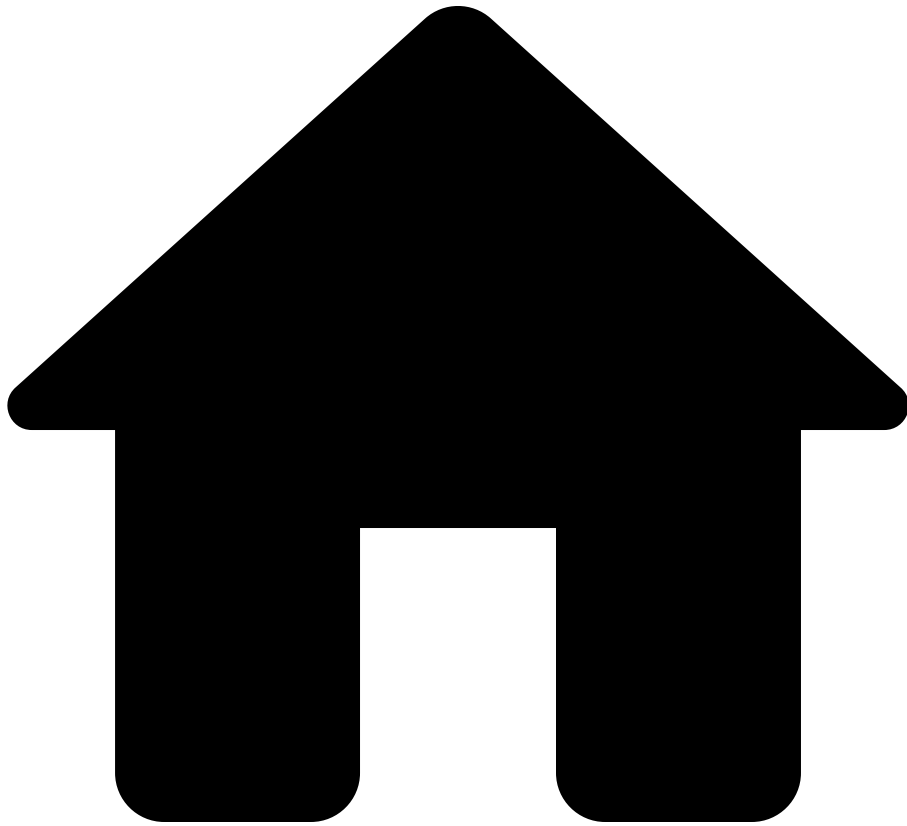
network devices outputs bluecat config items data
services snmp configurations snmpConfiguration
agentService v2c

See also[ââ](#)

Data model path

- [illegible]

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SnmpAgentV3

On this page

SnmpAgentV3

Fields

name	type	nullable	description
securityLevel	String	false	--
privtype	String	false	--
enable	Bool	false	--
username	String	true	--
authtype	String	false	--

Used by [1000](#)

network â devices â outputs â bluecat â config â items â data
â services â snmp â configurations â snmpConfiguration â
agentService â v3[â](#)

See also

Data model path

- [illegible]

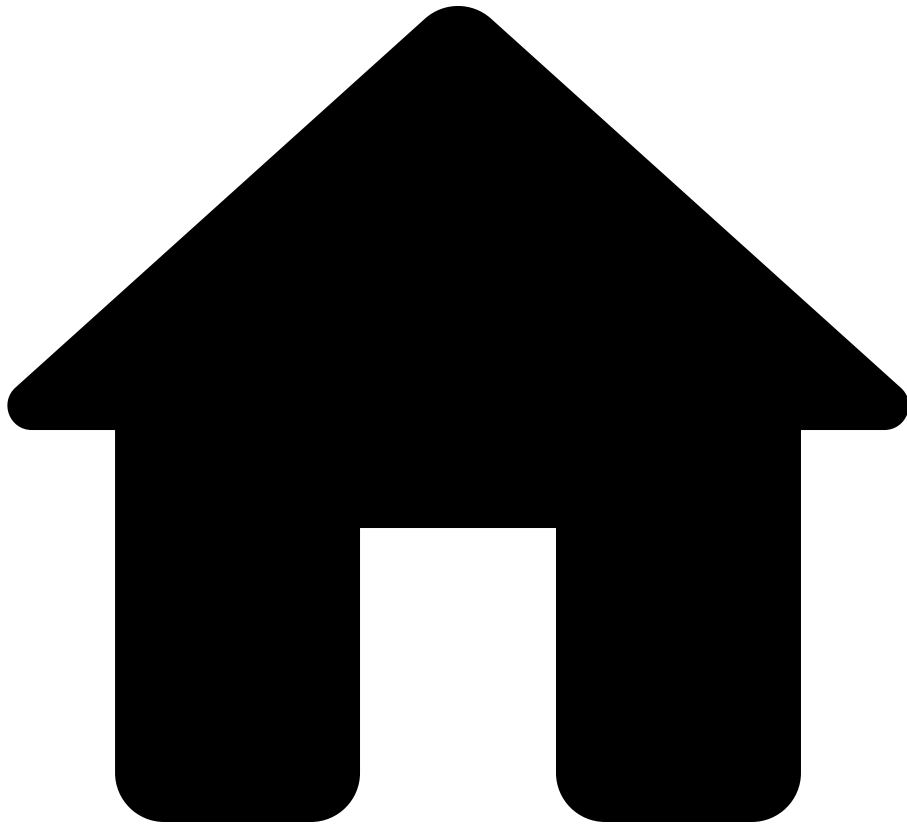
network â devices â outputs â bluecat â config â items â data
â services â snmp â configurations â snmpConfiguration â
trapService â trapServers â v3[â](#)

See also [ââ](#)

Data model path

- [illegible]

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SnmpConfigurationDetail

On this page

SnmpConfigurationDetail

Fields

name	type	nullable	description
enable	Bool	false	--
agentService	SnmpAgentService	false	--
trapService	SnmpTrapService	false	--

Used by

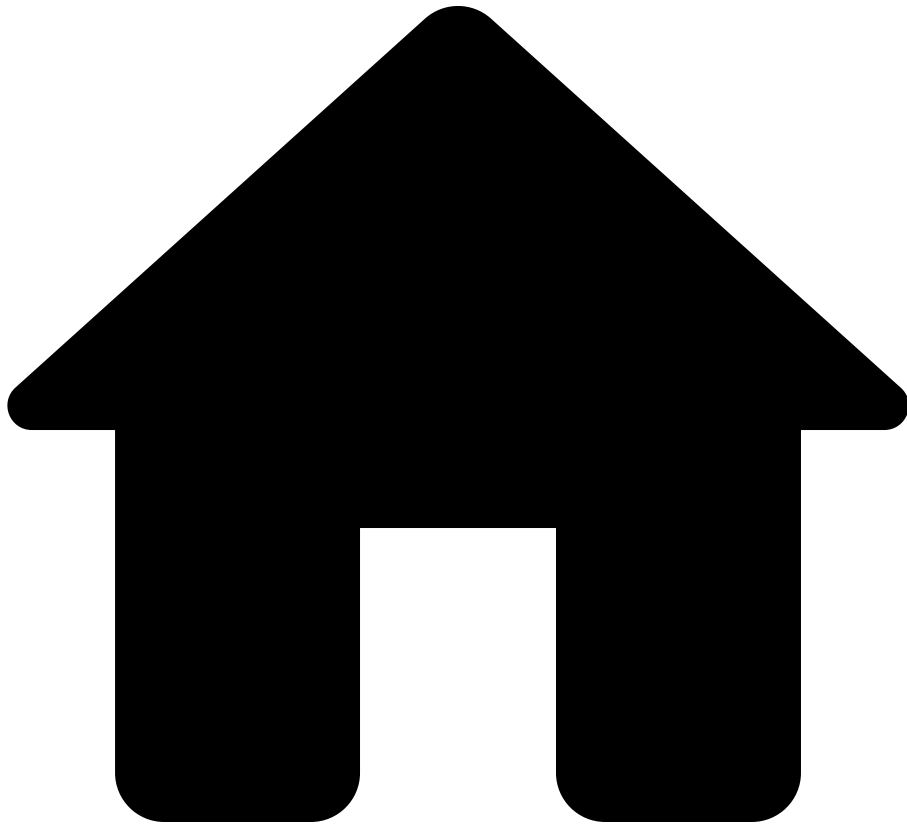
network devices outputs bluecat config items data services snmp configurations snmpConfiguration

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - outputs: [Outputs](#)
 - bluecat: [BluecatOutputs](#)
 - config: [BluecatConfig](#)
 - items: Bag< [BluecatConfigItem](#) >
 - data: [BluecatConfigItemData](#)
 - services: [BluecatServices](#)
 - snmp: [SnmpService](#)
 - configurations: Bag< [SnmpServiceConfiguration](#) >
 - snmpConfiguration: [SnmpConfigurationDetail](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SnmpService

On this page

SnmpService

Fields

| name | type | nullable | description |
|----------------|---|----------|-------------|
| configurations | Bag< SnmpServiceConfiguration > | false | -- |

Used by

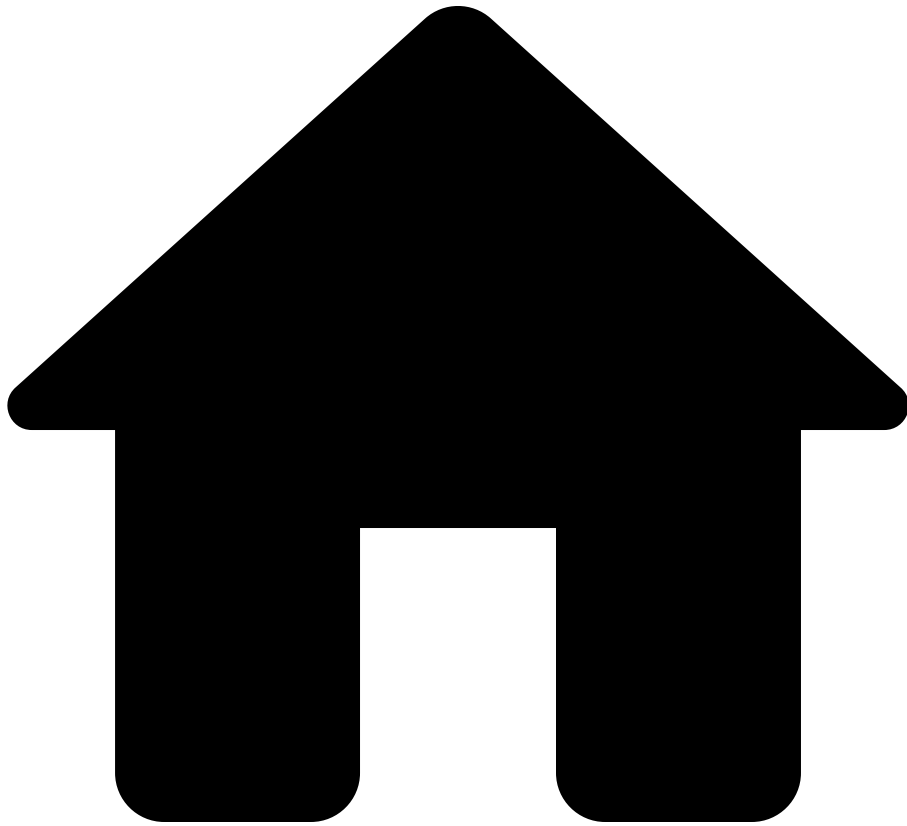
network devices outputs bluecat config items data
services snmp

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - outputs: [Outputs](#)
 - bluecat: [BluecatOutputs](#)
 - config: [BluecatConfig](#)
 - items: Bag< [BluecatConfigItem](#) >
 - data: [BluecatConfigItemData](#)
 - services: [BluecatServices](#)
 - snmp: [SnmpService](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SnmpServiceConfiguration

On this page

SnmpServiceConfiguration

Fields

| name | type | nullable | description |
|-------------------|---|----------|-------------|
| snmpConfiguration | SnmpConfigurationDetail | false | -- |

Used by

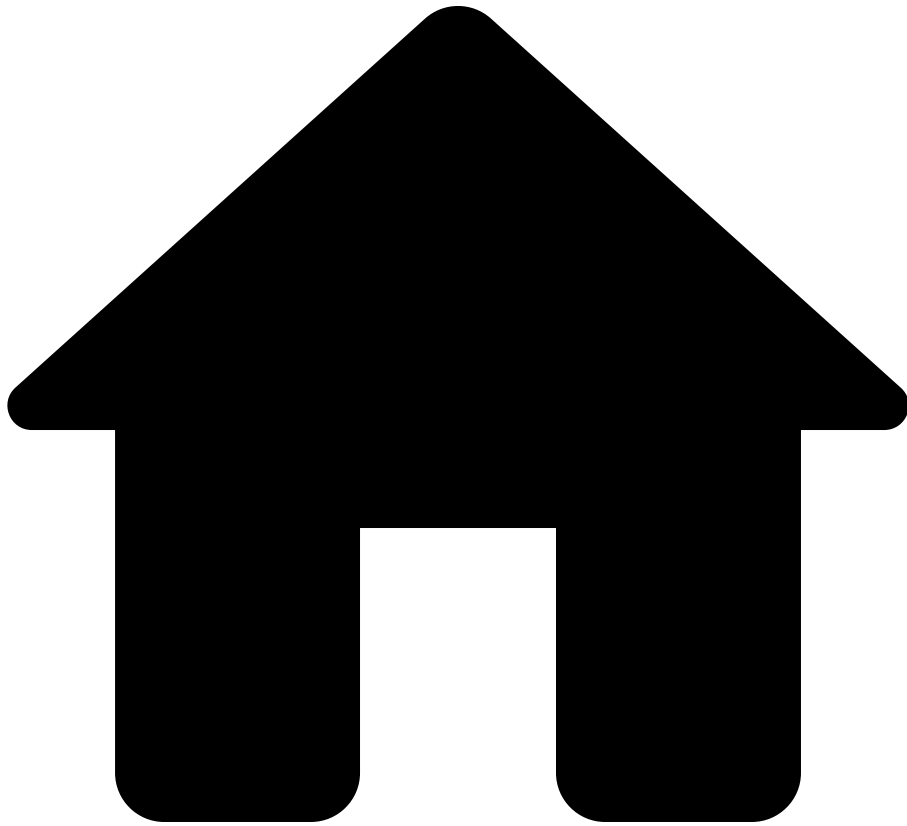
network devices outputs bluecat config items data
services snmp configurations

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - outputs: [Outputs](#)
 - bluecat: [BluecatOutputs](#)
 - config: [BluecatConfig](#)
 - items: Bag< [BluecatConfigItem](#) >
 - data: [BluecatConfigItemData](#)
 - services: [BluecatServices](#)
 - snmp: [SnmpService](#)
 - configurations: Bag< [SnmpServiceConfiguration](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SnmpSource

On this page

SnmpSource

Fields

| name | type | nullable | description |
|--------------|------------------------------------|----------|--|
| name | String | false | Unique name for the SNMP source; configured in Forward Enterprise. |
| snapshotInfo | DeviceSnapshotInfo | false | Metadata about the inclusion of this source in the snapshot. |
| profileName | String | false | The name of the profile from which this source was derived. |
| outputs | Bag< OidOutput > | false | -- |

Used by

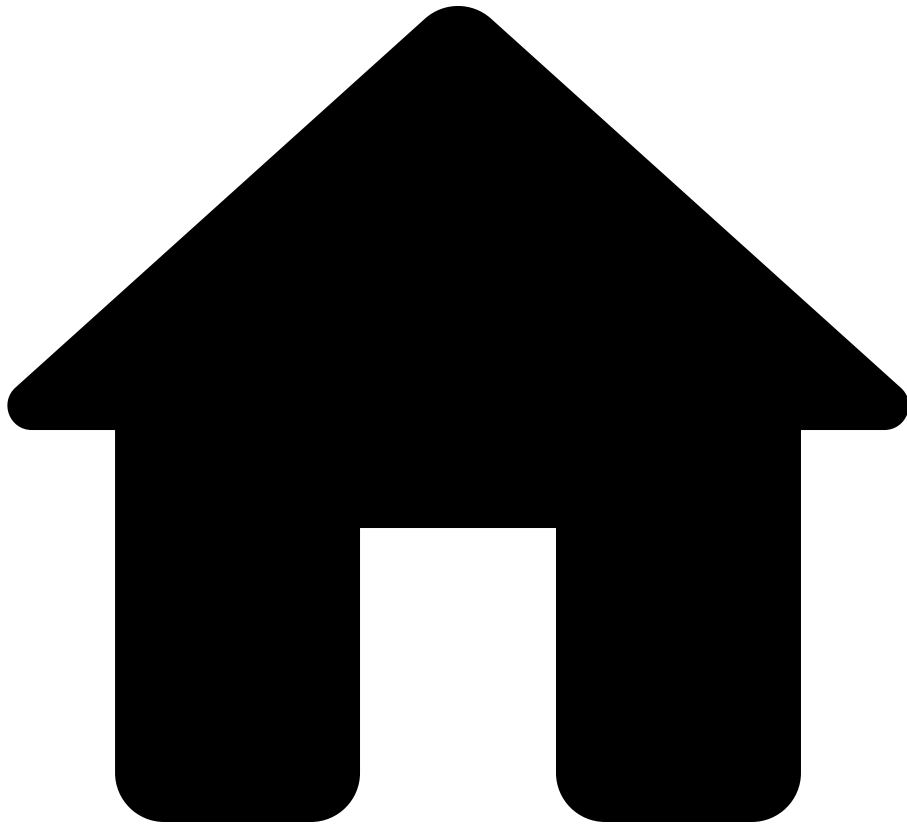
network externalSources snmpSources

See also

Data model path

- network: [Network](#)
 - externalSources: [ExternalSources](#)
 - snmpSources: Bag< [SnmpSource](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SnmpTrapServer

On this page

SnmpTrapServer

Fields

| name | type | nullable | description |
|---------|-----------------------------|----------|-------------|
| address | IpAddress | false | -- |
| port | Integer | false | -- |
| enable | Bool | false | -- |
| v3 | SnmpAgentV3 | true | -- |

Used by [1000](#)

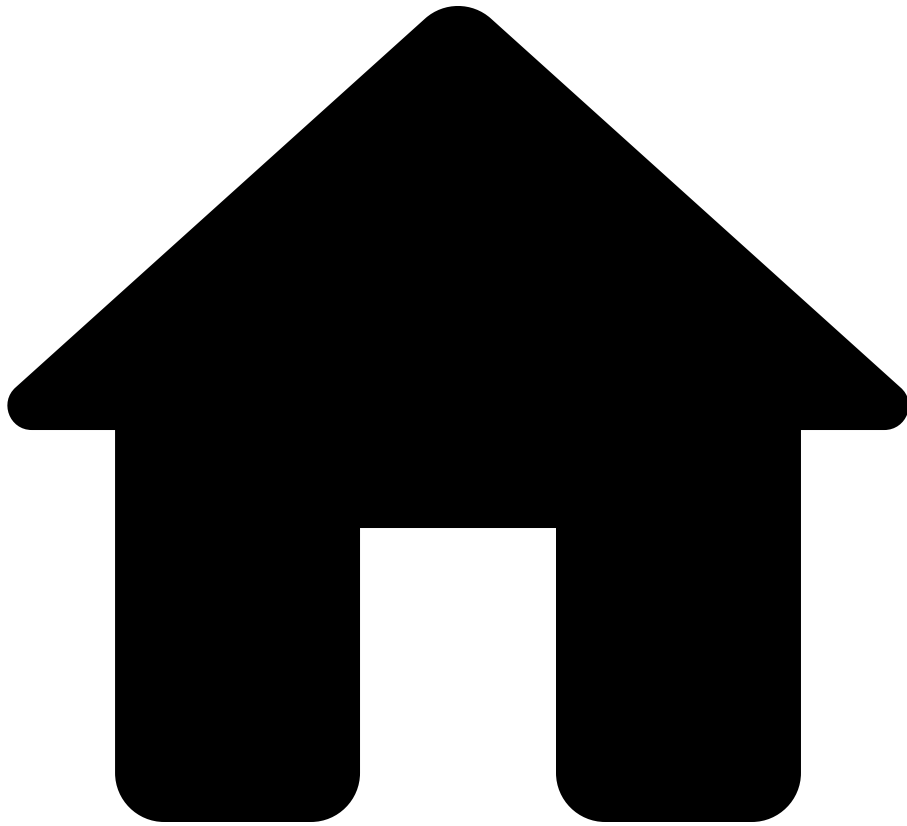
network → devices → outputs → bluecat → config → items → data
→ services → snmp → configurations → snmpConfiguration →
trapService → trapServers

See also [ââ](#)

Data model path

- [illegible]

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SnmpTrapService

On this page

SnmpTrapService

Fields

| name | type | nullable | description |
|-------------|---------------------------------------|----------|-------------|
| trapServers | Bag< SnmpTrapServer > | false | -- |

Used by [100](#)

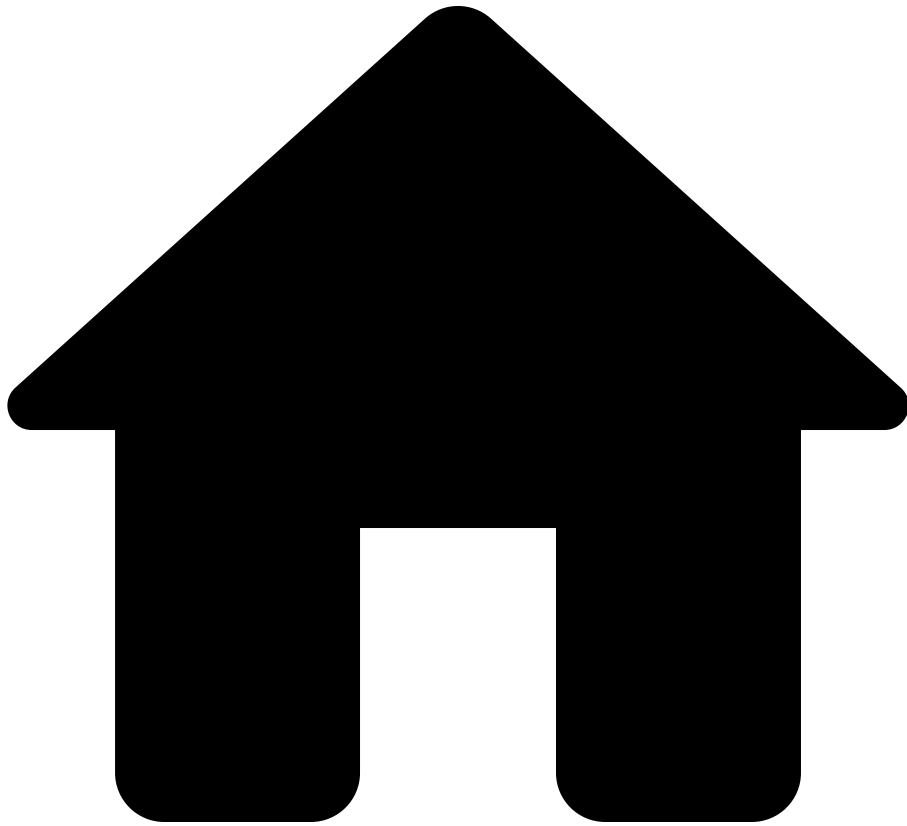
network devices outputs bluecat config items data
services snmp configurations snmpConfiguration
trapService

See also[ââ](#)

Data model path

- [illegible]

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SshConfigurationDetail

On this page

SshConfigurationDetail

Fields

| name | type | nullable | description |
|--------|---------------------------------|----------|-------------|
| tacacs | SshTacacsConfig | false | -- |
| enable | Bool | false | -- |

Used by an

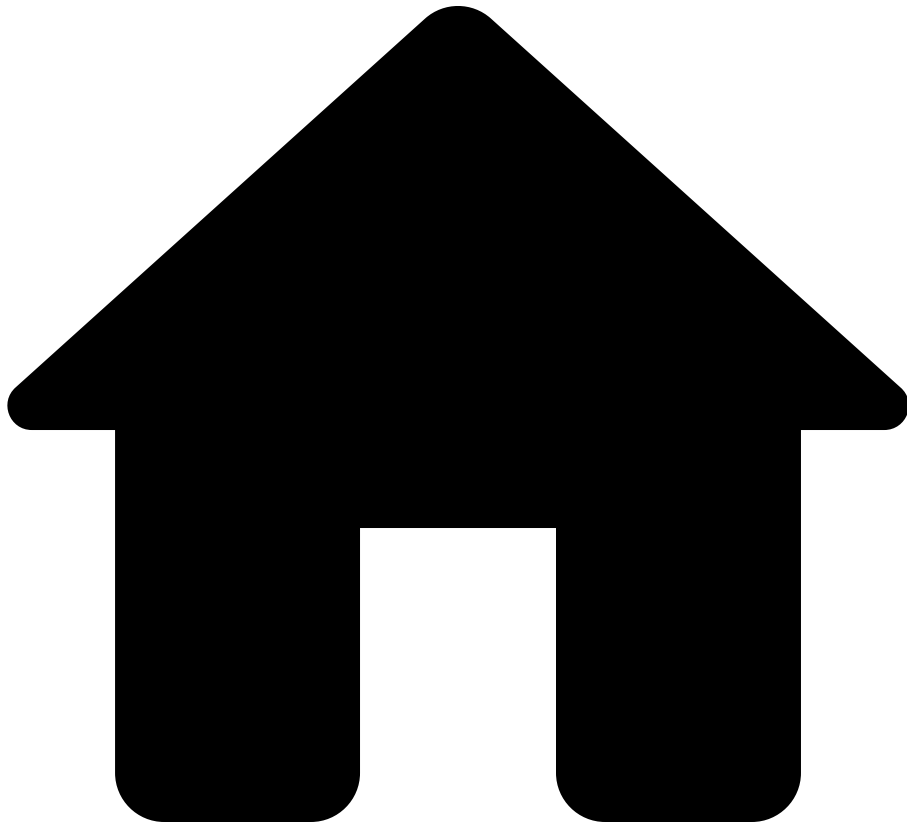
network â devices â outputs â bluecat â config â items â data
â services â ssh â configurations â sshConfigurationâ

See also [ââ](#)

Data model path

- [illegible]

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SshService

On this page

SshService

Fields

| name | type | nullable | description |
|----------------|--|----------|-------------|
| configurations | Bag< SshServiceConfiguration > | false | -- |

Used by

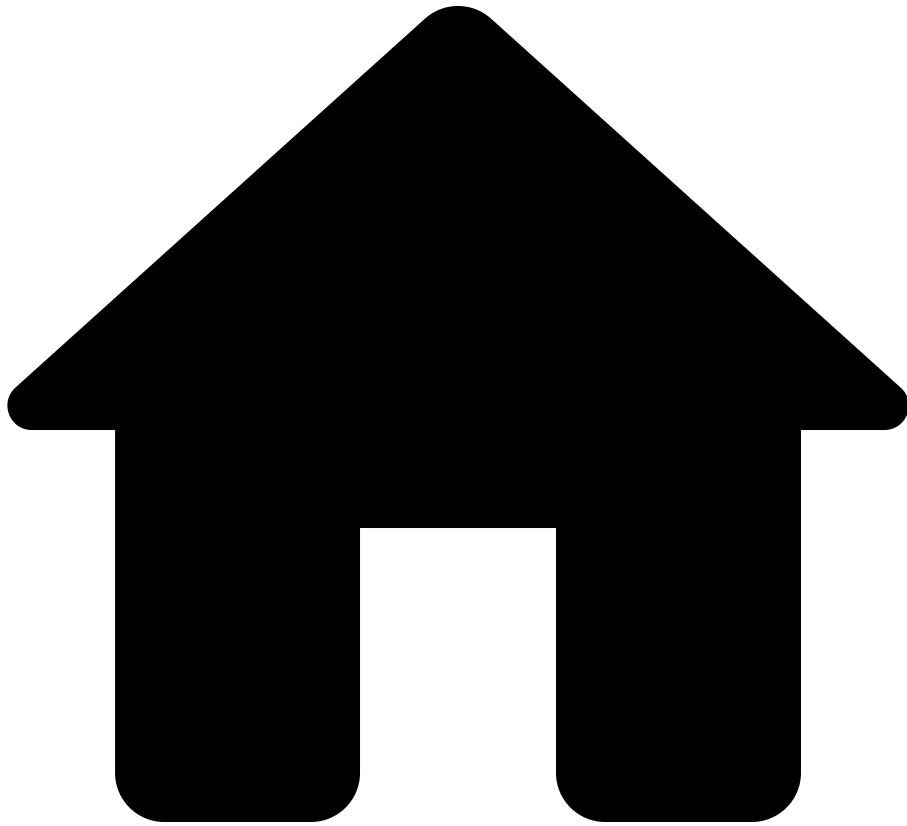
network devices outputs bluecat config items data
services ssh

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - outputs: [Outputs](#)
 - bluecat: [BluecatOutputs](#)
 - config: [BluecatConfig](#)
 - items: Bag< [BluecatConfigItem](#) >
 - data: [BluecatConfigItemData](#)
 - services: [BluecatServices](#)
 - ssh: [SshService](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SshServiceConfiguration

On this page

SshServiceConfiguration

Fields

| name | type | nullable | description |
|------------------|--|----------|-------------|
| sshConfiguration | SshConfigurationDetail | false | -- |

Used by

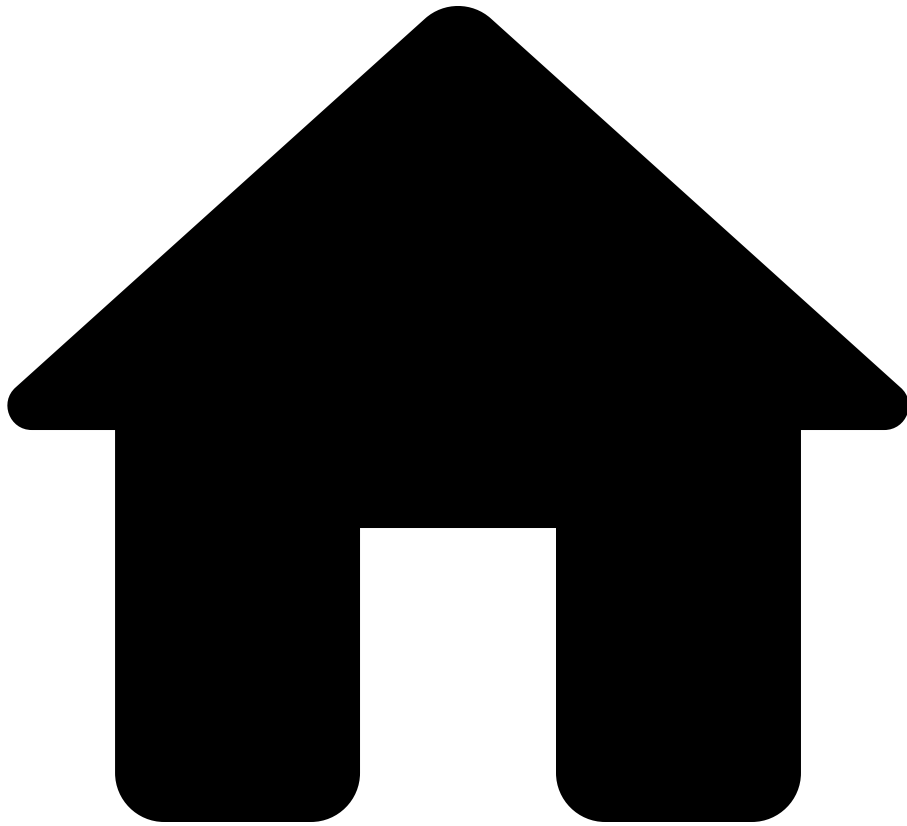
network devices outputs bluecat config items data
services ssh configurations

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - outputs: [Outputs](#)
 - bluecat: [BluecatOutputs](#)
 - config: [BluecatConfig](#)
 - items: Bag< [BluecatConfigItem](#) >
 - data: [BluecatConfigItemData](#)
 - services: [BluecatServices](#)
 - ssh: [SshService](#)
 - configurations: Bag< [SshServiceConfiguration](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SshTacacsConfig

On this page

SshTacacsConfig

Fields

| name | type | nullable | description |
|--------|------|----------|-------------|
| enable | Bool | false | -- |

Used by [1000](#)

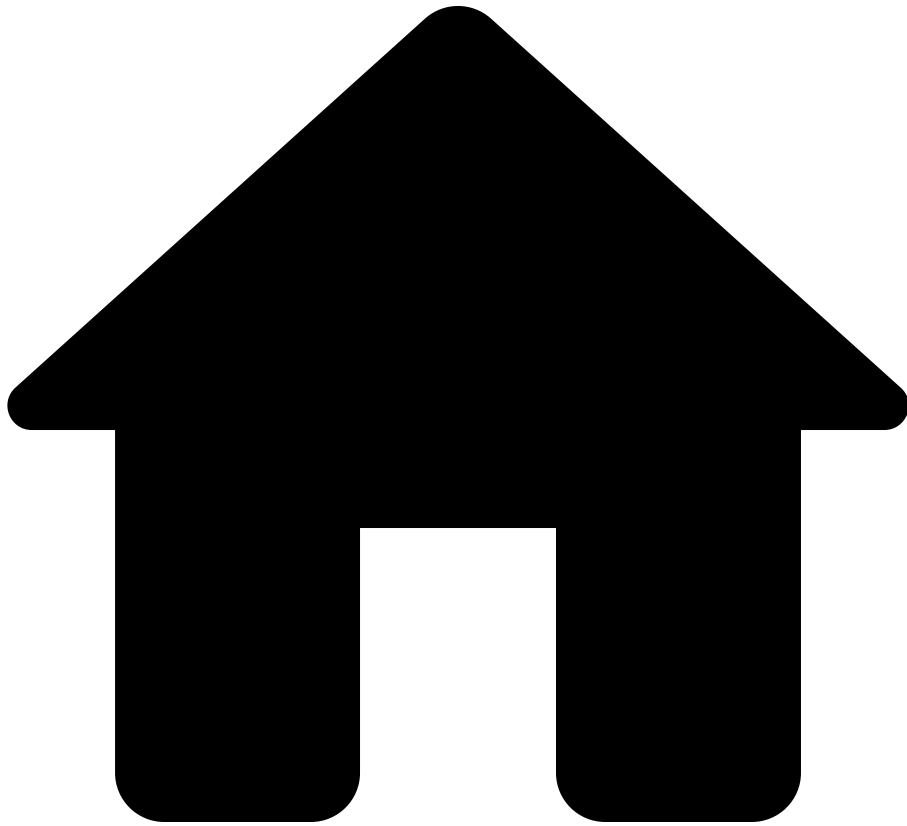
network â devices â outputs â bluecat â config â items â data
â services â ssh â configurations â sshConfiguration â
tacacsâ

See also[ââ](#)

Data model path

- [illegible]

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- StigDatabase

On this page

StigDatabase

Fields

| name | type | nullable | description |
|-------------------|---|----------|-------------|
| policy | StigPolicy | false | -- |
| parameterUsages | Bag< StigParameterUsage > | false | -- |
| defaultParameters | DefaultStigParameters | false | -- |
| guides | Bag< StigGuide > | false | -- |

Used by

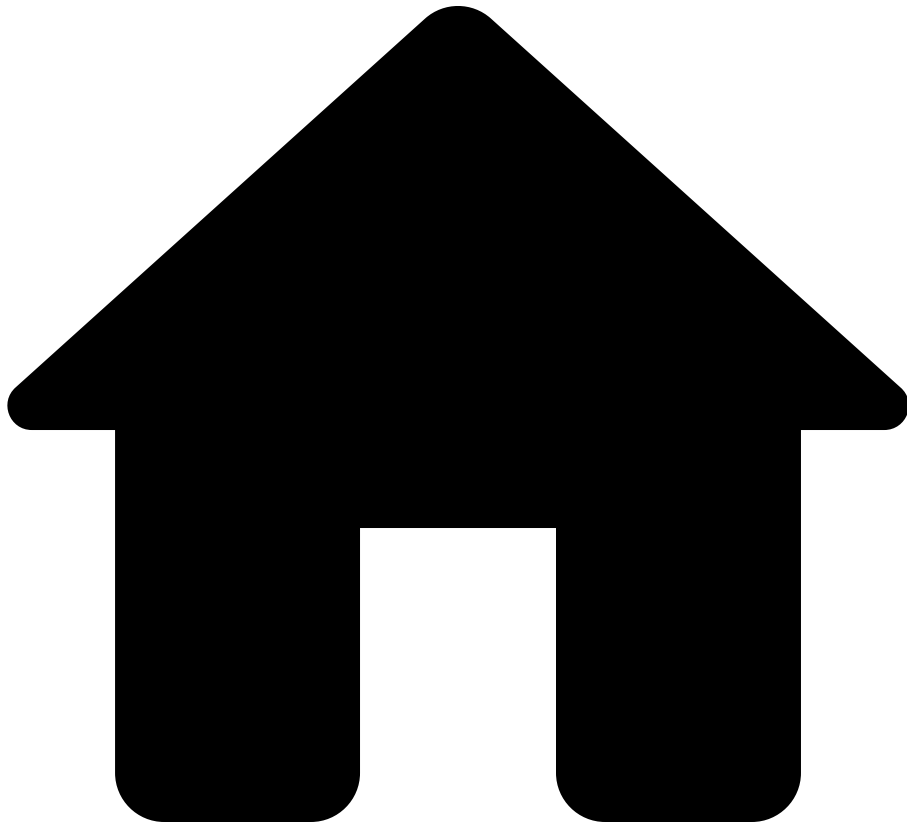
network [StigDatabase](#)

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- StigGuide

On this page

StigGuide

Fields

| name | type | nullable | description |
|----------|---|----------|--|
| name | String | false | The Forward Networks name of the STIG. For example, "Palo Alto Networks NDM" |
| versions | Bag< StigRuleCollection > | false | -- |

Used by

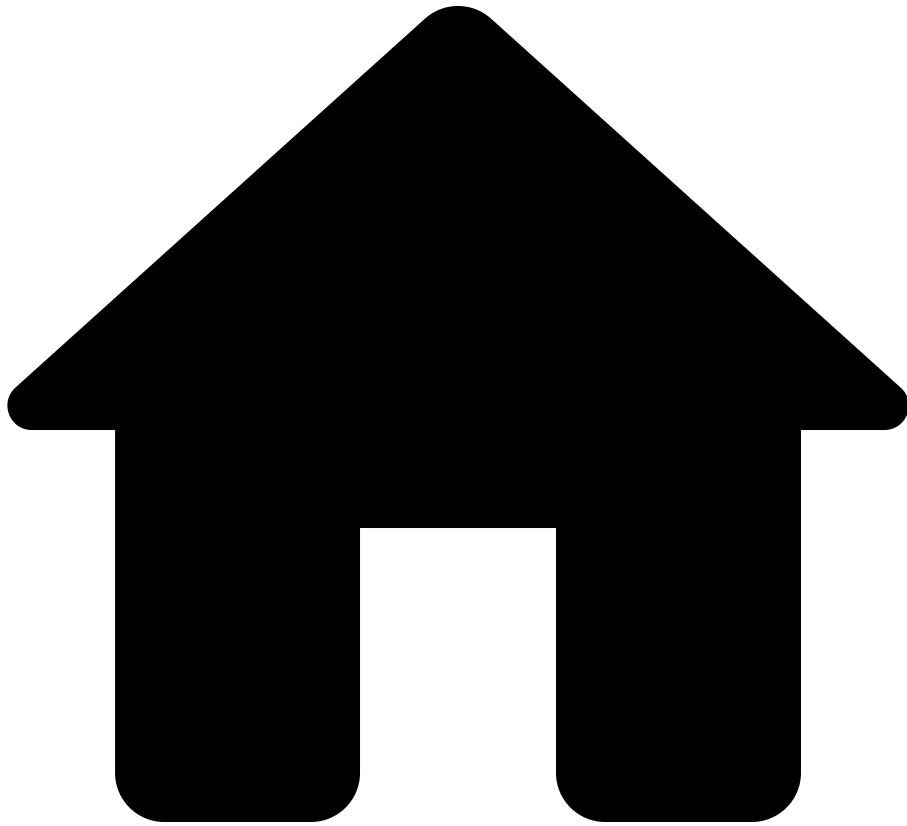
network [stigDatabase](#) [guides](#)

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - guides: Bag< [StigGuide](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- StigParameterUsage

On this page

StigParameterUsage

Fields

| name | type | nullable | description |
|---------------|---------------|----------|--|
| parameterName | String | false | The segmented name of the parameter as it appears in the policy file; for example, "device.allowedServices". |
| rules | Bag< String > | false | The list of vulnerability IDs (e.g. V-123456) of the rules that use the parameter named 'parameterName'. |

Used by

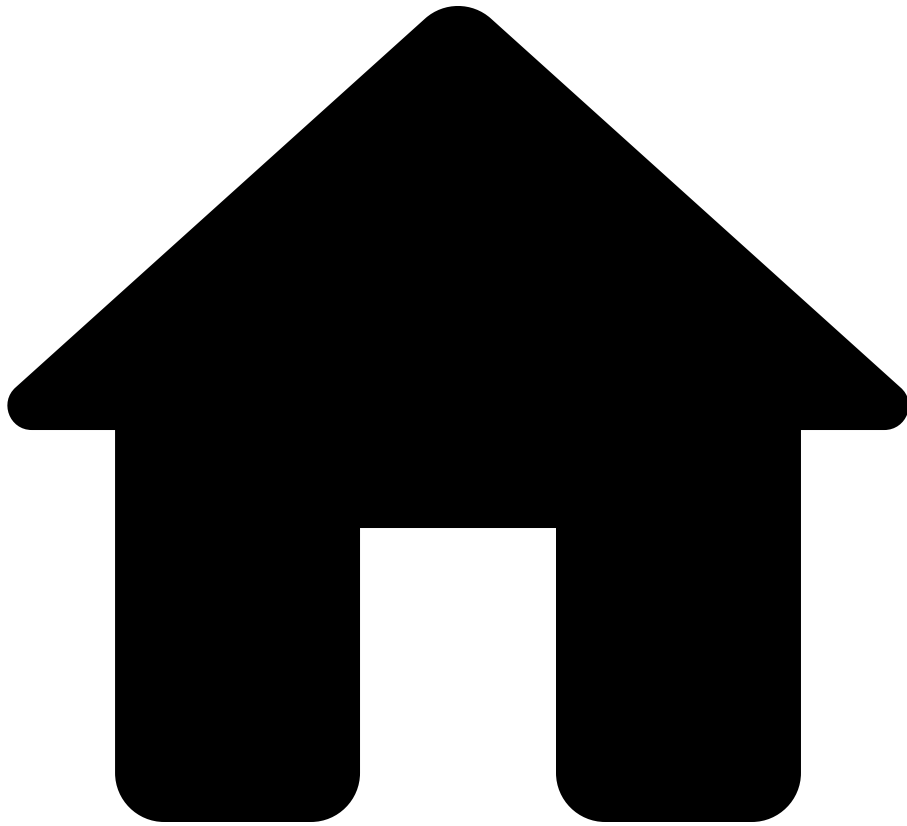
network stigDatabase parameterUsages

See also

Data model path

- network: Network
 - stigDatabase: StigDatabase
 - parameterUsages: Bag< StigParameterUsage >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- StigPolicy

On this page

StigPolicy

Fields

| name | type | nullable | description |
|--------------|---------------------------------|----------|-------------|
| devicePolicy | DevicePolicy | false | -- |
| ifacePolicy | InterfacePolicy | false | -- |
| vrfPolicy | VrfPolicy | false | -- |

Used by

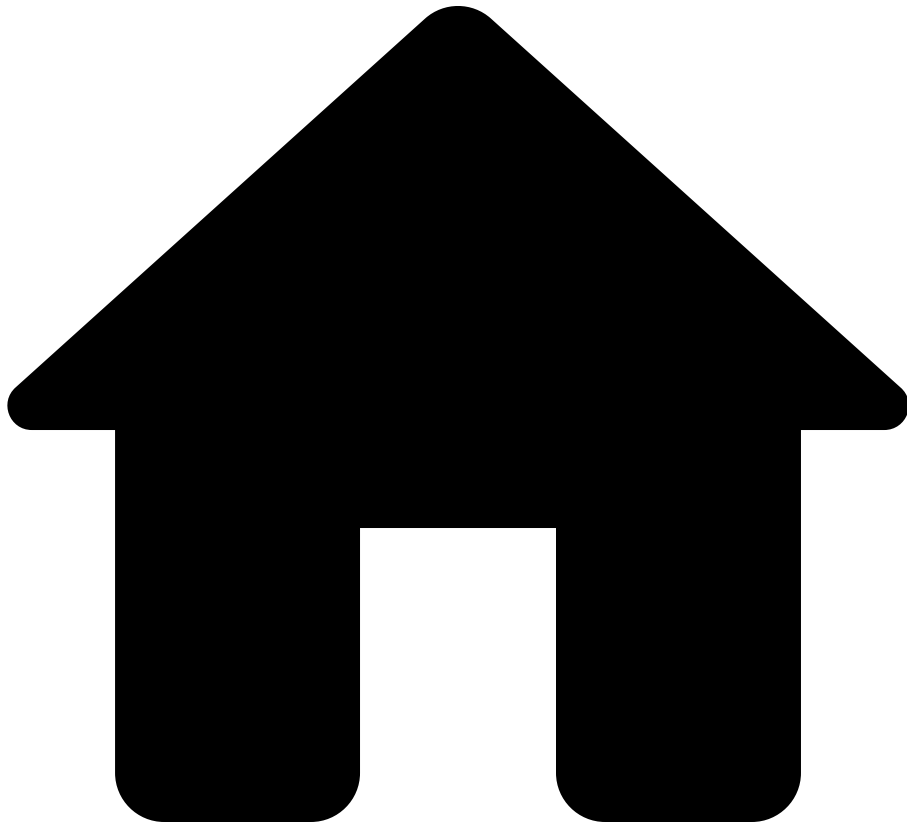
network [StigDatabase](#) policy

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- StigPolicyCondition

On this page

StigPolicyCondition

The condition (a set of constraints) under which a policy applies to a device in the network. The condition is true for a device when all constraint fields on this set are satisfied.

Fields

| name | type | nullable | description |
|-------------------|---------|----------|---|
| ordinal | Integer | false | The order in which this condition appears in the policy. Policy values are sourced from the first condition whose constraints are satisfied. |
| deviceNamePattern | String | false | A regular expression pattern. The constraint is satisfied for a device when the pattern matches its name. If empty, the constraint is satisfied for all devices. |
| ifaceNamePattern | String | false | A regular expression pattern. The constraint is satisfied for an interface when the pattern matches its name, or for a device when the pattern matches any interface on the device. If empty, the constraint is satisfied for all interfaces. |
| vrfName | String | false | A VRF name. The constraint is satisfied for a VRF when the name matches exactly. |

| name | type | nullable | description |
|---------------|---------------|----------|--|
| | | | If empty, the constraint is satisfied for all VRFs. |
| tagNames | Bag< String > | false | A set of tag names. The constraint is satisfied for a device when it is tagged by any of the provided names. If empty, the constraint is satisfied for all devices. |
| locationNames | Bag< String > | false | A set of location names. The constraint is satisfied for a device when its location is any of the provided names. If empty, the constraint is satisfied for all devices. |

Used by

network [StigDatabase](#) [policy](#) [devicePolicy](#) [allowedServices](#) [condition](#)

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - devicePolicy: [DevicePolicy](#)
 - allowedServices:
 - Bag< [AllowedServicesDevicePolicy](#) >
 - condition: [StigPolicyCondition](#)

network **stigDatabase** **policy** **devicePolicy**
allowedCertificationAuthorityEnrollmentUrls **condition**

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - devicePolicy: [DevicePolicy](#)
 - allowedCertificationAuthorityEnrollmentUrls:
Bag< [AllowedCertificationAuthorityEnrollmentUrlsDevice](#)
 - condition: [StigPolicyCondition](#)

network **stigDatabase** **policy** **devicePolicy**
managementSubnets **condition**

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - devicePolicy: [DevicePolicy](#)
 - managementSubnets:
Bag< [ManagementSubnetsDevicePolicy](#) >
 - condition: [StigPolicyCondition](#)

network **stigDatabase** **policy** **devicePolicy** **isProviderEdge**
condition

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - devicePolicy: [DevicePolicy](#)
 - isProviderEdge: Bag< [IsProviderEdgeDevicePolicy](#) >
 - condition: [StigPolicyCondition](#)

network **stigDatabase** **policy** **devicePolicy**
allowedBgpPeers **condition**

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - devicePolicy: [DevicePolicy](#)
 - allowedBgpPeers:
Bag< [AllowedBgpPeersDevicePolicy](#) >
 - condition: [StigPolicyCondition](#)

network **stigDatabase** **policy** **ifacePolicy**
requiresDefaultVlan **condition**

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - ifacePolicy: [InterfacePolicy](#)
 - requiresDefaultVlan:
Bag< [RequiresDefaultVlanInterfacePolicy](#) >
 - condition: [StigPolicyCondition](#)

network **stigDatabase** **policy** **ifacePolicy** **requiresMulticast**
condition

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - ifacePolicy: [InterfacePolicy](#)
 - requiresMulticast:
Bag< [RequiresMulticastInterfacePolicy](#) >
 - condition: [StigPolicyCondition](#)

network **stigDatabase** **policy** **ifacePolicy** **isHostGateway**
condition

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - ifacePolicy: [InterfacePolicy](#)
 - isHostGateway:
Bag< [IsHostGatewayInterfacePolicy](#) >
 - condition: [StigPolicyCondition](#)

network **stigDatabase** **policy** **ifacePolicy**
pseudowireVirtualCircuitId **condition**

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - ifacePolicy: [InterfacePolicy](#)
 - pseudowireVirtualCircuitId:
Bag< [VirtualCircuitIdInterfacePolicy](#) >
 - condition: [StigPolicyCondition](#)

network **stigDatabase** **policy** **ifacePolicy** **connectivity** **toExternalDevice** **condition**

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - ifacePolicy: [InterfacePolicy](#)
 - connectivity: [InterfaceConnectivityPolicy](#)
 - toExternalDevice:
 - Bag< [IsExternalInterfacePolicy](#) >
 - condition: [StigPolicyCondition](#)

network **stigDatabase** **policy** **ifacePolicy** **connectivity** **toAccessLayerSwitch** **condition**

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - ifacePolicy: [InterfacePolicy](#)
 - connectivity: [InterfaceConnectivityPolicy](#)
 - toAccessLayerSwitch:
 - Bag< [ConnectsToAccessLayerSwitchInterfacePolicy](#) >
 - condition: [StigPolicyCondition](#)

network **stigDatabase** **policy** **ifacePolicy** **connectivity** **toUntrustedDevice** **condition**

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - ifacePolicy: [InterfacePolicy](#)
 - connectivity: [InterfaceConnectivityPolicy](#)
 - toUntrustedDevice:
Bag< [IsUntrustedInterfacePolicy](#) >
 - condition: [StigPolicyCondition](#)

network **stigDatabase** **policy** **ifacePolicy** **connectivity** **toCoreLayer** **condition**

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - ifacePolicy: [InterfacePolicy](#)
 - connectivity: [InterfaceConnectivityPolicy](#)
 - toCoreLayer:
Bag< [ConnectsToCoreLayerInterfacePolicy](#) >
 - condition: [StigPolicyCondition](#)

network **stigDatabase** **policy** **ifacePolicy** **connectivity**
viaFiberOptics **condition**

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - ifacePolicy: [InterfacePolicy](#)
 - connectivity: [InterfaceConnectivityPolicy](#)
 - viaFiberOptics:
Bag< [ConnectsViaFiberOpticsInterfacePolicy](#) >
 - condition: [StigPolicyCondition](#)

network **stigDatabase** **policy** **ifacePolicy** **connectivity**
toCustomerEdgeDevice **condition**

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - ifacePolicy: [InterfacePolicy](#)
 - connectivity: [InterfaceConnectivityPolicy](#)
 - toCustomerEdgeDevice:
Bag< [ConnectsToCustomerEdgeDeviceInterfacePolicy](#) >
 - condition: [StigPolicyCondition](#)

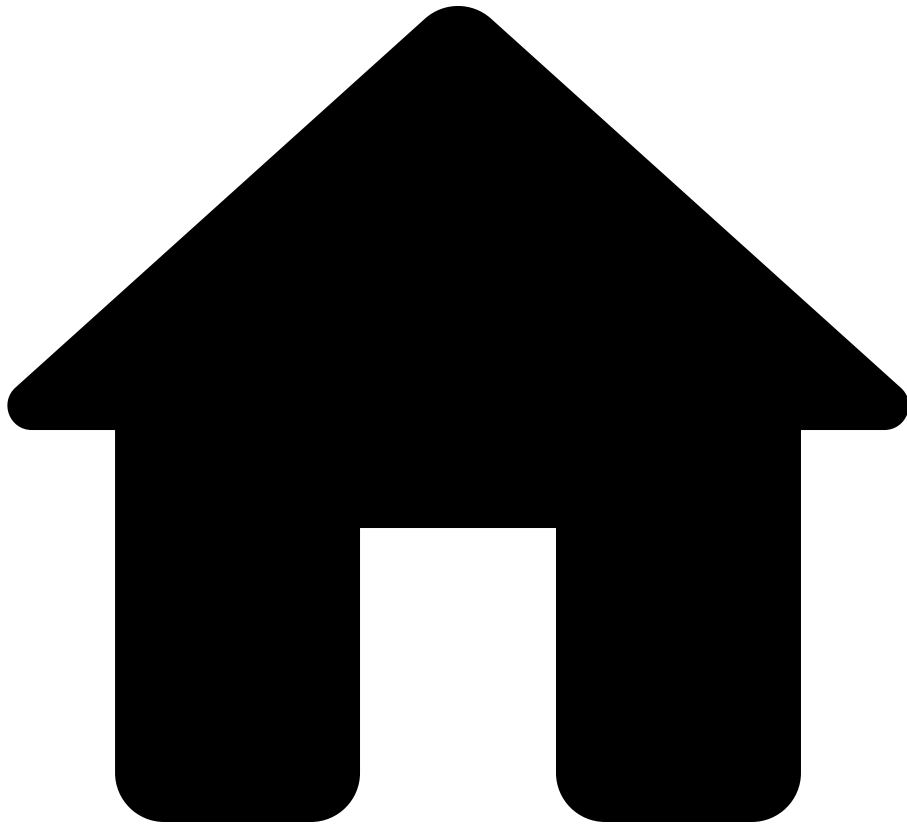
network **stigDatabase** **policy** **ifacePolicy** **connectivity** **toDodinBackbone** **condition**

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - ifacePolicy: [InterfacePolicy](#)
 - connectivity: [InterfaceConnectivityPolicy](#)
 - toDodinBackbone:
 - Bag< [ConnectsToDodinDeviceInterfacePolicy](#) >
 - condition: [StigPolicyCondition](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- StigRule

On this page

StigRule

Fields

| name | type | nullable | description |
|-----------------------|------------------------------|----------|--|
| id | String | false | The ruleId, such as SV-220623r863275_rule. |
| vulnerabilityId | String | false | -- |
| version | String | false | Rule version, such as CISC-L2-000020. |
| intent | String | false | -- |
| description | String | false | -- |
| checkContent | String | false | -- |
| fixText | String | false | -- |
| severity | StigSeverity | false | -- |
| legacyVulnerabilities | Bag< String > | false | -- |
| groupTitle | String | false | Group title, such as SRG-NET-000148-L2S-000015 |

Used by[↗](#)

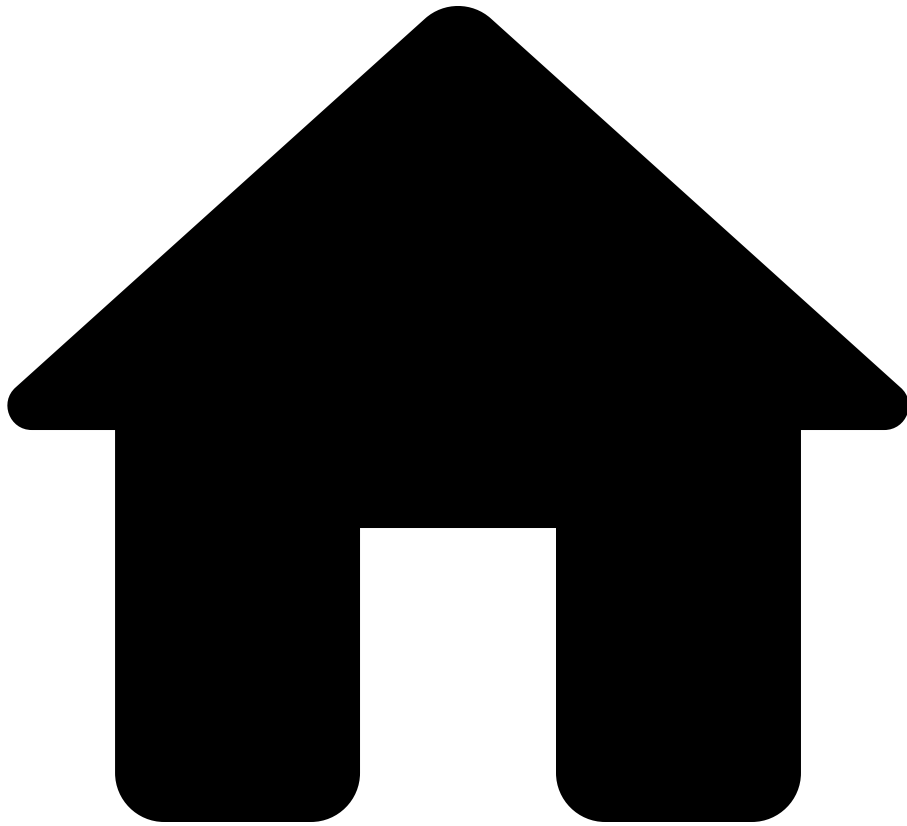
network [↗](#) **stigDatabase** [↗](#) **guides** [↗](#) **versions** [↗](#) **rules**[↗](#)

See also[↗](#)

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - guides: Bag< [StigGuide](#) >
 - versions: Bag< [StigRuleCollection](#) >
 - rules: Bag< [StigRule](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- StigRuleCollection

On this page

StigRuleCollection

Fields

| name | type | nullable | description |
|-------|---------------------------------|----------|---|
| id | String | false | The version identifier, such as "V1R4". |
| rules | Bag< StigRule > | false | -- |

Used by

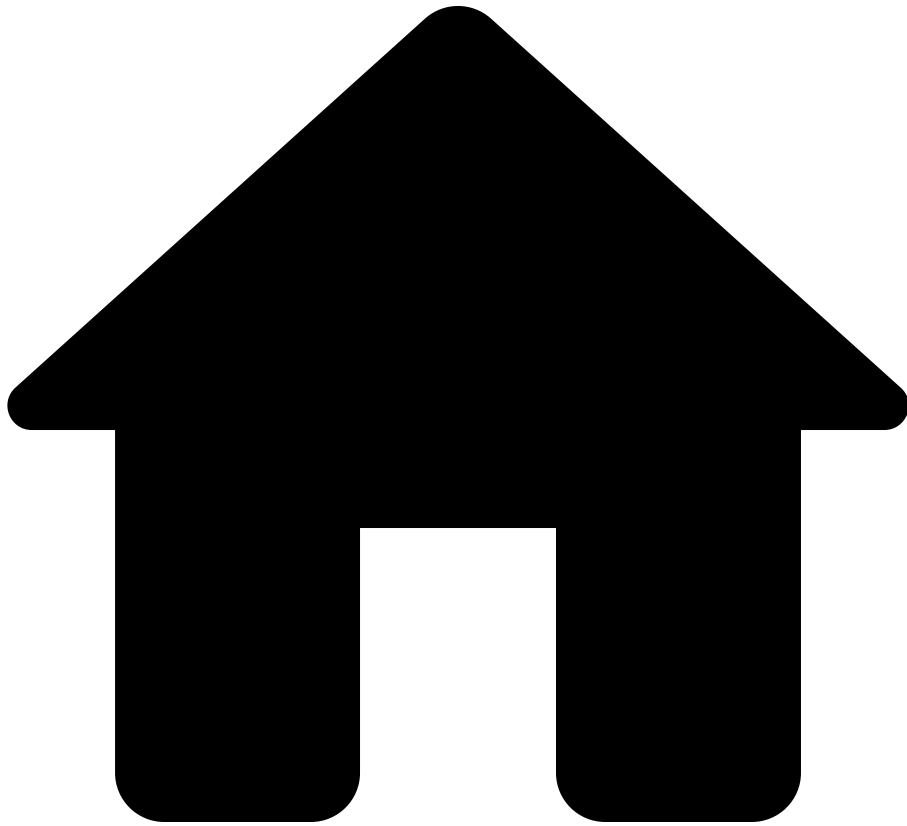
network [stigDatabase](#) [guides](#) [versions](#)

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - guides: Bag< [StigGuide](#) >
 - versions: Bag< [StigRuleCollection](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- StigSeverity

On this page

StigSeverity

Options

| name | description |
|--------|-------------|
| HIGH | -- |
| LOW | -- |
| MEDIUM | -- |

Used by

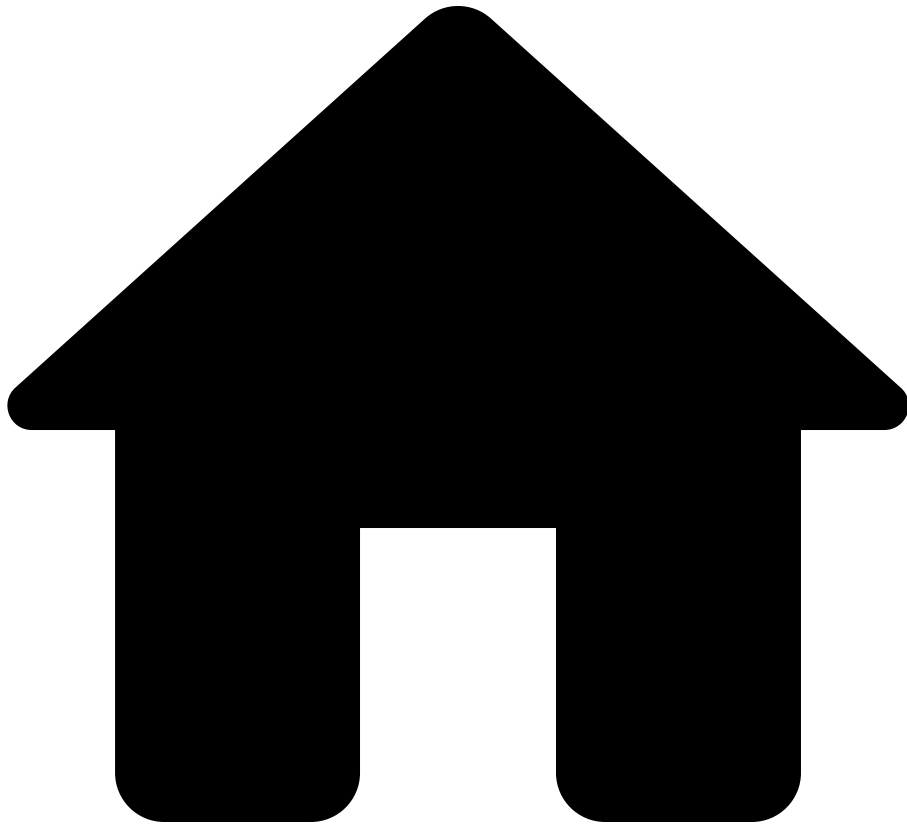
network **stigDatabase** **guides** **versions** **rules** **severity**

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - guides: Bag< [StigGuide](#) >
 - versions: Bag< [StigRuleCollection](#) >
 - rules: Bag< [StigRule](#) >
 - severity: [StigSeverity](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- Stp

On this page

Stp

Top-level container for spanning tree configuration and state data.

Fields

| name | type | nullable | description |
|-----------|-----------|----------|---|
| rstp | Rstp | false | Rapid Spanning-tree protocol configuration and operation data. |
| mstp | Mstp | false | Multi Spanning-tree protocol configuration and operation data. |
| rapidPvst | RapidPvst | false | Rapid per vlan Spanning-tree protocol configuration and operation data. |

Used by

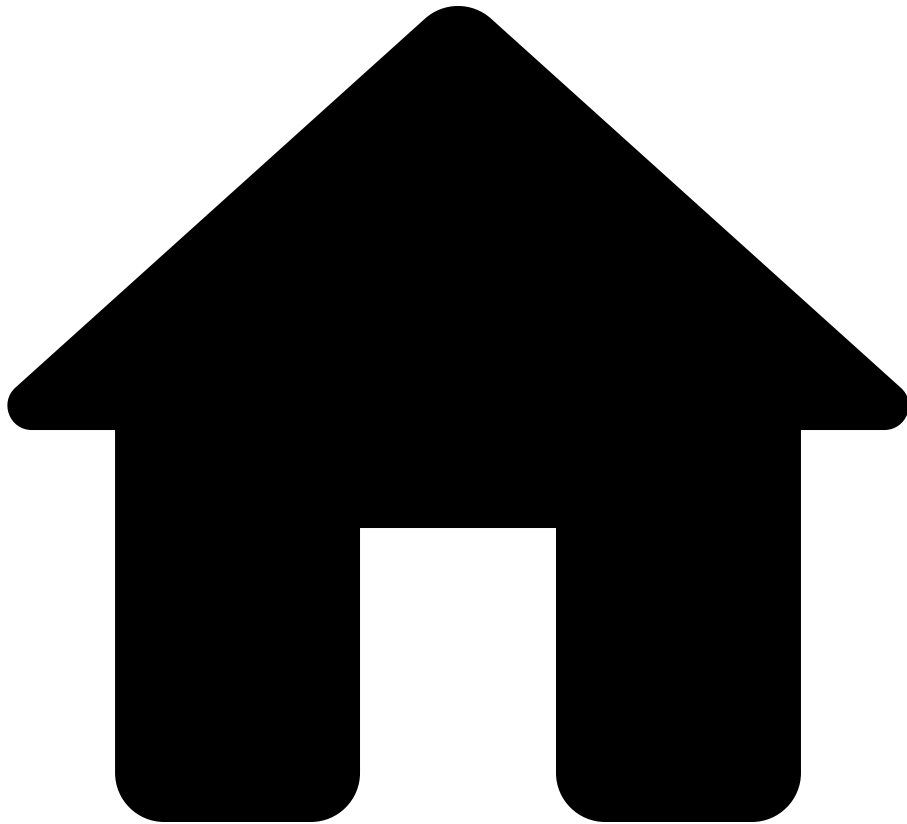
network devices stp

See also

Data model path

- network: Network
 - devices: Bag< Device >
 - stp: Stp

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SubInterface

On this page

SubInterface

Subinterface data for logical interfaces associated with a given interface.

Fields

| name | type | nullable | description |
|---------|-----------------------|----------|---|
| name | String | false | The system-assigned name for the sub-interface. This MAY be a combination of the base interface name and the subinterface index, or some other convention used by the system. |
| layer | Layer | false | Denotes whether this is a Layer 2 (physical) or Layer 3 (logical) interface. |
| aliases | Bag<String> | false | The set of all names that refer to this subinterface. For example, on Cisco, an interface named "eth1.100" might also be referred to as "Ethernet1.100" |

| name | type | nullable | description |
|-------------|----------------------------------|----------|---|
| | | | This field would then include both "eth1.100" and "Ethernet1.100" |
| description | String | true | A textual description of the interface. |
| adminStatus | AdminStatus | false | The desired state of the interface. Here, it reflects the administrative state as set by enabling or disabling the interface. |
| operStatus | OperStatus | false | The current operational state of the interface. |
| acls | IfaceAcls | false | Access control list details specific to this interface. |
| vlan | SubInterfaceVlan | true | -- |
| ipv4 | IfaceIpv4Info | false | Configuration and state for IPv4 interfaces |
| ipv6 | IfaceIpv6Info | false | Configuration and state for IPv6 interfaces |

| name | type | nullable | description |
|----------------------------------|---|-------------------|--|
| <code>switchedVlans</code> | SwitchedSubInterfaceVlans | <code>true</code> | Switched VLAN configuration of the subinterface. Non-null if this interface is a switch-subinterface. |
| <code>networkInstanceName</code> | <code>String</code> | <code>true</code> | The network instance (i.e. VRF) that this interface is part of. Non-null if this interface is not a switch-subinterface. |

Used by[â](#)

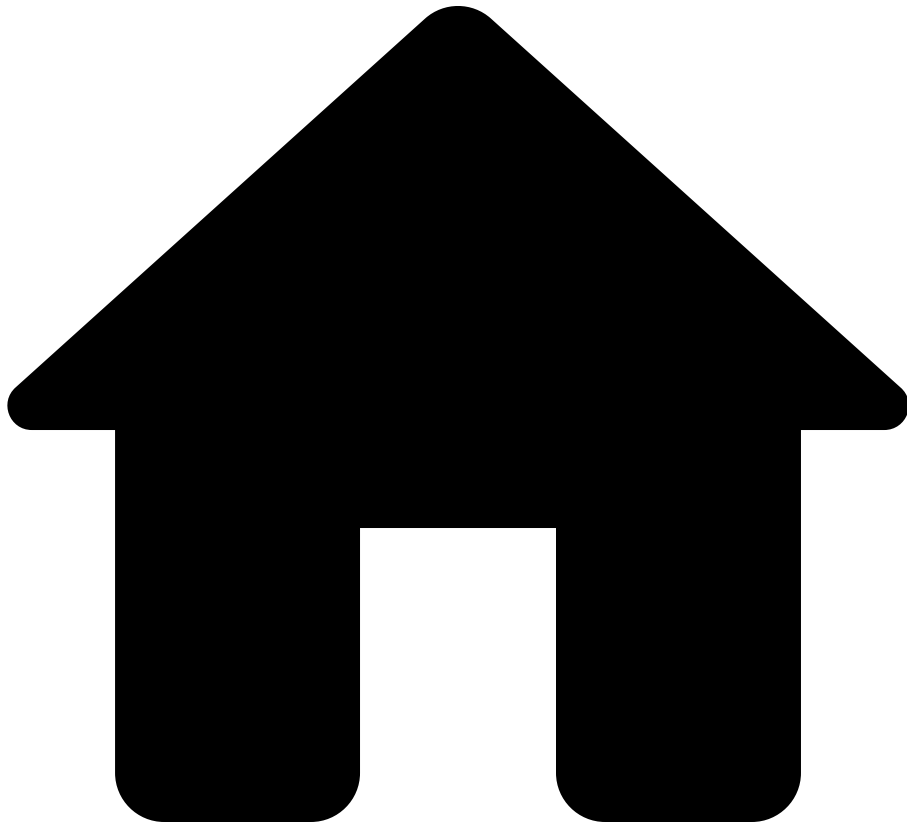
network [â](#) **devices** [â](#) **interfaces** [â](#) **subinterfaces**[â](#)

See also[â](#)

Data model path

- network: `Network`
 - devices: Bag< `Device` >
 - interfaces: Bag< `Iface` >
 - subinterfaces: Bag< `SubInterface` >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SubInterfaceVlan

On this page

SubInterfaceVlan

VLAN id (single or Q-in-Q) for a subinterface. The id is scoped to the subinterface, and could be repeated on different subinterfaces.

Options

| name | type | description |
|---------|---------|--|
| QINQ_ID | String | Type definition representing a single double-tagged/ QinQ VLAN identifier. The format of a QinQ VLAN-ID is x.y where X is the 'outer' VLAN identifier, and y is the 'inner' VLAN identifier. Both x and y must be valid VLAN IDs (1 <= vlan-id <= 4094) with the exception that y may be equal to a wildcard (*). In cases where y is set to the wildcard, this represents all inner VLAN identifiers where the outer VLAN identifier is equal to x. |
| VLAN_ID | Integer | A single VLAN ID configured for the subinterface. |

Used by

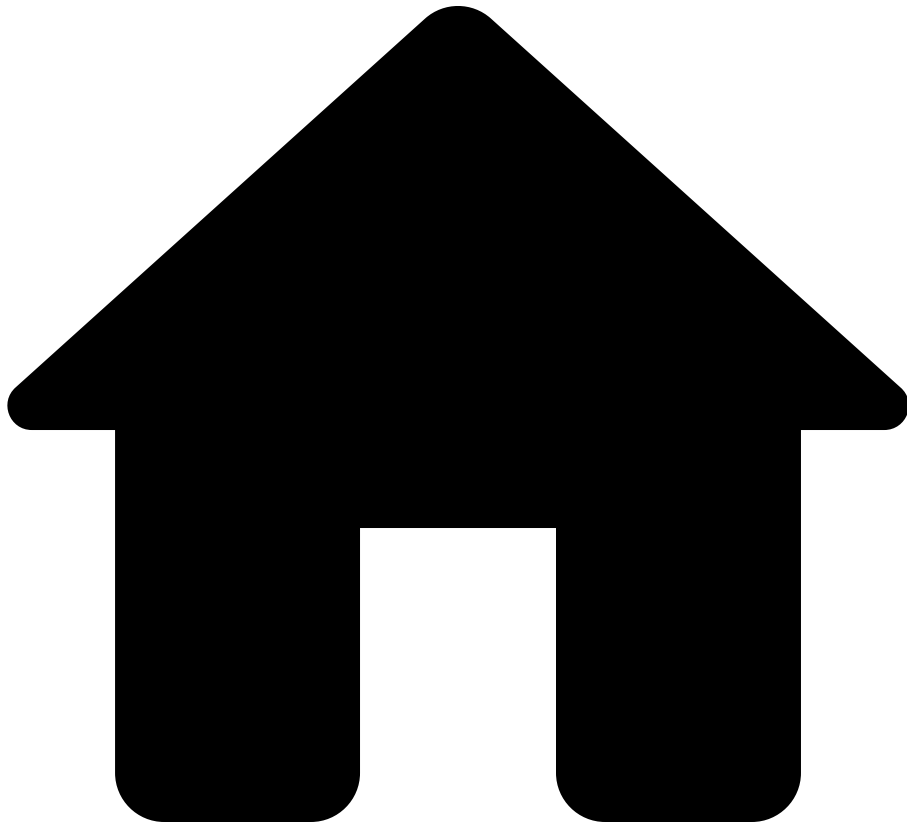
network devices interfaces subinterfaces vlan

See also

Data model path

- network: Network
 - devices: Bag< Device >
 - interfaces: Bag< Iface >
 - subinterfaces: Bag< SubInterface >
 - vlan: SubInterfaceVlan

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- Subnet

On this page

Subnet

Fields

| name | type | nullable | description |
|--------------------|-----------------------------------|----------|-------------|
| name | String | true | -- |
| id | String | false | -- |
| tags | Bag< String > | false | -- |
| addresses | Bag< IpSubnet > | false | -- |
| region | String | false | -- |
| availabilityZone | String | false | -- |
| ifaces | Bag< CloudIface > | false | -- |
| routeTableId | String | false | -- |
| serviceEndpointIds | Bag< String > | false | -- |

Used by

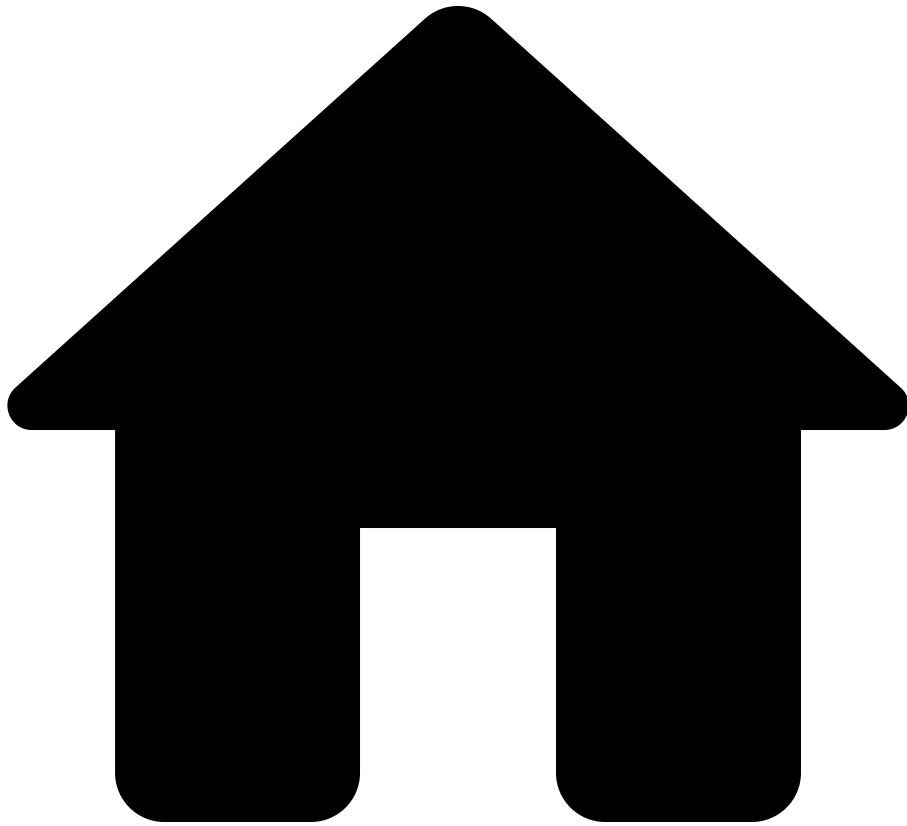
network [cloudAccounts](#) [vpcs](#) [subnets](#)

See also

Data model path

- network: [Network](#)
 - cloudAccounts: Bag< [CloudAccount](#) >
 - vpcs: Bag< [VpcData](#) >
 - subnets: Bag< [Subnet](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SwitchedSubInterfaceVlans

On this page

SwitchedSubInterfaceVlans

Fields

| name | type | nullable | description |
|------------|--------------------------------|----------|--|
| outerVlans | VlanIdSetOrAll | true | The outer VLANs configured on a switch-subinterface. |
| innerVlans | VlanIdSetOrAll | true | The inner VLANs configured on a switch-subinterface. |

Used by

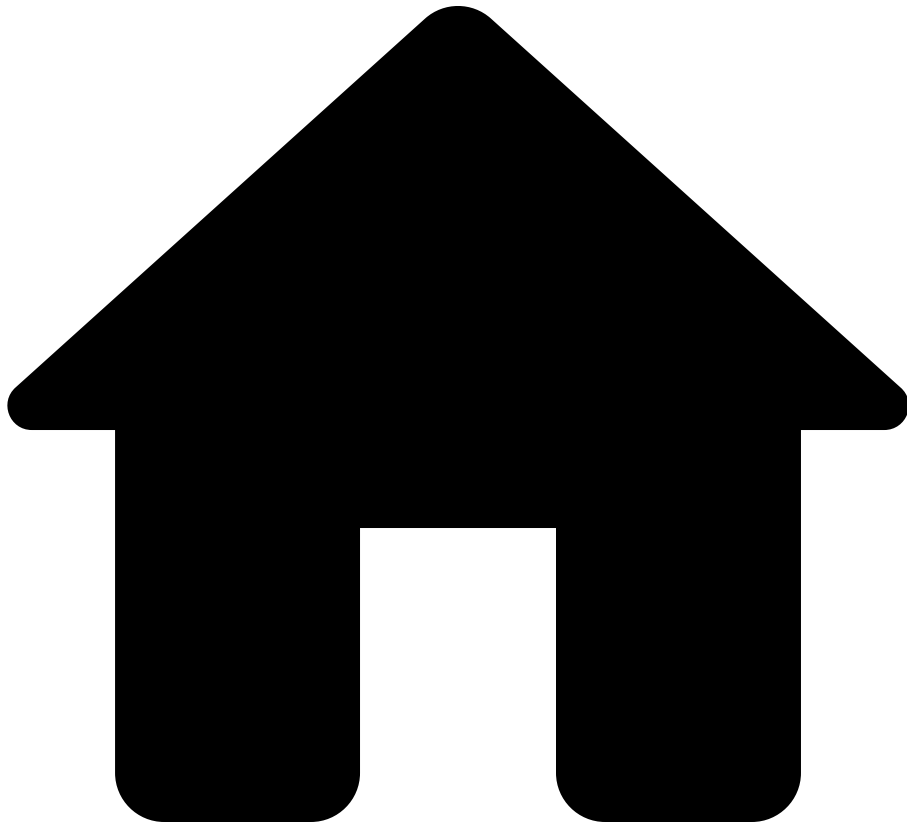
network devices interfaces subinterfaces
switchedVlans

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - interfaces: Bag< [Iface](#) >
 - subinterfaces: Bag< [SubInterface](#) >
 - switchedVlans: [SwitchedSubInterfaceVlans](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SwitchedVlan

On this page

SwitchedVlan

Type for VLAN interface-specific data on Ethernet interfaces. These are for standard L2, switched-style VLANs.

Fields

| name | type | nullable | description |
|---------------|------------------------------|----------|---|
| interfaceMode | VlanModeType | false | Access or trunk mode for VLANs. |
| nativeVlan | Integer | true | Native VLAN is valid for trunk mode interface. |
| accessVlan | Integer | true | Access VLAN assigned to the interface. |
| voiceVlan | Integer | true | Voice VLAN assigned to the interface. Not present on non-access interfaces. |
| defaultVlan | Integer | true | When present, frames tagged with this VLAN ID are accepted on the interface and are mapped to the same VLAN to which untagged traffic is mapped. Specifically, if the interface is an access interface, then the default VLAN is mapped to the access VLAN, while if the interface is a trunk interface, then the default VLAN is |

| name | type | nullable | description |
|------------|--------------------------------------|----------|---|
| | | | mapped to the native VLAN. |
| trunkVlans | Bag< VlanIdOrRange > | false | Specifies VLANs, or ranges thereof, that the interface may carry when in trunk mode. If not specified, all VLANs are allowed on the interface. Ranges are specified in the form x..y, where x<y - ranges are assumed to be inclusive (such that the VLAN range is x <= range <= y). |

Used by[â](#)

network [â](#) devices [â](#) interfaces [â](#) ethernet [â](#) switchedVlan[â](#)

See also[â](#)

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - interfaces: Bag< [Iface](#) >
 - ethernet: [Ethernet](#)
 - switchedVlan: [SwitchedVlan](#)

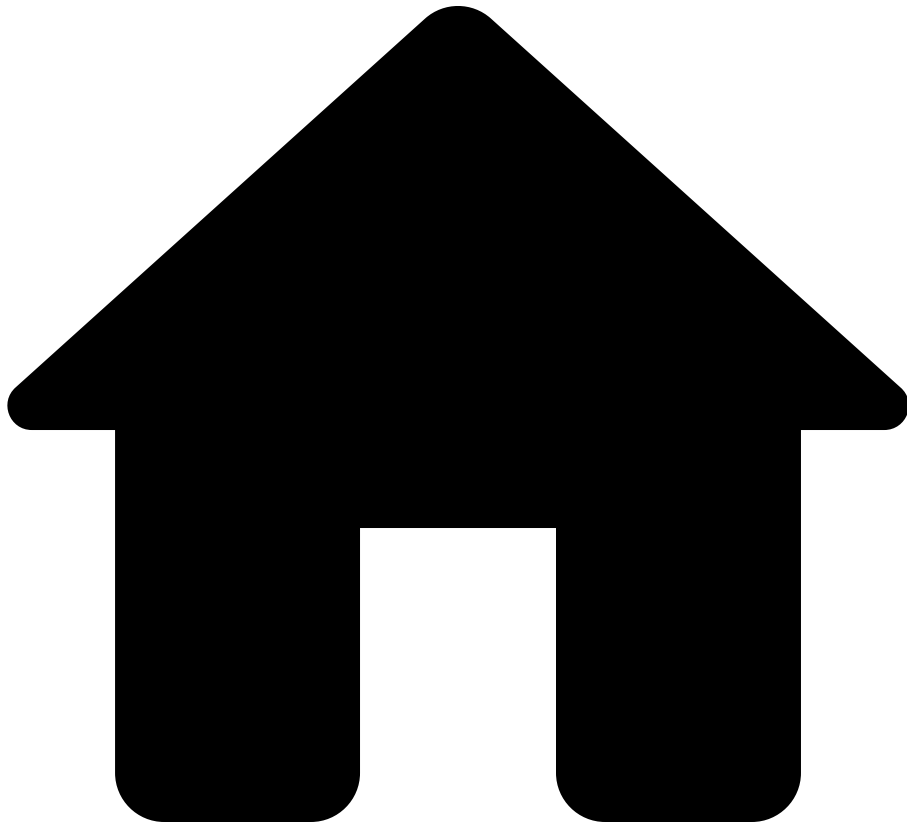
network **devices** **interfaces** **aggregation** **switchedVlan**

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - interfaces: Bag< [Iface](#) >
 - aggregation: [Aggregation](#)
 - switchedVlan: [SwitchedVlan](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SyslogConfigurationDetail

On this page

SyslogConfigurationDetail

Fields

| name | type | nullable | description |
|----------|-------------------------------------|----------|-------------|
| qradar | QRadarConfig | false | -- |
| arcsight | ArcsightConfig | false | -- |
| servers | Bag< SyslogServer > | false | -- |

Used by an

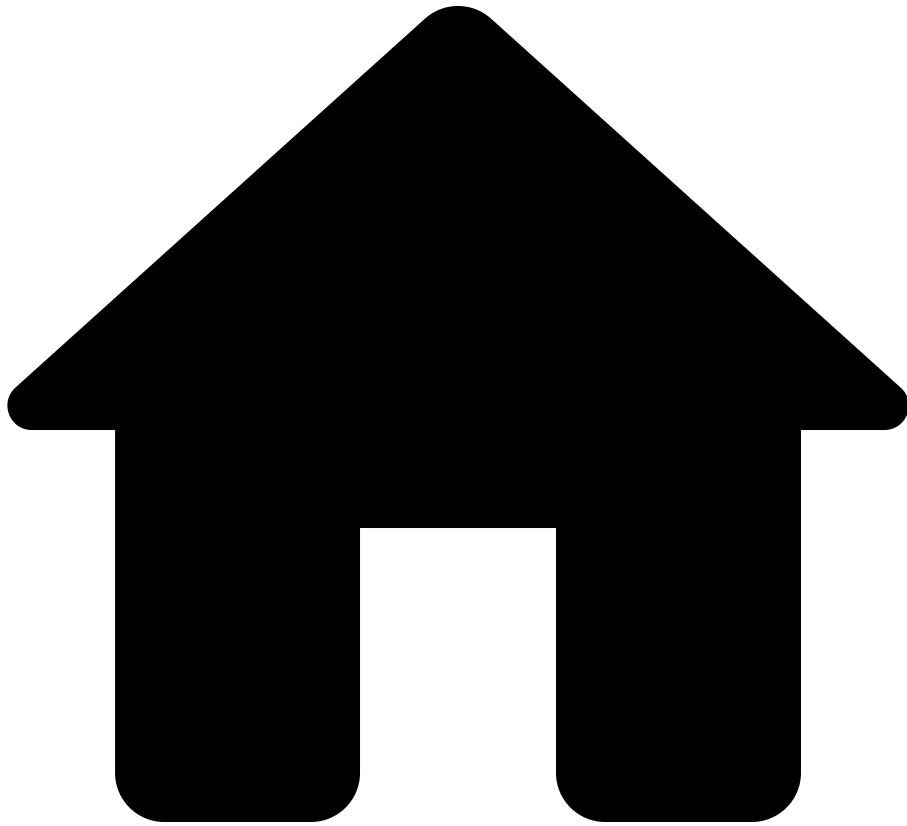
network > devices > outputs > bluecat > config > items > data
> services > syslog > configurations > syslogConfiguration

See also[ââ](#)

Data model path

- [illegible]

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SyslogServer

On this page

SyslogServer

Fields

| name | type | nullable | description |
|-----------|-----------|----------|-------------|
| ip | IpAddress | false | -- |
| port | Integer | false | -- |
| transport | String | false | -- |

Used by â

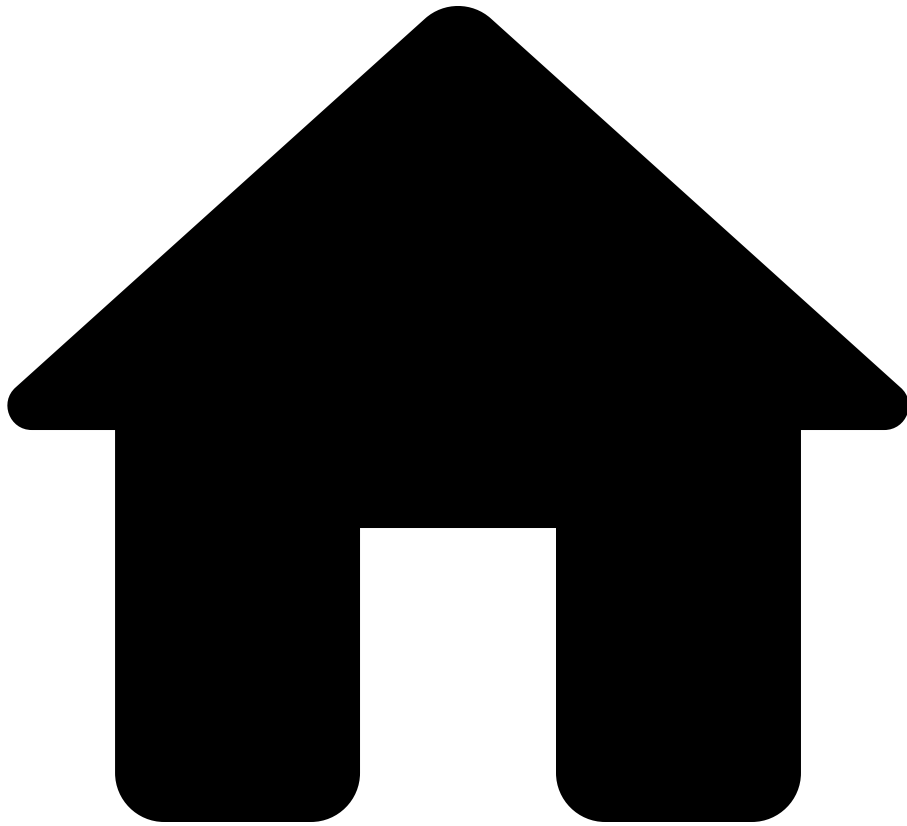
network â devices â outputs â bluecat â config â items â data
â services â syslog â configurations â syslogConfiguration â
serversâ

See also [ââ](#)

Data model path

- [illegible]

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SyslogService

On this page

SyslogService

Fields

| name | type | nullable | description |
|----------------|---|----------|-------------|
| configurations | Bag< SyslogServiceConfiguration > | false | -- |

Used by

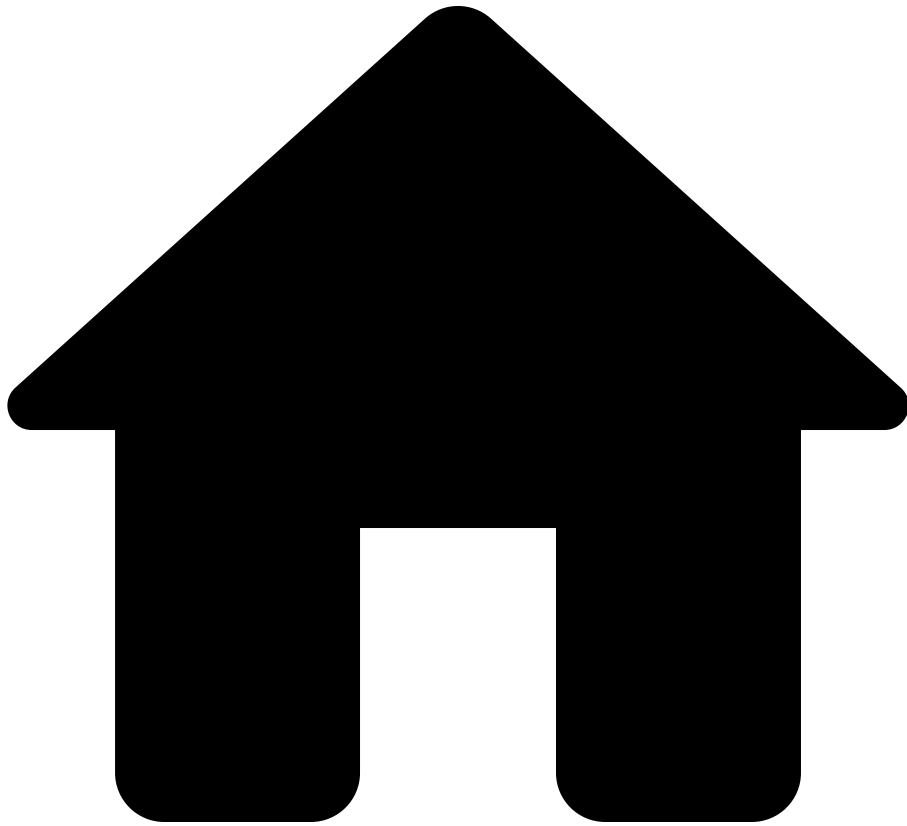
network devices outputs bluecat config items data
services syslog

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - outputs: [Outputs](#)
 - bluecat: [BluecatOutputs](#)
 - config: [BluecatConfig](#)
 - items: Bag< [BluecatConfigItem](#) >
 - data: [BluecatConfigItemData](#)
 - services: [BluecatServices](#)
 - syslog: [SyslogService](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- SyslogServiceConfiguration

On this page

SyslogServiceConfiguration

Fields

| name | type | nullable | description |
|---------------------|---|----------|-------------|
| syslogConfiguration | SyslogConfigurationDetail | false | -- |

Used by

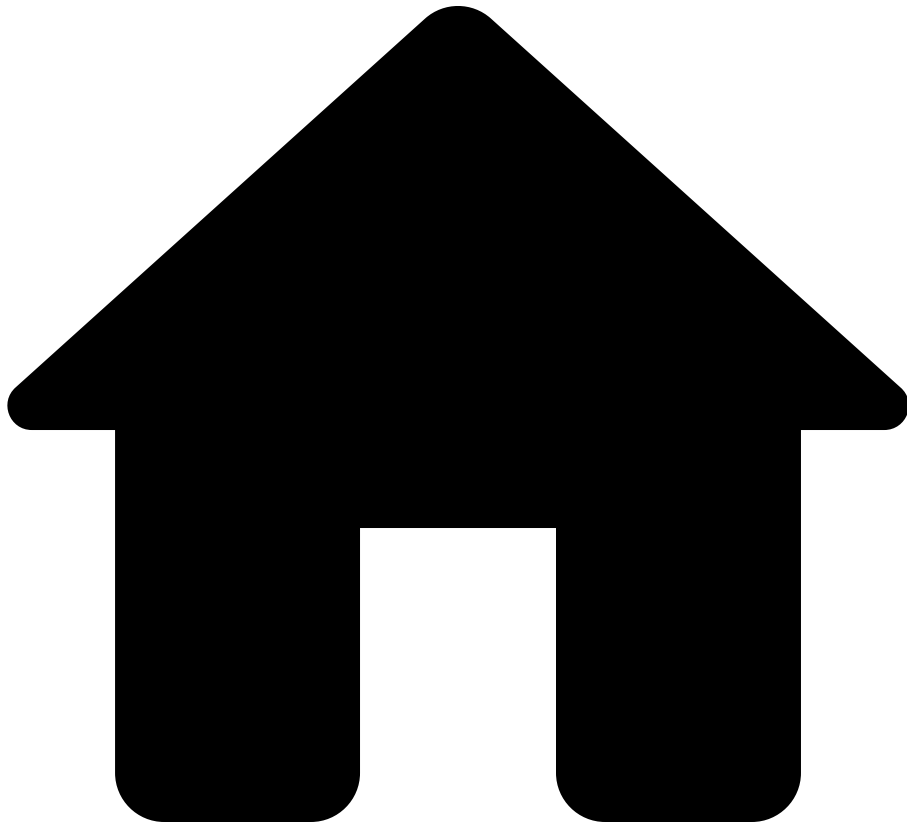
network devices outputs bluecat config items data
services syslog configurations

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - outputs: [Outputs](#)
 - bluecat: [BluecatOutputs](#)
 - config: [BluecatConfig](#)
 - items: Bag< [BluecatConfigItem](#) >
 - data: [BluecatConfigItemData](#)
 - services: [BluecatServices](#)
 - syslog: [SyslogService](#)
 - configurations: Bag< [SyslogServiceConfiguration](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- System

On this page

System

Fields

| name | type | nullable | description |
|---------------|---------------|----------|---|
| hostNames | Bag< String > | false | Configured host names on the device |
| otherNames | Bag< String > | false | Other names that the device is also known by in the snapshot. Usually obtained from configuration settings such as <code>hostname</code> and <code>lldp system-name</code> . Other names can be advertised to neighbors and show up in lldp/cdp links. |
| physicalName | String | false | Name of the physical device of which this modeled device is a part. Differs from the device name only if the device is virtual. For devices that are actually virtual contexts within a physical device, this value will be the name of the containing physical device. |
| uptimeSeconds | Integer | true | Uptime for this device in seconds. Present for systems that were successfully collected and have one of these operating systems: APIC, FXOS, IOS, IOS_XE, IOS_XR, NXOS, SG, |

| name | type | nullable | description |
|--------|----------|----------|--|
| | | | ASA, ARISTA_EOS, JUNOS, SRX, NETSCREEN, FORTINET, F5, PAN_OS, CHECKPOINT. |
| uptime | Duration | true | Uptime for this device.
Present for systems that were successfully collected and have one of these operating systems: APIC, FXOS, IOS, IOS_XE, IOS_XR, NXOS, SG, ASA, ARISTA_EOS, JUNOS, SRX, NETSCREEN, FORTINET, F5, PAN_OS, CHECKPOINT, DELL_OS6, DELL_OS9, DELL_OS10, ARUBA_AOS_CX. |

Used by

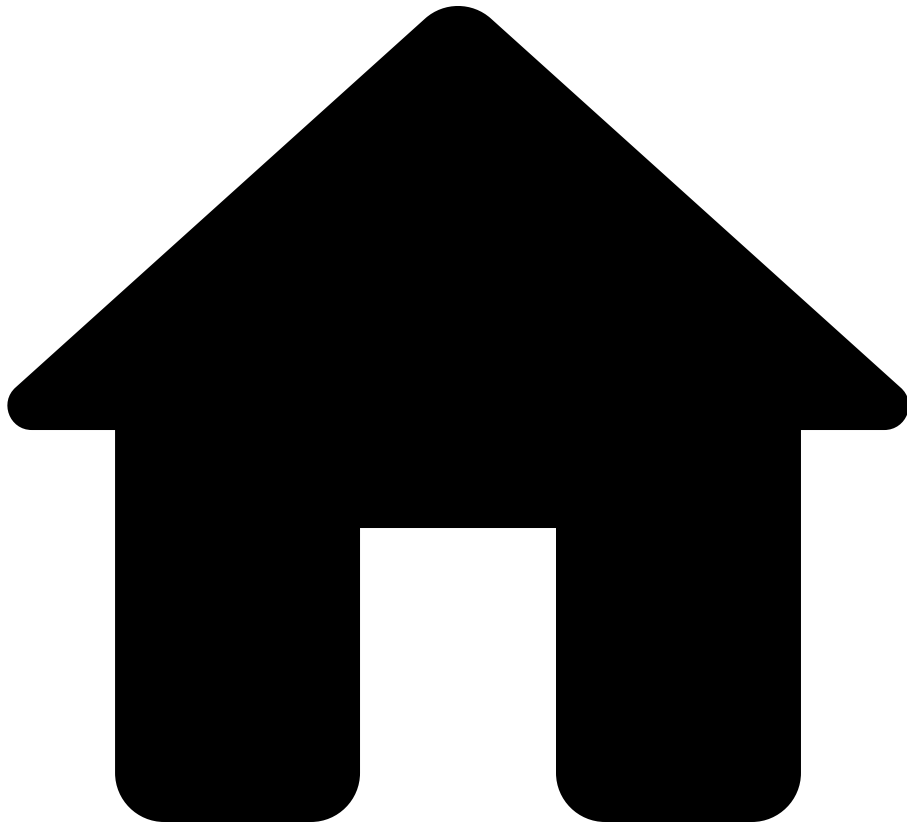
network devices system

See also

Data model path

- network: Network
 - devices: Bag< Device >
 - system: System

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- TeTunnel

On this page

TeTunnel

Configuraton parameters relating to a TE-tunnel interface.

Fields

| name | type | nullable | description |
|---------------------|--|----------|---|
| nextHops | Bag< TeTunnelNextHop > | false | Next hops for the tunnel. |
| networkInstanceName | String | false | The network instance (i.e. VRF) that this interface is part of. |

Used by

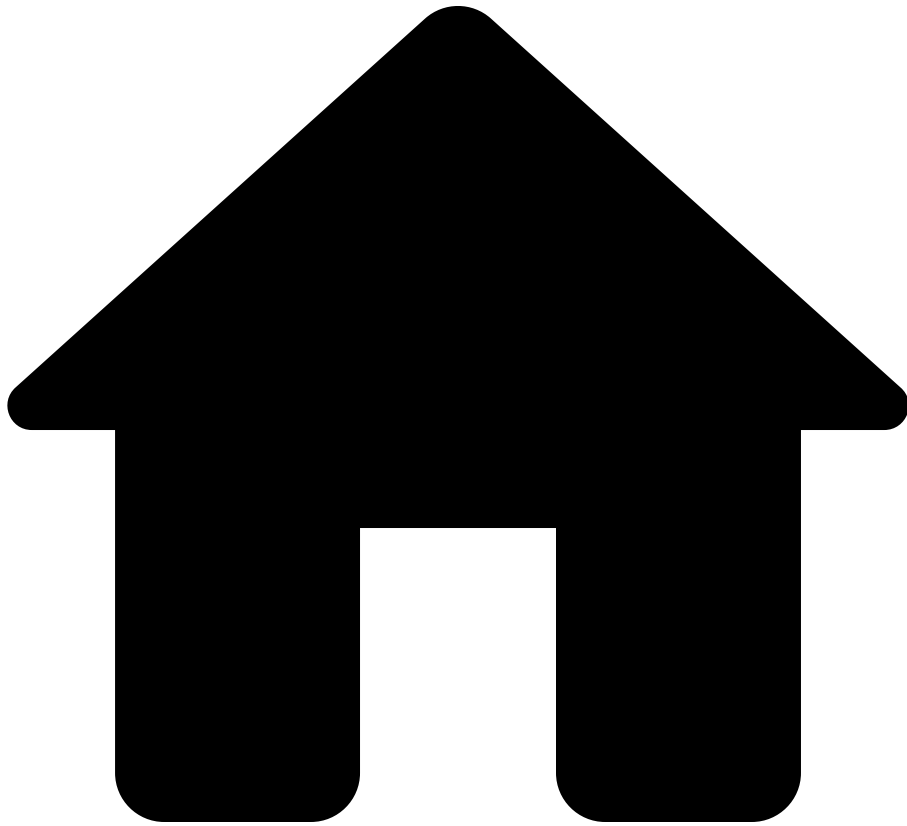
network devices interfaces teTunnel

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - interfaces: Bag< [Iface](#) >
 - teTunnel: [TeTunnel](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- TeTunnelNextHop

On this page

TeTunnelNextHop

Fields

| name | type | nullable | description |
|---------------|----------------|----------|---|
| interfaceName | String | false | Output interface for the packet. |
| ipAddress | IpAddress | false | Next hop IP address for the outgoing packet. |
| mplsLabels | Bag< Integer > | false | MPLS labels to push on the outgoing patchket. |

Used by

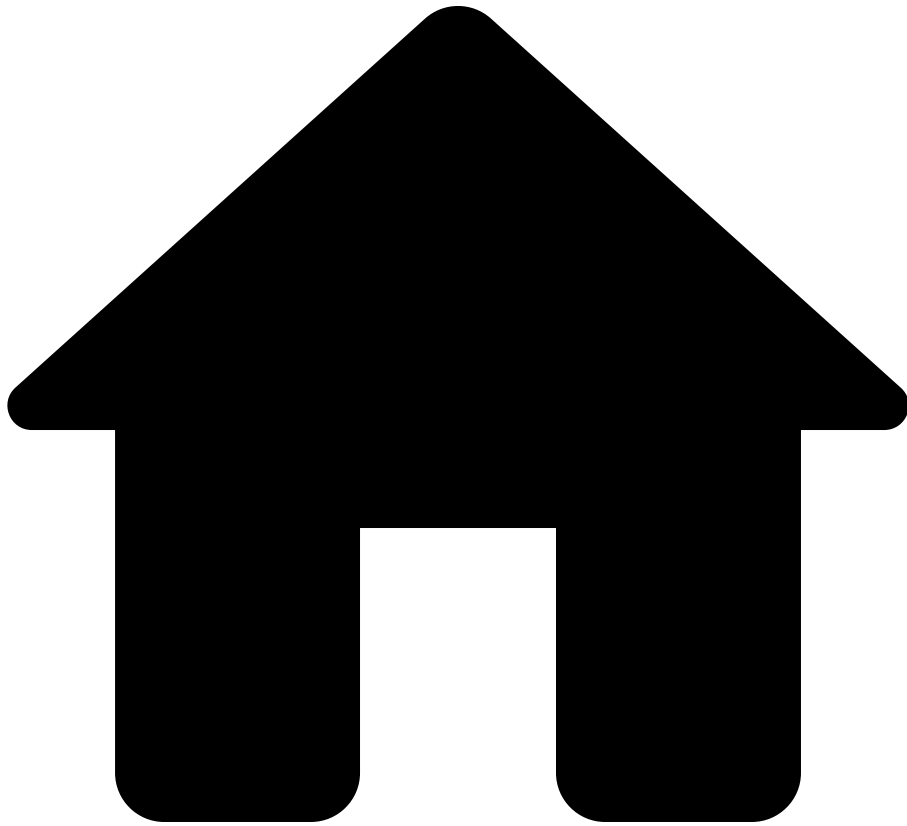
network devices interfaces teTunnel nextHops

See also

Data model path

- network: Network
 - devices: Bag< Device >
 - interfaces: Bag< Iface >
 - teTunnel: TeTunnel
 - nextHops: Bag< TeTunnelNextHop >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- TransitGateway

On this page

TransitGateway

Fields

| name | type | nullable | description |
|-------------|---|----------|-------------|
| name | String | true | -- |
| id | String | false | -- |
| tags | Bag< String > | false | -- |
| region | String | true | -- |
| attachments | Bag< TransitGatewayAttachment > | false | -- |
| routeTables | Bag< TransitGatewayRouteTable > | false | -- |

Used by

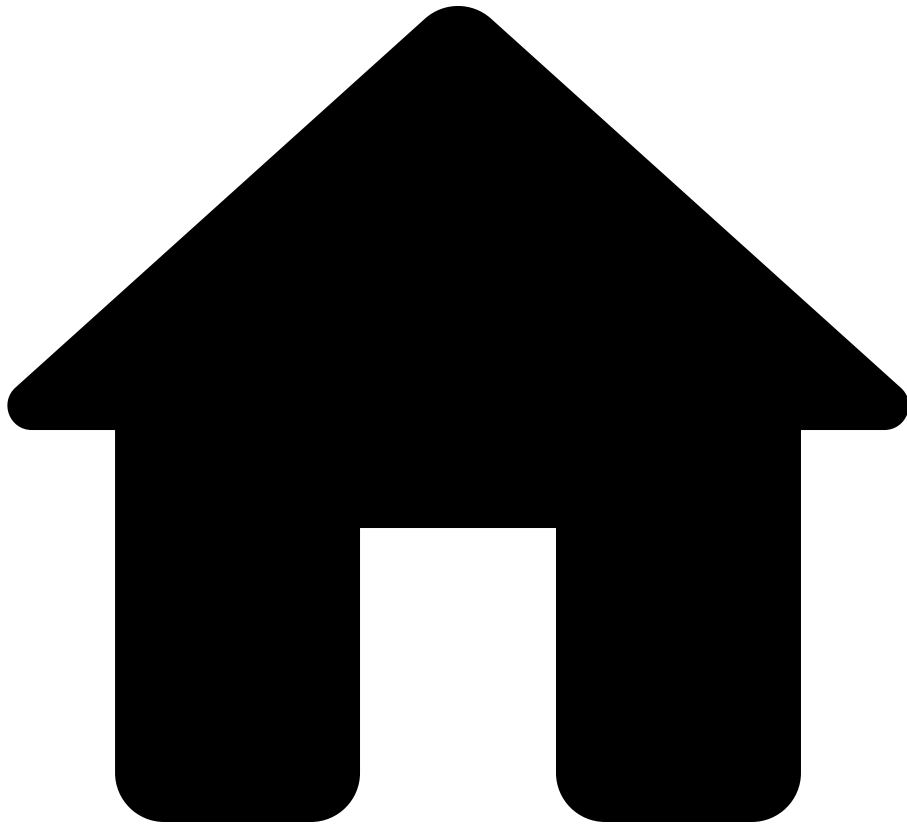
network cloudAccounts transitGateways

See also

Data model path

- network: [Network](#)
 - cloudAccounts: Bag< [CloudAccount](#) >
 - transitGateways: Bag< [TransitGateway](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- TransitGatewayAttachment

On this page

TransitGatewayAttachment

Fields

| name | type | nullable | description |
|----------------|---|----------|-------------|
| name | String | true | -- |
| id | String | false | -- |
| tags | Bag< String > | false | -- |
| attachmentType | TransitGatewayAttachmentType | false | -- |
| routeTables | Bag< TransitGatewayRouteTable > | false | -- |

Used by

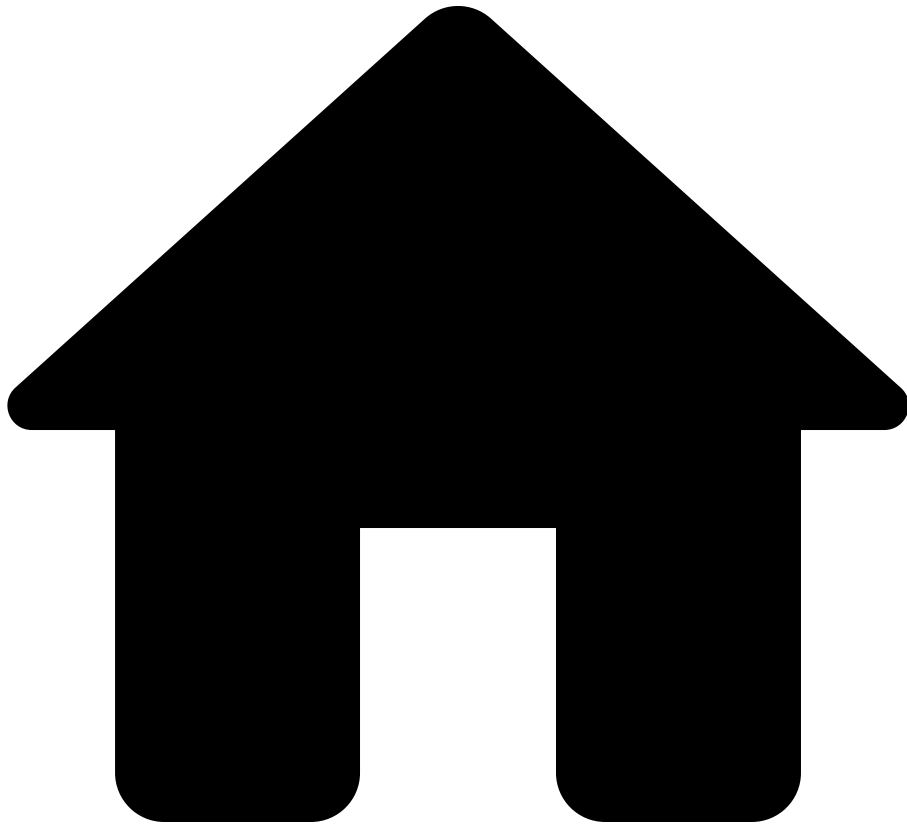
network cloudAccounts transitGateways attachments

See also

Data model path

- network: [Network](#)
 - cloudAccounts: Bag< [CloudAccount](#) >
 - transitGateways: Bag< [TransitGateway](#) >
 - attachments: Bag< [TransitGatewayAttachment](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- TransitGatewayAttachmentType

On this page

TransitGatewayAttachmentType

Options

| name | description |
|-------------|-------------|
| CONNECT | -- |
| DC | -- |
| TGW_PEERING | -- |
| VPC | -- |
| VPN | -- |

Used by

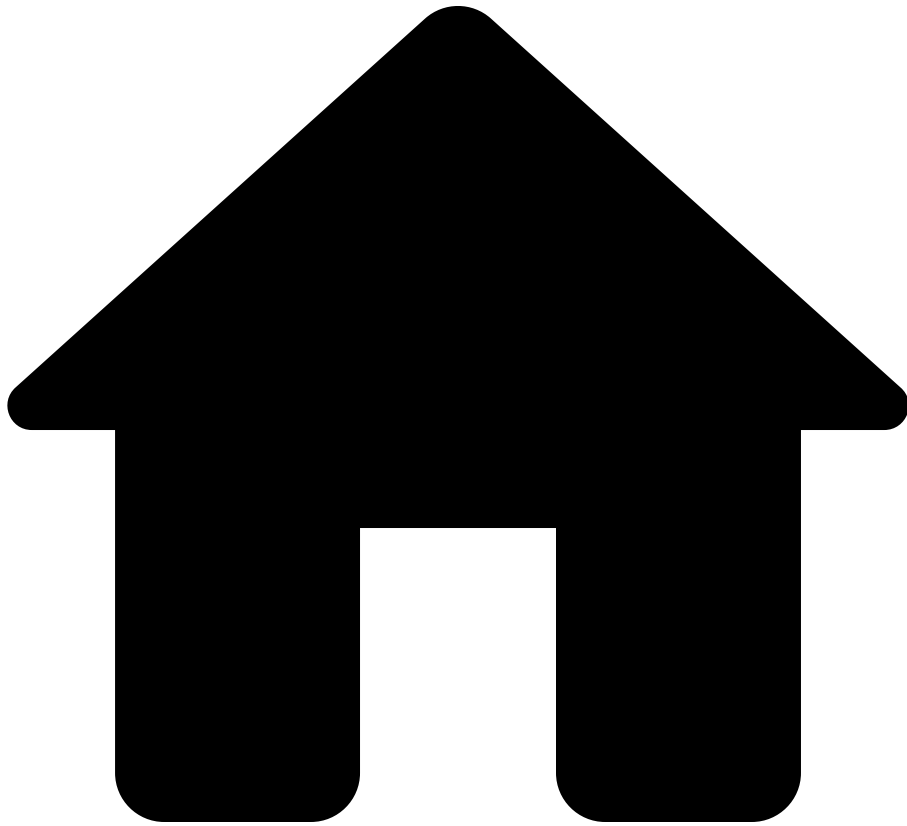
network cloudAccounts transitGateways attachments attachmentType

See also

Data model path

- network: Network
 - cloudAccounts: Bag< CloudAccount >
 - transitGateways: Bag< TransitGateway >
 - attachments: Bag< TransitGatewayAttachment >
 - attachmentType: TransitGatewayAttachmentType

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- TransitGatewayRoute

On this page

TransitGatewayRoute

Fields

| name | type | nullable | description |
|----------------------|--------------------------------|----------|--|
| prefixes | Bag< IpSubnet > | false | -- |
| routeType | CloudRouteType | false | -- |
| nextHopAttachmentIds | Bag< String > | false | Route with null nextHopAttachmentId drops traffic. |

Used by

network cloudAccounts transitGateways attachments routeTables routes

See also

Data model path

- network: [Network](#)
 - cloudAccounts: Bag< [CloudAccount](#) >
 - transitGateways: Bag< [TransitGateway](#) >
 - attachments: Bag< [TransitGatewayAttachment](#) >
 - routeTables: Bag< [TransitGatewayRouteTable](#) >
 - routes: Bag< [TransitGatewayRoute](#) >

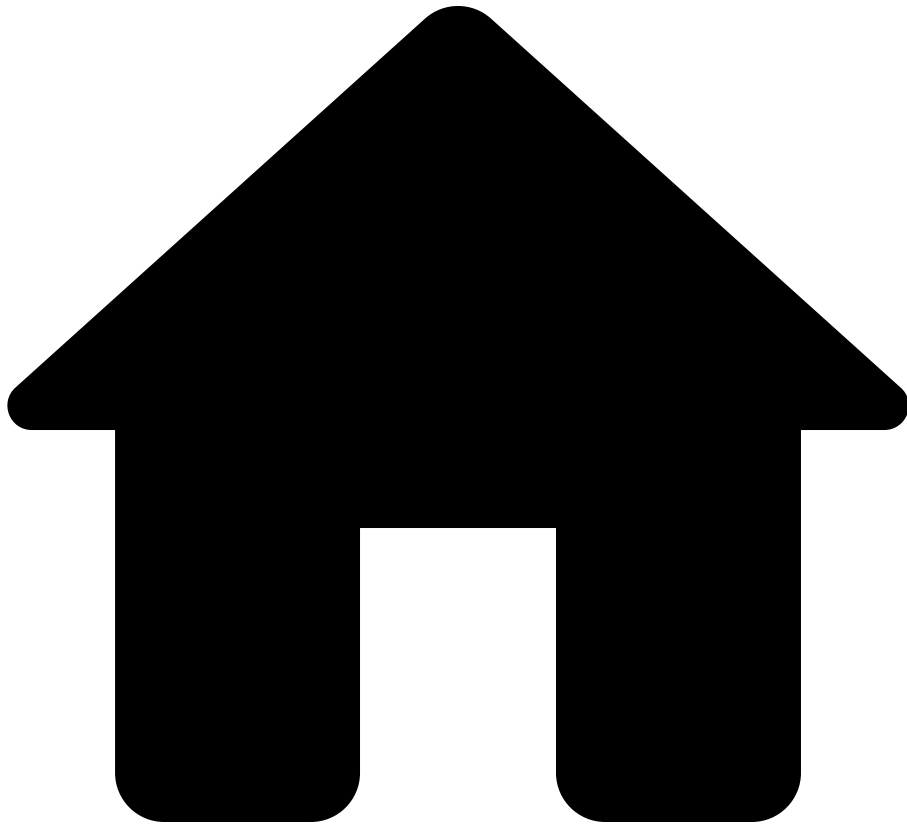
network **cloudAccounts** **transitGateways** **routeTables** **routes**

See also

Data model path

- network: [Network](#)
 - cloudAccounts: Bag< [CloudAccount](#) >
 - transitGateways: Bag< [TransitGateway](#) >
 - routeTables: Bag< [TransitGatewayRouteTable](#) >
 - routes: Bag< [TransitGatewayRoute](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- TransitGatewayRouteTable

On this page

TransitGatewayRouteTable

Fields

| name | type | nullable | description |
|--------|--|----------|-------------|
| name | String | false | -- |
| id | String | false | -- |
| tags | Bag< String > | false | -- |
| routes | Bag< TransitGatewayRoute > | false | -- |

Used by

network [cloudAccounts](#) [transitGateways](#) [attachments](#) [routeTables](#)

See also

Data model path

- network: [Network](#)
 - cloudAccounts: Bag< [CloudAccount](#) >
 - transitGateways: Bag< [TransitGateway](#) >
 - attachments: Bag< [TransitGatewayAttachment](#) >
 - routeTables: Bag< [TransitGatewayRouteTable](#) >

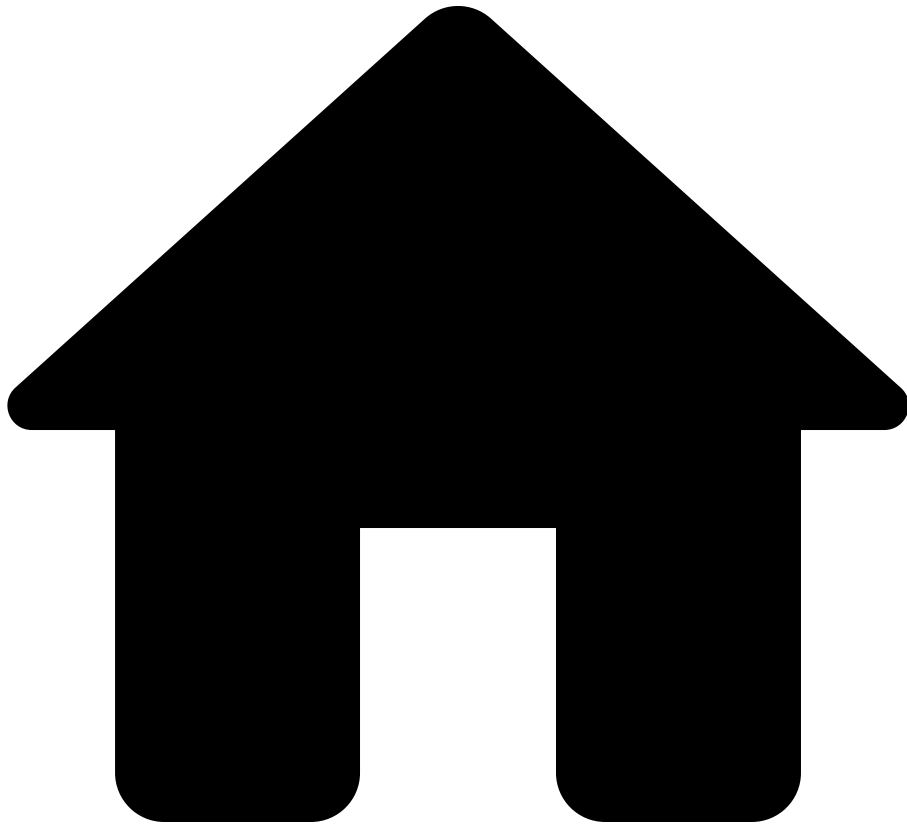
network **cloudAccounts** **transitGateways** **routeTables**

See also

Data model path

- network: [Network](#)
 - cloudAccounts: Bag< [CloudAccount](#) >
 - transitGateways: Bag< [TransitGateway](#) >
 - routeTables: Bag< [TransitGatewayRouteTable](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- Tunnel

On this page

Tunnel

Configuraton parameters relating to a tunnel interface.

Fields

| name | type | nullable | description |
|---------------------|-------------------------------|----------|---|
| src | IpAddress | false | The source address for the tunnel. |
| dst | IpAddress | false | The destination address for the tunnel. |
| ipv4 | IfaceIpv4Info | false | -- |
| ipv6 | IfaceIpv6Info | false | -- |
| networkInstanceName | String | false | The network instance (i.e. VRF) that this interface is part of. |

Used by

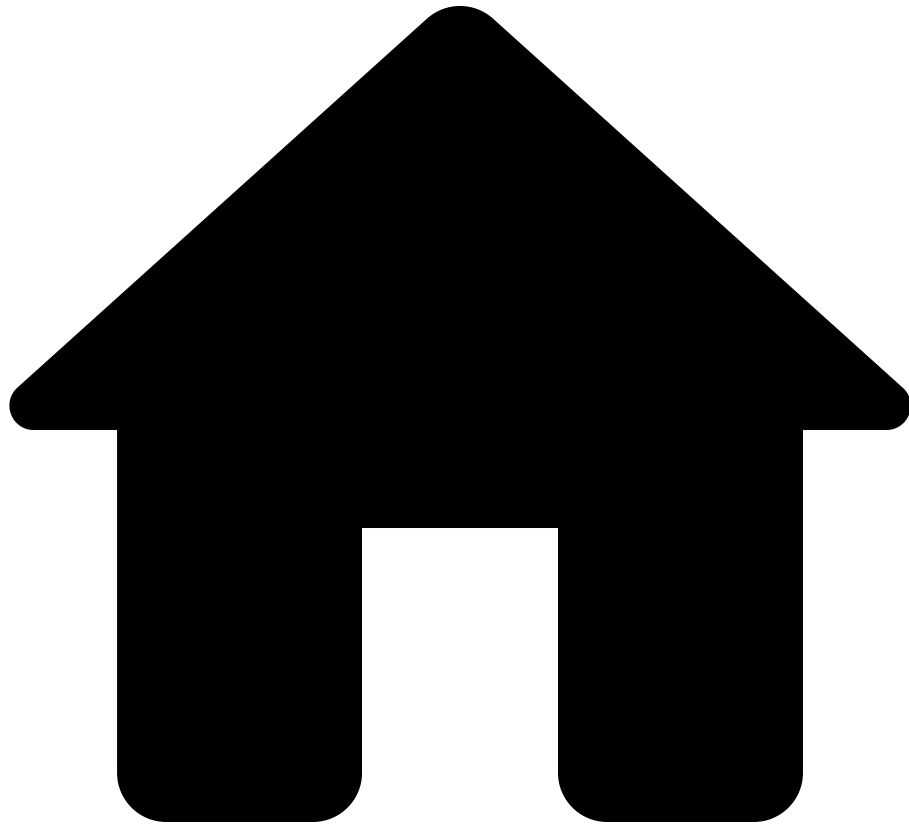
network devices interfaces tunnel

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - interfaces: Bag< [Iface](#) >
 - tunnel: [Tunnel](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- Url

On this page

Url

Fields

| name | type | nullable | description |
|------|--------|----------|-------------|
| host | String | false | -- |
| path | String | false | -- |

Used by

network **cloudAccounts** **vpcs** **loadBalancers** **loadBalancerRules** **backends**

See also

Data model path

- network: **Network**
 - cloudAccounts: Bag< **CloudAccount** >
 - vpcs: Bag< **VpcData** >
 - loadBalancers: Bag< **LoadBalancer** >
 - loadBalancerRules: Bag< **LoadBalancerRule** >
 - backends: Bag< **Backend** >

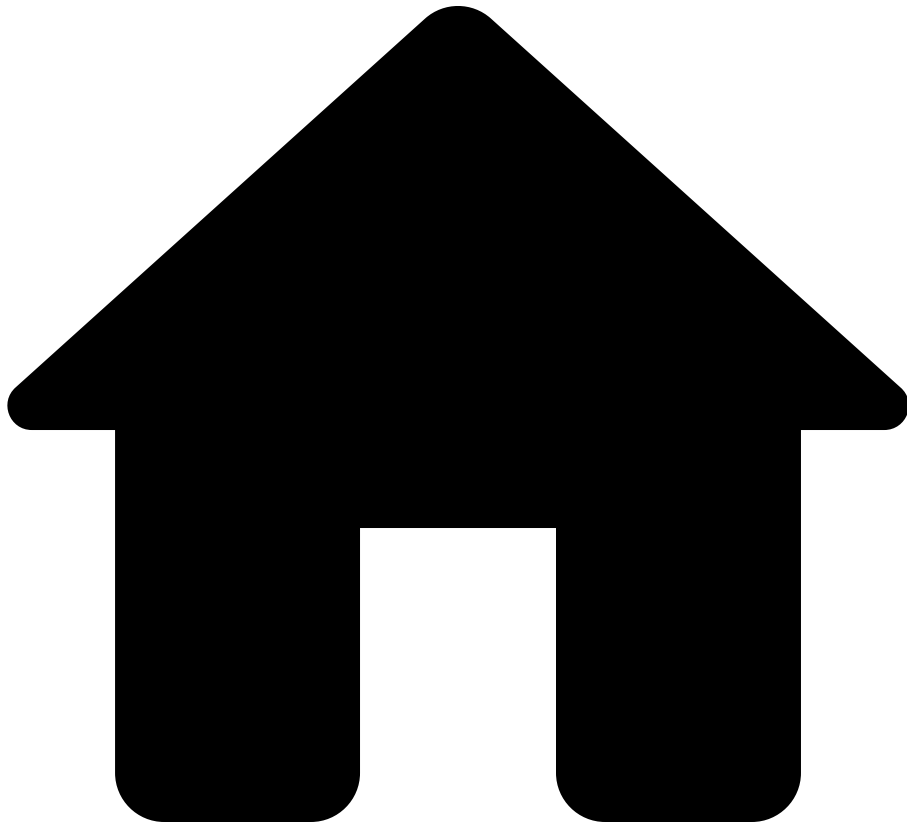
network **cloudAccounts** **externalLoadBalancers**
loadBalancerRules **backends**

See also

Data model path

- network:
 - cloudAccounts: Bag<
 - externalLoadBalancers: Bag<
 - loadBalancerRules: Bag<
 - backends: Bag<

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- Vendor

On this page

Vendor

Options

| name | description |
|--------------|-------------|
| A10 | -- |
| ALKIRA | -- |
| AMAZON | -- |
| ARISTA | -- |
| ARUBA | -- |
| AVAYA | -- |
| AVI_NETWORKS | -- |
| AZURE | -- |
| BLUECAT | -- |
| BROCADE | -- |
| CHECKPOINT | -- |
| CISCO | -- |
| CITRIX | -- |
| CUMULUS | -- |
| DELL | -- |
| EDGE_CORE | -- |
| EXTREME | -- |
| F5 | -- |
| FORCEPOINT | -- |
| FORTINET | -- |

| name | description |
|--------------------|-------------|
| FORWARD_CUSTOM | -- |
| GENERAL_DYNAMICS | -- |
| GOOGLE | -- |
| HP | -- |
| HUAWEI | -- |
| JUNIPER | -- |
| LINUX_GENERIC | -- |
| NOKIA | -- |
| PALO_ALTO_NETWORKS | -- |
| PENSANDO | -- |
| PICA8 | -- |
| RIVERBED | -- |
| SILVER_PEAK | -- |
| SYMANTEC | -- |
| T128 | -- |
| UNKNOWN | -- |
| VERSA | -- |
| VIASAT | -- |
| VMWARE | -- |

Used by

network **devices** **platform** **vendor**

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - platform: [Platform](#)
 - vendor: [Vendor](#)

network **cveDatabase** **cves** **criteria** **vendor**

See also

Data model path

- network: [Network](#)
 - cveDatabase: [CveDatabase](#)
 - cves: Bag< [Cve](#) >
 - criteria: Bag< [CveProductCriteria](#) >
 - vendor: [Vendor](#)

network **cveDatabase** **cves** **vendorInfos** **vendor**

See also

Data model path

- network: [Network](#)
 - cveDatabase: [CveDatabase](#)
 - cves: Bag< [Cve](#) >
 - vendorInfos: Bag< [VendorCveInfo](#) >
 - vendor: [Vendor](#)

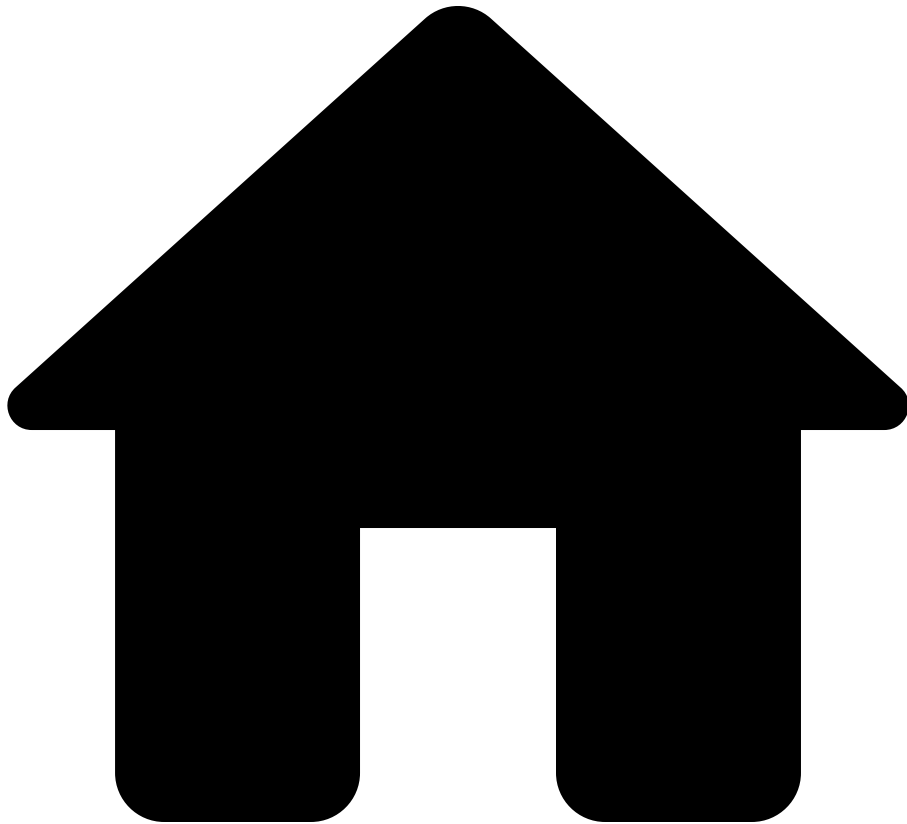
network **cveDatabase** **cves** **vendorInfos** **criteria** **vendor**

See also

Data model path

- network: [Network](#)
 - cveDatabase: [CveDatabase](#)
 - cves: Bag< [Cve](#) >
 - vendorInfos: Bag< [VendorCveInfo](#) >
 - criteria: Bag< [CveProductCriteria](#) >
 - vendor: [Vendor](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VendorCveInfo

On this page

VendorCveInfo

Fields

| name | type | nullable | description |
|-----------------|--------------------------|----------|---|
| vendor | Vendor | false | -- |
| publicationDate | Date | true | The date that this vendor published an advisory for this CVE. Absent if no publication date is known. |
| url | String | true | The URL to the security advisory for this CVE from this vendor. |
| severity | Severity | false | The severity corresponding to baseScoreV4 if that score is known. Otherwise, the severity corresponding to baseScoreV3 if that score is known. Otherwise, the |

| name | type | nullable | description |
|-----------------|------|----------|--|
| | | | severity corresponding to baseScoreV2 if that score is known. Otherwise, no score is known and this value is NONE. |
| hasKnownExploit | Bool | true | Whether this vendor has reported exploitation of this CVE in their security advisory (the URL in the url field). More specifically, the value is true if the vendor has reported exploitation of this CVE in their security advisory, and the value is false if the vendor has reported that there is no known |

| name | type | nullable | description |
|-----------|-------|----------|---|
| | | | <p>exploitation of this CVE in their security advisory or the vendor does not say anything one way or the other about known exploitation of this CVE in their security advisory. Absent if Forward Networks does not know what the vendors says about exploitation of this CVE in their security advisory. Also, the value will never be false for now.</p> |
| baseScore | Float | true | <p>The base score in version 4 of the CVSS if that score is known. Otherwise, the</p> |

| name | type | nullable | description |
|-------------|-------|----------|--|
| | | | <p>base score in version 3 of the CVSS if that score is known.</p> <p>Otherwise, the base score in version 2 of the CVSS if that score is known.</p> <p>Otherwise, no score is known and this value is absent.</p> |
| baseScoreV2 | Float | true | <p>The base severity score in version 2 of the CVSS.</p> <p>Absent if it does not exist or is not known.</p> |
| baseScoreV3 | Float | true | <p>The base severity score in version 3 of the CVSS.</p> <p>Absent if it does not exist or is not known.</p> |
| baseScoreV4 | Float | true | <p>The base severity score</p> |

| name | type | nullable | description |
|-------------|---|----------|---|
| | | | in version 4 of the CVSS.
Absent if it does not exist or is not known. |
| description | String | true | -- |
| criteria | Bag< CveProductCriteria > | false | -- |

Used by[â](#)

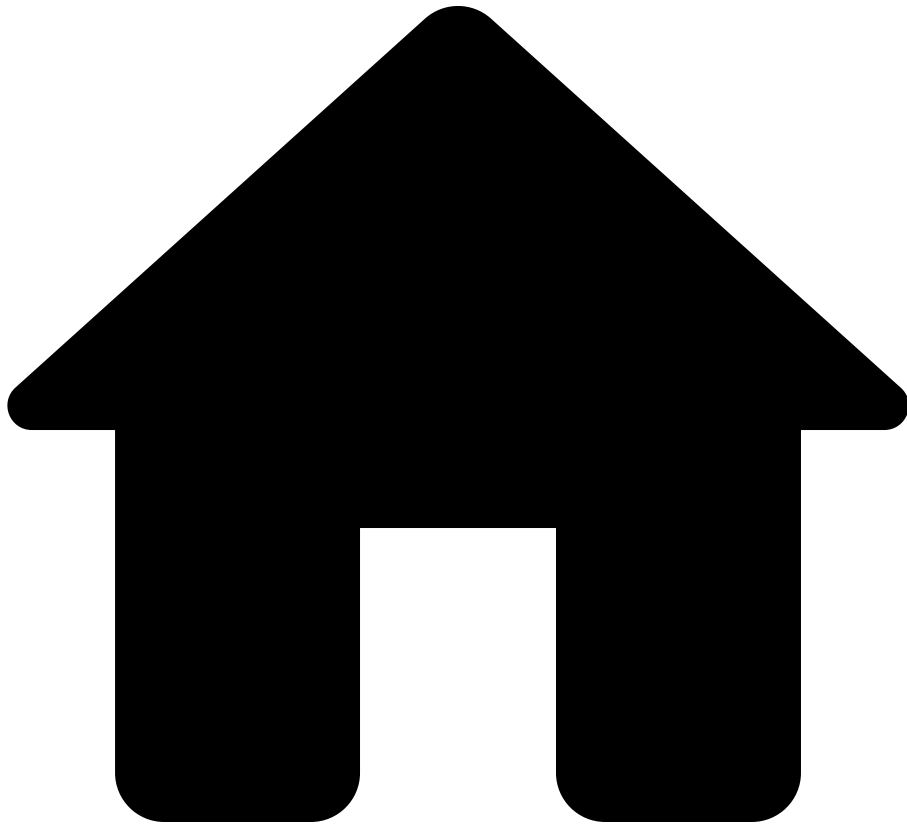
network [â](#) **cveDatabase** [â](#) **cves** [â](#) **vendorInfos**[â](#)

See also[â](#)

Data model path

- network: [Network](#)
 - cveDatabase: [CveDatabase](#)
 - cves: Bag< [Cve](#) >
 - vendorInfos: Bag< [VendorCveInfo](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VirtualCircuitIdInterfacePolicy

On this page

VirtualCircuitIdInterfacePolicy

Fields

| name | type | nullable | description |
|-----------|-------------------------------------|----------|---|
| condition | StigPolicyCondition | false | -- |
| value | Integer | false | The virtual circuit ID (VC ID) assigned to the interface. |

Used by

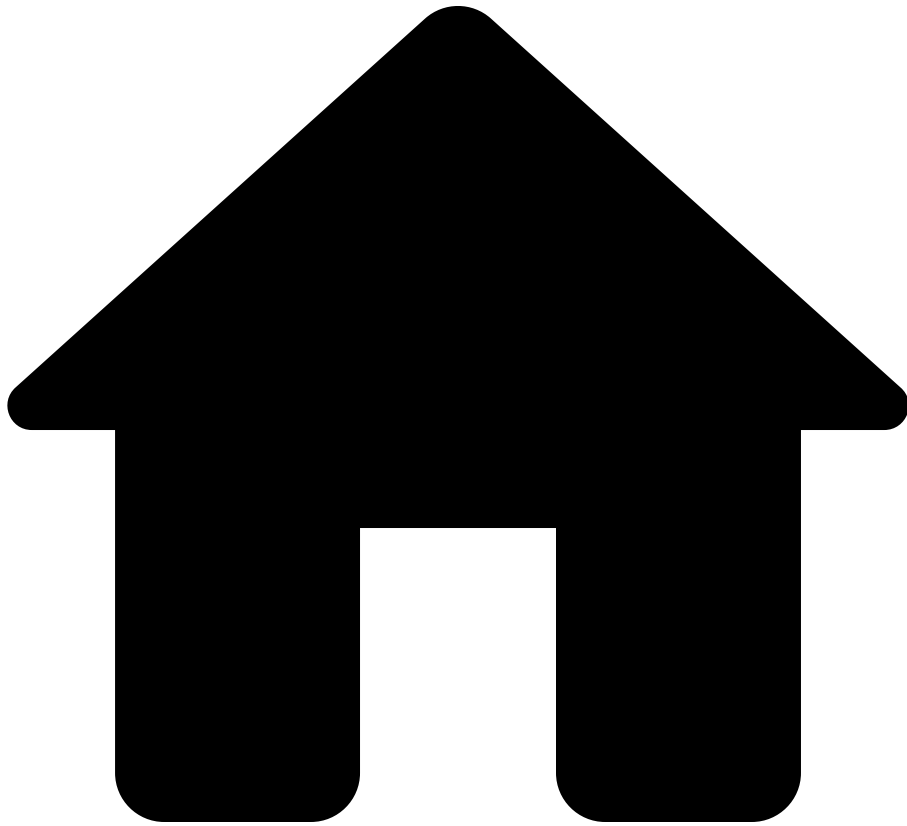
network [stigDatabase](#) [policy](#) [ifacePolicy](#) [pseudowireVirtualCircuitId](#)

See also

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - ifacePolicy: [InterfacePolicy](#)
 - pseudowireVirtualCircuitId:
Bag< [VirtualCircuitIdInterfacePolicy](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- Vlan

On this page

Vlan

Fields

| name | type | nullable | description |
|-------------|---------------|----------|--------------------------------|
| vlanId | Integer | false | -- |
| name | String | true | Name for this vlan. |
| memberNames | Bag< String > | false | Interfaces carrying this vlan. |

Used by

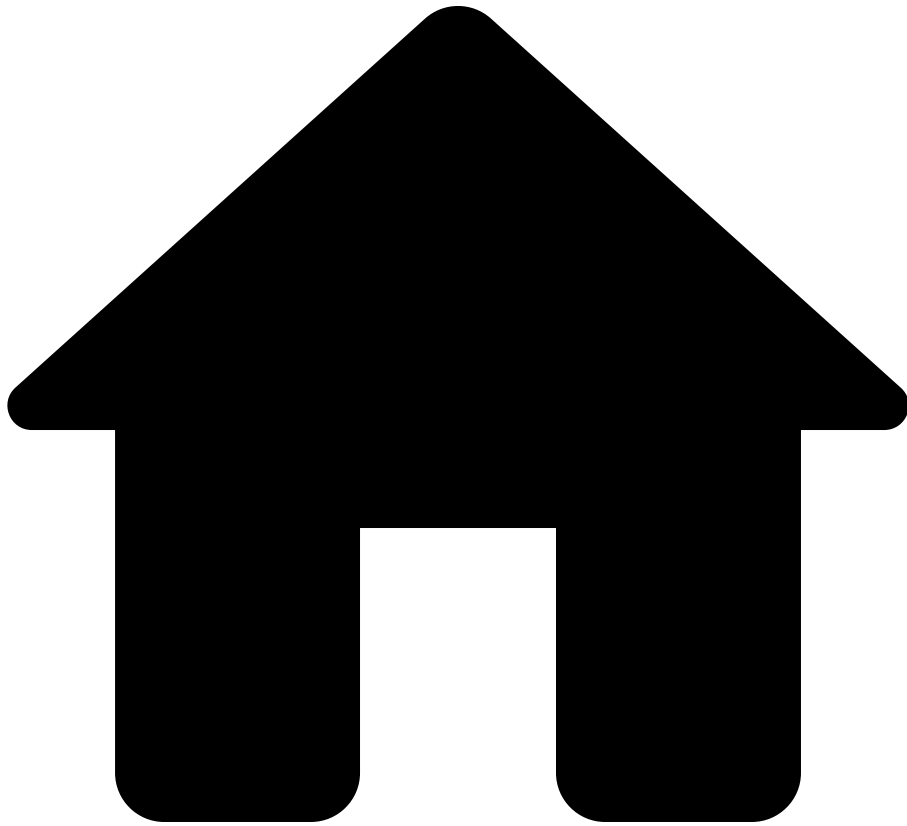
network devices networkInstances vlans

See also

Data model path

- network: Network
 - devices: Bag< Device >
 - networkInstances: Bag< NetworkInstance >
 - vlans: Bag< Vlan >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VlanIdOrRange

On this page

VlanIdOrRange

Options

| name | type | description |
|-------|----------|---|
| ID | Integer | Type definition representing a single-tagged VLAN. |
| RANGE | VlanPair | Type definition representing a range of single-tagged VLANs. A range is specified as x..y where x and y are valid VLAN IDs (1 <= vlan-id <= 4094). The range is assumed to be inclusive, such that any VLAN-ID matching x <= VLAN-ID <= y falls within the range. |

Used by

network devices interfaces ethernet switchedVlan trunkVlans

See also

Data model path

- network: Network
 - devices: Bag< Device >
 - interfaces: Bag< Iface >
 - ethernet: Ethernet
 - switchedVlan: SwitchedVlan
 - trunkVlans: Bag< VlanIdOrRange >

network **devices** **interfaces** **aggregation** **switchedVlan** **trunkVlans**

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - interfaces: Bag< [Iface](#) >
 - aggregation: [Aggregation](#)
 - switchedVlan: [SwitchedVlan](#)
 - trunkVlans: Bag< [VlanIdOrRange](#) >

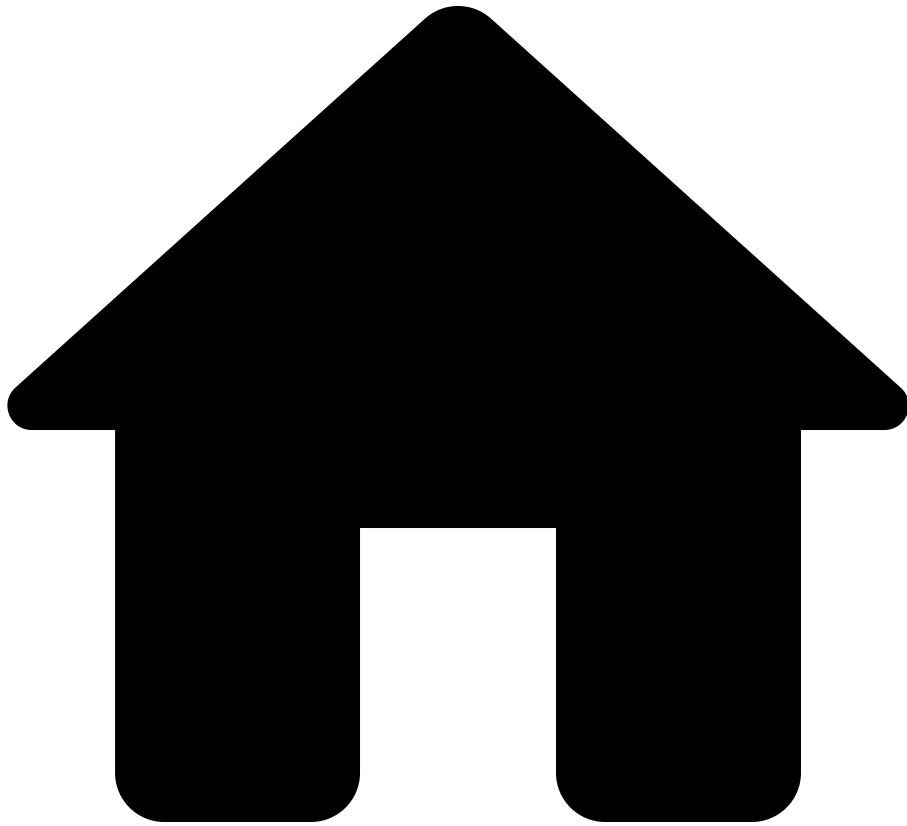
network **devices** **stp** **mstp** **instances** **vlans**

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - stp: [Stp](#)
 - mstp: [Mstp](#)
 - instances: Bag< [MstInstance](#) >
 - vlans: Bag< [VlanIdOrRange](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VlanIdSet

On this page

VlanIdSet

Fields

| name | type | nullable | description |
|--------|----------------|----------|-------------|
| v lans | Bag< Integer > | false | -- |

Used by

network devices interfaces subinterfaces switchedVlans
outerVlans

See also

Data model path

- network: Network
 - devices: Bag< Device >
 - interfaces: Bag< Iface >
 - subinterfaces: Bag< SubInterface >
 - switchedVlans: SwitchedSubInterfaceVlans
 - outerVlans: VlanIdSetOrAll

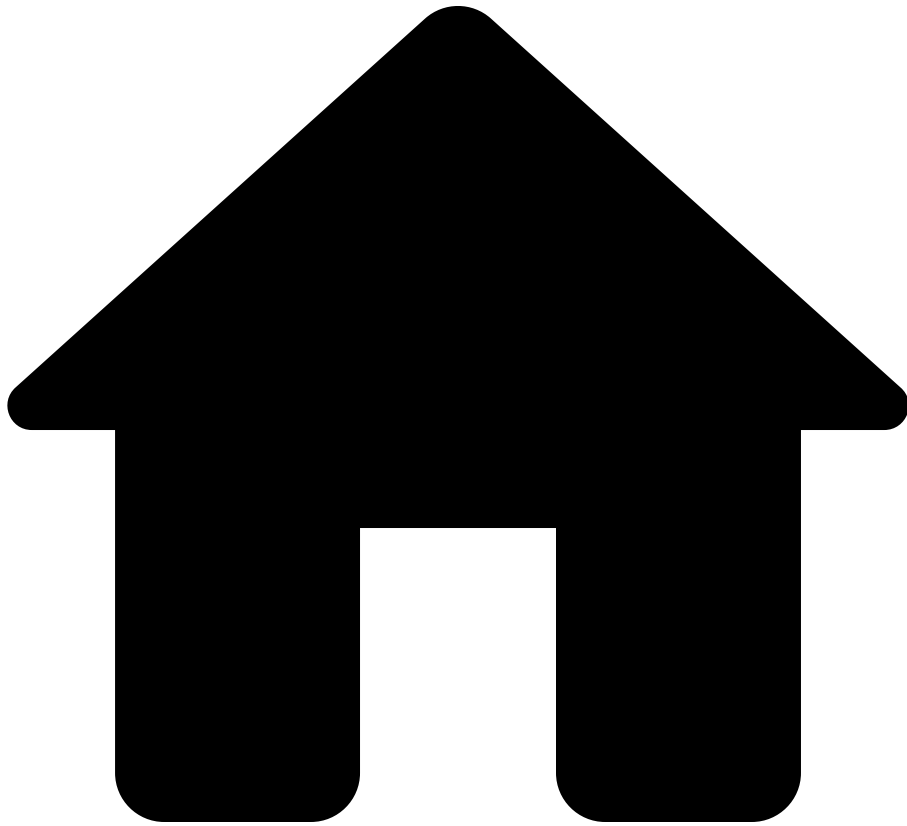
network devices interfaces subinterfaces switchedVlans
innerVlans

See also

Data model path

- network: Network
 - devices: Bag< Device >
 - interfaces: Bag< Iface >
 - subinterfaces: Bag< SubInterface >
 - switchedVlans: SwitchedSubInterfaceVlans
 - innerVlans: VlanIdSetOrAll

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VlanIdSetOrAll

On this page

VlanIdSetOrAll

Options

| name | type | description |
|------|---------------------------|----------------------|
| ALL | -- | All VLAN IDs. |
| IDS | VlanIdSet | Individual VLAN IDs. |

Used by

network **devices** **interfaces** **subinterfaces** **switchedVlans** **outerVlans**

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - interfaces: Bag< [Iface](#) >
 - subinterfaces: Bag< [SubInterface](#) >
 - switchedVlans: [SwitchedSubInterfaceVlans](#)
 - outerVlans: [VlanIdSetOrAll](#)

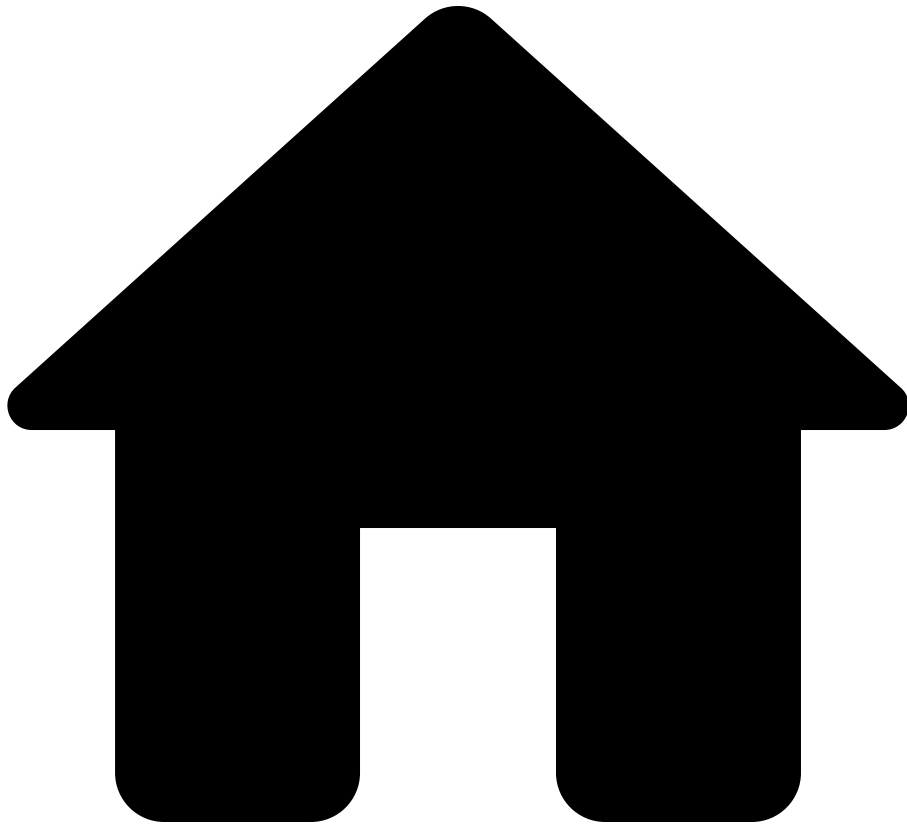
network **devices** **interfaces** **subinterfaces** **switchedVlans**
innerVlans

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - interfaces: Bag< [Iface](#) >
 - subinterfaces: Bag< [SubInterface](#) >
 - switchedVlans: [SwitchedSubInterfaceVlans](#)
 - innerVlans: [VlanIdSetOrAll](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VlanModeType

On this page

VlanModeType

VLAN interface mode (trunk or access).

Options

| name | description |
|--------|--|
| ACCESS | Access mode VLAN interface (No 802.1q header). |
| TRUNK | Trunk mode VLAN interface. |

Used by

network devices interfaces ethernet switchedVlan interfaceMode

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - interfaces: Bag< [Iface](#) >
 - ethernet: [Ethernet](#)
 - switchedVlan: [SwitchedVlan](#)
 - interfaceMode: [VlanModeType](#)

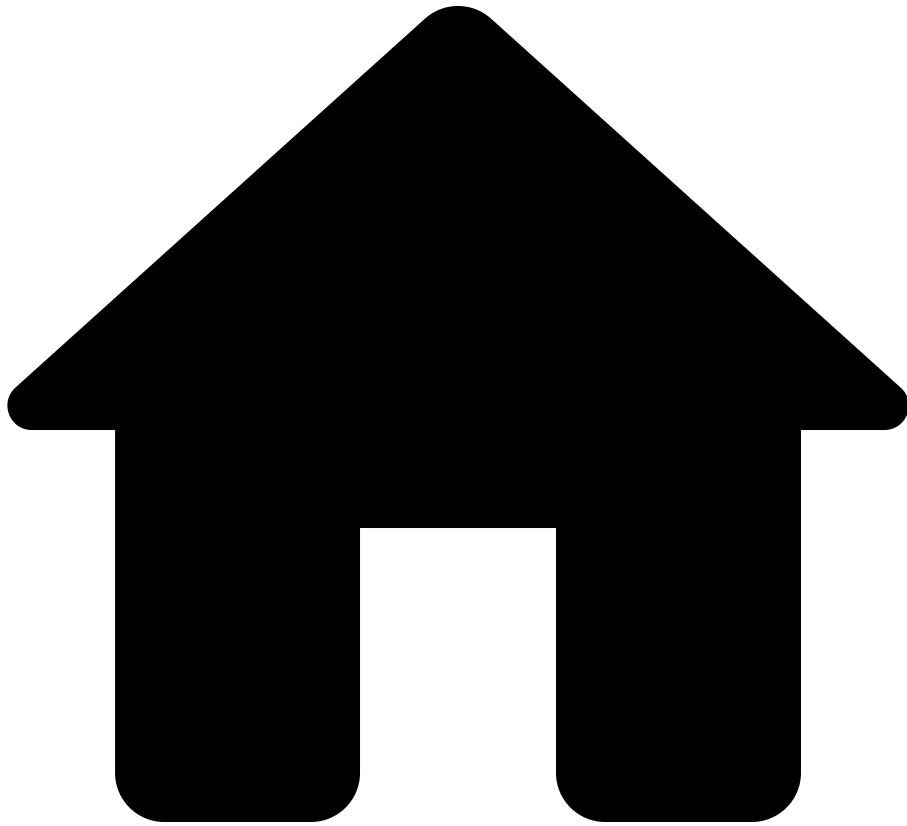
network devices interfaces aggregation switchedVlan interfaceMode

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - interfaces: Bag< [Iface](#) >
 - aggregation: [Aggregation](#)
 - switchedVlan: [SwitchedVlan](#)
 - interfaceMode: [VlanModeType](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VlanPair

On this page

VlanPair

Fields

| name | type | nullable | description |
|------|---------|----------|-------------|
| from | Integer | false | -- |
| to | Integer | false | -- |

Used by

network devices interfaces ethernet switchedVlan trunkVlans

See also

Data model path

- network: Network
 - devices: Bag< Device >
 - interfaces: Bag< Iface >
 - ethernet: Ethernet
 - switchedVlan: SwitchedVlan
 - trunkVlans: Bag< VlanIdOrRange >

network **devices** **interfaces** **aggregation** **switchedVlan** **trunkVlans**

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - interfaces: Bag< [Iface](#) >
 - aggregation: [Aggregation](#)
 - switchedVlan: [SwitchedVlan](#)
 - trunkVlans: Bag< [VlanIdOrRange](#) >

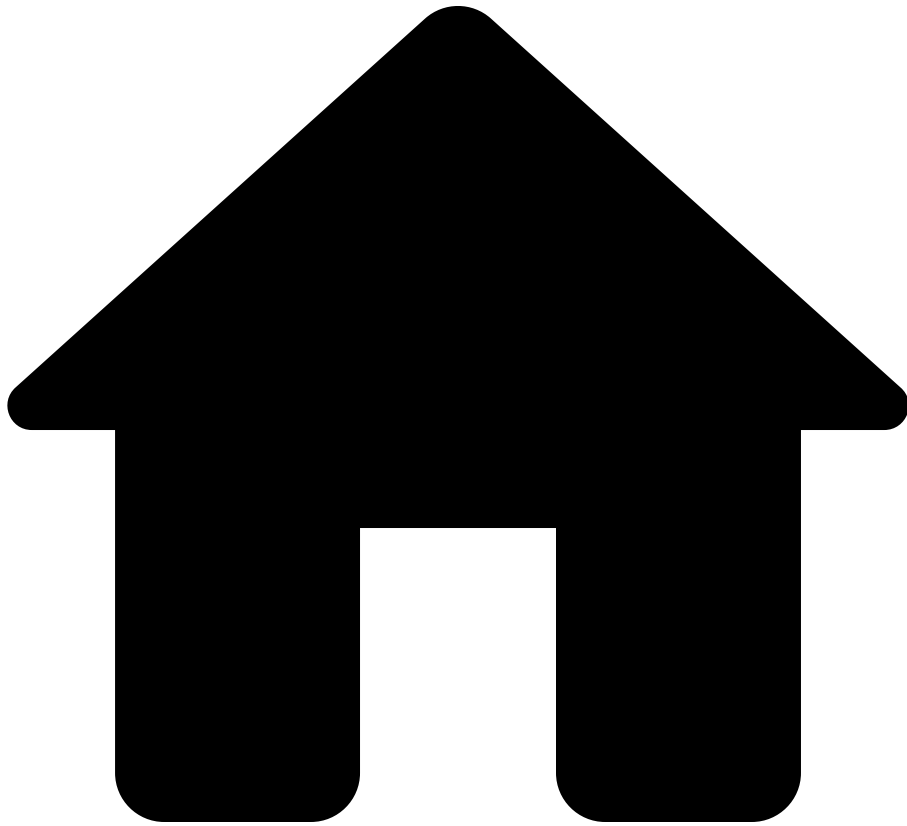
network **devices** **stp** **mstp** **instances** **vlans**

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - stp: [Stp](#)
 - mstp: [Mstp](#)
 - instances: Bag< [MstInstance](#) >
 - vlans: Bag< [VlanIdOrRange](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VlanRange

On this page

VlanRange

Fields

| name | type | nullable | description |
|------|---------|----------|---|
| from | Integer | false | The lower bound (inclusive) of the range. |
| to | Integer | false | The upper bound (inclusive) of the range. |

Used by

network devices vlans ranges

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - vlans: Bag< [NamedVlanCollection](#) >
 - ranges: Bag< [VlanRange](#) >

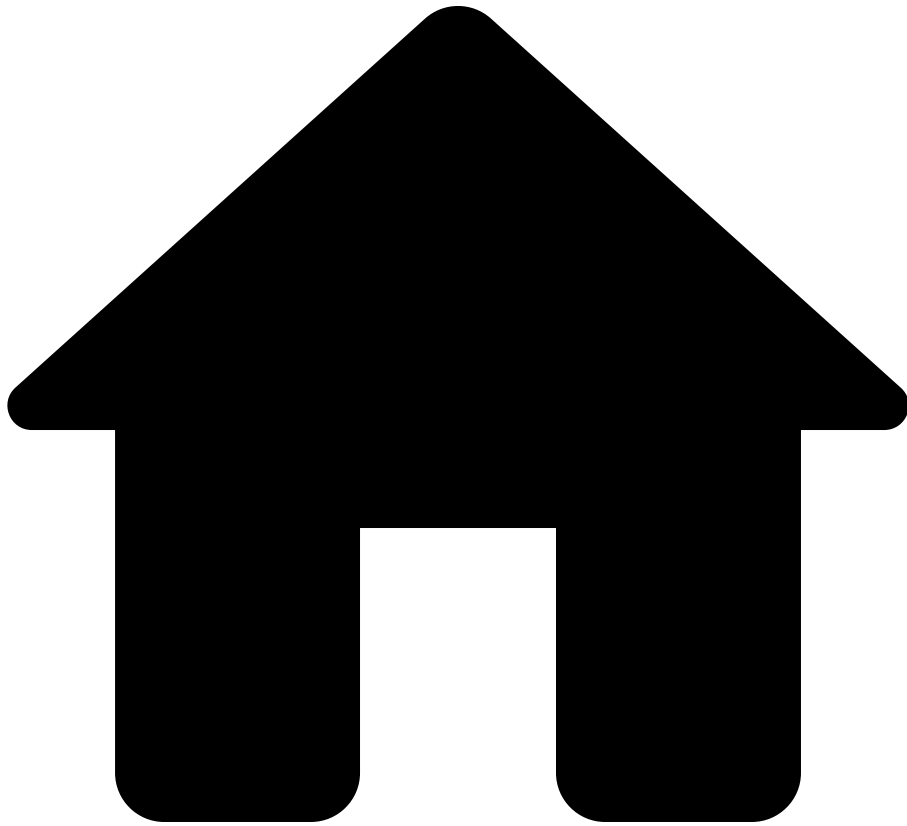
network devices hosts vlans

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - hosts: Bag< [DeviceHost](#) >
 - vlans: Bag< [VlanRange](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- Vpc

On this page

Vpc

Fields

| name | type | nullable | description |
|--------------|----------------|----------|-------------|
| id | Integer | false | -- |
| portName | String | false | -- |
| isUp | Bool | false | -- |
| isConsistent | Bool | false | -- |
| activeVlans | Bag< Integer > | false | -- |

Used by

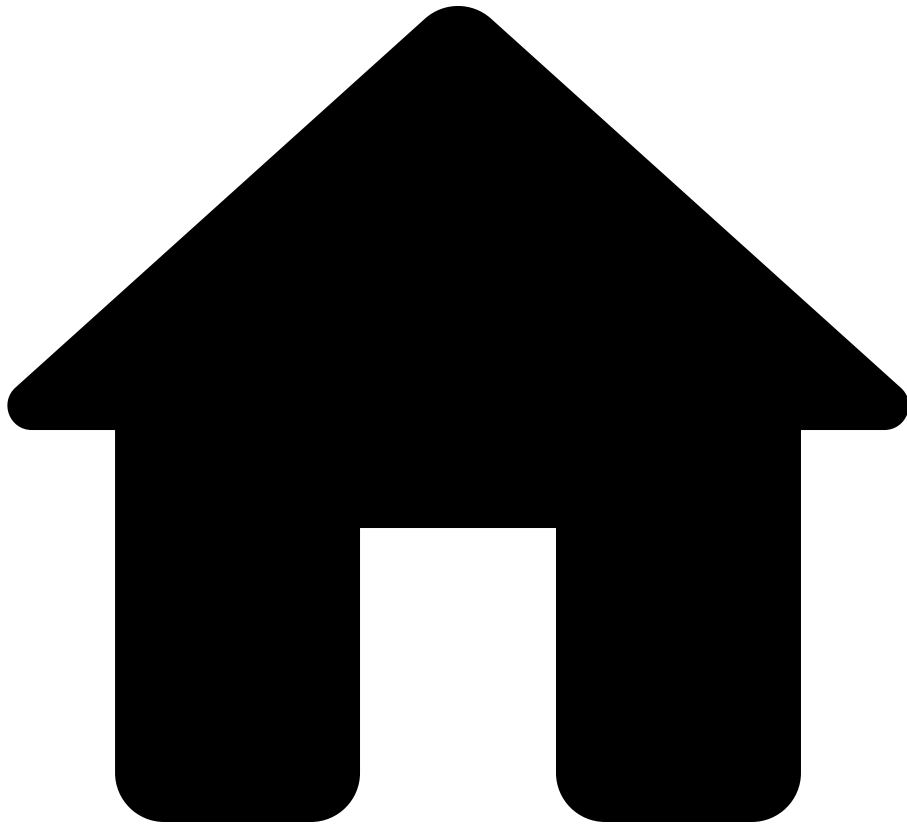
network devices ha vpc vpcStates

See also

Data model path

- network: Network
 - devices: Bag< Device >
 - ha: Ha
 - vpc: VpcDomain
 - vpcStates: Bag< Vpc >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VpcData

On this page

VpcData

Fields

| name | type | nullable | description |
|------------------|--|----------|---|
| name | String | true | Never null for GCP. |
| id | String | false | -- |
| tags | Bag< String > | false | -- |
| cloudRegions | Bag< String > | false | -- |
| cloudType | CloudType | false | -- |
| subnets | Bag< Subnet > | false | -- |
| computeInstances | Bag< ComputeInstance > | false | -- |
| serviceEndpoints | Bag< ServiceEndpoint > | false | The field serviceEndpoints is AWS only. |
| securityGroups | Bag< SecurityGroup > | false | -- |
| networkAcls | Bag< NetworkAcl > | false | -- |
| routeTables | Bag< RouteTable > | false | -- |
| vpnGateways | Bag< VpnGateway > | false | -- |
| dcGatewayIds | Bag< String > | false | -- |
| inetGateways | Bag< InetGateway > | false | -- |
| loadBalancers | Bag< LoadBalancer > | false | -- |
| natGateways | Bag< NatGateway > | false | -- |
| vpcPeerings | Bag< VpcPeering > | false | -- |
| ipv4CidrBlocks | Bag< IpSubnet > | false | -- |

| name | type | nullable | description |
|----------------|--------------------------------------|----------|-------------|
| ipv6CidrBlocks | Bag< IpSubnet > | false | -- |
| firewalls | Bag< CloudFirewall > | false | -- |

Used by

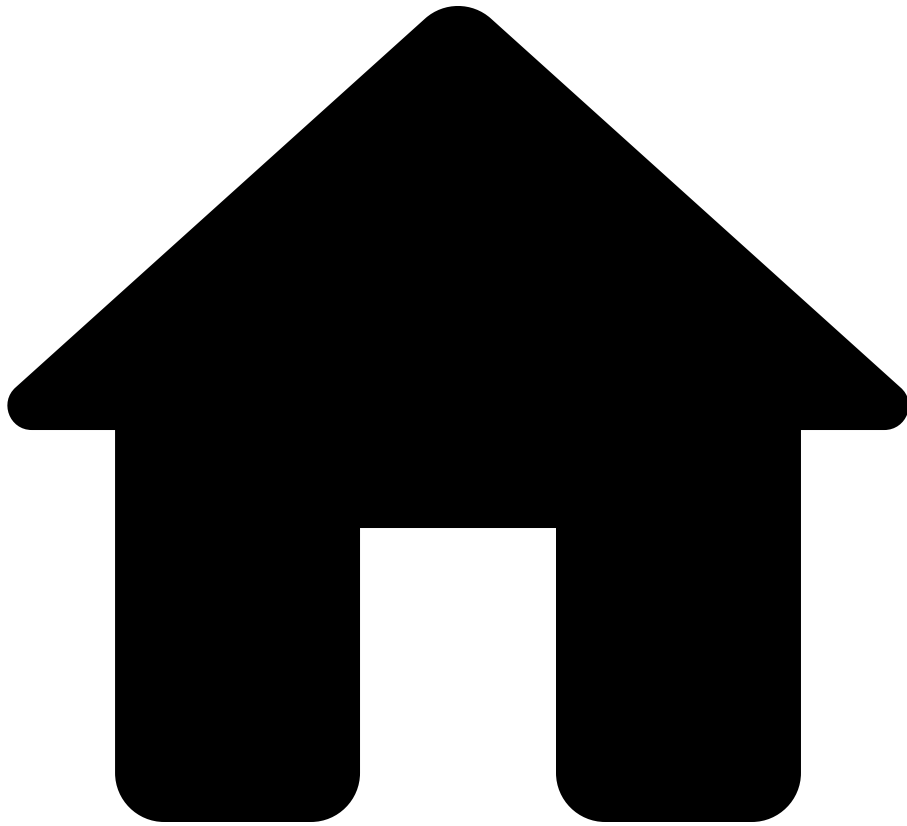
network cloudAccounts vpcs

See also

Data model path

- network: [Network](#)
 - cloudAccounts: Bag< [CloudAccount](#) >
 - vpcs: Bag< [VpcData](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VpcDomain

On this page

VpcDomain

Fields

| name | type | nullable | description |
|---------------------------------|---------------------------------|----------|-------------|
| domainId | Integer | false | -- |
| peerStatus | PeerStatus | true | -- |
| vpcKeepAliveStatus | KeepAliveStatus | true | -- |
| vpcRole | VpcRole | true | -- |
| peerGatewayEnabled | Bool | false | -- |
| gracefulConsistencyCheckEnabled | Bool | false | -- |
| autoRecoveryStatusEnabled | Bool | false | -- |
| peerLinkStatus | PeerLink | true | -- |
| vpcStates | Bag< Vpc > | false | -- |

Used by

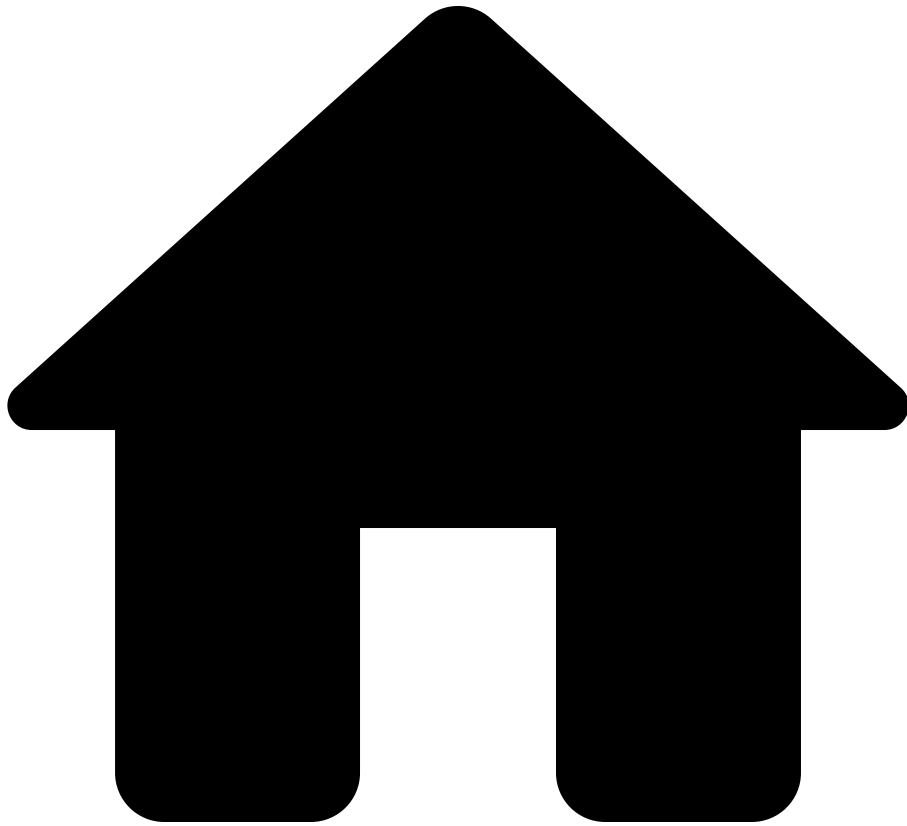
network devices ha vpc

See also

Data model path

- network: [Network](#)
 - devices: Bag< [Device](#) >
 - ha: [Ha](#)
 - vpc: [VpcDomain](#)

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VpcPeering

On this page

VpcPeering

Fields

| name | type | nullable | description |
|------------------|-------------|----------|-------------|
| name | String | false | -- |
| id | String | false | -- |
| tags | Bag<String> | false | -- |
| sourceVpcId | String | false | -- |
| destinationVpcId | String | true | -- |

Used by

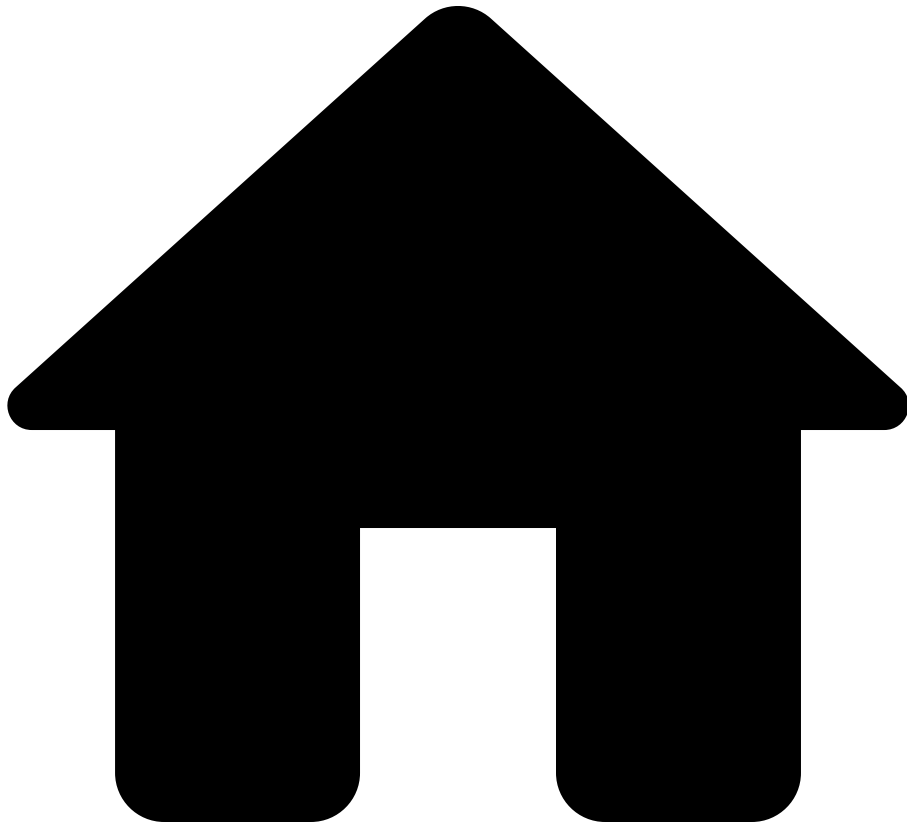
network cloudAccounts vpcs vpcPeerings

See also

Data model path

- network: Network
 - cloudAccounts: Bag<CloudAccount>
 - vpcs: Bag<VpcData>
 - vpcPeerings: Bag<VpcPeering>

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VpcRole

On this page

VpcRole

Options

| name | description |
|-----------------------|-------------|
| OPERATIONAL_PRIMARY | -- |
| OPERATIONAL_SECONDARY | -- |
| PRIMARY | -- |
| SECONDARY | -- |

Used by

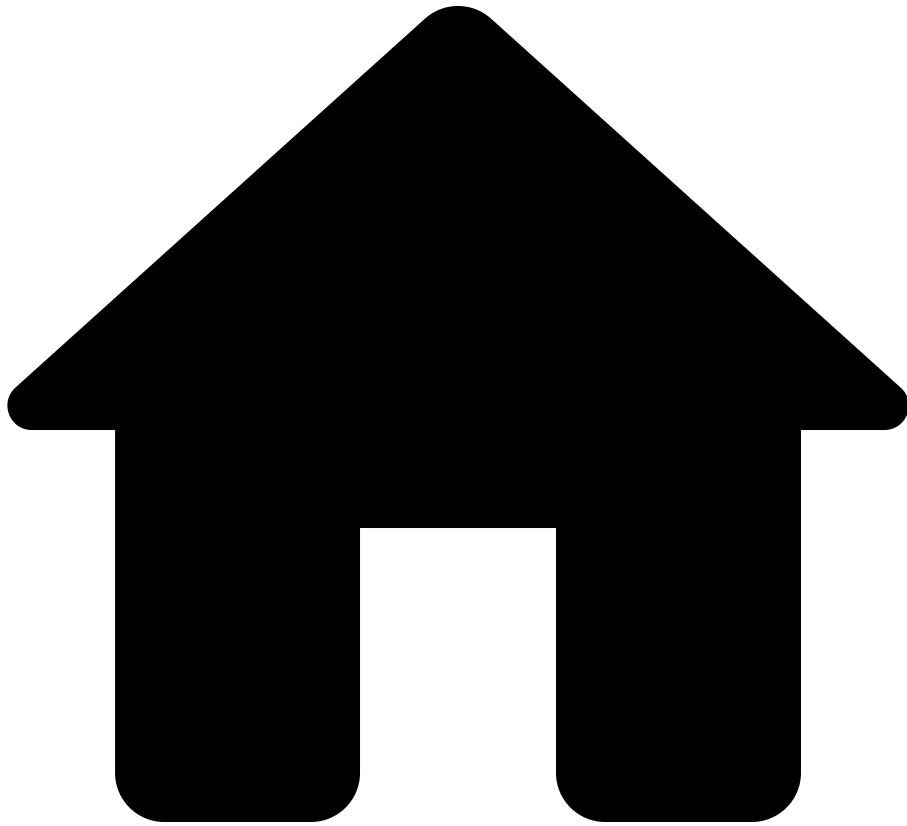
`network` `devices` `ha` `vpc` `vpcRole`

See also

Data model path

- network: `Network`
 - devices: Bag< `Device` >
 - ha: `Ha`
 - vpc: `VpcDomain`
 - vpcRole: `VpcRole`

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VpnConnection

On this page

VpnConnection

Fields

| name | type | nullable | description |
|--------------|------------------|----------|-------------|
| name | String | true | -- |
| id | String | false | -- |
| tags | Bag< String > | false | -- |
| isUp | Bool | false | -- |
| tunnelSrcIps | Bag< IPAddress > | false | -- |
| tunnelDstIps | Bag< IPAddress > | false | -- |

Used by

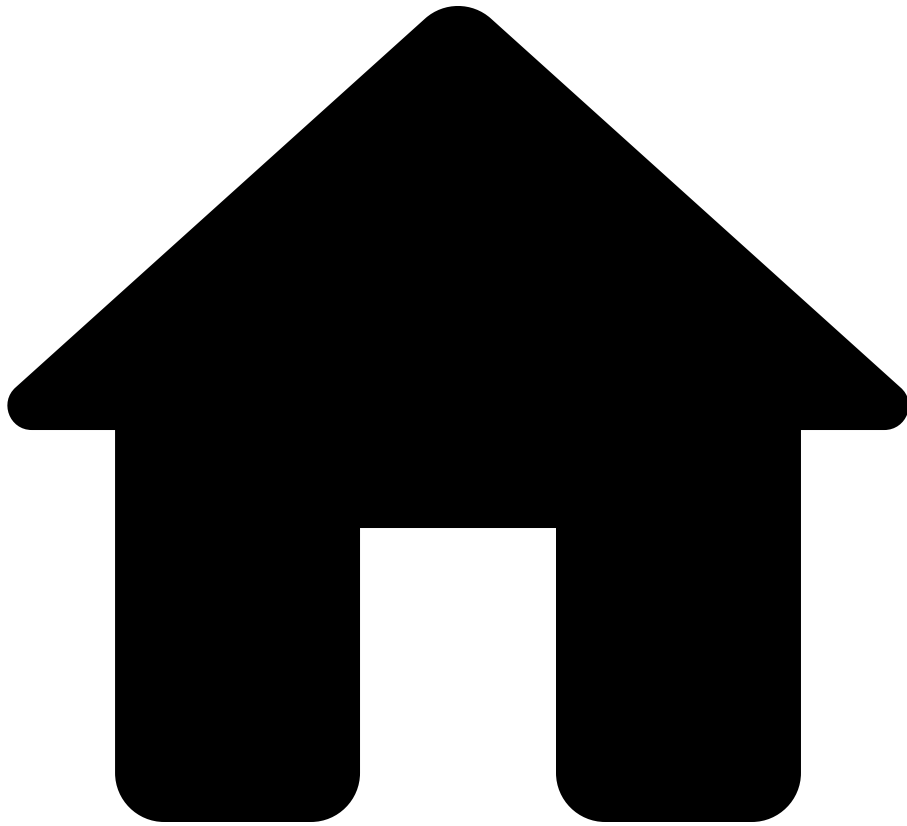
network cloudAccounts vpcs vpnGateways vpnConnections

See also

Data model path

- network: Network
 - cloudAccounts: Bag< CloudAccount >
 - vpcs: Bag< VpcData >
 - vpnGateways: Bag< VpnGateway >
 - vpnConnections: Bag< VpnConnection >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VpnGateway

On this page

VpnGateway

Fields

| name | type | nullable | description |
|----------------|--------------------------------------|----------|-------------|
| name | String | true | -- |
| id | String | false | -- |
| tags | Bag< String > | false | -- |
| region | String | true | -- |
| vpnConnections | Bag< VpnConnection > | false | -- |

Used by

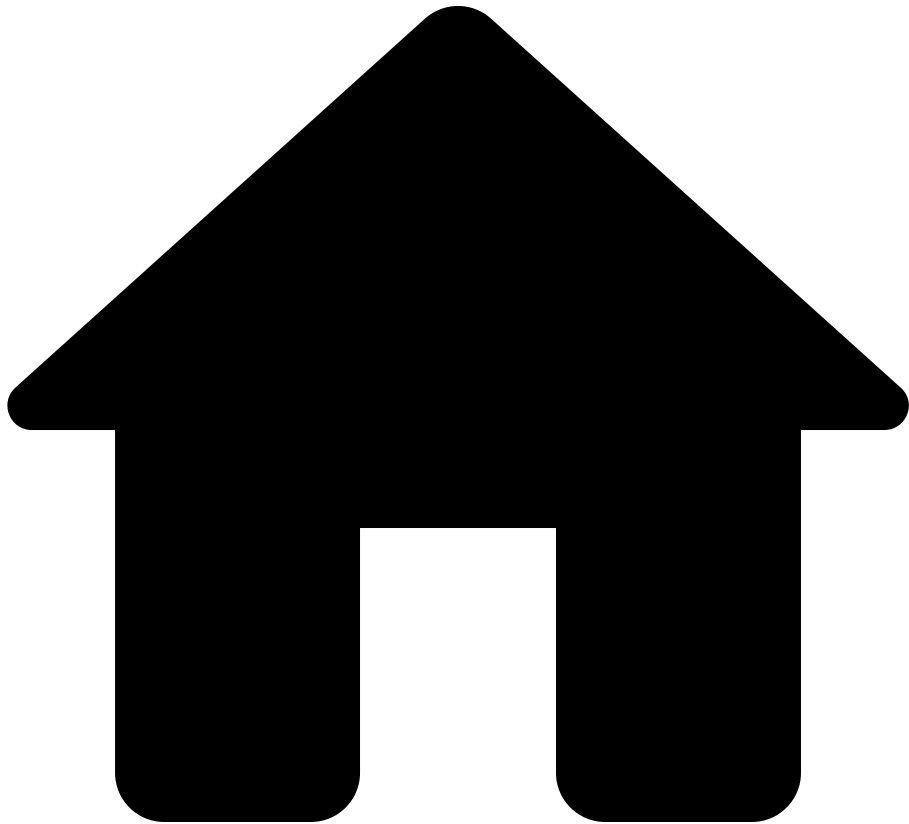
network [cloudAccounts](#) [vpcs](#) [vpnGateways](#)

See also

Data model path

- network: [Network](#)
 - cloudAccounts: Bag< [CloudAccount](#) >
 - vpcs: Bag< [VpcData](#) >
 - vpnGateways: Bag< [VpnGateway](#) >

.



- Reference
- [NQE Language](#)
- [Data Model](#)
- VrfPolicy

On this page

VrfPolicy

Fields

| name | type | nullable | description |
|------|------|----------|-------------|
|------|------|----------|-------------|

Used by[↗](#)

`network` [↗](#) `stigDatabase` [↗](#) `policy` [↗](#) `vrfPolicy`[↗](#)

See also[↗](#)

Data model path

- network: [Network](#)
 - stigDatabase: [StigDatabase](#)
 - policy: [StigPolicy](#)
 - vrfPolicy: [VrfPolicy](#)

ðᳵᳵᳵᳵᳵᳵ address

[address\(subnet IpAddress](#)

ðᳵᳵᳵᳵᳵᳵ all

[Returns true if all elements of the given collection booleans are true.](#)

ðᳵᳵᳵᳵᳵᳵ any

[Returns true if any element of the given collection booleans is true.](#)

ðᳵᳵᳵᳵᳵᳵ bag

[bag\(list Bag](#)

ðᳵᳵᳵᳵᳵᳵ blockDiff

[blockDiff\(config PatternBlocks\) T, blocks Integer}](#)

ðᳵᳵᳵᳵᳵᳵ blockMatches

[blockMatches\(config PatternBlocks\) : Bag](#)

ðᳵᳵᳵᳵᳵᳵ blockPattern

[blockPattern\(text PatternBlocks](#)

ðŸŒŠðŸŒŠðŸŒŠ broadcastAddress

[broadcastAddress\(subnet IpAddress](#)

ðŸŒŠðŸŒŠðŸŒŠ ceil

[ceil\(number Integer](#)

ðŸŒŠðŸŒŠðŸŒŠ date

[date\(text Date](#)

ðŸŒŠðŸŒŠðŸŒŠ days

[days\(value Duration](#)

ðŸŒŠðŸŒŠðŸŒŠ distinct

[For any type T,](#)

ðŸŒŠðŸŒŠðŸŒŠ duration

[duration\(text Duration](#)

ðŸŒŠðŸŒŠðŸŒŠ extractJson

[extractJsonT : T for most types T. One example that T cannot be nor can contain is some OneOf](#)

၎င်း၏ findInterface

[findInterface\(device String\) Iface, subInterface: SubInterface?}??](#)

၎င်း၏ float

[float\(number Float](#)

၎င်း၏ floor

[floor\(number Integer](#)

၎င်း၏ fromTo

[fromTo\(a Integer\) : List](#)

၎င်း၏ hasBlockMatch

[hasBlockMatch\(config PatternBlocks\) : Bool](#)

၎င်း၏ hasMatch

[hasMatch\(text Regex\) : Bool](#)

၎င်း၏ hours

[hours\(value Duration](#)

ðᳵᳵᳵᳵᳵᳵ int

[int\(number Integer](#)

ðᳵᳵᳵᳵᳵᳵ intersect

[intersect\(set1 Set\) : Set for any type T.](#)

ðᳵᳵᳵᳵᳵᳵ ipAddress

[ipAddress\(str IpAddress](#)

ðᳵᳵᳵᳵᳵᳵ ipAddressSet

[ipAddressSet\(ipSubnets Set](#)

ðᳵᳵᳵᳵᳵᳵ ipSubnet

[ipSubnet\(str IpSubnet](#)

ðᳵᳵᳵᳵᳵᳵ ipv4Address

[ipv4Address\(num IpAddress](#)

ðᳵᳵᳵᳵᳵᳵ isEmpty

[isEmpty\(set Bool](#)

ðᳵᳵᳵᳵᳵᳵ isIPv4

[isIPv4\(address Bool](#)

ðᳵᳵᳵᳵᳵᳵ isIPv6

[isIPv6\(address Bool](#)

ðᳵᳵᳵᳵᳵᳵ isPresent

[isPresent\(value Bool for any type T.](#)

ðᳵᳵᳵᳵᳵᳵ join

[Returns the result of concatenating together the elements in list of type String using delimiter between each](#)

ðᳵᳵᳵᳵᳵᳵ length

[For any type T,](#)

ðᳵᳵᳵᳵᳵᳵ limit

[limit\(list Number\) : List](#)

ðᳵᳵᳵᳵᳵᳵ macAddress

[macAddress\(str MacAddress](#)

ðᳵᳵᳵᳵᳵᳵ match

[match\(text Regex\) : T?](#)

ðᳵᳵᳵᳵᳵᳵ matches

[matches\(str String\) : Bool](#)

ðᳵᳵᳵᳵᳵᳵ max

[For any type T,](#)

ðᳵᳵᳵᳵᳵᳵ maxBy

[For any types T and K,](#)

ðᳵᳵᳵᳵᳵᳵ mergeBlocks

[mergeBlocks\(blockPattern PatternBlocks](#)

ðᳵᳵᳵᳵᳵᳵ min

[For any type T,](#)

ðᳵᳵᳵᳵᳵᳵ minBy

[For any types T and K,](#)

minutes

[minutes\(value Duration](#)

networkAddress

[networkAddress\(subnet IpAddress](#)

order

[For any type T,](#)

orderBy

[For any types T and K,](#)

ouiAssignee

[ouiAssignee\(mac String?](#)

ouiVendors

[ouiVendors\(mac Bag](#)

parseConfigBlocks

[parseConfigBlocks\(os String\) : List](#)

ðᳵᳵᳵᳵᳵᳵ patternMatch

[patternMatch\(str Pattern\) : T?](#)

ðᳵᳵᳵᳵᳵᳵ patternMatches

[patternMatches\(config Pattern\) : Bag](#)

ðᳵᳵᳵᳵᳵᳵ prefix

[prefix\(str Integer\) : String](#)

ðᳵᳵᳵᳵᳵᳵ regex

[regex\(text Regex](#)

ðᳵᳵᳵᳵᳵᳵ regexMatches

[regexMatches\(text Regex\) : List](#)

ðᳵᳵᳵᳵᳵᳵ replace

[replace\(text String, replacement String](#)

ðᳵᳵᳵᳵᳵᳵ replaceMatches

[replaceMatches\(text String, replacement String](#)

ðᳵᳵᳵᳵiᳵ replaceRegexMatches

[replaceRegexMatches\(text Regex, replacement String](#)

ðᳵᳵᳵiᳵ round

[round\(number Integer](#)

ðᳵᳵᳵiᳵ seconds

[seconds\(value Duration](#)

ðᳵᳵᳵiᳵ sourceConfigText

[sourceConfigText\(value String for any type T.](#)

ðᳵᳵᳵiᳵ splitJsonObjects

[splitJsonObjects\(json List](#)

ðᳵᳵᳵiᳵ substring

[substring\(str Integer, end String](#)

ðᳵᳵᳵiᳵ suffix

[suffix\(str Integer\) : String](#)

đề thi sum

The sum of a collection of integer numbers or a collection of floating-point numbers.

điều kiện the

For any type T,

ð[][][i] timestamp

timestamp(text Timestamp

toLowerCase

toLowerCase(str String

ð[][][]i toNumber

toNumber(ipAddr Integer

ð[i] toString

toString(value String for any type T.

đi toStringTruncate

toStringTruncate(value String for any type T.

ðŹŹŹŹŹŹ toUpperCase

[toUpperCase\(str String](#)

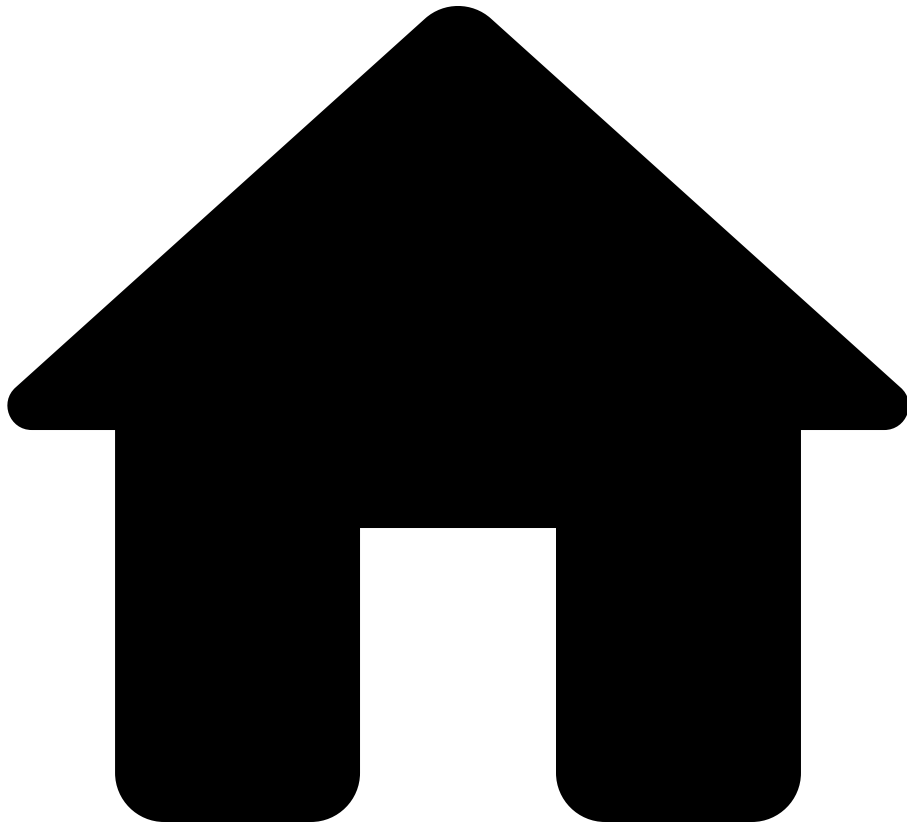
ðŹŹŹŹŹŹ withInfoStatus

[withInfoStatus\(value InfoStatus\)](#)

ðŹŹŹŹŹŹ withOption

[The withOption functions change the behavior of a block pattern.](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- address

On this page

address

```
address(subnet: IpSubnet) : IpAddress
```

Returns the IP address of a subnet. For example,

```
address(ipSubnet("1.2.3.4/24")) is ipAddress("1.2.3.4") .
```

See also[â](#)

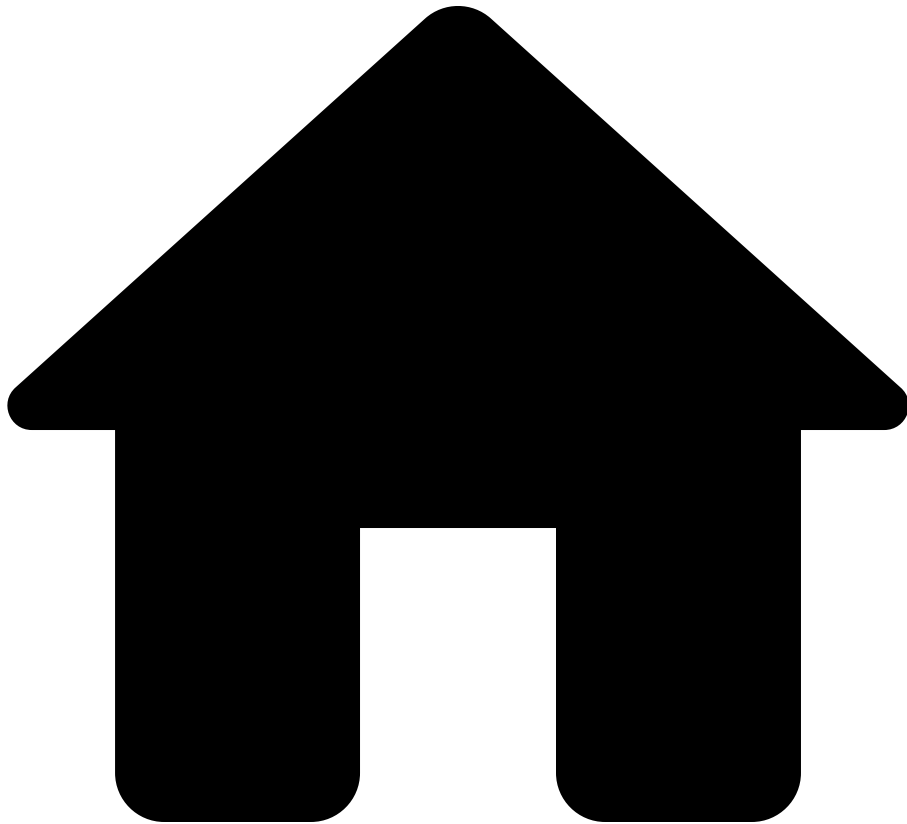
Types[â](#)

- [IpAddress](#)
- [IpSubnet](#)

Functions[â](#)

- [ipAddress](#)
- [ipSubnet](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- all

On this page

all

```
all(booleans: Bag<Bool>) : Bool
```

Returns `true` if all elements of the given collection `booleans` are `true`.

- If the `booleans` collection is empty, returns `true`.
- A `null` element is considered `false`.
- Note that this method accepts both `Lists` and `Bags` since [Lists subtype Bags](#)

Examples

```
emptyList =
  foreach x in fromTo(1, 0)
  select true;

[
  { allWithOnlyTrues: all([true, true]), /* ==> true */
    allWithOnlyFalses: all([false, false]), /* ==> false */
    allWithSomeTrues: all([true, false, true]), /* ==> false */
  }
]
```

The following example query finds for each device if *all* password-policy config satisfies long password length:

```
pattern = ``
  password-policy minimum-length {minLength:number}
``;

LONG_PASSWORD_LENGTH = 15;

foreach device in network.devices
let config = device.files.config
select {
  Device: device.name,
  OnlyLongPassword: if hasBlockMatch(config, pattern)
    then all(foreach match in
blockMatches(config, pattern)
```

```
select match.data.minLength >=
LONG_PASSWORD_LENGTH)
else false
}
```

See also

[â€¦](#)

Types

[â€¦](#)

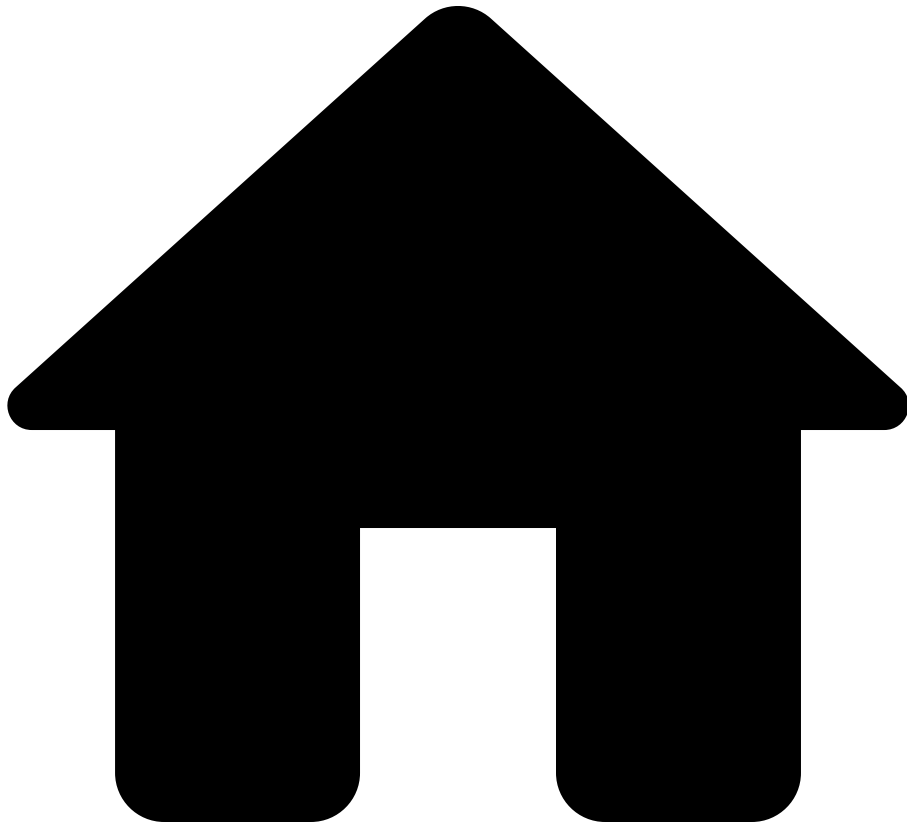
- [Bool](#)
- [Collections](#)

Functions

[â€¦](#)

- [any](#)
- [min](#)
- [max](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- any

On this page

any

```
any(booleans: Bag<Bool>) : Bool
```



```
LONG_PASSWORD_LENGTH)  
}
```

See also[â](#)

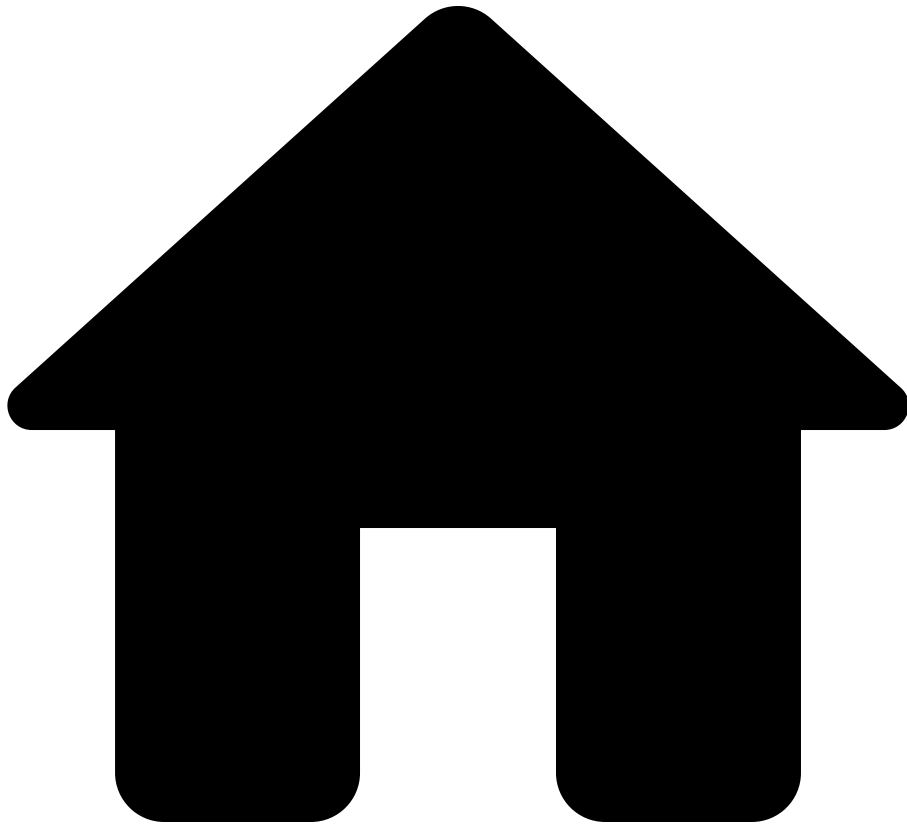
Types[â](#)

- [Bool](#)
- [Collections](#)

Functions[â](#)

- [all](#)
- [min](#)
- [max](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- bag

On this page

bag

```
bag(list: List<T>) : Bag<T>
```

Returns a bag containing the same elements as the input list, but without any ordering guarantees.

Examples[↗](#)

Basic Usage[↗](#)

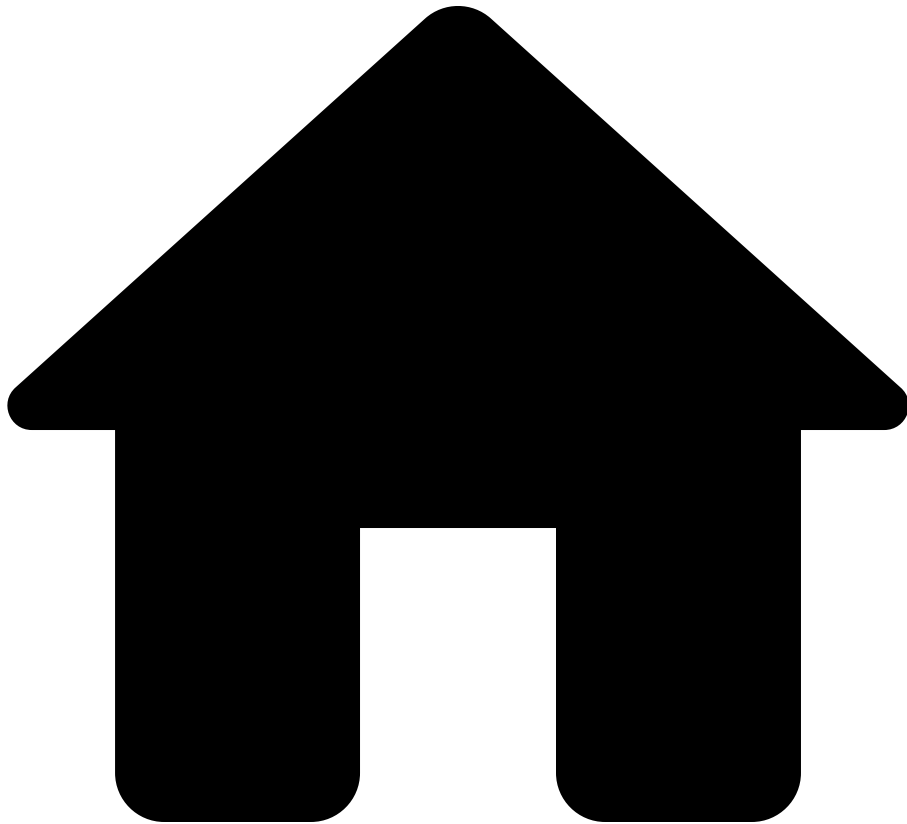
```
x = [3, 1, 2];  
y = bag(x); // Bag containing 1, 2, 3 (order undefined)
```

See Also[↗](#)

Guides[↗](#)

- [Collections](#) - Comprehensive guide on Lists, Bags, and ordering

.

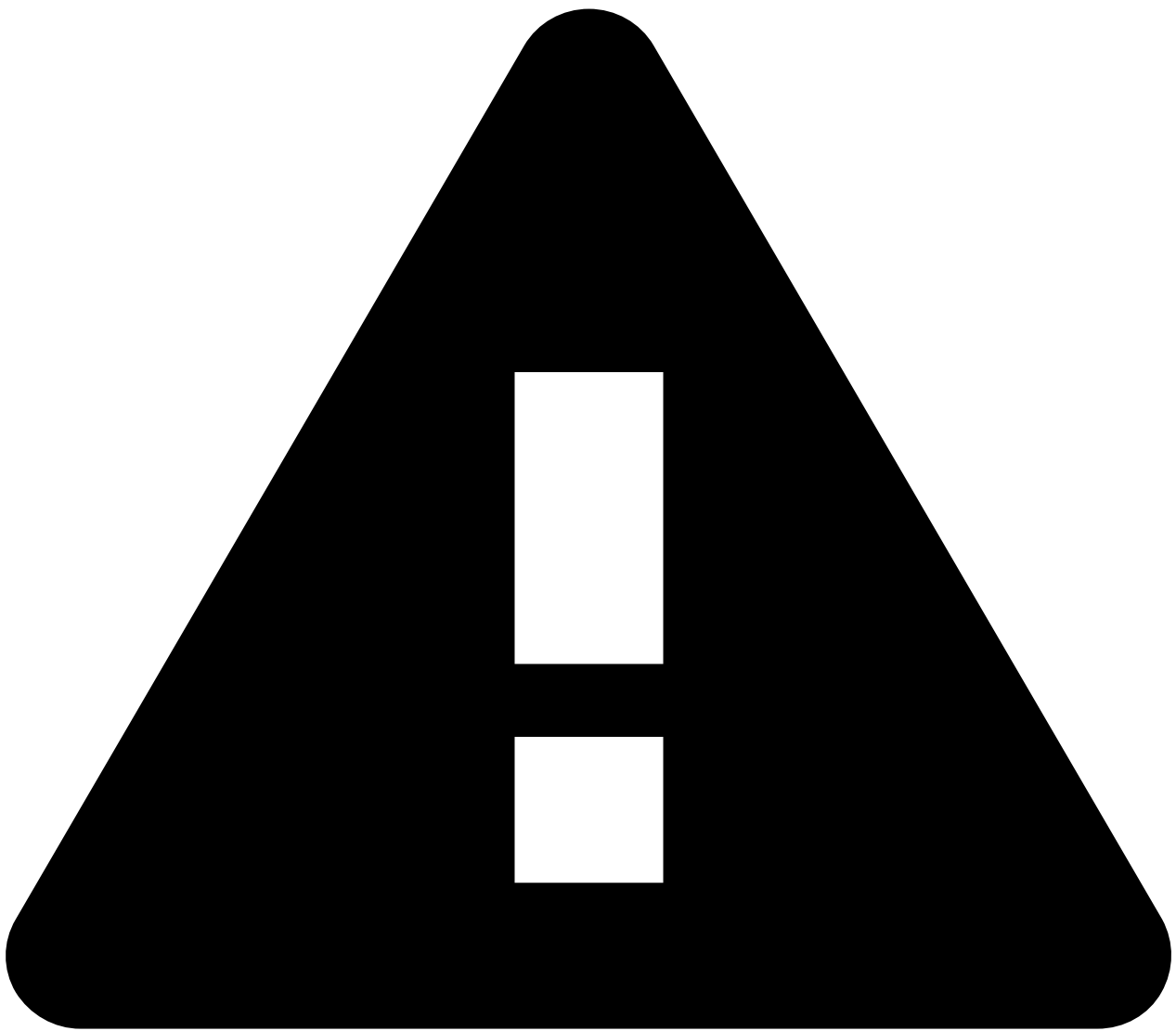


- Reference
- [NQE Language](#)
- [Standard Library](#)
- blockDiff

On this page

blockDiff

```
blockDiff(config: List<ConfigLine>, pattern: PatternBlocks<T>) :  
{data: T, blocks: MatchBlocks, diffCount: Integer}
```



Deprecated Bag overload

As a transitional measure for backwards compatibility, `blockDiff` also accepts a `Bag<ConfigLine>` as its first argument. Use `orderBy()` or `order()` to create an argument with type `List<ConfigLine>`. Eventually, the Bag overload will be removed and a type-checking error will be emitted instead.

See also

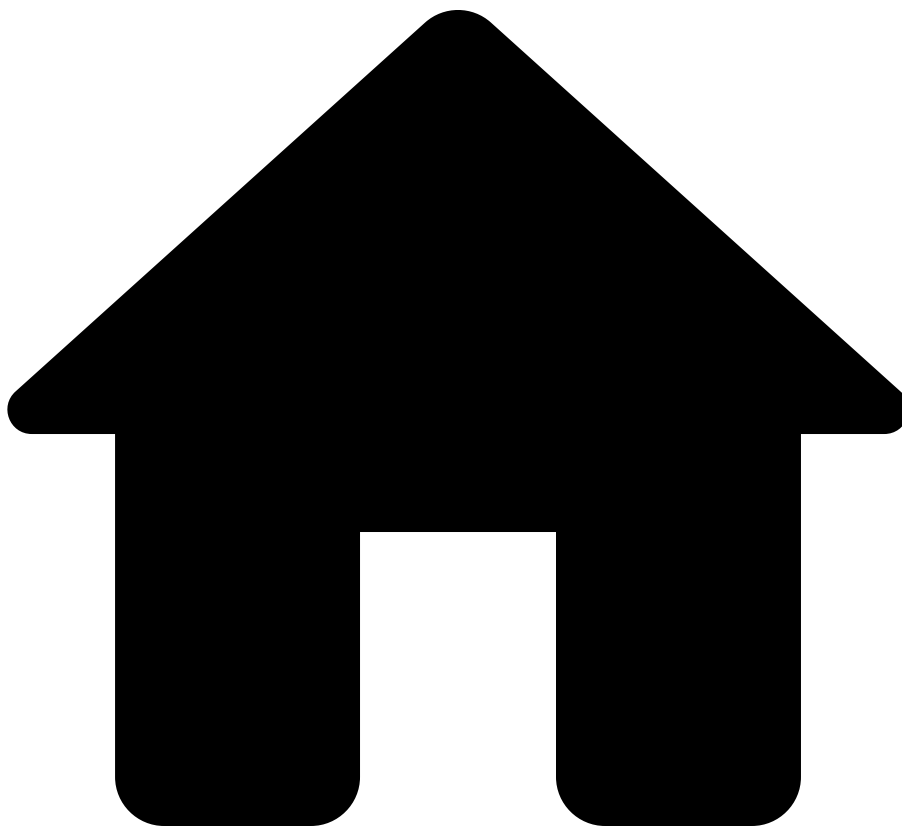
[â](#)

Functions

[â](#)

- [blockMatches](#)

.

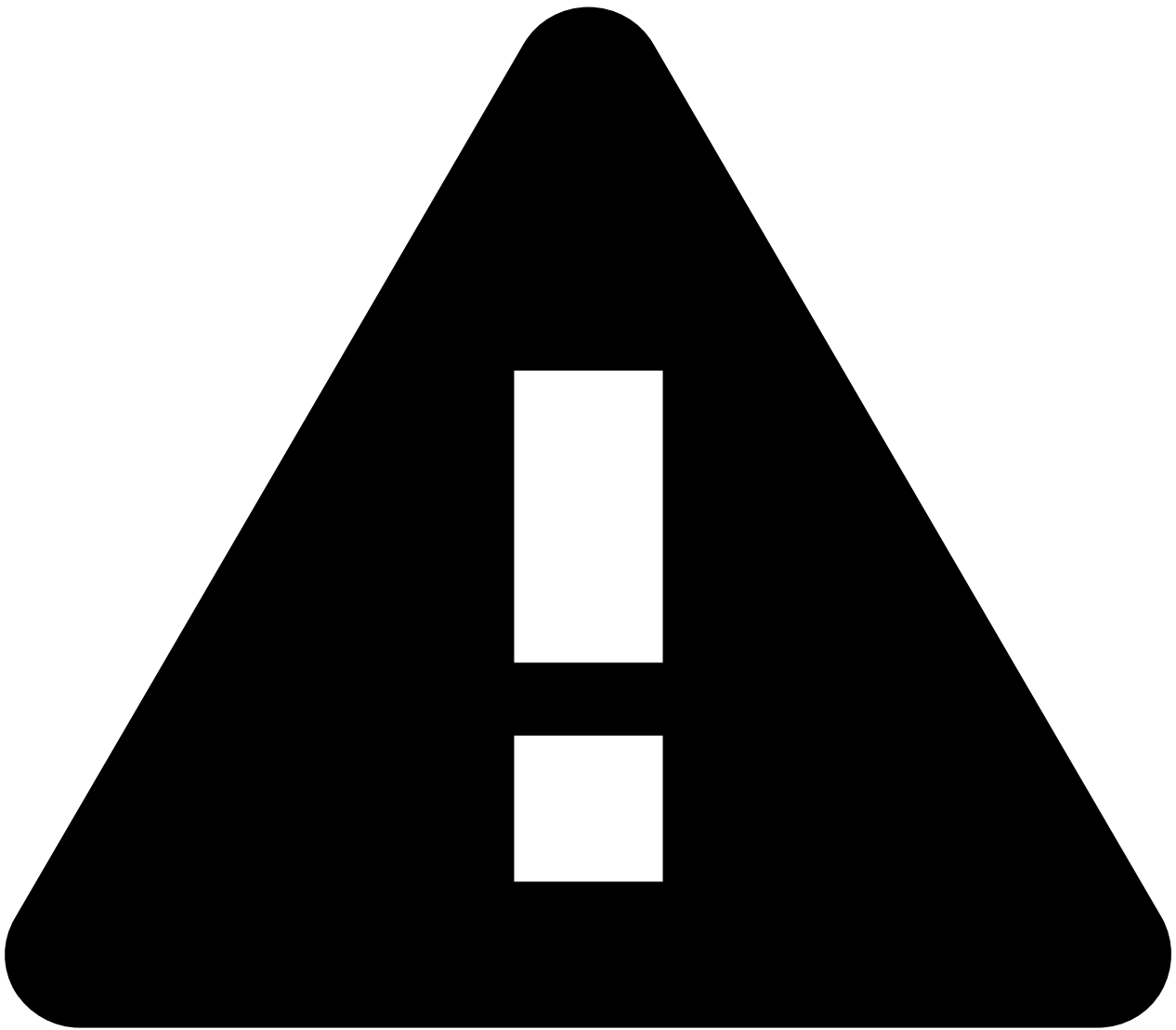


- Reference
- [NQE Language](#)
- [Standard Library](#)
- blockMatches

On this page

blockMatches

```
blockMatches(config: List<ConfigLine>, pattern:  
PatternBlocks<T>) : Bag<{data: T, blocks: MatchBlocks, diffCount:  
Integer}>
```



Deprecated Bag overload

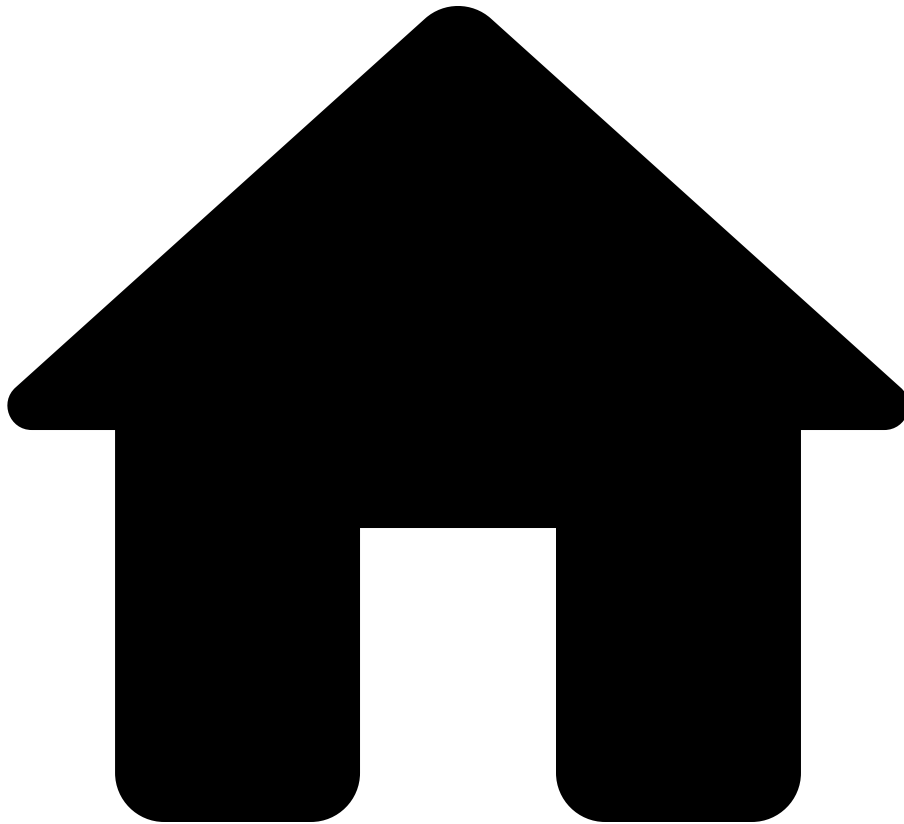
As a transitional measure for backwards compatibility, `blockMatches` also accepts a `Bag<ConfigLine>` as its first argument. Use `orderBy()` or `order()` to create an argument with type `List<ConfigLine>`. Eventually, the Bag overload will be removed and a type-checking error will be emitted instead.

See also

Functions

- [hasBlockMatch](#)
- [blockDiff](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- blockPattern

On this page

blockPattern

```
blockPattern(text: String) : PatternBlocks<{}>
```

The `blockPattern(text)` function can be used to parse a block pattern from a string. For example, the following query matches a set of patterns against device

configurations, where the patterns are obtained from HTTP sources using the `blockPattern` function:

```
foreach device in network.devices
foreach httpSource in network.externalSources.httpSources
foreach endpoint in httpSource.endpoints
where endpoint.statusCode == 200
let patternSource = endpoint.responseBody
where isPresent(patternSource)
foreach match
  in blockMatches(device.files.config,
blockPattern(patternSource))
select { deviceName: device.name, patternSource,
matchedBlocks: match.blocks }
```

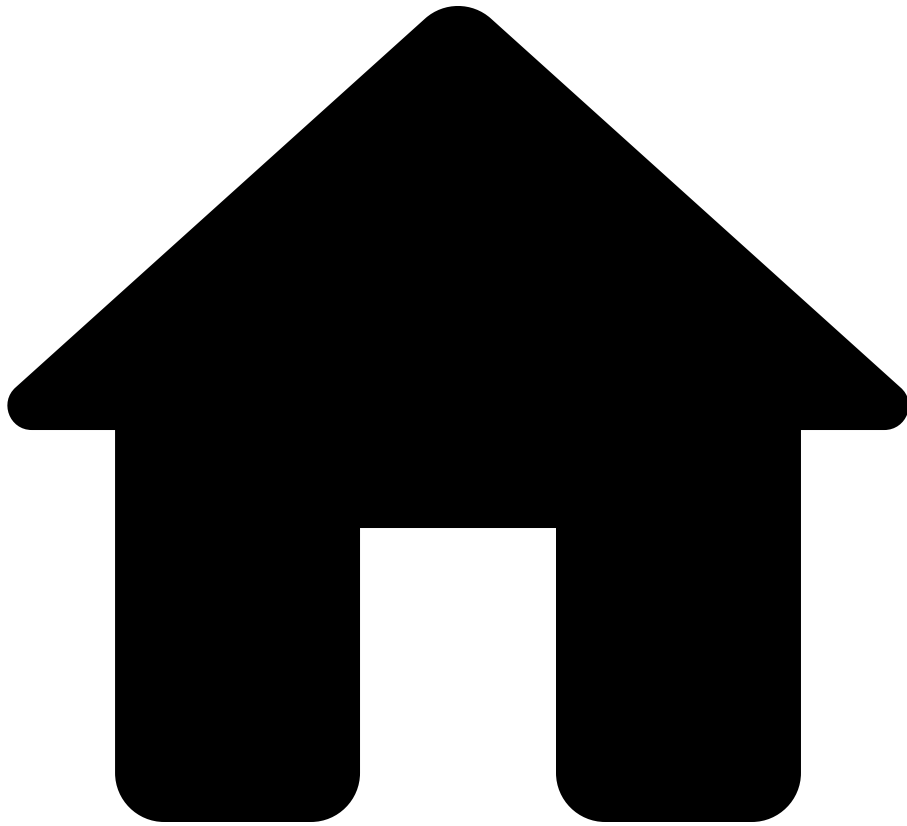
Note that the pattern source must not contain any capture groups.

See also [â](#)

Types [â](#)

- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- broadcastAddress

On this page

broadcastAddress

```
broadcastAddress(subnet: IpSubnet) : IpAddress
```

Returns the broadcast IP address for a subnet. That is, the IP address for a subnet, with all host bits set to 1. For example,

```
broadcastAddress(ipSubnet("1.2.3.4/24")) is  
ipAddress("1.2.3.255") .
```

See also[â€‹](#)

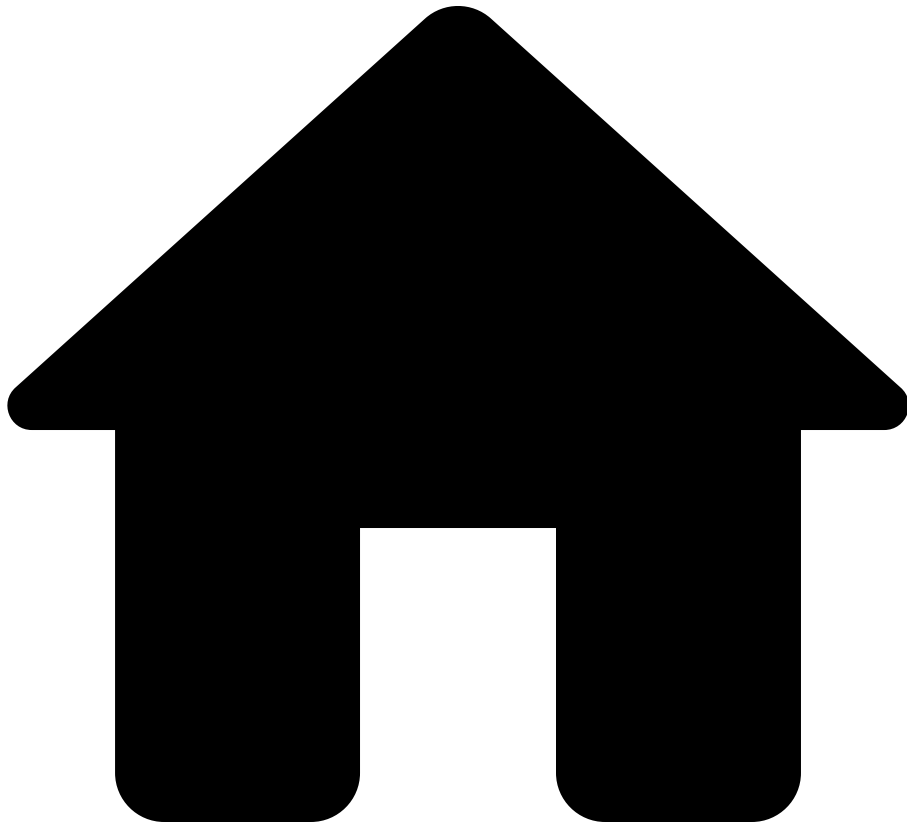
Types[â€‹](#)

- [IpAddress](#)
- [IpSubnet](#)

Functions[â€‹](#)

- [ipAddress](#)
- [ipSubnet](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- `ceil`

On this page

ceil

```
ceil(number: Float) : Integer
```

Gets an integer from a float by *rounding up* to the nearest larger integer; e.g.,

```
int(2.4) results in 3 .
```

See also [a](#)

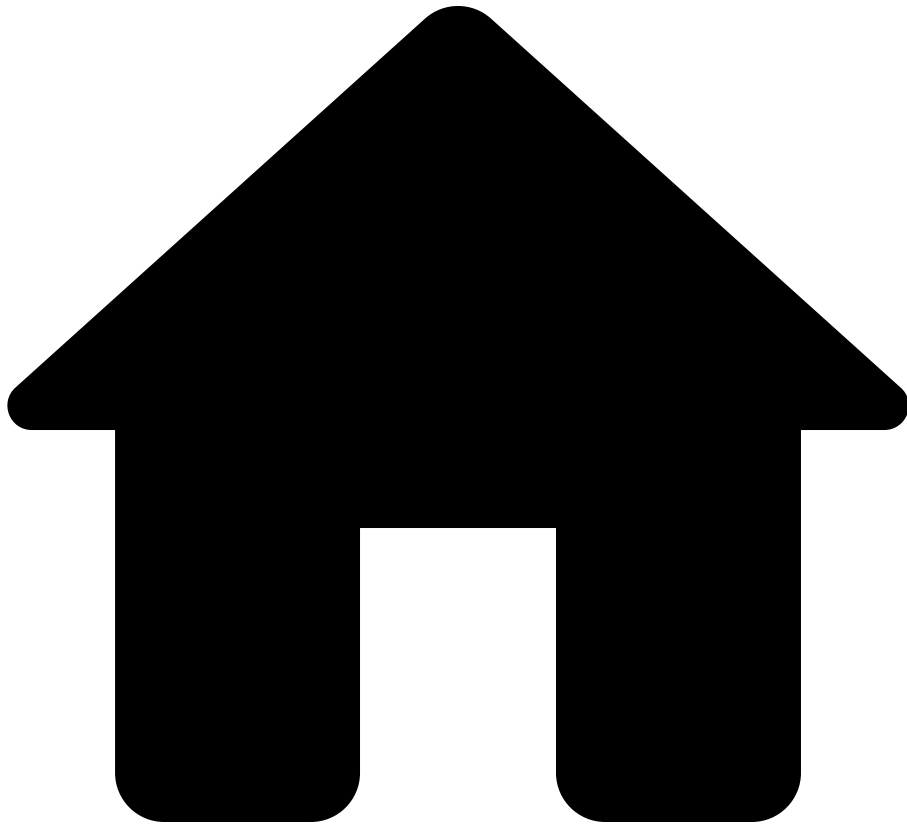
Functions [a](#)

- [float](#)
- [int](#)
- [floor](#)
- [round](#)
- [toNumber](#)

Types [a](#)

- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- date

On this page

date

```
date(text: String) : Date
```

Obtains a `Date` from a `String` that represents a date according to the ISO 8601 standard. For example, `date("1929-10-29")` returns a `Date` that represents October 29, 1929.

This function will raise an error if the `String` deviates from the ISO 8601 standard for dates or if the `String` is absent.

```
date(timestamp: Timestamp) : Date
```

Obtains a `Date` from a `Timestamp` by returning the date portion of a `Timestamp` when placed in the UTC timezone.

This function will raise an error if the resulting date exceeds the range of `Date` or if the `Timestamp` is absent.

See also

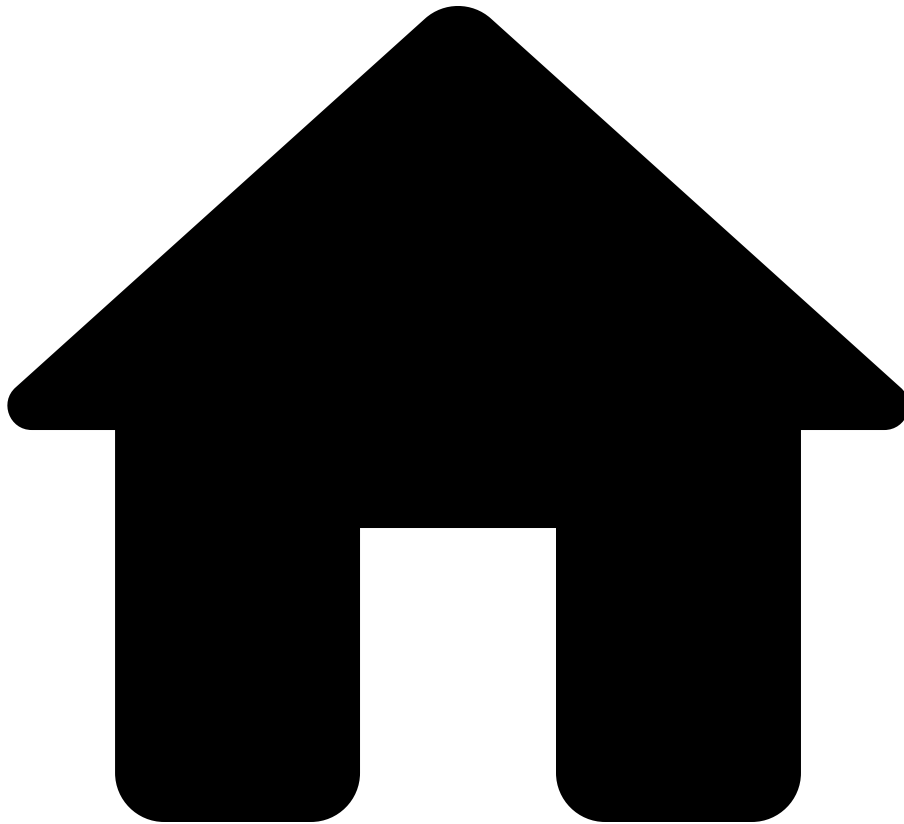
Functions

- [timestamp](#)
- [duration](#)
- [days](#)
- [hours](#)
- [minutes](#)
- [seconds](#)

Types

- [Time](#)
- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- days

On this page

days

`days(value: Integer) : Duration`

Obtains a `Duration` representing the given number of days. For example, `days(3)` returns a `Duration` that represents three days. The value can be negative. For example, `days(-2)` returns a `Duration` that represents negative two days.

This function will raise an error if the absolute value of the input is greater than `106751991167300` .

See also[â](#)

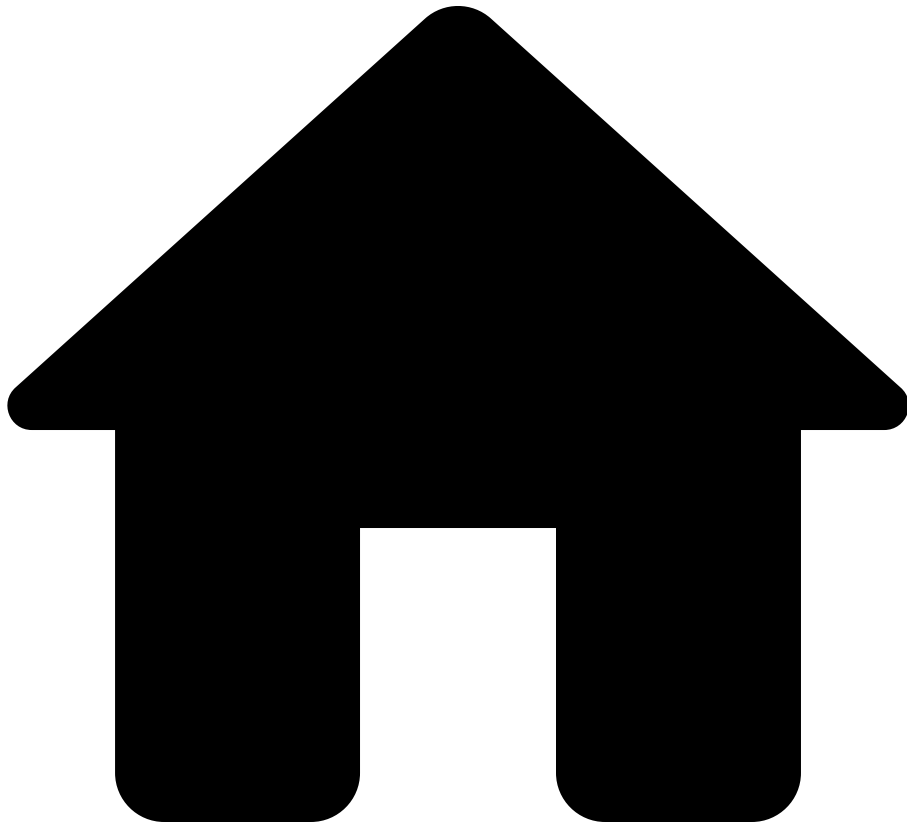
Functions[â](#)

- [date](#)
- [timestamp](#)
- [duration](#)
- [hours](#)
- [minutes](#)
- [seconds](#)

Types[â](#)

- [Time](#)
- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- distinct

On this page

distinct

For any type `T`,

```
distinct(collection: List<T>) : List<T>
distinct(collection: Bag<T>)  : Bag<T>
```

Returns the unique items in `collection` .

- For lists, `distinct` deduplicates the collection while preserving the order of the items in the original list: The first occurrence of an element is kept and all other subsequent occurrences are dropped.

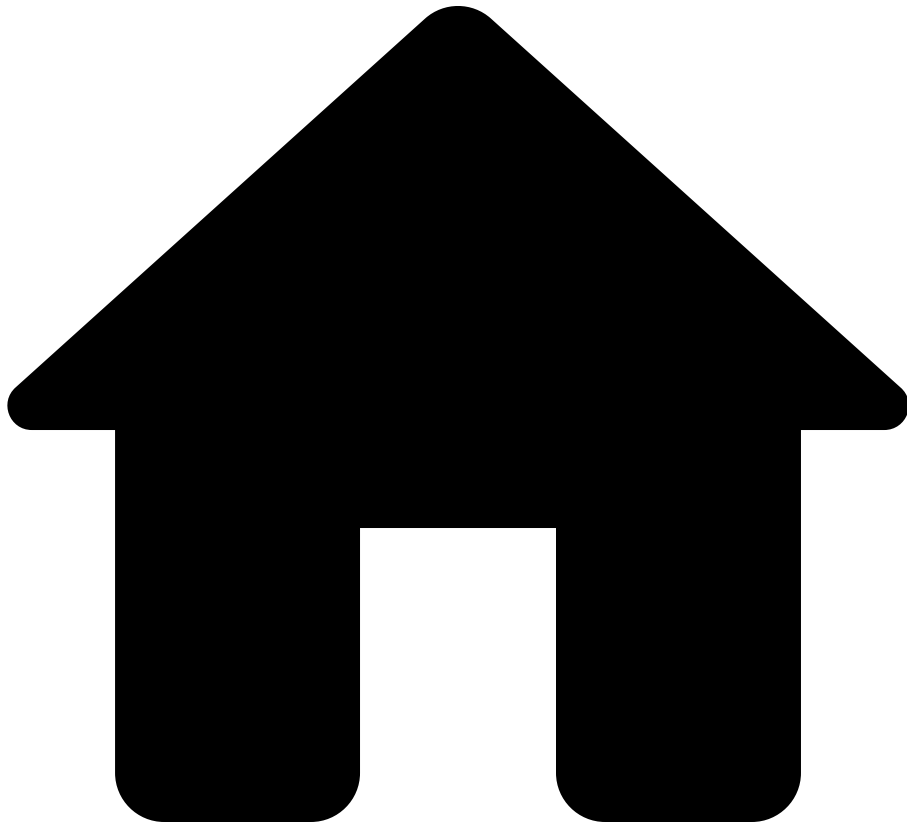
```
list      = [1, 2, 3, 1, 4, 2];  
uniqueList = distinct(list); // [1, 2, 3, 4]
```

See also [â](#)

Types [â](#)

- [Collections](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- duration

On this page

duration

`duration(text: String) : Duration`

Obtains a `Duration` from a `String` that represents a duration according to the ISO 8601 standard. For example, `duration("PT23H56M4S")` returns a `Duration` that represents 23 hours, 56 minutes, and 4 seconds.

This function will raise an error if the string significantly deviates from the ISO 8601 standard. A permitted deviation is for strings that represent a negative interval of time. This function accepts strings like `"-P1DT12H"` , `"P-1DT-12H"` , and `"P-2DT12H"` , which all represent -36 hours.

This function will also raise an error if the string represents a duration with a nonzero amount of nanoseconds, such as the string `"PT0.5S"` .

See also[â](#)

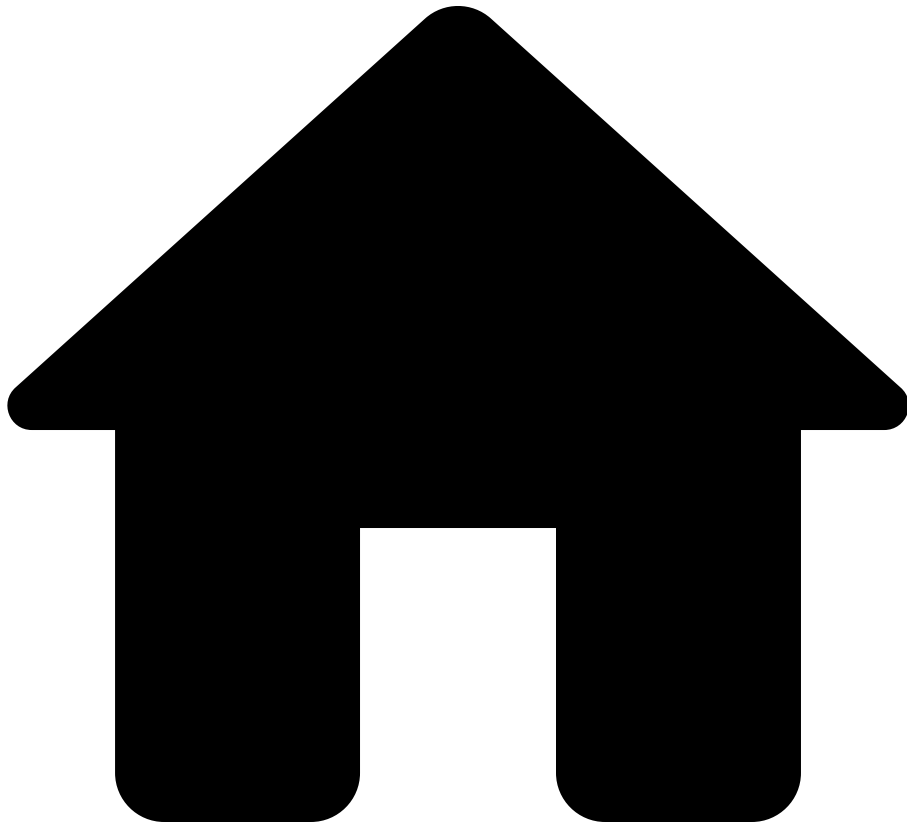
Functions[â](#)

- [date](#)
- [timestamp](#)
- [days](#)
- [hours](#)
- [minutes](#)
- [seconds](#)

Types[â](#)

- [Time](#)
- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- `extractJson`

extractJson

`extractJson[T](json: String) : T` for most types `T`. One example that `T` cannot be nor can contain is some `OneOf` type.

NQE values can be extracted from a JSON string using the `extractJson` function.

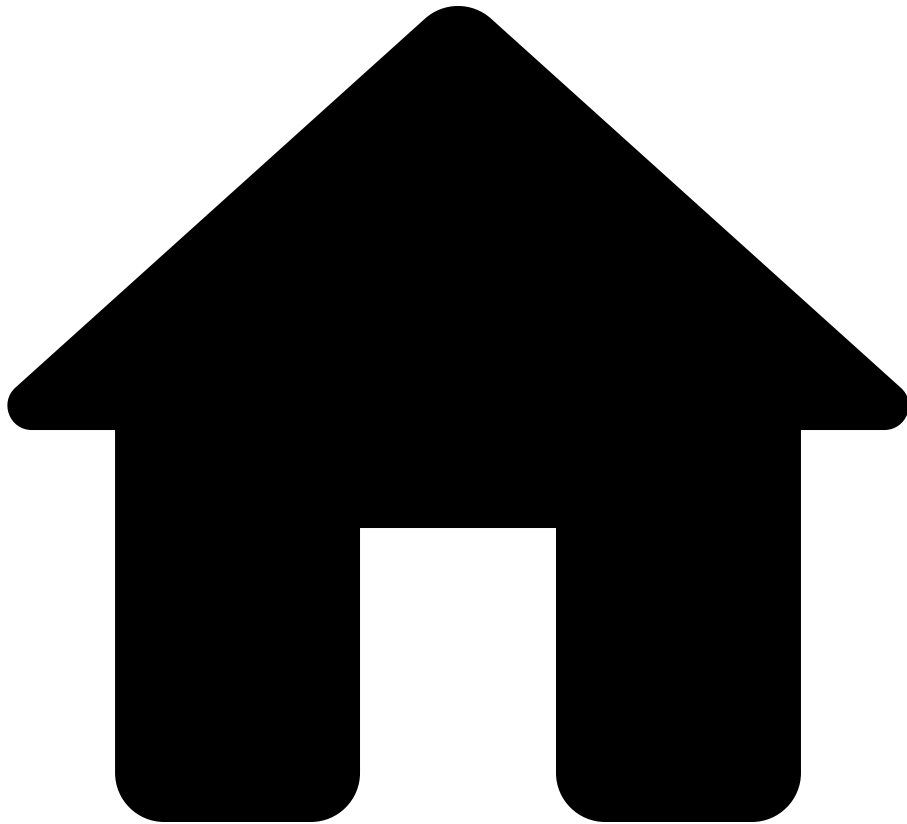
```
extractJson[Integer]("1")
```

The above expression will return the NQE value `1` of type `Integer`. The `String` value that contains the JSON value (`"1"` above) is passed to `extractJson` as a regular argument. The type of NQE value to be extracted from the JSON value is written between square brackets prior to the function call's parentheses (`Integer` in the above example).

The following are examples of expressions with `extractJson` calls and the NQE value that results from evaluating each expression.

- `extractJson[Integer]("1") => 1`
- `extractJson[Boolean]("true") => true`
- `extractJson[String]("\"foo\") => \"foo\"`
- `extractJson[IpAddress]("\"192.168.0.1\") => 192.168.0.1`
- `extractJson[{}](\"{\}") => {}`
- `extractJson[{}](\"{\\\"x\\\" : 1}\") => {}`
- `extractJson[{x: Integer}](\"{\\\"x\\\" : 1}\") => { x: 1 }`
- `extractJson[{x: Integer, y: Boolean}](\"{\\\"x\\\": 1, \\\"y\\\": true}\") => { x: 1, y: true }`
- `extractJson[{x?: Integer}](\"{\}") => { x: null }`
- `extractJson[{x?: Integer}](\"{\\\"x\\\": null}\") => { x: null }`
- `extractJson[List<Integer>](\"[1,2]\") => [1, 2]`
- `extractJson[Bag<Integer>](\"[1,2]\") => bag([1, 2])`
- `extractJson[List<{x: Integer}>](\"[{\\\"x\\\":1},{\\\"x\\\":2}]\") => [{ x: 1 }, { x: 2 }]`

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- findInterface

findInterface

```
findInterface(device: Device, ifaceName: String) : { interface: Iface, subInterface: SubInterface?}?
```

A single device's configuration and state files may refer to the same interface by slightly different names. For example, on a Cisco NXOS device, a port channel might appear in the configuration as "interface port- channel3", while the output of a "show interface"

command on the device may refer to the same interface as "Po3", using an abbreviated form and different capitalization of letters.

The normalized data model for interfaces uses a canonical, normalized name for each interface. The `findInterface` function enables you to lookup an interface in the normalized data model using a name that might not be normalized; for example, you can use it to look up an interface using "port-channel3" as it appears in the config, even though it is named "po3" in the normalized data model.

Specifically, `findInterface` takes two arguments: a device and an interface name. For example, `findInterface(device, "port-channel3")` returns:

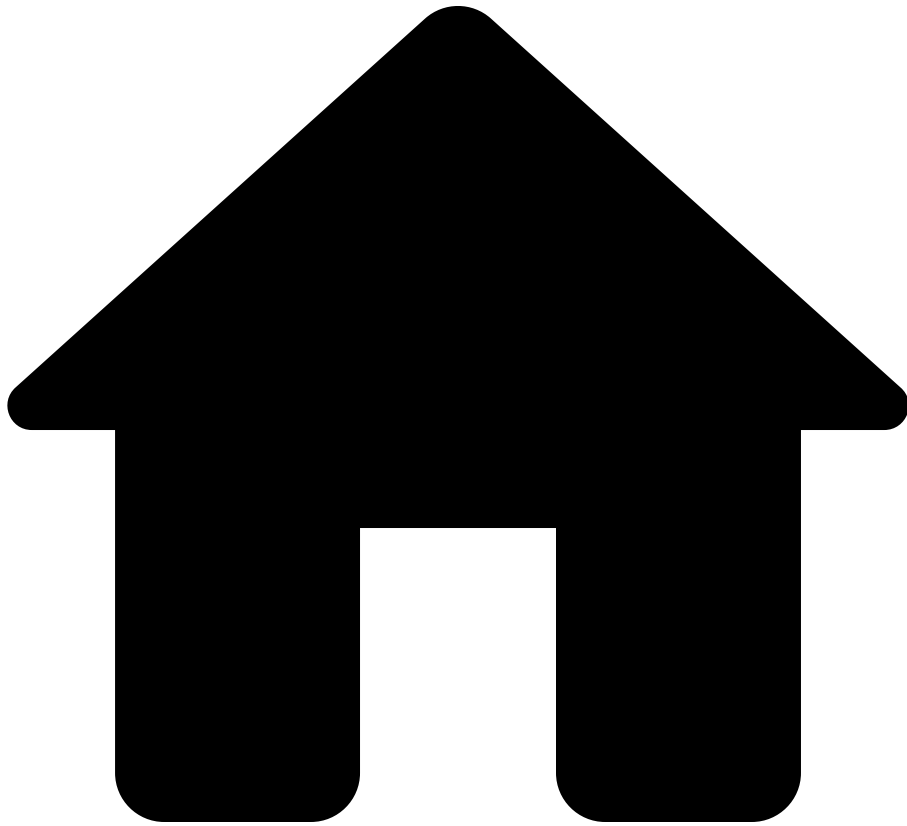
- `null` if "port-channel3" returned no matches across the device's interfaces and the sub-interfaces of all the device's interfaces.
- `{ interface: Iface, subInterface: null }` if "port-channel3" matched one of the device's interfaces, but none of that interface's sub-interfaces were matched.
- `{ interface: Iface, subInterface: SubInterface }` if "port-channel3" matched one of the device's interfaces' sub-interfaces. In this case the matched sub-interface's parent interface is also returned.

As an example, the following query find interfaces on NXOS devices that have some ip ospf configuration and MTU setting above 9000. The query finds relevant interfaces in the configuration, and then uses `findInterface` to lookup the MTU setting in the normalized data model based on the interface name as it appears in the configuration.

```
foreach device in network.devices
where device.platform.os == OS.NXOS
foreach match in patternMatches(device.files.config,
`interface {ifaceName:string}`)
where length(patternMatches(match.line.children, `ip ospf`)) >
0
let ifaceMatch = findInterface(device, match.data.ifaceName)
where isPresent(ifaceMatch) && ifaceMatch.interface.mtu > 9000
select {
  deviceName: device.name,
  iface: ifaceMatch.interface.name,
```

```
mtu: ifaceMatch.interface.mtu  
}
```

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- float

On this page

float

```
float(number: Integer) : Float
```

Converts an integer-valued expression to a floating-point number.

Some examples, where `x` and `y` are assumed to be variables that have integers assigned to them:

- `float(x)`
- `float((x + y) / 100.)`
- `float(sum(fromTo(1,100)))`

See also [â](#)

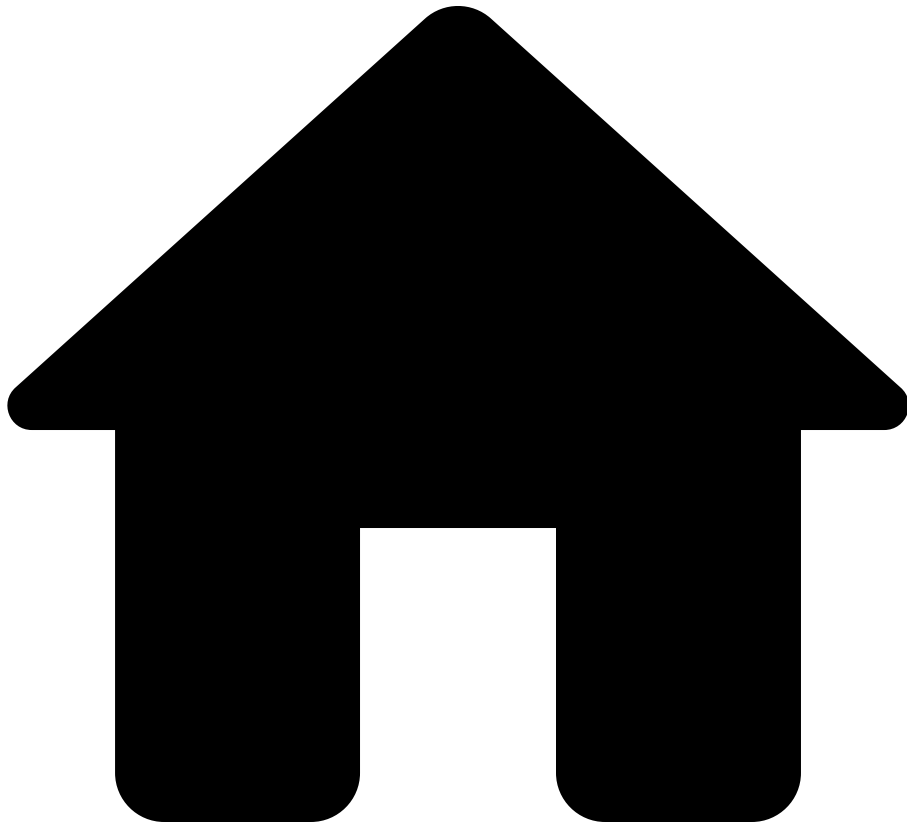
Functions [â](#)

- [int](#)
- [floor](#)
- [ceil](#)
- [round](#)

Types [â](#)

- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- floor

On this page

floor

`floor(number: Float) : Integer`

Gets an integer from a float by *rounding down* to the nearest lower; e.g., `int(2.9)` results in `2`.

See also [a](#)

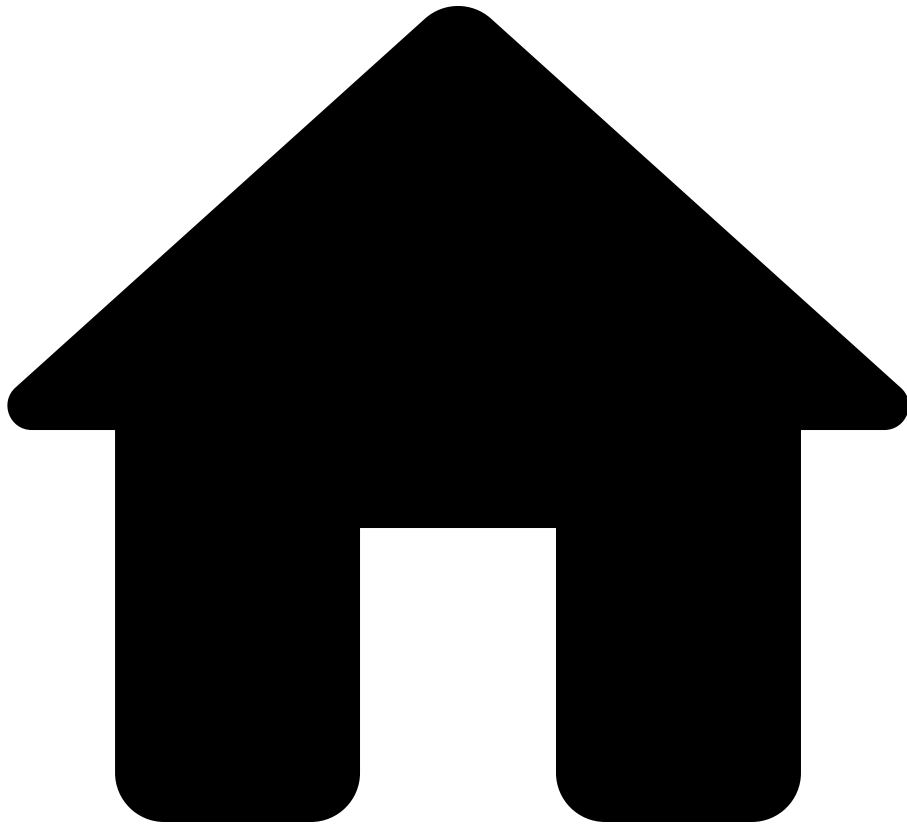
Functions [a](#)

- [float](#)
- [int](#)
- [ceil](#)
- [round](#)
- [toNumber](#)

Types [a](#)

- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- fromTo

On this page

fromTo

```
fromTo(a: Integer, b: Integer) : List<Integer>
```

Returns an ordered list of consecutive integers from `a` to `b` (inclusive). The list is in ascending order.

- For example, `fromTo(1, 3)` returns `[1, 2, 3]`.
- If the second argument is smaller than the first, as in `fromTo(3, 1)`, the returned value is the empty list.

```
fromTo(a: IPAddress, b: IPAddress) : List<IPAddress>
```

A list of IP addresses in a range can be constructed with `fromTo(a, b)`, which equals the list of IP addresses starting from `a` and ending with `b`. For example:

- `fromTo(ipAddress("1.2.2.254"), ipAddress("1.2.3.1"))` is same as `[1.2.2.254, 1.2.2.255, 1.2.3.0, 1.2.3.1]`
- `fromTo(ipAddress("2001:0db8:85a3:0000:0000:8a2e:0370:ffff"), ipAddress("2001:0db8:85a3:0000:0000:8a2e:0371:001"))` is same as `[2001:0db8:85a3:0000:0000:8a2e:0370:ffff, 2001:0db8:85a3:0000:0000:8a2e:0370:ffff, 2001:0db8:85a3:0000:0000:8a2e:0371:0, 2001:0db8:85a3:0000:0000:8a2e:0371:1]`.

If the second argument is smaller than the first, as in

`fromTo(ipAddress("1.2.2.4"), ipAddress("0.0.0.0"))`, the returned value is the empty list.

This function will raise an error:

- if `a` and `b` are not of same IP address family. For example, `fromTo(ipAddress("1.2.2.254"), ipAddress("2001:0db8:85a3:0000:0000:8a2e:0370:ffff"))`.
- if number of IP addresses between `a` and `b` is 2147483647 or greater.

See also

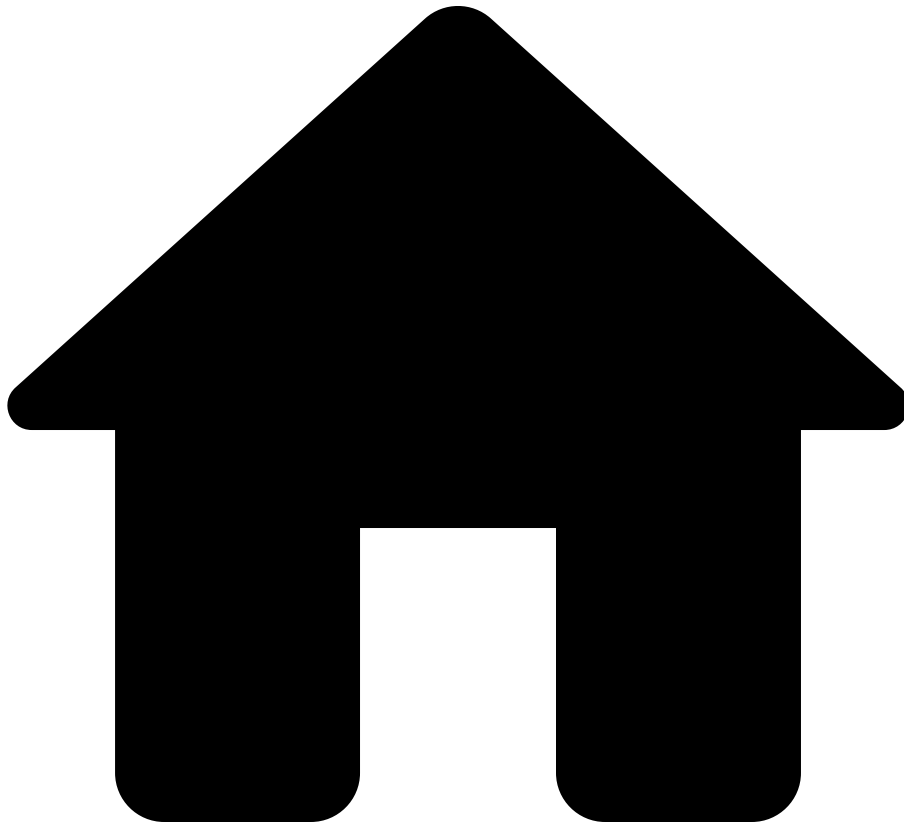
Guides

- [Collections](#) - Working with ordered lists
- [Integer](#)
- [IpAddress](#)

Functions

- [ipAddress](#) - Create IP address from string

.



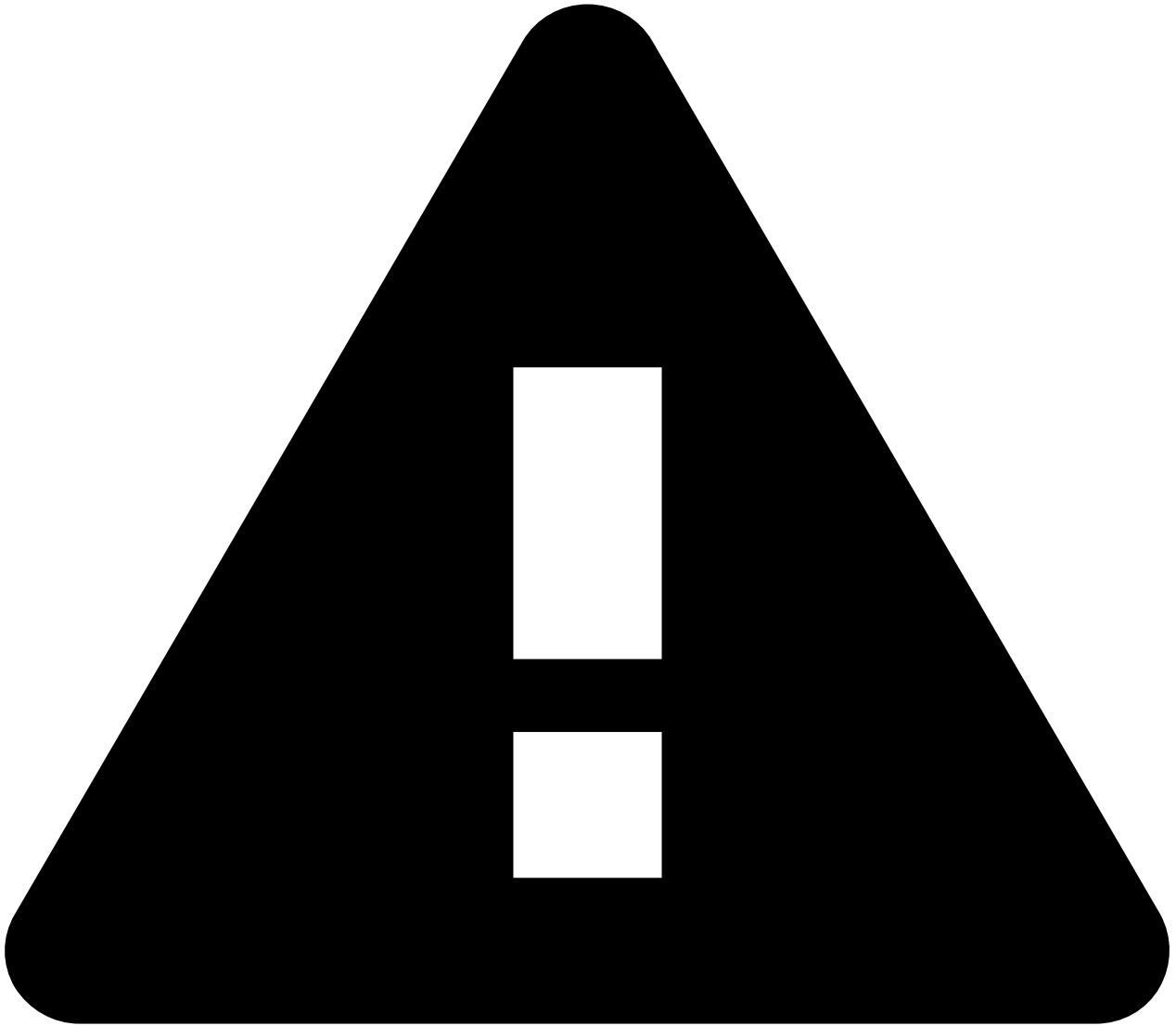
- Reference
- [NQE Language](#)
- [Standard Library](#)
- hasBlockMatch

On this page

hasBlockMatch

```
hasBlockMatch(config: List<ConfigLine>, pattern:  
PatternBlocks<T>) : Bool
```

If [blockMatches](#) returns a nonempty collection on the same input, then this function returns `true` . Otherwise, this function returns `false` .



Deprecated Bag overload

As a transitional measure for backwards compatibility, `hasBlockMatch` also accepts a `Bag<ConfigLine>` as its first argument. Use `orderBy()` or `order()` to create an argument with type `List<ConfigLine>`. Eventually, the Bag overload will be removed and a type-checking error will be emitted instead.

See also

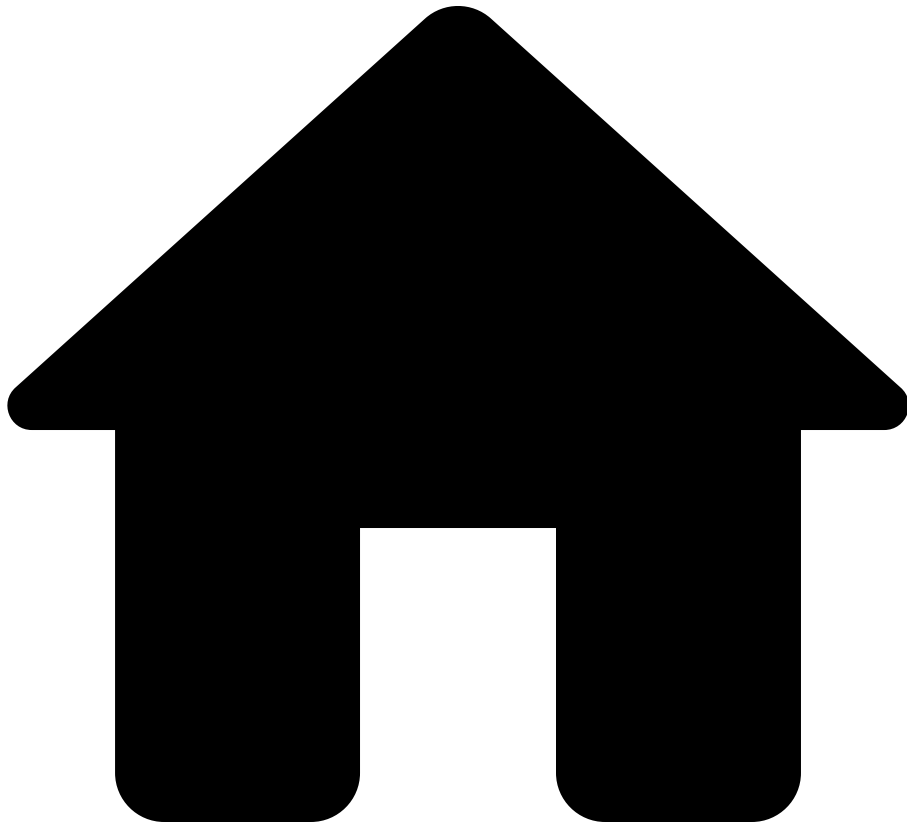
[ââ](#)

Functions

[ââ](#)

- [blockMatches](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- hasMatch

On this page

hasMatch

```
hasMatch(text: String, regex: Regex<T>) : Bool
```

Checks whether `text` matches the structure specified by `regex`. The entirety of `text` must match.

Examples

| Example | Result |
|--|--------|
| <code>hasMatch("", re``)</code> | true |
| <code>hasMatch(" ", re``)</code> | false |
| <code>hasMatch("Name: Jane, Age: 32", re`Name: (\w+), Age: (\d+)`)</code> | true |
| <code>hasMatch("", re`Name: (\w+), Age: (\d+)`)</code> | false |
| <code>hasMatch("192.168.0.1", re`\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b`)</code> | true |
| <code>hasMatch("1.2.3x4", re`\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b`)</code> | false |
| <code>hasMatch("device-config.txt", re`[^ ]*\.txt`)</code> | true |
| <code>hasMatch("device config.org", re`[^ ]*\.txt`)</code> | false |
| <code>hasMatch("2357", re`(?<age:Integer>\w+)`)</code> | true |
| <code>hasMatch("3x", re`(?<age:Integer>\w+)`)</code> | false |
| <code>hasMatch("3.14", re`(?<age:Float>\d+\.\d+)`)</code> | true |
| <code>hasMatch("3.14", re`(?<age:Float>\w+)`)</code> | false |
| <code>hasMatch("2.4.6.8", re`(?<age:IpAddress>\d.\d.\d.\d)`)</code> | true |
| <code>hasMatch("a.b.c.d", re`(?<age:IpAddress>\w+)`)</code> | false |

See also

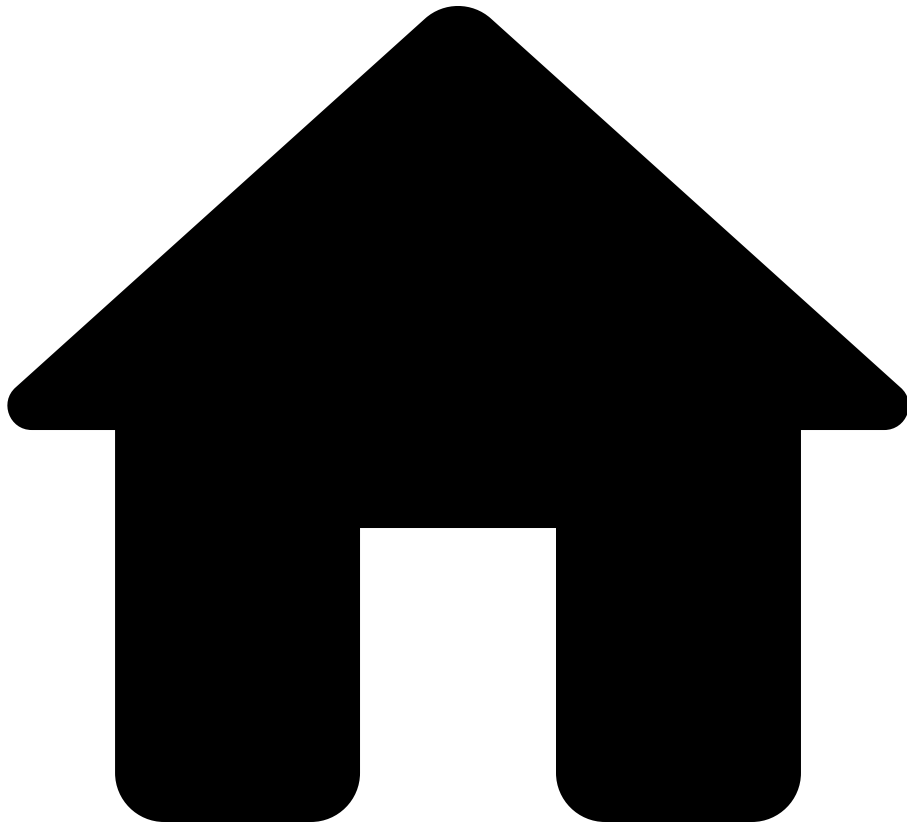
Functions

- [match](#): Gets a record of information about text that matches the structure specified by a regex, or otherwise `null`.
- [regexMatches](#): Gets the list of all substring matches of a given regex.

Types [☒☒](#)

See [the regular expression guide](#).

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- hours

On this page

hours

`hours(value: Integer) : Duration`

Obtains a `Duration` representing the given number of hours. For example, `hours(3)` returns a `Duration` that represents three hours. The value can be

negative. For example, `hours(-2)` returns a `Duration` that represents negative two hours.

This function will raise an error if the absolute value of the input is greater than `2562047788015215`.

See also

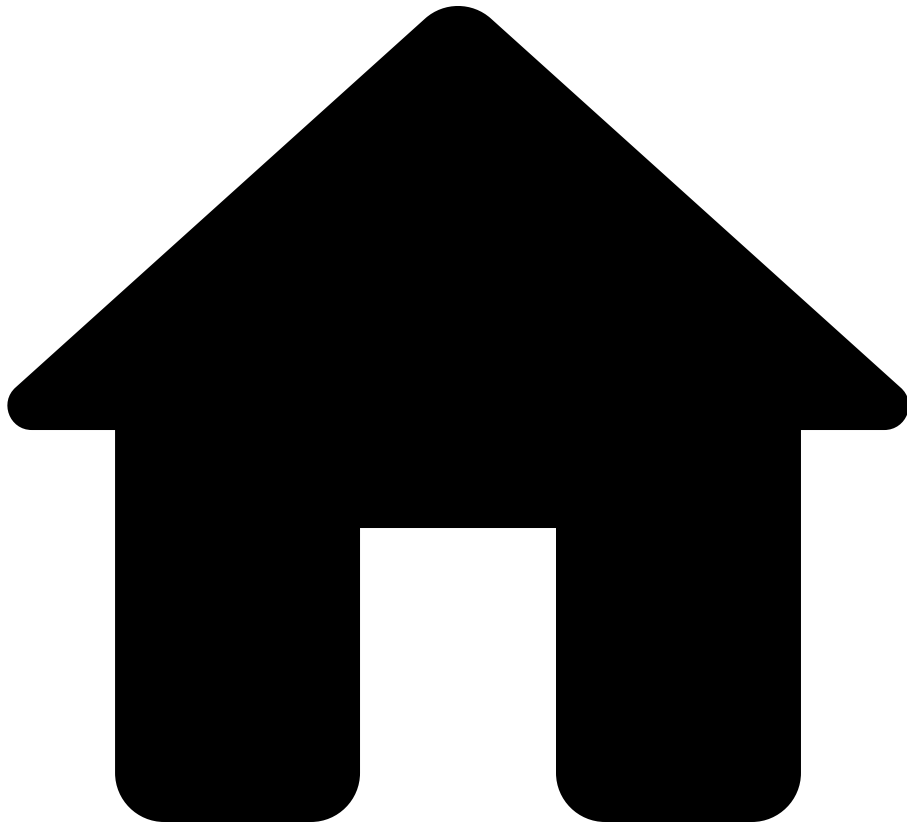
Functions

- [date](#)
- [timestamp](#)
- [duration](#)
- [days](#)
- [minutes](#)
- [seconds](#)

Types

- [Time](#)
- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- int

On this page

int

```
int(number: Float) : Integer
```

Truncates a floating-point number to obtain an integer; e.g., `int(2.9)` results in `2`.

If we have a named *float* `x`, then we can convert it to an *integer* by writing `int(x)`.

See also[ââ](#)

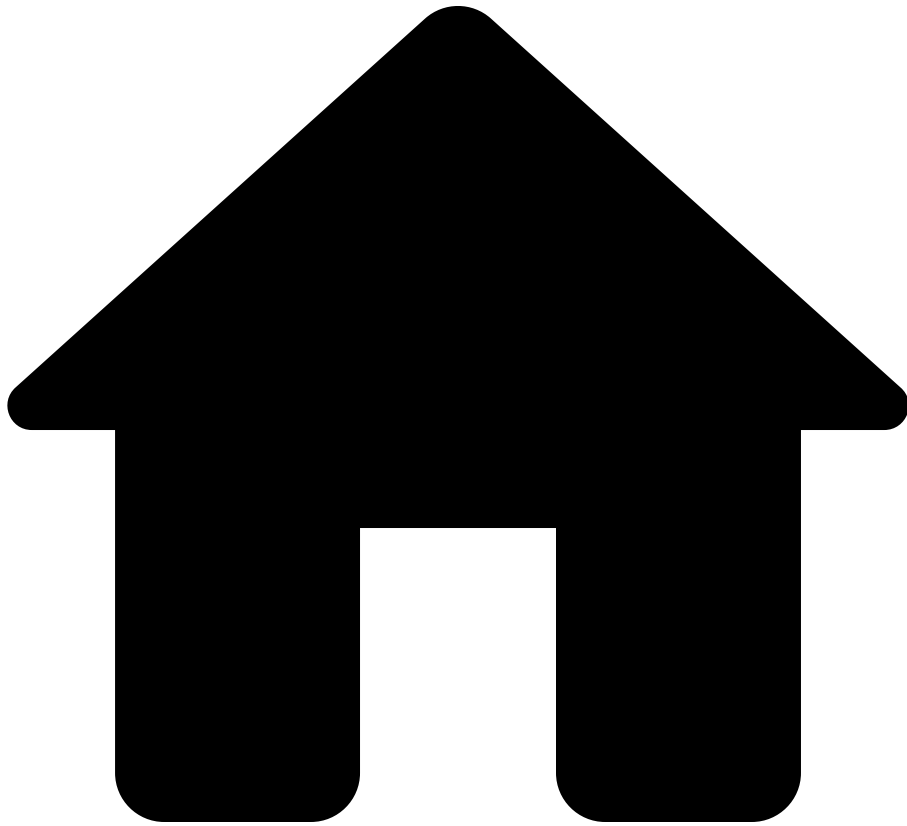
Functions[ââ](#)

- [float](#)
- [floor](#)
- [ceil](#)
- [round](#)
- [toNumber](#)

Types[ââ](#)

- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- intersect

On this page

intersect

`intersect(set1: Set<T>, set2: Set<T>) : Set<T>` for any type `T`.

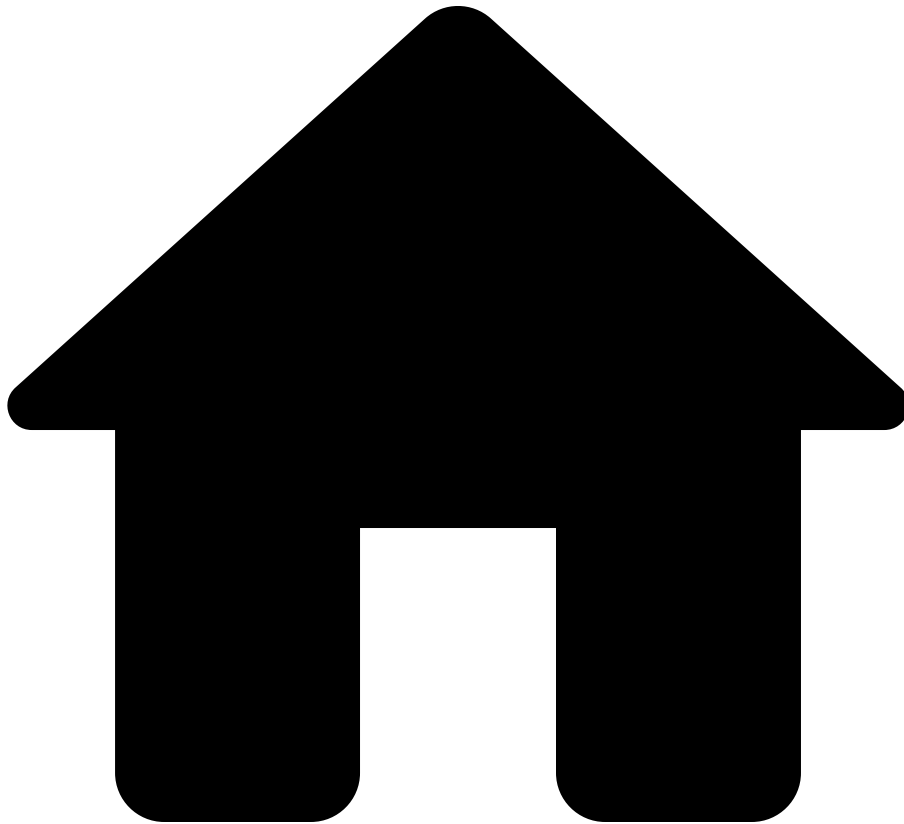
Returns the intersection of `set1` and `set2`.

See also [ââ](#)

Types [ââ](#)

- [Set](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- ipAddress

On this page

ipAddress

```
ipAddress(str: String) : IPAddress
```

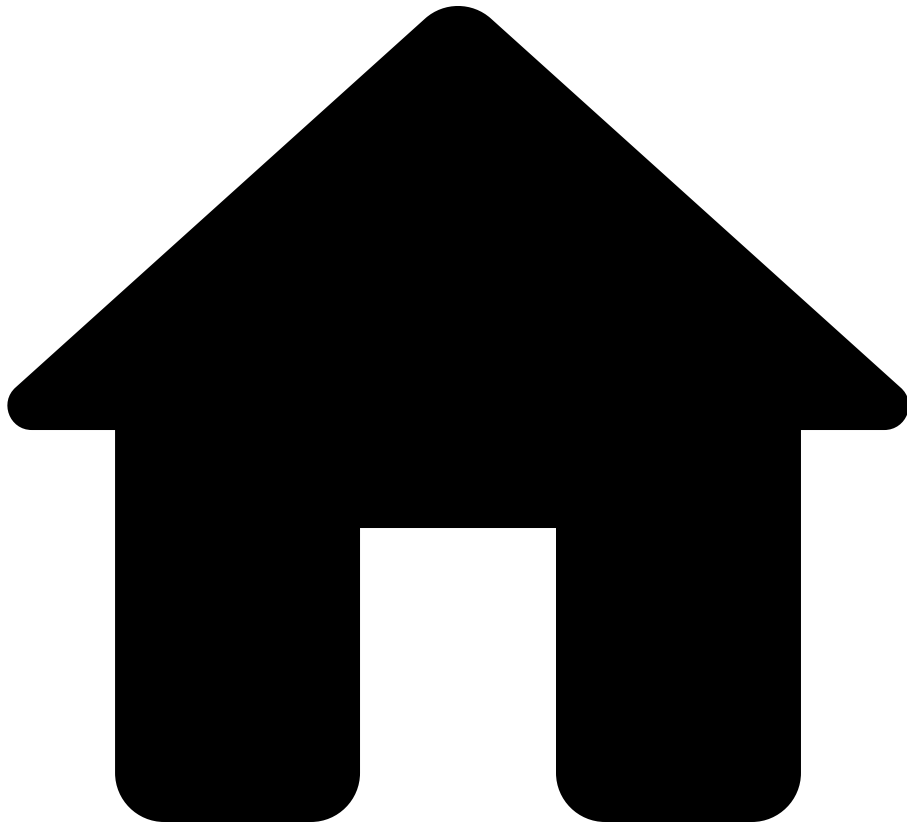
An IP address can be written like this: `ipAddress("1.2.3.4")`.

See also [a](#)

Types [a](#)

- [ipv4Address](#)
- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- ipAddressSet

On this page

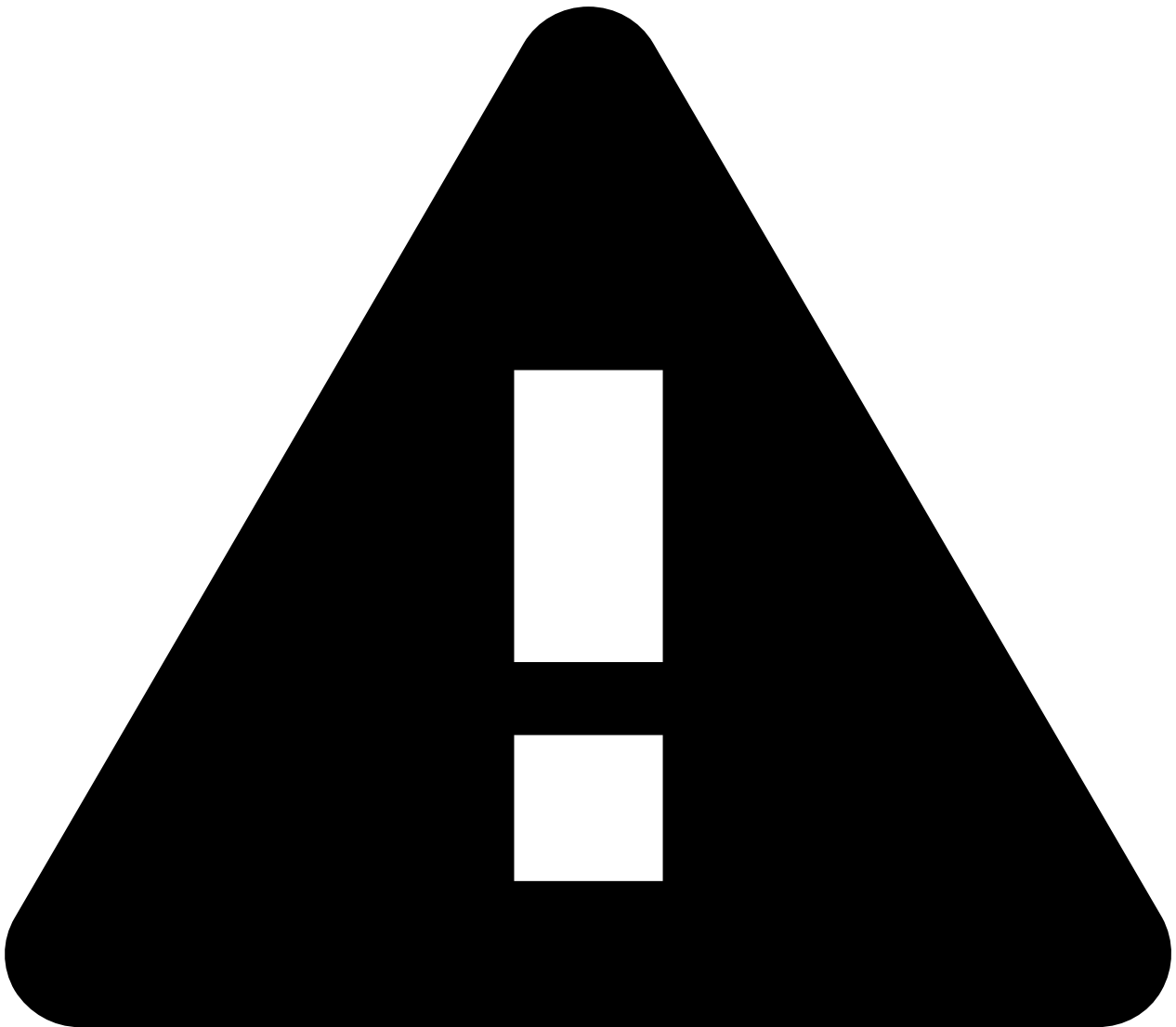
ipAddressSet

```
ipAddressSet(ipSubnets: Bag<IpSubnet>) : Set<IpAddress>
```

Returns a set of IP addresses from a collection of subnets, such as

```
ipAddressSet([ipSubnet("1.2.3.4/24"), ipSubnet("5.6.7.8/24")]) or
```

`ipAddressSet([ipSubnet("1234::/64"), ipSubnet("abcd::/64")])`. The space of IPv4 addresses is disjoint from the space of IPv6 addresses.



warning

A runtime error will occur if more than 20,000 subnets are required to express either the IPv4 addresses or the IPv6 addresses and the set is returned from a query or passed to [toString](#) or [toStringTruncate](#).

A bag of [IpSubnet](#) s is different than a set of [IpAddress](#) es. For example, if `subnets = bag([ipSubnet("1.2.3.0/32"), ipSubnet("1.2.3.1/32")])`, then

- `toString(subnets)` returns `"[1.2.3.0/32, 1.2.3.1/32]"` but

- `toString(ipAddressSet(subnets))` returns `"[1.2.3.0/31]"`.

See also[â](#)

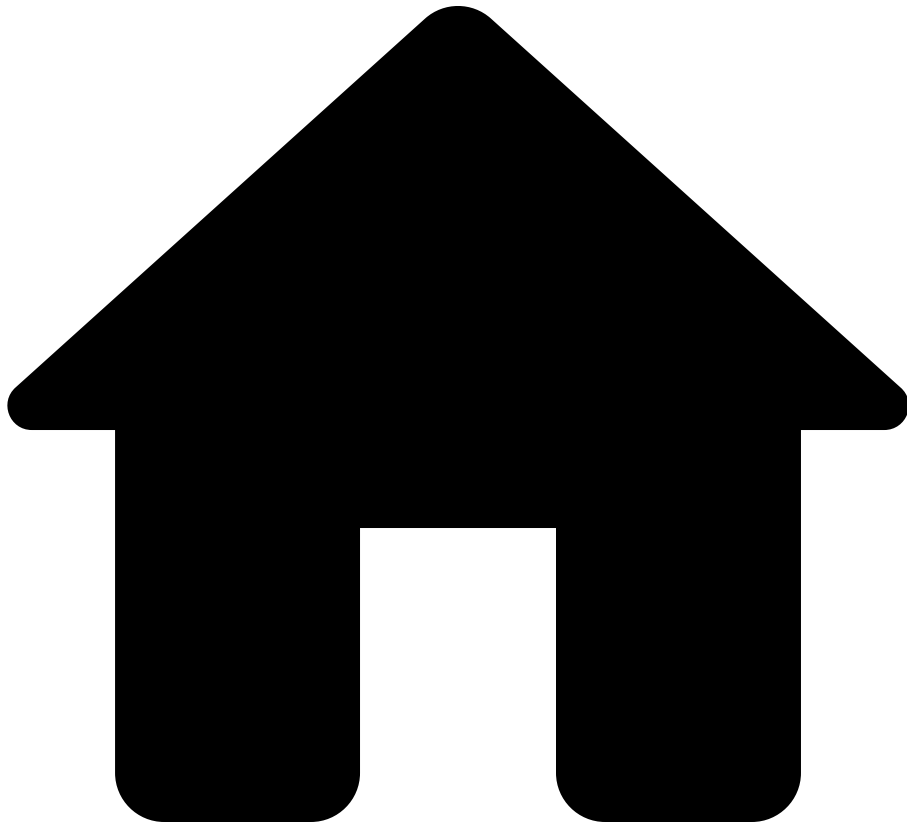
Types[â](#)

- [IpSubnet](#)
- [Collections](#)
- [Set](#)
- [IpAddress](#)

Functions[â](#)

- [ipSubnet](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- ipSubnet

On this page

ipSubnet

```
ipSubnet(str: String) : IpSubnet
```

Returns a subnet from a string, such as `ipSubnet("1.2.3.4/24")` .

```
ipSubnet(ipAddr: Ipv4Address, prefixLength: Integer): IpSubnet
```

Returns a subnet from an IP address and length, such as

```
ipSubnet(ipAddress("1.2.3.4"), 24) .
```

```
ipSubnet(ipAddr: Ipv4Address, netMask: Ipv4Address): IpSubnet
```

Returns a subnet from an IP address and a net mask represented as an IP address, such as `ipSubnet(ipAddress("1.2.3.4"), ipAddress("255.255.128.0"))`, which is equivalent to `ipSubnet("1.2.3.4/17")`. This function will raise an error if the net mask parameter (second parameter) is not a valid subnet mask (an IP address that consists of some number of leading 1 bits followed by all 0 bits).

See also

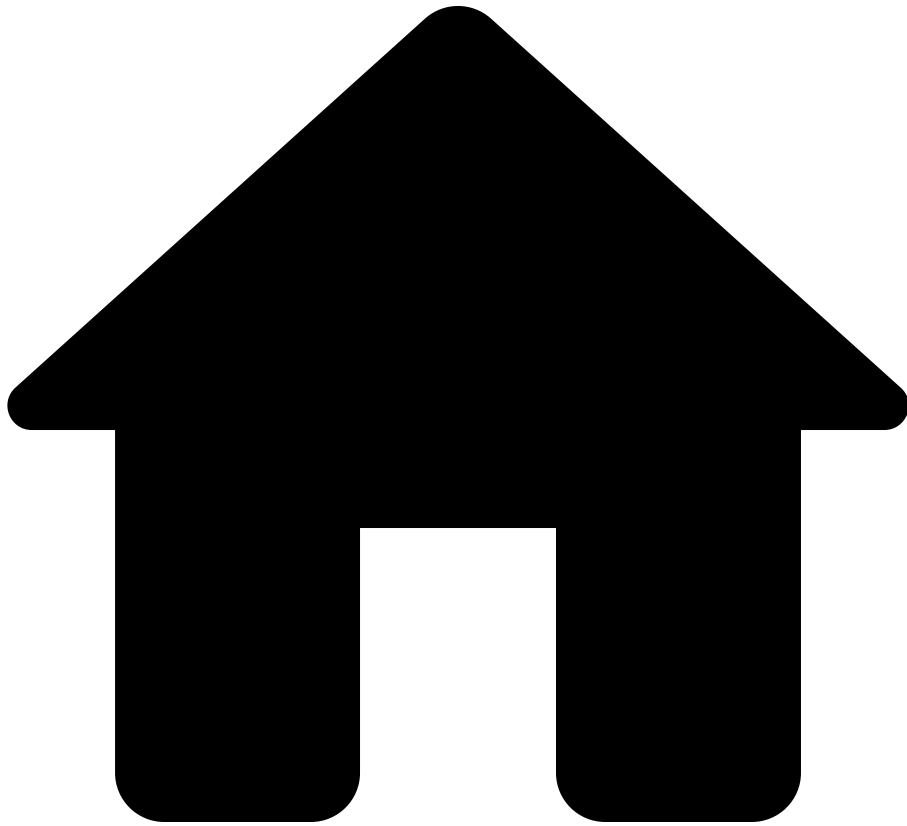
Types

- [String](#)
- [IpSubnet](#)

Functions

- [ipAddress](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- ipv4Address

On this page

ipv4Address

```
ipv4Address(num: Integer) : IPAddress
```

Constructs an IPv4 address from an integer in the range `0` to `(2 ^ 32) - 1`. For example,

1. `ipv4Address(0)` denotes `0.0.0.0`
2. `ipv4Address(4)` denotes `0.0.0.4`
3. `ipv4Address(1234)` denotes `0.0.4.210`
4. `ipv4Address(123456789)` denotes `7.91.205.21`
5. `ipv4Address((2 ^ 32) - 1)` denotes `255.255.255.255`

For numbers outside the range `0..2^32-1`, `ipv4Address` throws an error.

Whereas `ipv4Address` constructs IP addresses from *integers*, there is the [ipAddress](#) method to construct IP addresses from *strings*. Moreover, we can construct integers from IP addresses using [toNumber](#).

Properties

The [ipv4Address](#) function and the [toNumber](#) are inverses: `toNumber(ip) == num` is true exactly when `ip == ipv4Address(num)` is true, for `ip` an IPv4 address and `num` an integer in `0..2^32-1`. For example:

- `ipv4Address(toNumber(ipAddress("1.2.3.4"))) == ipAddress("1.2.3.4")`
- `toNumber(ipv4Address(1234)) == 1234`

See also

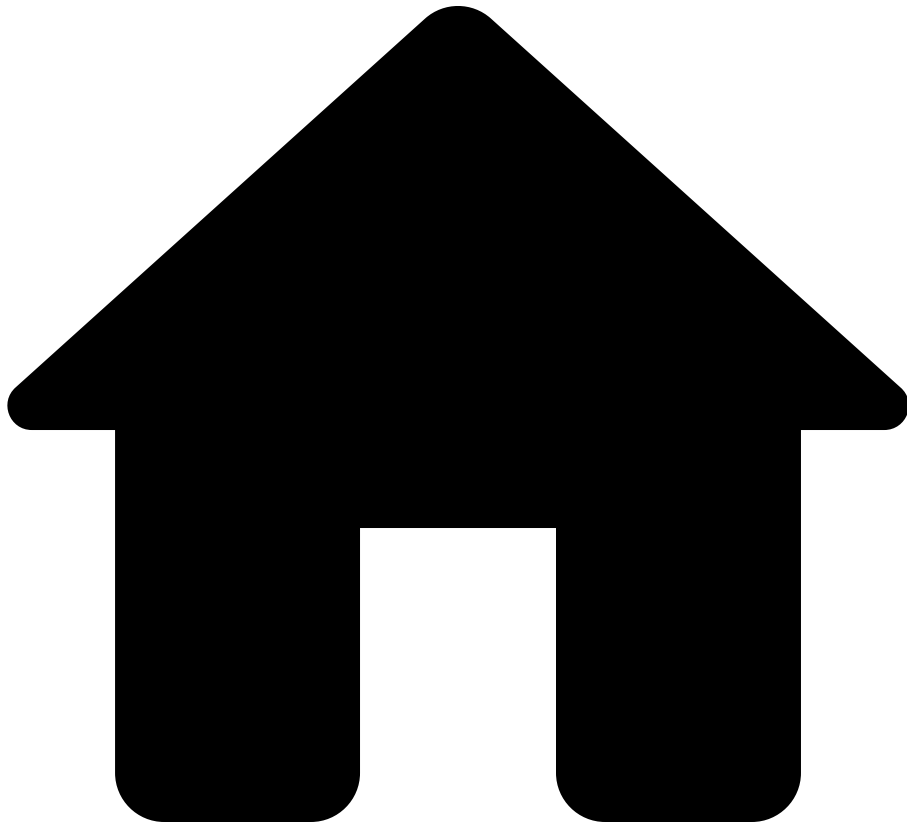
Types

- [ipAddress](#)
- [String](#)

Functions

- [toNumber](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- isEmpty

On this page

isEmpty

```
isEmpty(set: Set<IpAddress>) : Bool
```

Returns a `Bool` indicating if `set` contains no elements.

For example,

- `isEmpty(ipAddressSet([ipSubnet("1.2.3.4/32")]))` returns `false` while
- `isEmpty(intersect(ipAddressSet([ipSubnet("1.2.3.4/32")]), ipAddressSet([ipSubnet("5.6.7.8/32")])))` returns `true`.

`isEmpty(str: String) : Bool`

Returns a `Bool` indicating if `str` has no characters. For example, `isEmpty("")` returns `true` while `isEmpty("hello")` returns `false`.

`isEmpty(collection: Bag<T>) : Bool` for any type `T`.

Returns a `Bool` indicating if a collection contains no elements.

- Note that this method accepts both `Lists` and `Bags`, since [Lists subtype Bags](#).

For example, `isEmpty([1, 2, 3])` returns `false` while `isEmpty(foreach x in fromTo(1, 0) select x)` returns `true`.

See also

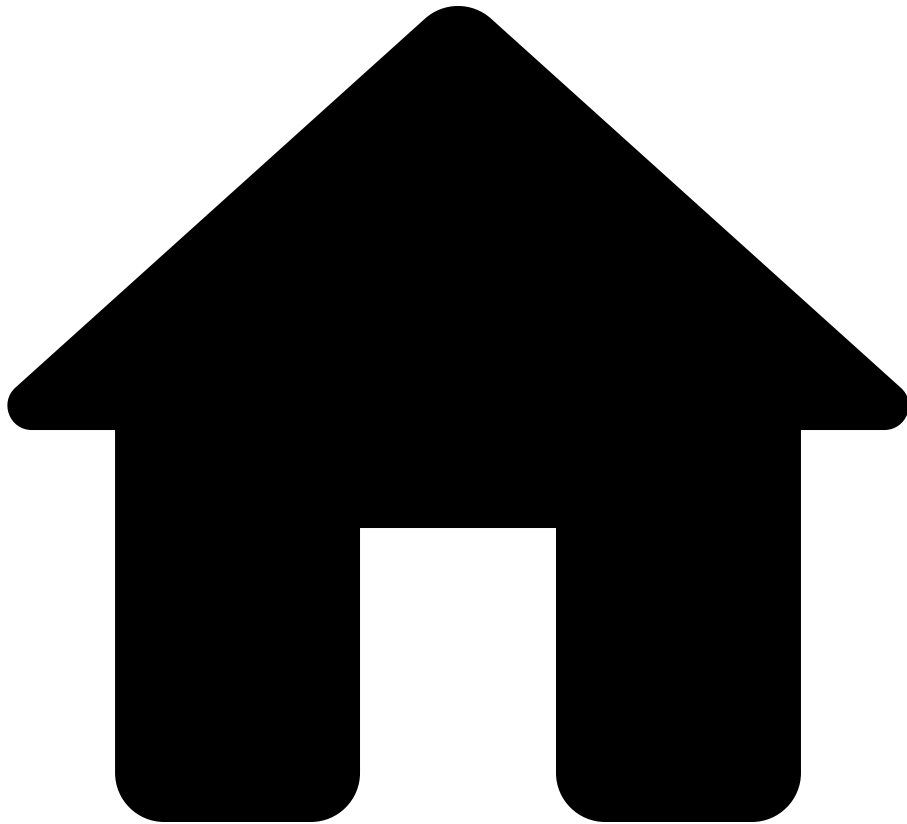
Types

- [Set](#)
- [Collections](#)
- [String](#)
- [Bool](#)

Functions

- [ipAddressSet](#)
- [ipSubnet](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- isIPv4

On this page

isIPv4

```
isIPv4(address: IPAddress) : Bool
```

Tests whether the specified `IPAddress` is an IPv4 address. For example,

```
isIPv4(ipAddress("1.2.3.4")) returns true and
```



```
isIPv4(ipAddress("2001:0db8:85a3:0000:0000:8a2e:0370:7334"))  
returns false .
```

This function will raise an error if `address` is absent.

```
isIPv4(subnet: IpSubnet) : Bool
```

Tests whether the specified `IpSubnet` is an IPv4 subnet. For example,

```
isIPv4(ipSubnet("192.168.1.0/24")) returns true and  
isIPv4(ipSubnet("2001:0db8:abcd:0000::/56")) returns false .
```

This function will raise an error if `subnet` is absent.

An example usage would be the following query that gets all used IPv4 VPC subnets:

```
getVpcIPv4Subnets =  
  foreach cloudAccount in network.cloudAccounts  
  foreach vpc in cloudAccount  
  foreach subnet in vpc.subnets  
  foreach address in subnet.addresses  
  where isIPv4(address)  
  select address;
```

See also[â](#)

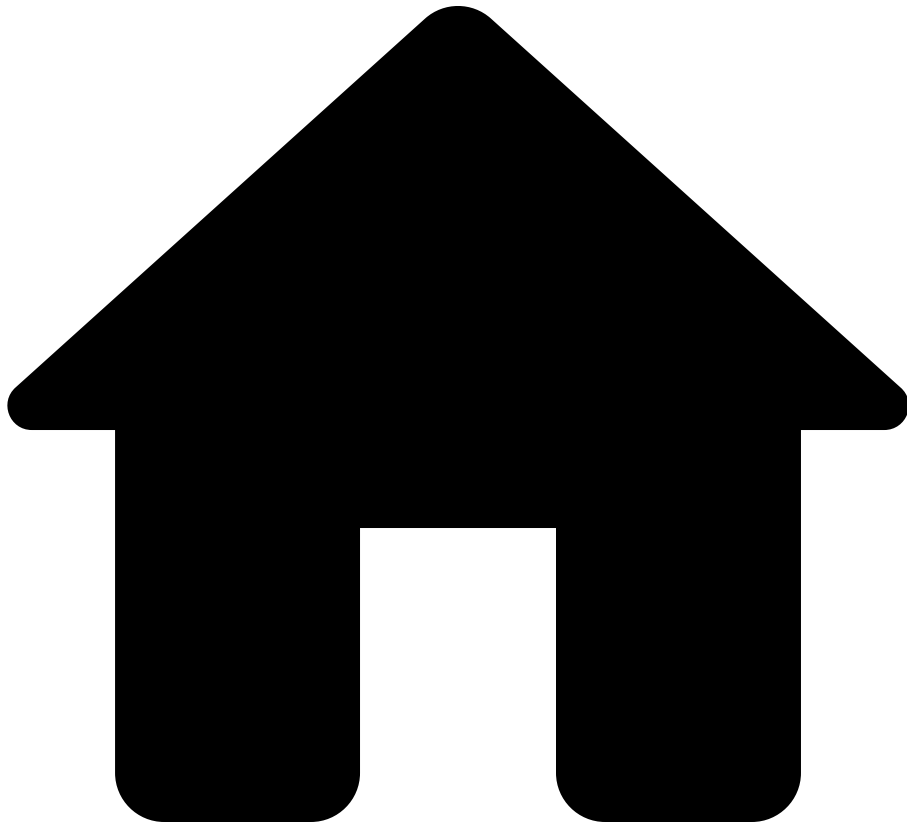
Types[â](#)

- [IpAddress](#)
- [IpSubnet](#)
- [Bool](#)

Functions[â](#)

- [ipAddress](#)
- [ipSubnet](#)
- [isIPv6](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- isIPv6

On this page

isIPv6

```
isIPv6(address: IPAddress) : Bool
```

Tests whether the specified `IPAddress` is an IPv6 address. For example,

```
isIPv6(ipAddress("2001:0db8:85a3:0000:0000:8a2e:0370:7334"))
```

returns `true` and `isIPv6(ipAddress("1.2.3.4"))` returns `false`.

This function will raise an error if `address` is absent.

```
isIPv6(subnet: IpSubnet) : Bool
```

Tests whether the specified `IpSubnet` is an IPv6 subnet. For example, `isIPv6(ipSubnet("2001:0db8:abcd:0000::/56"))` returns `true` and `isIPv6(ipSubnet("192.168.1.0/24"))` returns `false`.

This function will raise an error if `subnet` is absent.

An example usage would be the following query that gets all used IPv6 VPC subnets:

```
getVpcIPv6Subnets =  
  foreach cloudAccount in network.cloudAccounts  
  foreach vpc in cloudAccount  
  foreach subnet in vpc.subnets  
  foreach address in subnet.addresses  
  where isIPv6(address)  
  select address;
```

See also

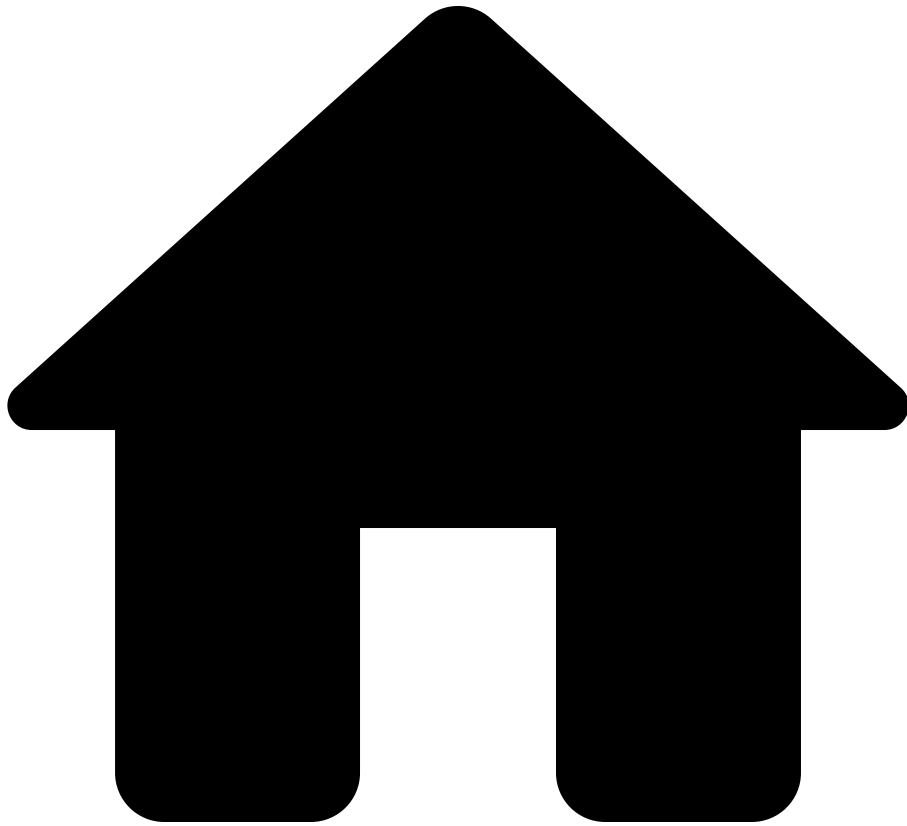
Types

- [IpAddress](#)
- [IpSubnet](#)
- [Bool](#)

Functions

- [ipAddress](#)
- [ipSubnet](#)
- [isIPv4](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- isPresent

On this page

isPresent

`isPresent(value: T) : Bool` for any type `T`.

Returns `true` if a value is non-`null` and `false` otherwise. (Values like `[]`, `""`, `0` and `false` are not equal to `null` and return `true`.)

Examples:

- `isPresent("")` returns `true`
- `isPresent(0)` returns `true`
- `isPresent(null: Integer)` returns `false`
- `foreach l in network.locations select {x: isPresent(l.city)}`
returns `true` for each city in `network.locations` that has a value, and
`false` if the city is undefined.

See also

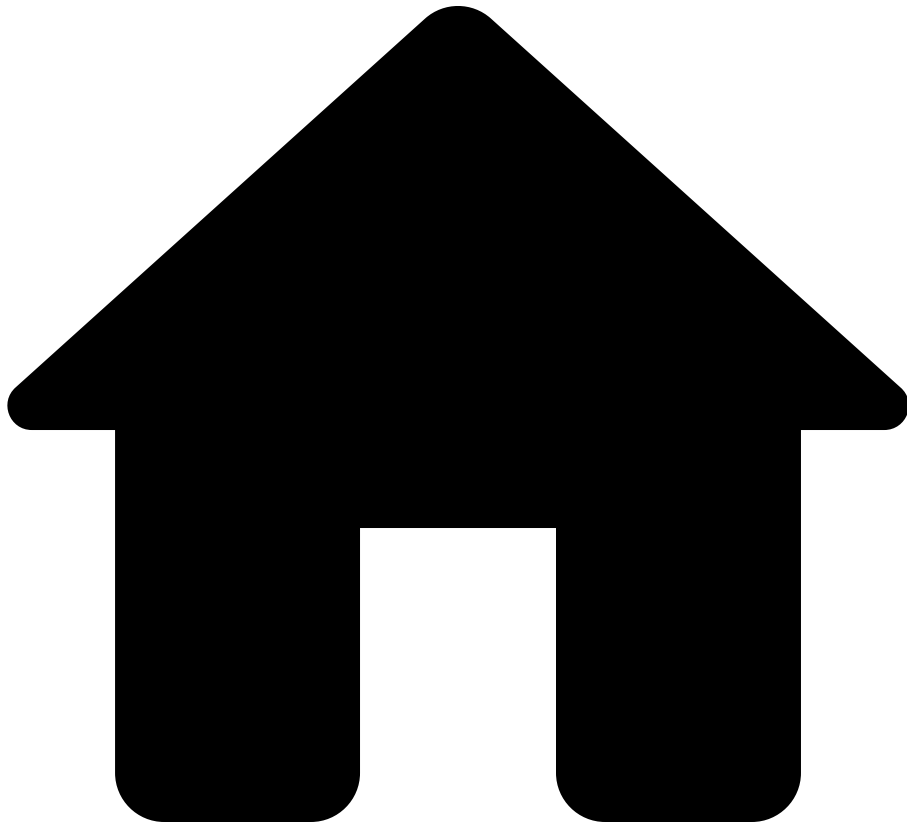
Functions

- [isEmpty](#)

Data Model

- [Location](#)

.



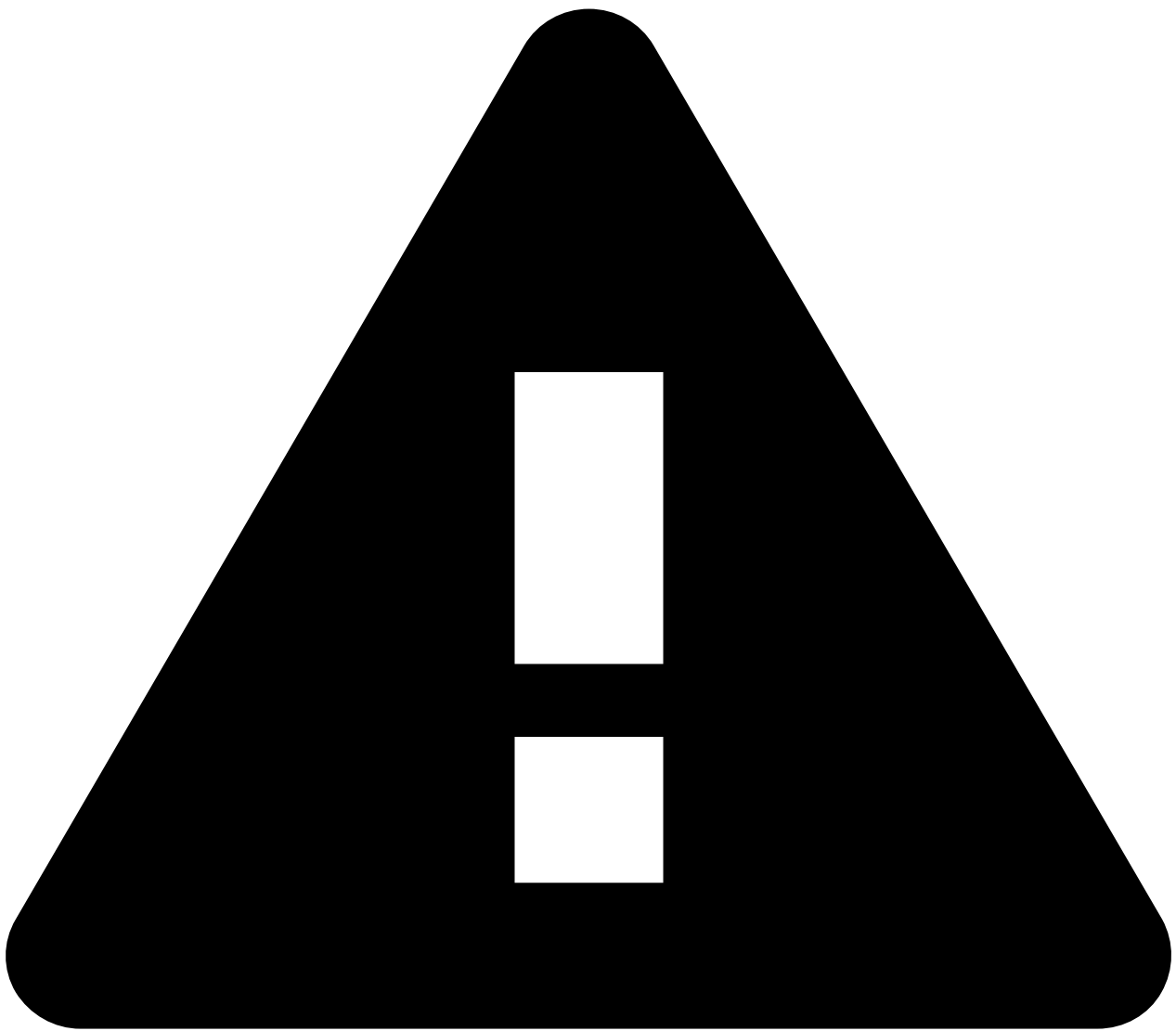
- Reference
- [NQE Language](#)
- [Standard Library](#)
- join

On this page

join

```
join(delimiter: String, list: List<String>) : String
```

Returns the result of concatenating together the elements in `list` of type `String` using `delimiter` between each element.



Deprecated Bag overload

As a transitional measure for backwards compatibility, `join` also accepts a `Bag<String>` as its second argument. Because Bags have non-deterministic ordering, the order of joined elements may be unpredictable. Convert the Bag to a List using `orderBy()` or `order()` to get deterministic results. In due time, the Bag overload will be removed and a type-checking error will be emitted instead.

Examples

```
join("", ["a", "b", "c"]) == "abc"
join(" ", ["a", "b", "c"]) == "a b c"
```

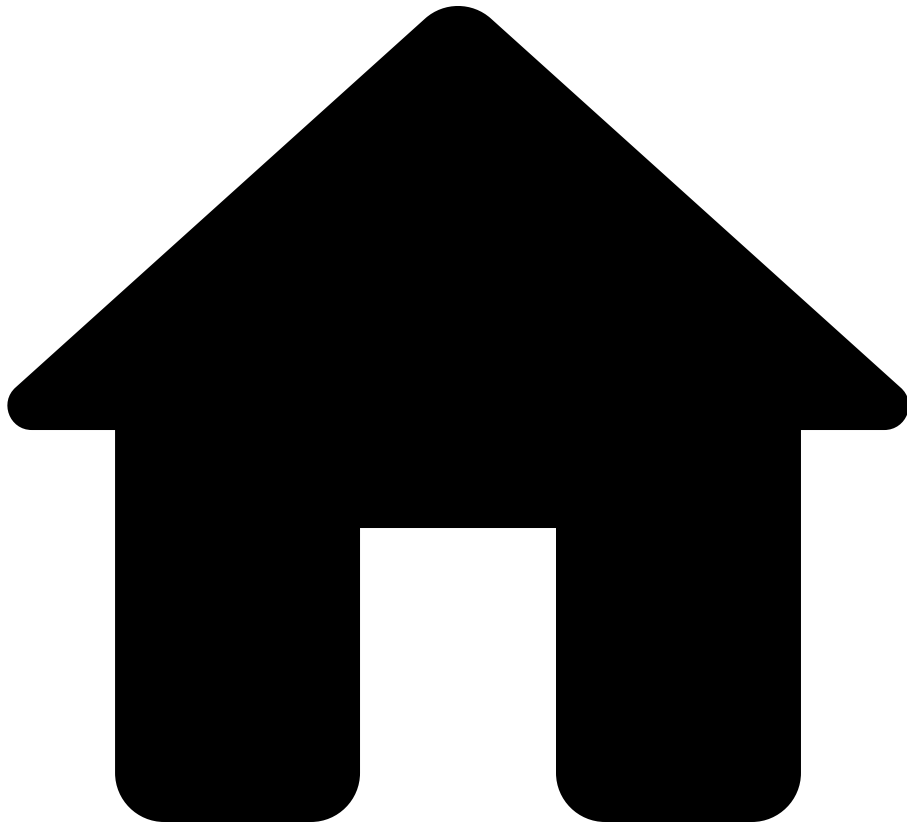
```
join(", ", ["a", "b", "c"])      == "a, b, c"  
join(" and ", ["a", "b", "c"])  == "a and b and c"
```

See also[â](#)

Types[â](#)

- [String](#)
- [Collections](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- length

On this page

length

For any type `T`,

```
length(collection: Bag<T>) : Integer
```

Returns the length of a collection. For example, `length(["a", "b"])` is `2`.

- Note that this method accepts both `Lists` and `Bags` —since [Lists subtype Bags](#)

```
length(str: String): Integer
```

Returns the number of characters in a string. For example, `length("abc")` is `3`.

```
length(subnet: IpSubnet): Integer
```

Returns the length of the subnet. For example,

```
length(ipSubnet("1.2.3.4/24")) is 24 .
```

See also

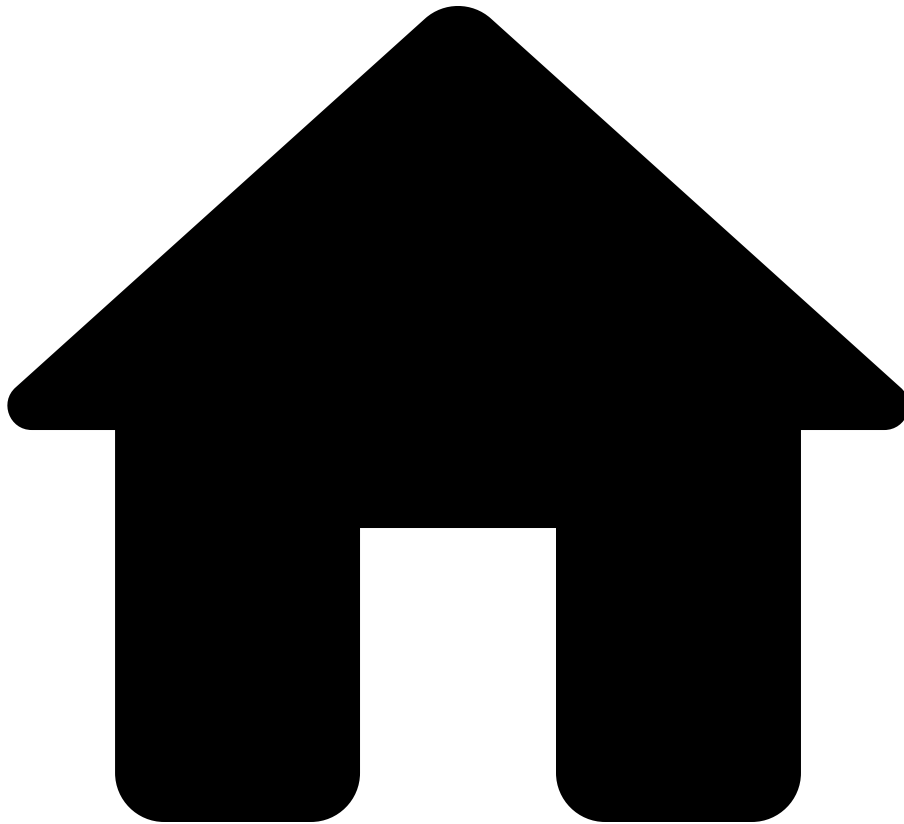
Types

- [Collections](#)
- [Integer](#)
- [String](#)
- [IpSubnet](#)

Functions

- [ipSubnet](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- limit

On this page

limit

```
limit(list: List<T>, n: Number) : List<T>
```

Gets the sublist of `list` consisting of the first `n` elements of `list`.

- The maximum number of elements to return, `n`, must be non-negative.

- If `n` is greater than the collection size, returns all elements.
- If `n` is 0, returns an empty collection

Examples

Basic Usage

```
numbers      = [1, 2, 3, 4, 5];  
firstThree   = limit(numbers, 3); // [1, 2, 3]
```

With order by Clause

```
limit(  
  foreach device in network.devices  
  select { Name: device.name }  
  order by Name desc,  
  10  
)
```

Using Comprehension Syntax

The `limit` clause in a comprehension provides a more readable alternative:

```
foreach device in network.devices  
select { Name: device.name }  
order by Name desc  
limit 10
```

See Also

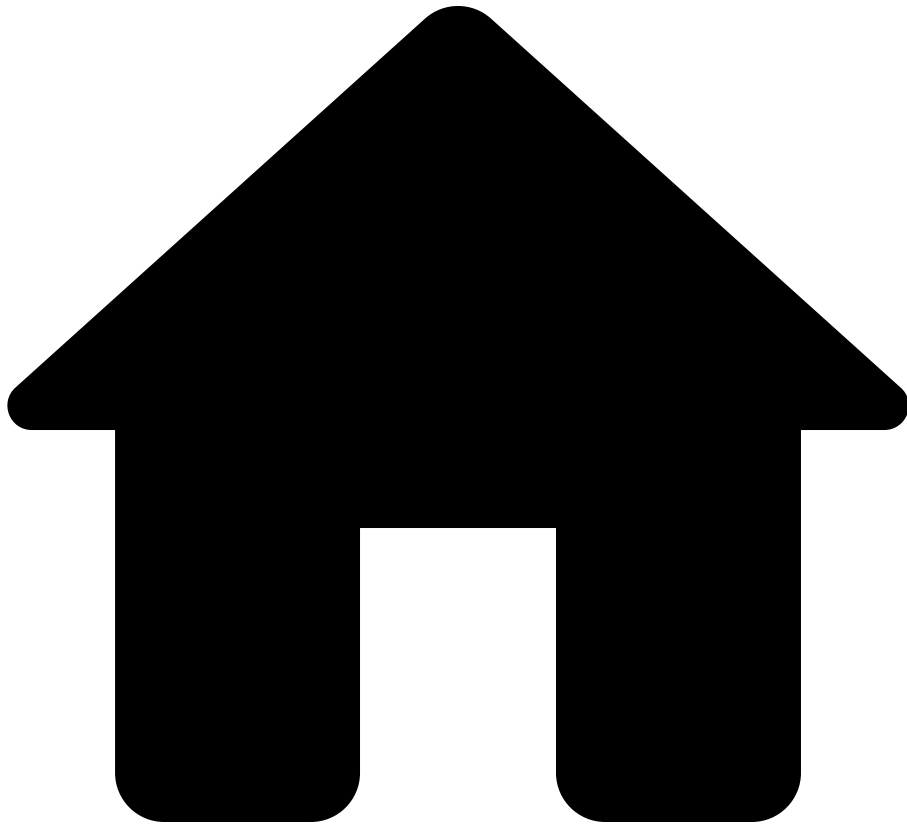
Guides

- [Collections](#) - Comprehensive guide on Lists, Bags, and ordering
- [Comprehensions](#) - Using limit clause

Functions [↗](#)

- [orderBy](#) - Sort a collection by a key function
- [length](#) - Get the length of a collection

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- `macAddress`

On this page

macAddress

```
macAddress(str: String) : MacAddress
```

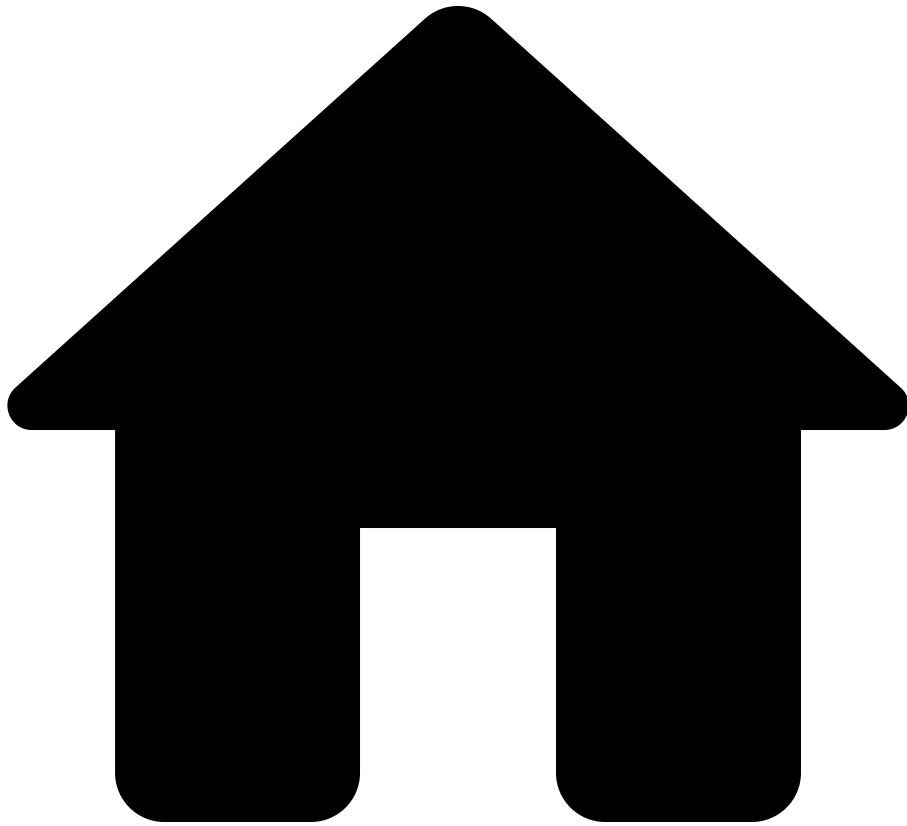
MAC addresses can be written like this: `macAddress("11:22:33:44:55:66")`.

See also [ââ](#)

Types [ââ](#)

- [MacAddress](#)
- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- match

On this page

match

```
match(text: String, regex: Regex<T>) : T?
```

Gets a record of captured data if `text` matches the structure specified by `regex`, or otherwise `null`.

See [the regular expression guide](#).

Examples

| Example | text | regex |
|---------|------------------------|--|
| 1 | "Jane is 32 years old" | re`Jane is 32 years old` |
| 2 | "Jane is 33 years old" | re`Jane is 32 years old` |
| 3 | "Name: Jane, Age: 32" | re`Name: (\w+), Age: (\d+)` |
| 4 | "Name: Jane, Age: 32" | re`Name: (?<name>\w+), Age: (?<age:Integer>\d+)` |
| 5 | "Jane is 32 years old" | re`(?<age:Integer>\d+) (?: (?<name>\w+))` |
| 6 | "32" | re`(?<age:Integer>\d+) (?: (?<name>\w+))` |
| 7 | "3x2" | re`(?<age:Integer>\d+) (?: (?<name>\w+))` |

Regexes with anonymous captures have match result record with indices as keys, and all values are of type `String`.

- Matches of *unnamed* capture groups are referenced with string indices, such as `"1"` and not numbers such as `1`.
- For instance, in example 3 above, the regex has type `Regex<{"1": String, "2": String}>`. As such, `match("Name: Jane, Age: 32", re`Name: (\w+), Age: (\d+)`)[ "2" ]` is how we access the second capture group, which results in the `String` value `"32"`.

Regexes with *named* captures have match result record with the declared names as keys, and all values are of declared types

- For instance, in example 4 above, the regex has type `Regex<{name: String, age: Integer}>`, and so `match("Name: Jane, Age: 32", re`Name: (?<name>\w+), Age: (?<age:Integer>\d+)`).age` is how we access the capture group *named* `age`, resulting in the `Integer` value `32`.
- Moreover, notice that the capture group `name` has no declared type, so its type defaults to `String`.

See also [â](#)

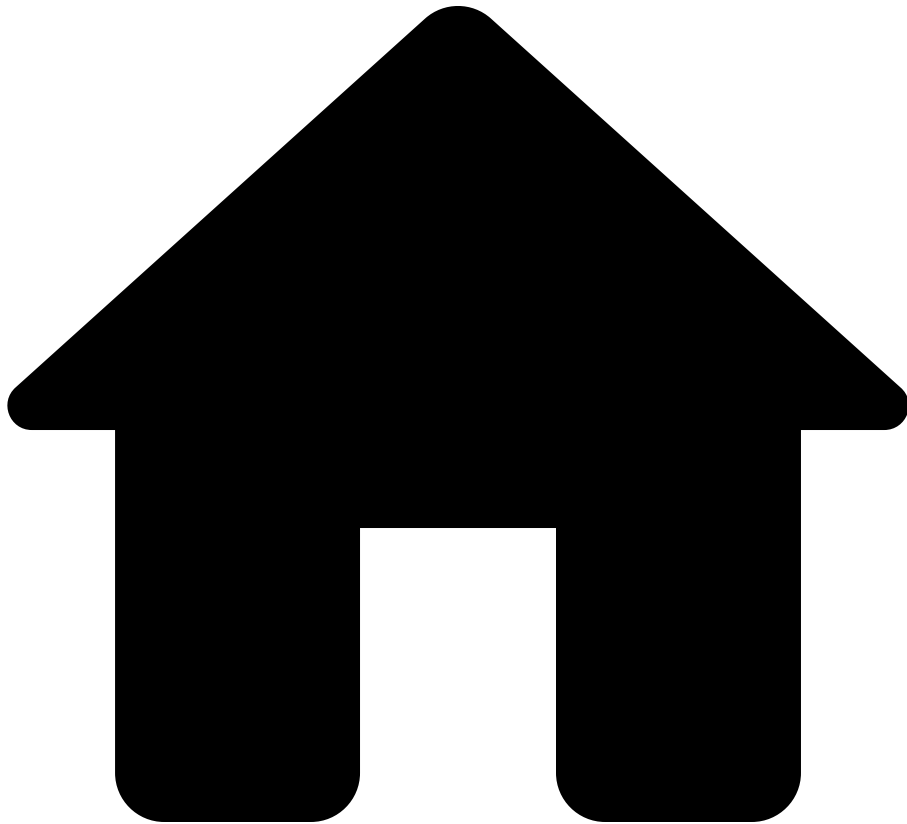
Functions [â](#)

- [hasMatch](#): Checks whether a string matches the structure specified by a regex or not.
- [regexMatches](#): Gets the list of all substring matches of a given regex.

Types [â](#)

See [the regular expression guide](#).

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- matches

On this page

matches

```
matches(str: String, glob: String) : Bool
```

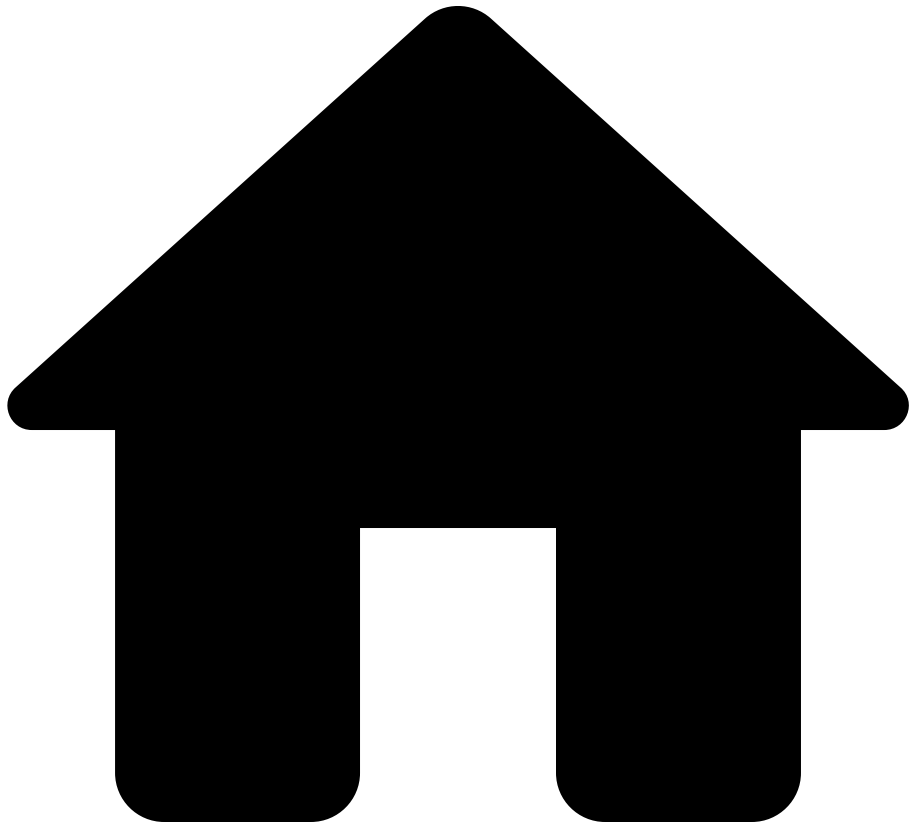
Returns `true` if and only if `str` matches `glob`, case-sensitively. For example, `matches("def", "def")` and `matches("abcdefefg", "*def*")` both return `true`.

See also [a](#)

Types [a](#)

- [String](#)
- [Bool](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- max

On this page

max

For any type `T`,

```
max(collection: Bag<T>) : T?
```

Returns the greatest item in the `collection`, or `null` if it is empty.

- The greatest item in a collection is selected with the same order used for comparing items with relational expressions such as less-than (`<`) and greater-than (`>`).
- See how [comparisons](#) work for more detail on the order of values.
- Note that this method accepts both `Lists` and `Bags` since [Lists subtype Bags](#)

See also[ââ](#)

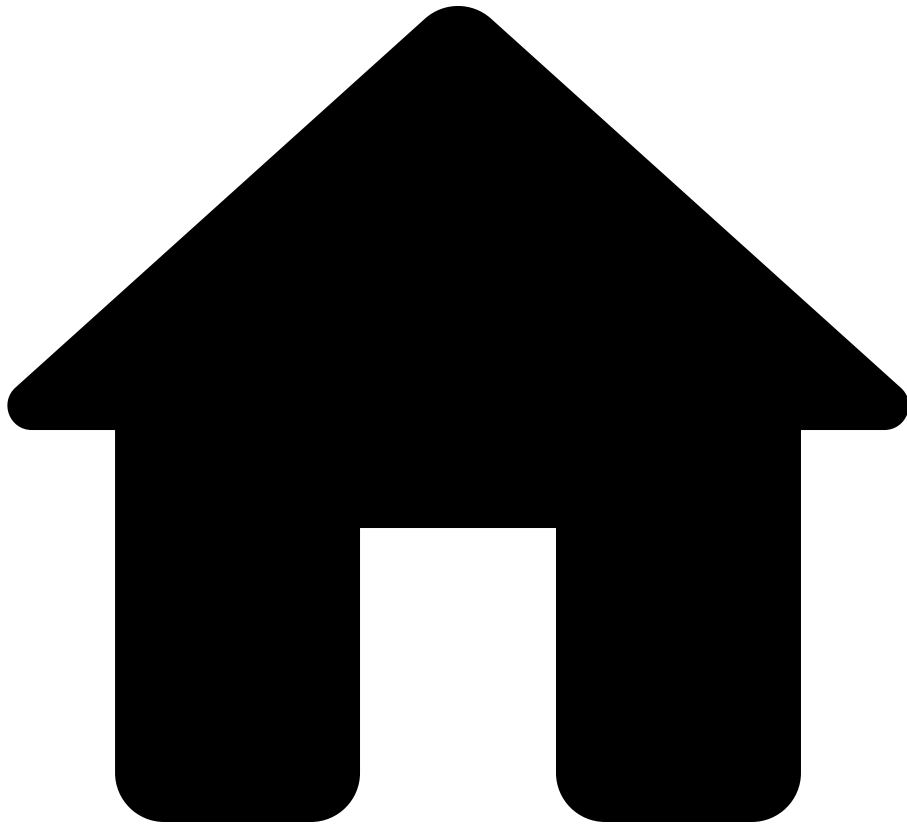
Types[ââ](#)

- [Collections](#)

Functions[ââ](#)

- [maxBy](#)
- [min](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- `maxBy`

On this page

maxBy

For any types `T` and `K`,

```
maxBy(collection: Bag<T>, keySelector: T -> K) : T?
```

Gets the greatest item in the `collection`, where items are ordered by the value returned by applying `keySelector` to the item.

- For example, if the following function has been declared `getRoutePrefix(route) = route.prefix;`, and if `routes` is a variable referring to some list of `IpEntry` records, then `maxBy(routes, getRoutePrefix)` evaluates to the route with the largest prefix (aka subnet). See how [comparisons](#) work for more on how subnets are ordered.
- Similar to `max` and `min`, this function returns `null` if there are no items in the collection.
- Note that this method accepts both `Lists` and `Bags` —since [Lists subtype Bags](#)

See also[â](#)

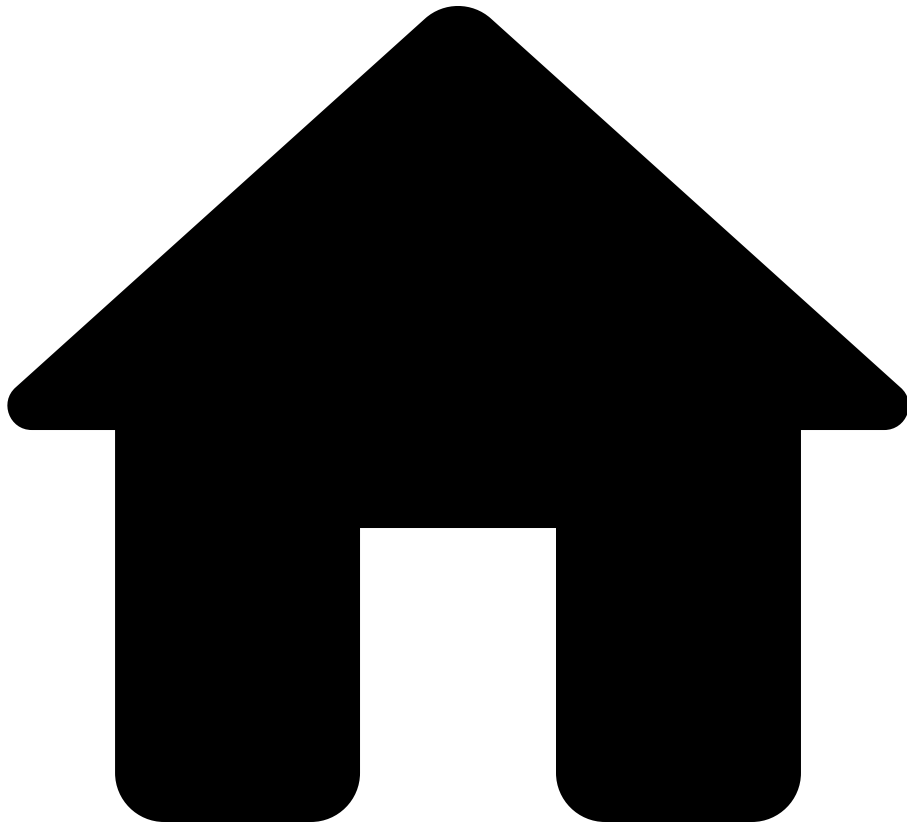
Types[â](#)

- [Collections](#)

Functions[â](#)

- [max](#)
- [minBy](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- mergeBlocks

mergeBlocks

```
mergeBlocks(blockPattern: PatternBlocks<{}>) : PatternBlocks<{}>
```

`mergeBlocks(blockPattern)` combines the patterns and sub-patterns in `blockPattern` based on common roots. This can be useful if the pattern contains multiple sub-patterns that each specify part of the desired block properties. For example, consider the following `pattern1` :

```

pattern1 = ```
interface GigabitEthernet1/0/1
  switchport mode access
  hsrp 30
    preempt
router bgp
interface GigabitEthernet1/0/1
  description Important port
  hsrp 30
    priority 50
```;

```

The value of `mergeBlocks(pattern1)` will be the following pattern, `pattern2`, which has all sub-blocks of the interface combined into one block:

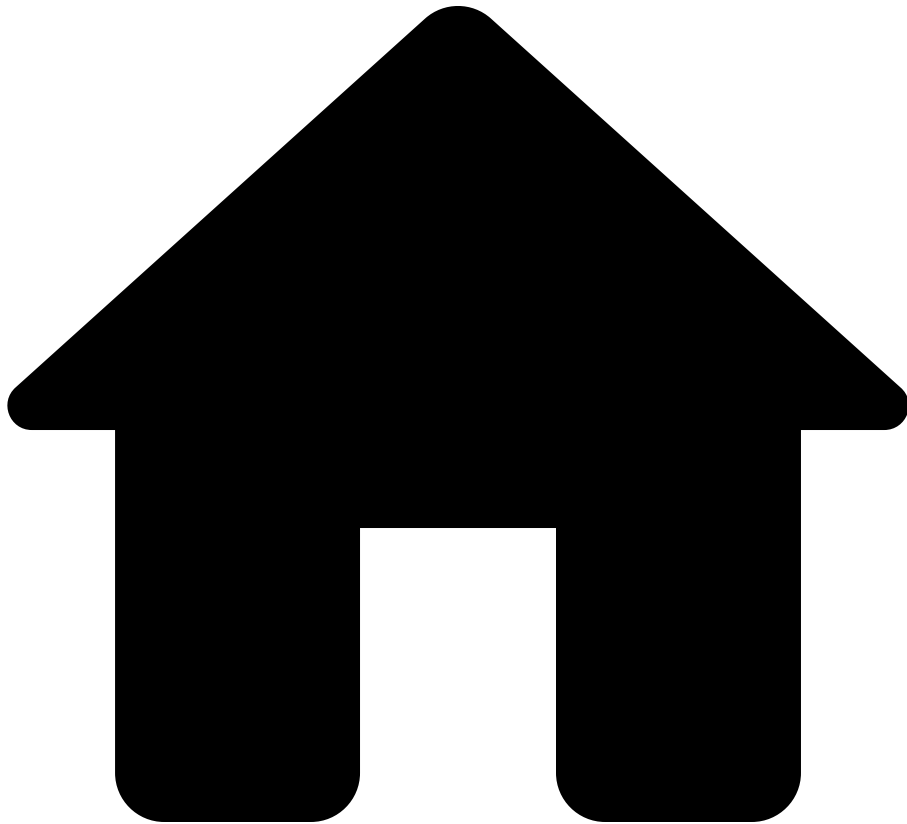
```

pattern2 = ```
interface GigabitEthernet1/0/1
 switchport mode access
 hsrp 30
 preempt
 priority 50
 description Important port
router bgp
```;

```

Note that `mergeBlocks(pattern)` works at all nesting levels of a block pattern. In other words, it will merge blocks not just at the root level, but also will apply block merging for the merged subtrees of a block. This can be seen in the above example, where the multiple `hsrp 30` have been merged together into a single block.

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- min

On this page

min

For any type `T`,

```
min(collection: Bag<T>) : T?
```

Gets the least item in the `collection`, or `null` if there are no items in the collection.

- The least item in a collection is selected with the same order used for comparing items with relational expressions such as less-than (`<`) and greater-than (`>`).
- See how [comparisons](#) work for more detail on the order of values.
- Note that this method accepts both `Lists` and `Bags` since [Lists subtype Bags](#)

See also[â](#)

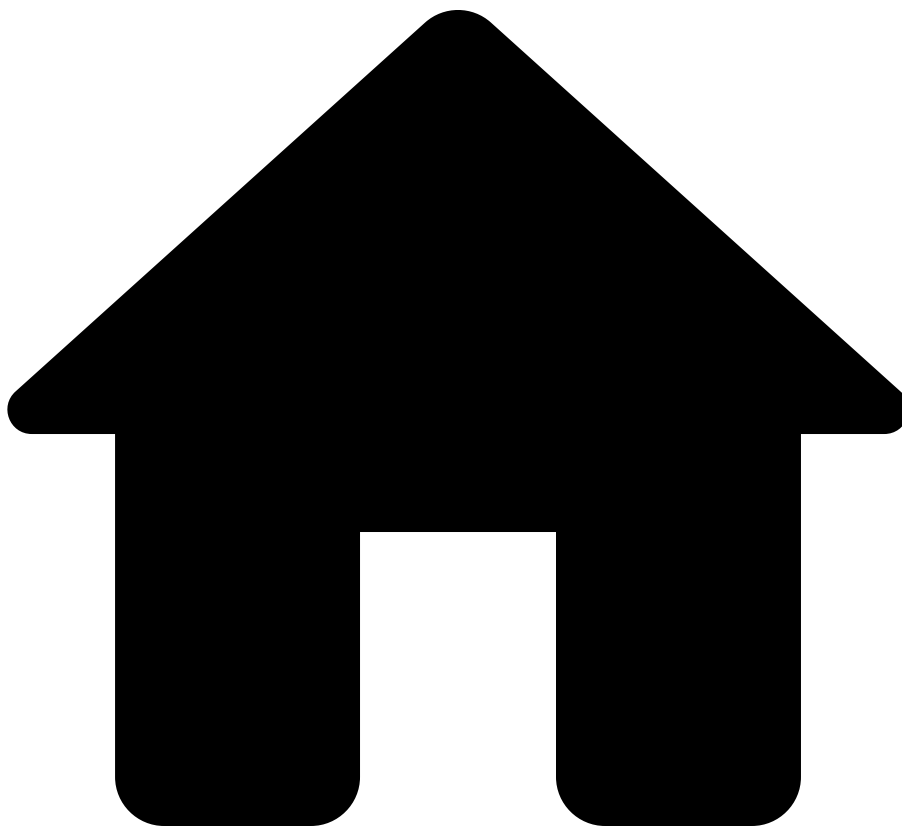
Types[â](#)

- [Collections](#)

Functions[â](#)

- [minBy](#)
- [max](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- minBy

On this page

minBy

For any types `T` and `K`,

```
minBy(collection: Bag<T>, keySelector: T -> K) : T?
```

Gets the least item in the `collection`, where items are ordered by the value returned by applying `selector` to each item.

- For example, if the following function has been declared `getRoutePrefix(route) = route.prefix;`, and if `routes` is a variable referring to some list of `IpEntry` records, then `minBy(routes, getRoutePrefix)` evaluates to the route with the smallest prefix (aka subnet). See how [comparisons](#) work for more on how subnets are ordered.
- Similar to `max` and `min`, this function returns `null` if there are no items in the collection.
- Note that this method accepts both `Lists` and `Bags` —since [Lists subtype Bags](#)

See also[â€‹](#)

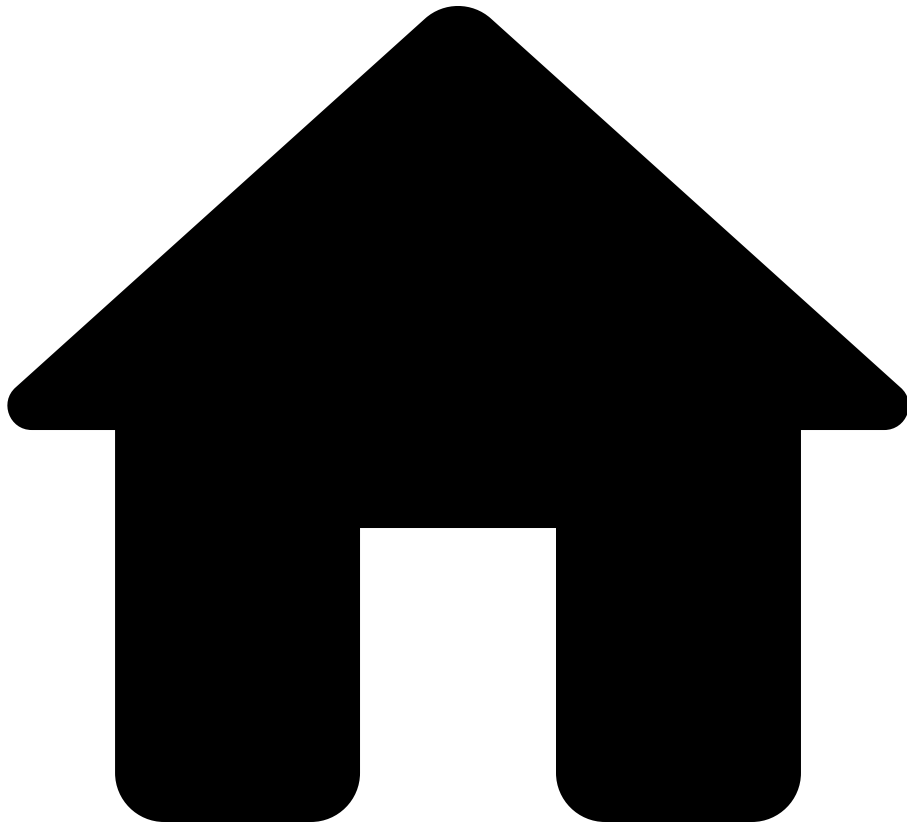
Types[â€‹](#)

- [Collections](#)

Functions[â€‹](#)

- [min](#)
- [maxBy](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- minutes

On this page

minutes

```
minutes(value: Integer) : Duration
```

Obtains a `Duration` representing the given number of minutes. For example, `minutes(3)` returns a `Duration` that represents three minutes. The value can be

negative. For example, `minutes(-2)` returns a `Duration` that represents negative two minutes.

This function will raise an error if the absolute value of the input is greater than `153722867280912930`.

See also

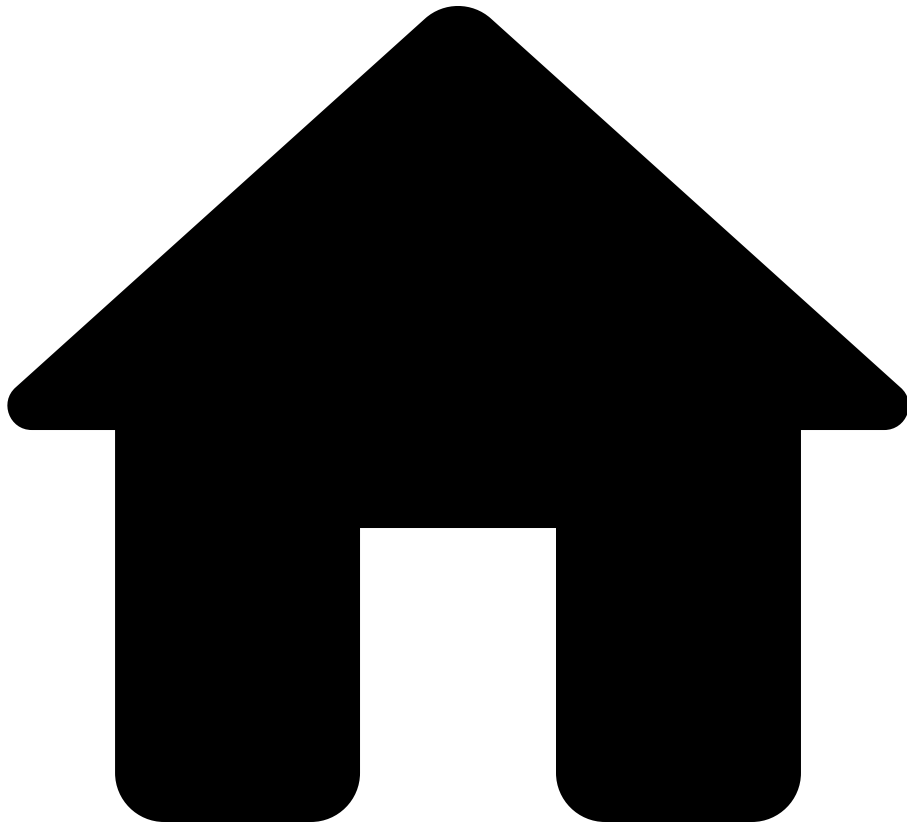
Functions

- [date](#)
- [timestamp](#)
- [duration](#)
- [days](#)
- [hours](#)
- [seconds](#)

Types

- [Time](#)
- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- `networkAddress`

On this page

networkAddress

```
networkAddress(subnet: IpSubnet) : IpAddress
```

Returns the network IP address for the subnet, that is the IP address for the subnet, with all host bits set to 0. For example, `networkAddress(ipSubnet("1.2.3.4/24"))` is `ipAddress("1.2.3.0")`.

See also[â](#)

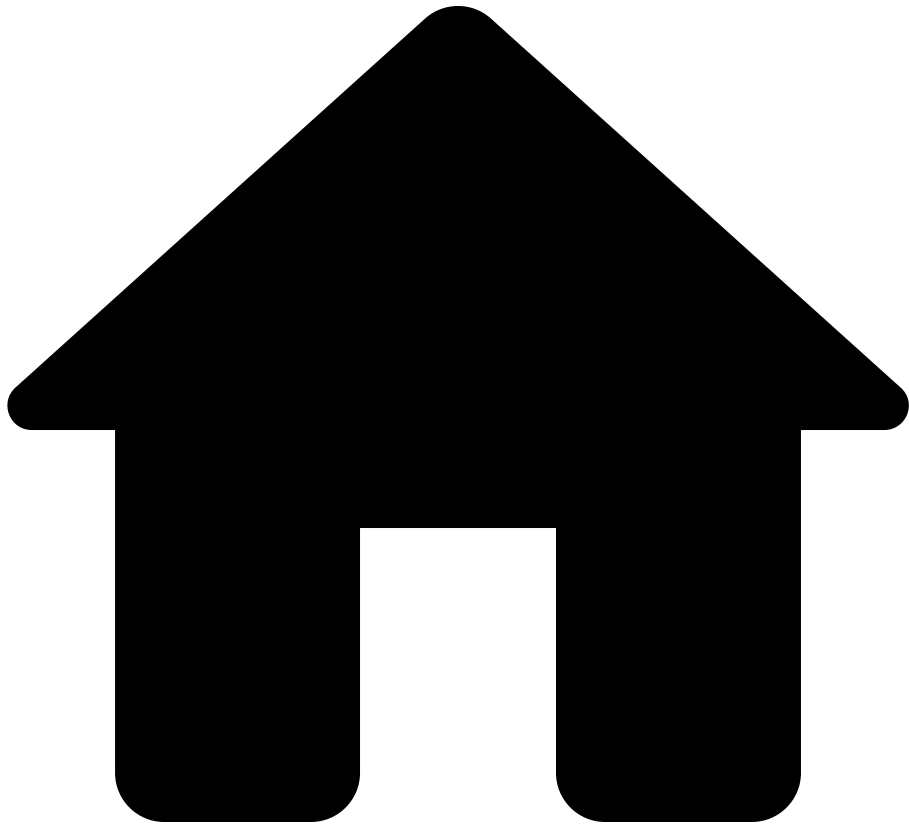
Types[â](#)

- [IpSubnet](#)
- [IpAddress](#)

Functions[â](#)

- [ipSubnet](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- order

On this page

order

For any type `T`,

```
order(collection: Bag<T>) : List<T>
```

Sorts a collection (in ascending order).

- The collection is ordered by the natural order on elements.
 - That is, the same order used for comparing items with relational expressions such as less-than (`<`) and greater-than (`>`).
 - See [comparisons](#) for more detail on the order of values.
- Note: `x <= y` if and only if `order([x, y])[0] == x` and `order([x, y])[1] == y`.
- Note that this method accepts both `Lists` and `Bags` —since [Lists subtype Bags](#)

Example

Sort a list of numbers in ascending order:

```
numbers = [3, 1, 4, 1, 5];  
order(numbers) // Returns [1, 1, 3, 4, 5]
```

This is equivalent to the more verbose, albeit more flexible, comprehension:

```
numbers = [3, 1, 4, 1, 5];  
  
foreach n in numbers  
select n  
order by n ascending
```

Notes

To sort in *descending order*, use the `order by` comprehension clause. See [collections](#) for more detail.

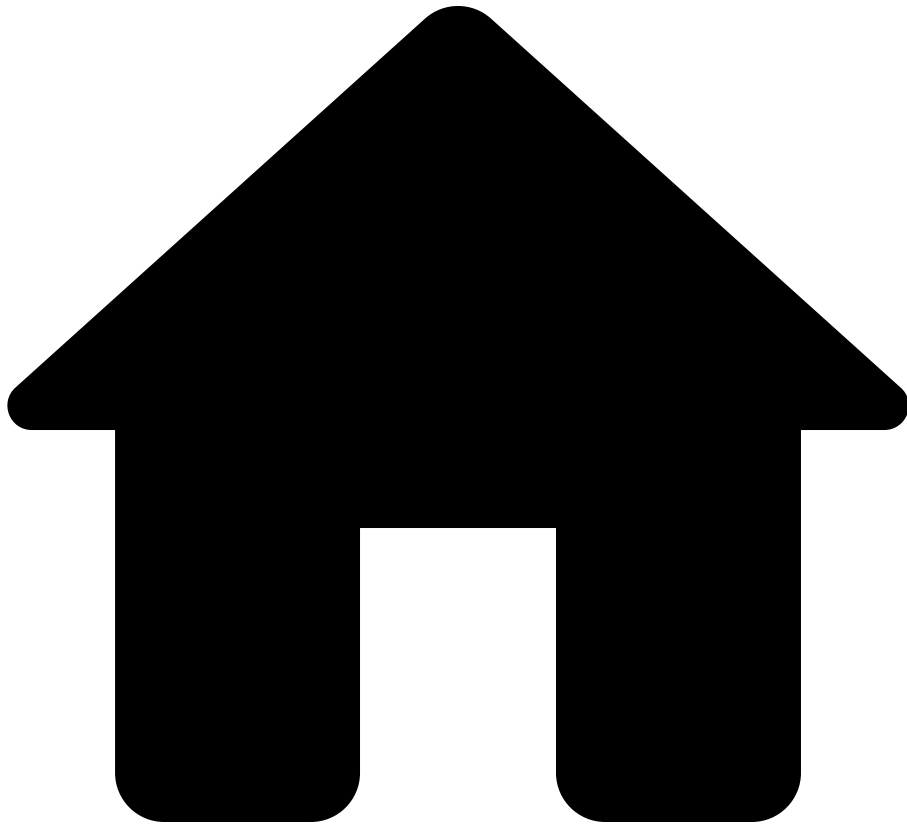
See Also

Guides

- [Collections](#) - Comprehensive guide on Lists, Bags, and ordering

- [Comprehensions](#) - Using order by clause

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- orderBy

On this page

orderBy

For any types `T` and `K`,

```
orderBy(collection: Bag<T>, keySelector: T -> K) : List<T>
```

Sorts a collection by applying a key selector function to each element and ordering by the resulting keys.

- The collection is sorted in ascending order based on the values returned by `keySelector`.
- For descending order, consider using the `order by` clause with `desc` modifier in a [comprehension](#), or negate numeric keys.
- See [comparisons](#) for more detail on the order of values.
- Note that this method accepts both `Lists` and `Bags` since [Lists subtype Bags](#)

Example

Sorting Devices by Name

```
getName(device) = device.name;  
orderBy(network.devices, getName);
```

This is equivalent to the more verbose, albeit more flexible, comprehension:

```
foreach device in network.devices  
let name = device.name  
select device  
order by name ascending
```

Notes

- For sorting by multiple keys or for descending order, consider using the `order by` clause in a comprehension instead.
- Elements with equal keys maintain their relative order from the original list (stable sort).

See Also[â](#)

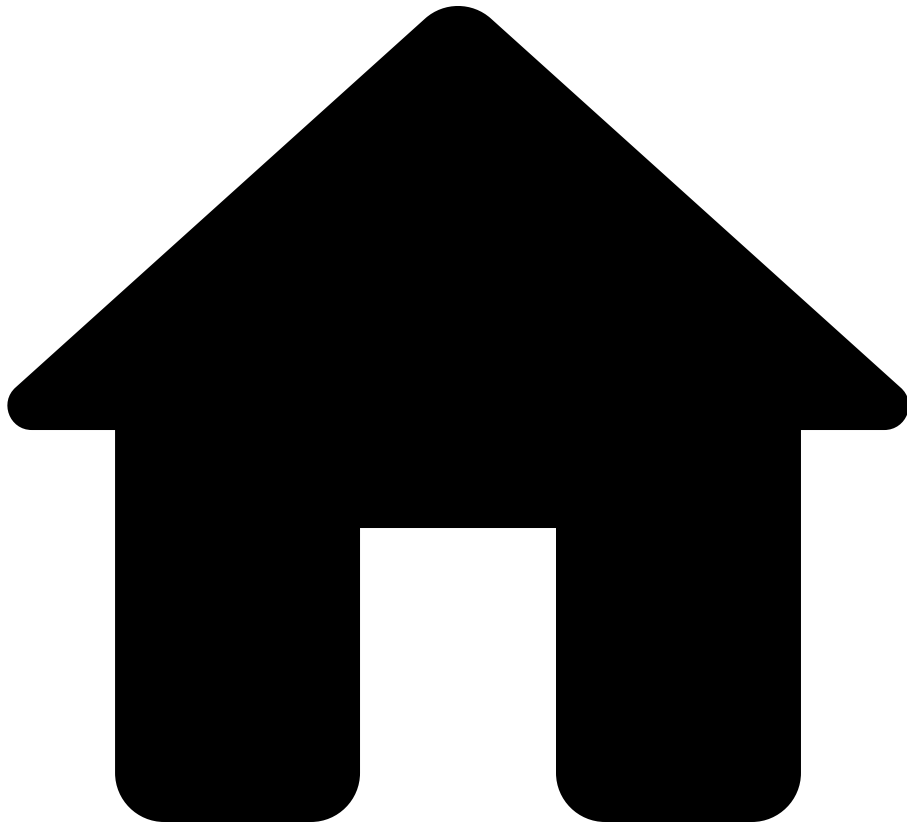
Guides[â](#)

- [Collections](#) - Comprehensive guide on Lists, Bags, and ordering
- [Comprehensions](#) - Using order by clause

Functions[â](#)

- [limit](#) - Limit the number of elements in a list
- [max](#) - Get the maximum element
- [min](#) - Get the minimum element
- [maxBy](#) - Get element with maximum key
- [minBy](#) - Get element with minimum key

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- ouiAssignee

On this page

ouiAssignee

```
ouiAssignee(mac: MacAddress) : String?
```

Returns the name of the OUI assignee (as a `String`) associated with a MAC address. For example, `ouiAssignee(macAddress("00:55:DA:50:00:00"))` returns the

string `"Nanoleaf"` . The function can return a `null` value if the assignee for the MAC address is not known.

See also

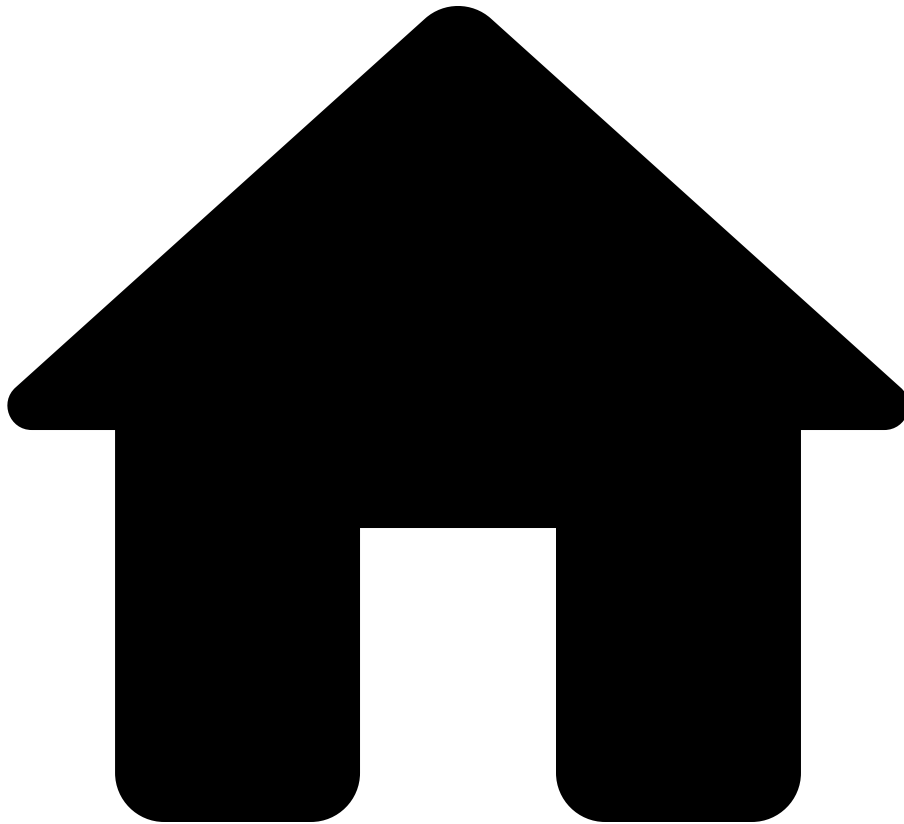
Types

- [MacAddress](#)
- [Collections](#)

Functions

- [ouiVendors](#)
- [macAddress](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- ouiVendors

On this page

ouiVendors

```
ouiVendors(mac: MacAddress) : Bag<Vendor>
```

Returns a bag of `Vendor` s associated with a MAC address. For example,

```
ouiVendors(macAddress("00:05:00:00:00:0D")) returns  
bag([Vendor.CISCO]) and
```

```
ouiVendors(macAddress("F8:0D:AC:FF:FF:FF")) returns  
bag([Vendor.HP, Vendor.ARUBA]) .
```

See also[â](#)

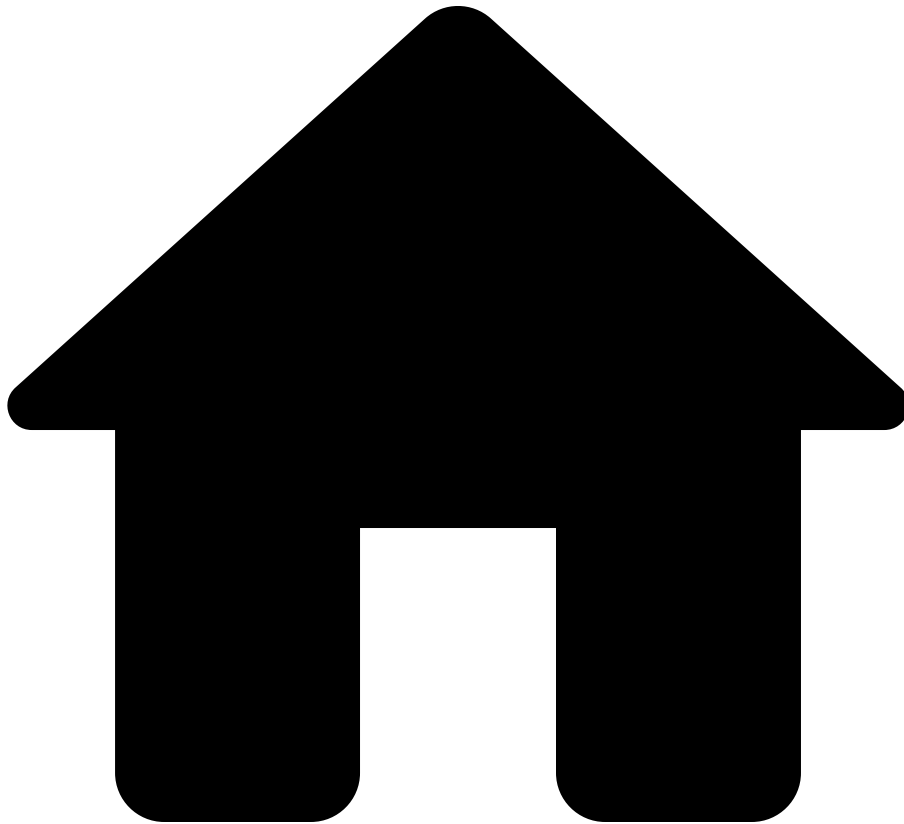
Types[â](#)

- [MacAddress](#)
- [Collections](#)

Functions[â](#)

- [ouiAssignee](#)
- [macAddress](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- parseConfigBlocks

On this page

parseConfigBlocks

```
parseConfigBlocks(os: OS, text: String) : List<ConfigLine>
```

The `parseConfigBlocks(os, text)` function can be used to parse a collection of configuration blocks from a string that holds a device configuration. In other words, an expression like `parseConfigBlocks(OS.NXOS, "interface eth1\n no`

`shut\n")` could be used to parse some NXOS configuration in the given string. The resulting value can then be used with various pattern-matching functions.

This function may be useful to process the response to custom commands, for example the response to command "show running-config all".

Not all lines in the given `text` correspond to some `ConfigLine` in the output. The lines that are ignored depend on the value of `os`. Here are all the ways that an input line is ignored.

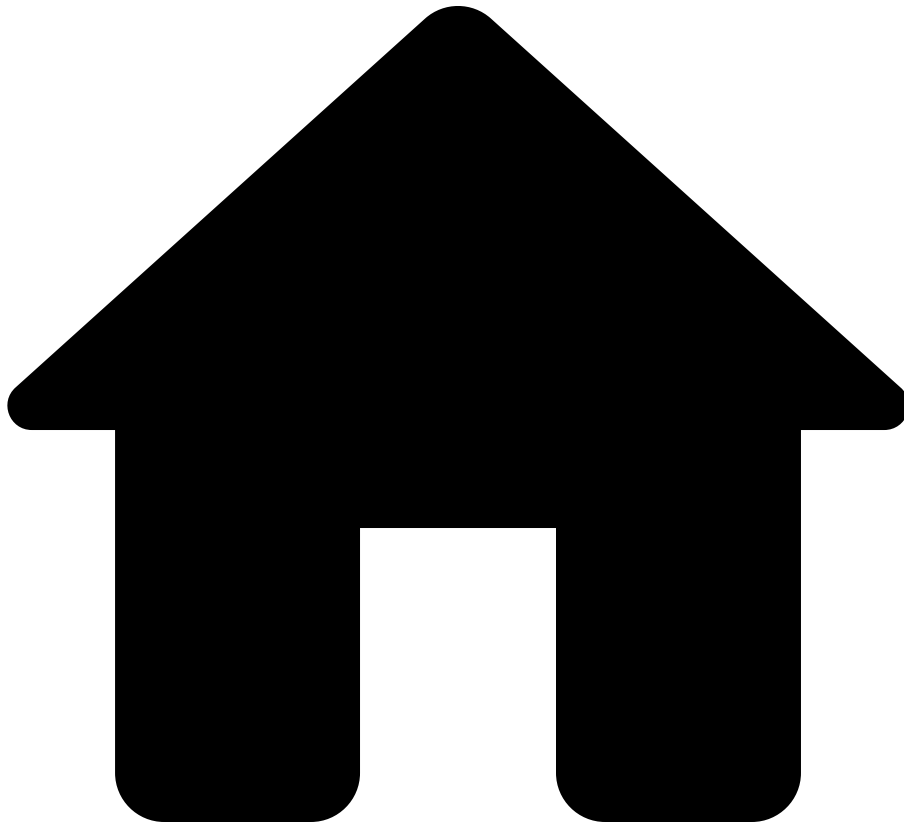
- `OS.PAN_OS`
 - No lines are ignored.
- For all other values of `os`, a line is ignored if any of the following are true.
 - The line is blank.
 - The line starts with
 - `!`,
 - `#`,
 - `;`,
 - `%`,
 - `building configuration...` (ignoring case), or
 - `current configuration :` (ignoring case).
 - The line starts with `Using` and contains `bytes`, such as `Using 2016 out of 260088 bytes!`.

See also [a](#)

Types [a](#)

- [OS](#)
- [String](#)
- [Collections](#)
- [ConfigLine](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- patternMatch

On this page

patternMatch

```
patternMatch(str: String, pattern: Pattern<T>) : T?
```

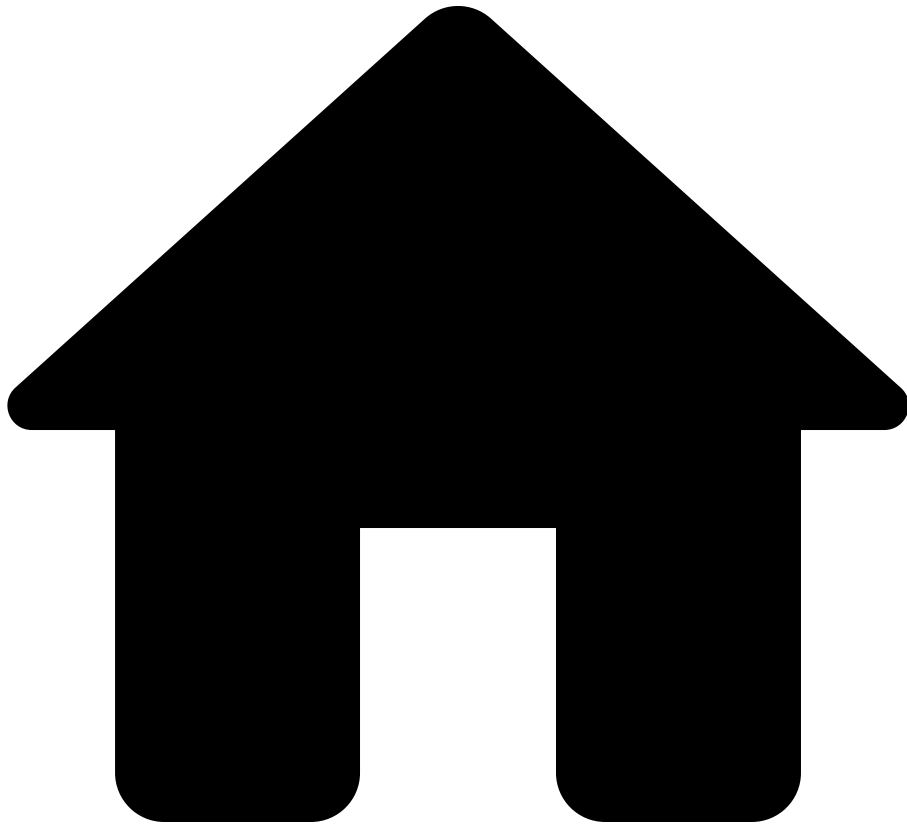
See [the data extraction guide](#).

See also [a](#)

Types [a](#)

- [String](#)

.



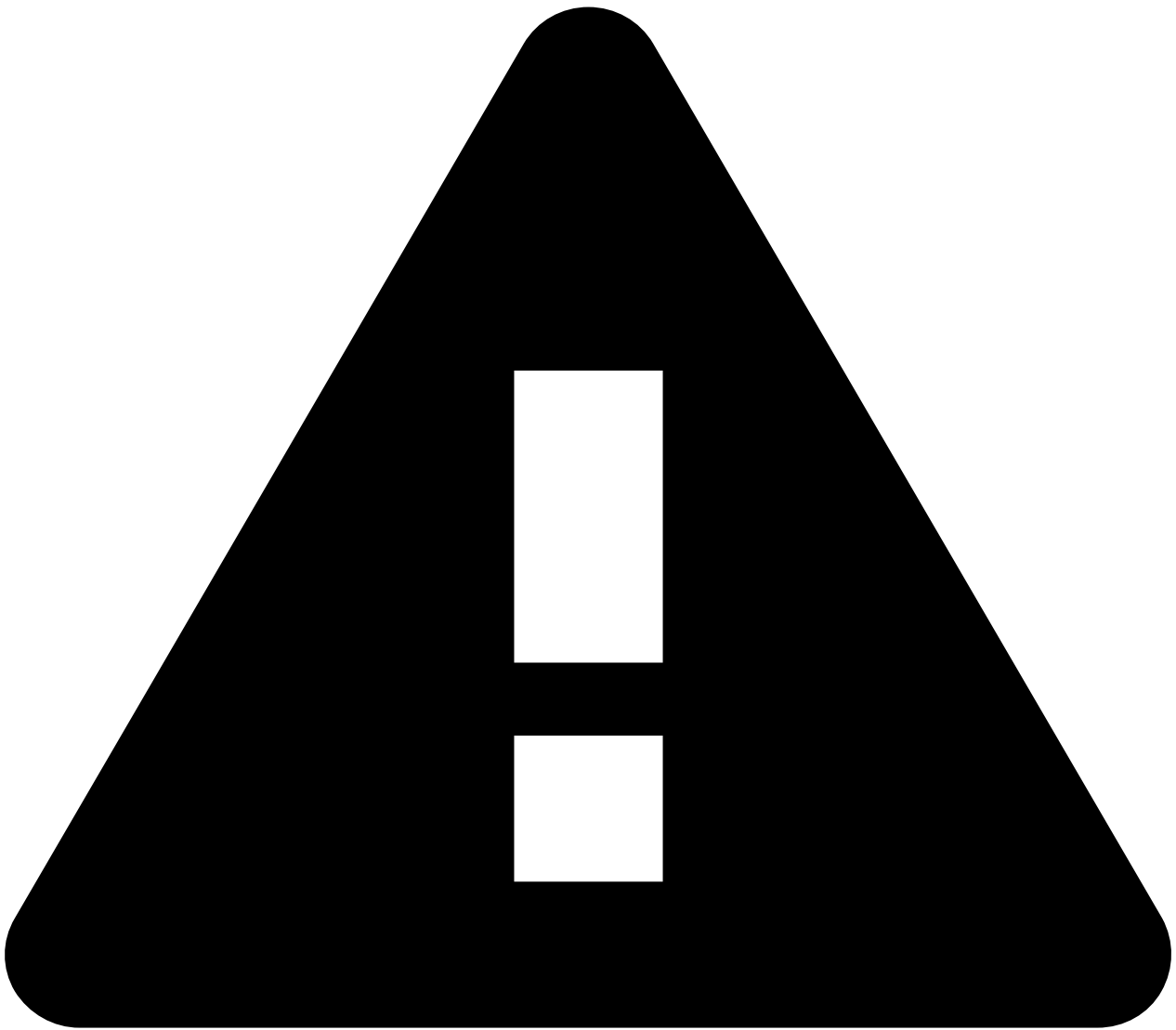
- Reference
- [NQE Language](#)
- [Standard Library](#)
- patternMatches

On this page

patternMatches

```
patternMatches(config: List<ConfigLine>, pattern: Pattern<T>) :  
Bag<{line: ConfigLine, data: T}>
```

See [the data extraction guide](#).



Deprecated Bag overload

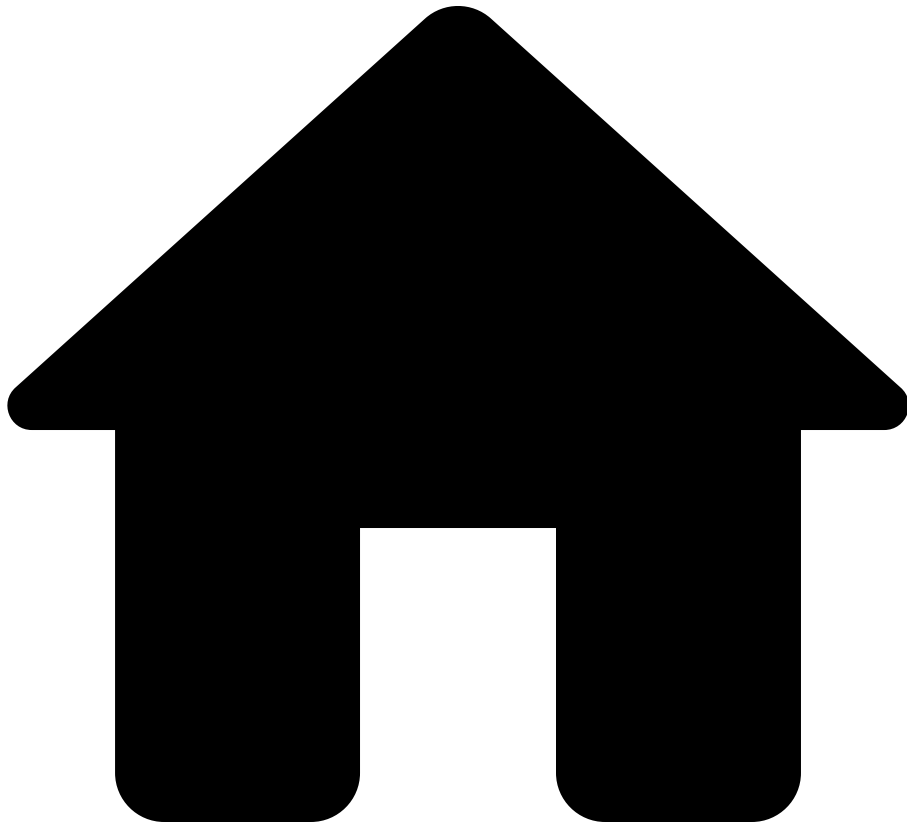
As a transitional measure for backwards compatibility, `patternMatches` also accepts a `Bag<ConfigLine>` as its first argument. Use `orderBy()` or `order()` to create an argument with type `List<ConfigLine>`. Eventually, the Bag overload will be removed and a type-checking error will be emitted instead.

See also

Types

- [String](#)
- [Collections](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- prefix

On this page

prefix

```
prefix(str: String, n: Integer) : String
```

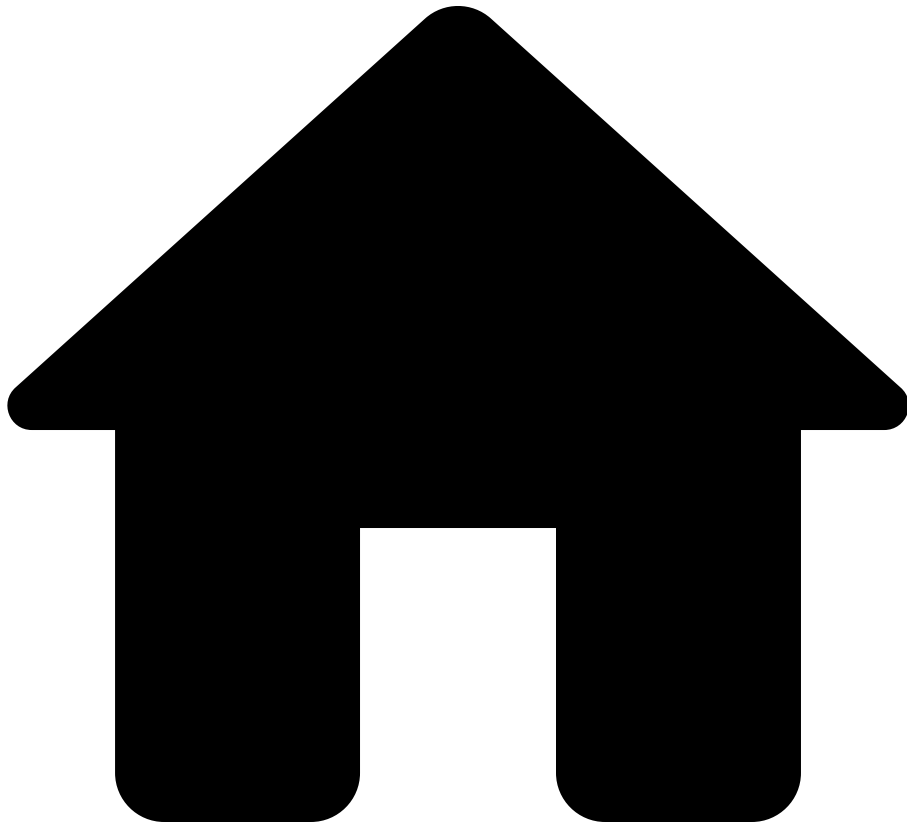
Returns first `n` characters of string `s`. For example, `prefix("abcdef", 3)` is `"abc"`, `prefix("abcdef", 0)` is `" "` (empty string), and `prefix("abcdef", 10)` is `"abcdef"`.

See also [ââ](#)

Types [ââ](#)

- [String](#)
- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- regex

On this page

regex

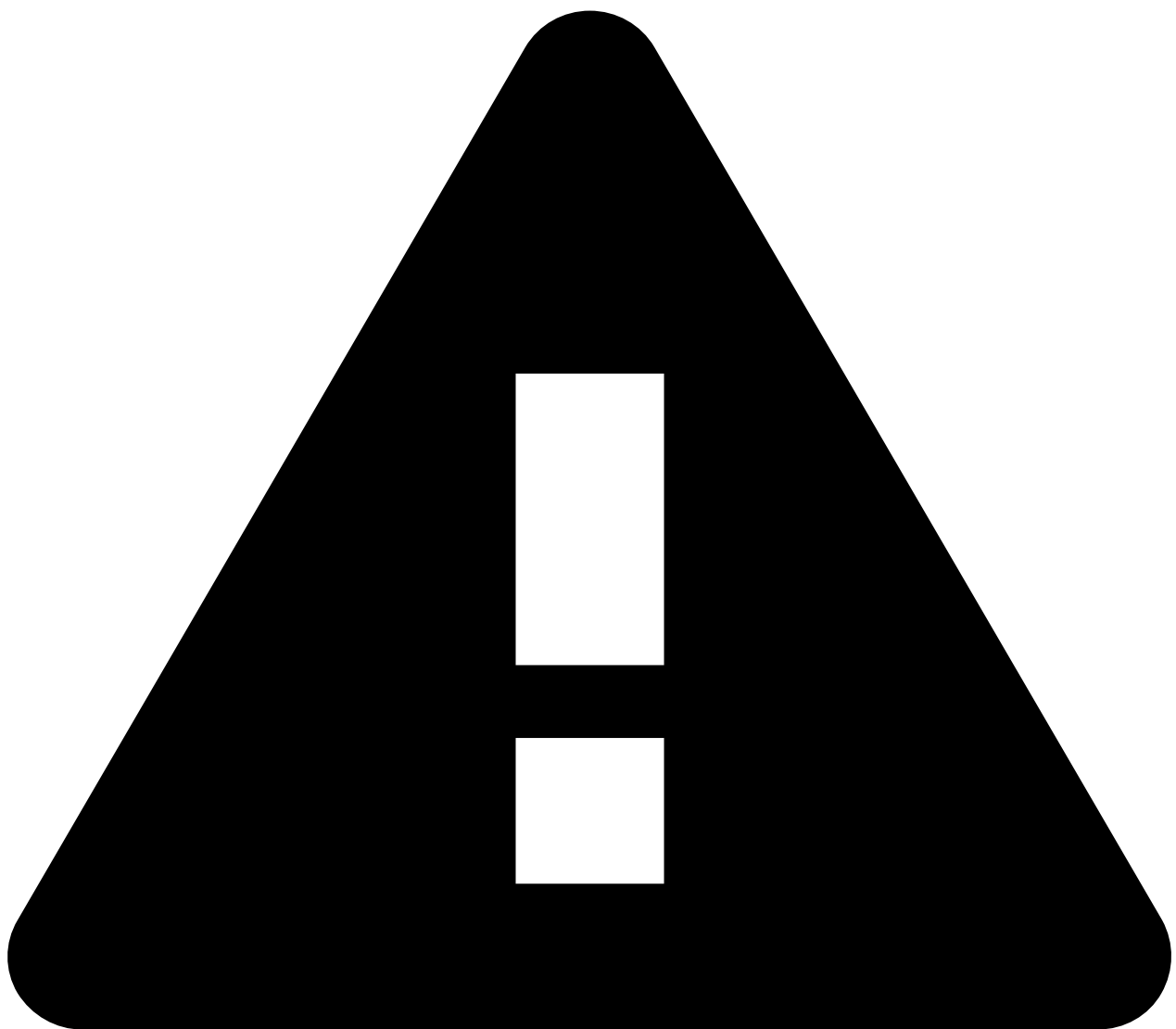
```
regex(text: String) : Regex<{}>
```

Converts `text` into a regular expression. If `text` is not a valid regular expression, the query terminates with an error.

The `regex(text)` function is particularly useful when the regular expression is passed in to the query or if it is dynamically computed by the query. For example:

```
@query
query(subIfaceIndex: String) =
  foreach device in network.devices
  foreach interface in device.interfaces
  let isInteresting = hasMatch(interface.name, regex("eth.*/" +
subIfaceIndex))
  select { deviceName: device.name, isInteresting }
```

In contrast, the `re` syntax can only be used when the regular expression can be statically embedded in the query source code.



caution

Regular expressions created with this function do not capture any data, even if they have capture groups.

Examples

| Example | Result |
|--|------------|
| <code>regex("[a-z][0-9]")</code> | A valid re |
| <code>regex("(?<word>\\w+)")</code> | The valid |
| <code>regex("(?<word>\\w+")</code> | Will thro |

| Example | Result |
|---|-------------|
| <code>regexMatches("2024 or 1974", regex("(?<year:Integer>\\d{4})"))</code> | A list of t |

Backslashes⚠️

Care must be taken when using the backslash character `\` in strings passed to the `regex` function. It may be escaped *twice*: Once when constructing the string, and once when converted to a regular expression. Because of this, a `\` will be interpreted as escaping the subsequent character if not escaped itself. To use a backslash `\`, escape it with a second backslash `\\`. (Backslashes in regex *literals* do *not* require double escaping.)

This behavior is best illustrated by an example:

```
foreach phrase in [" cat"]
select {
  "Works as intended, using literal syntax":
  hasMatch(phrase, re`\\scat`),
  "Works as intended, using regex and escapes":
  hasMatch(phrase, regex("\\scat")),
  "Whoops, forgot to escape the slashes":
  hasMatch(phrase, regex("\\scat")),
  "Another case where we need to escape slashes": hasMatch("\\
\\", regex("\\\\\\")) == hasMatch("\\\\", re`\\`)
```

This query results in one row with values `true, true, false, true`. The first two have the same output, but the third one is erroneous.

See also [â](#)

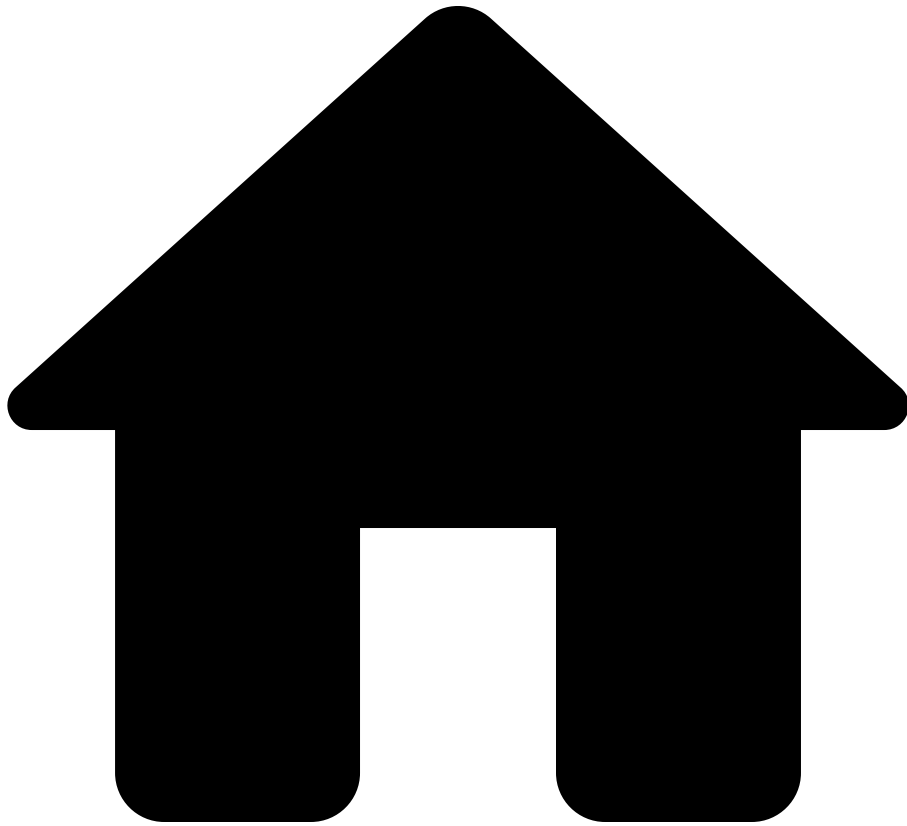
Functions [â](#)

- [hasMatch](#): Checks whether a string matches the structure specified by a regex or not.

Types [â](#)

See [the regular expression guide](#).

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- regexMatches

On this page

regexMatches

```
regexMatches(text: String, regex: Regex<T>) : List<{string: String, start: Integer, end: Integer, data: T}>
```

Gets a list of all matches of `regex` within `text`, or the empty list if no matches occur.

Matches are returned in the order they appear in the text (left to right), maintaining their sequential positions.

Each match is a record with the following fields:

- `string: String` *The substring that is matched.*
- `start: Integer` *Zero-based character offset in `text` where the match begins.*
- `end: Integer` *The first character offset index after the match ends.*
- `data: T` *The capture groups. This is a record value.*

Examples with no capture groups^{[1](#)[2](#)}

| text |
|---|
| "hello world" |
| "abc123def456" |
| "I have 2 apples and 3 bananas" |
| "A1 B2 C3" |
| "12/05/2024 and 20/08/1991 but not 02/01/1974" |
| "Me: jim@place.com, You: jam@company.org, Them: timothy@hilltop.school" |
| "Contact me at 555-123-4567 or 444-987-6543" |
| "The year is 2024, not 2014 nor 2004 nor 1994 nor 1974" |

text

Examples with capture groups [ââ](#)

text

"hello world"

"abc123def456"

"I have 2 apples and 13.582 bananas"

"A1 B2 C3"

"12/05/2024 and 20/08/1991 but not 02/01/1974"

"Me: jim@place.com, You: jam@company.org, Them: timothy@hilltop.school"

"Contact me at 555-123-4567 or 444-987-6543"

"The year is 2024, not 2014 nor 2004 nor 1994 nor 1974"

See also[a](#)

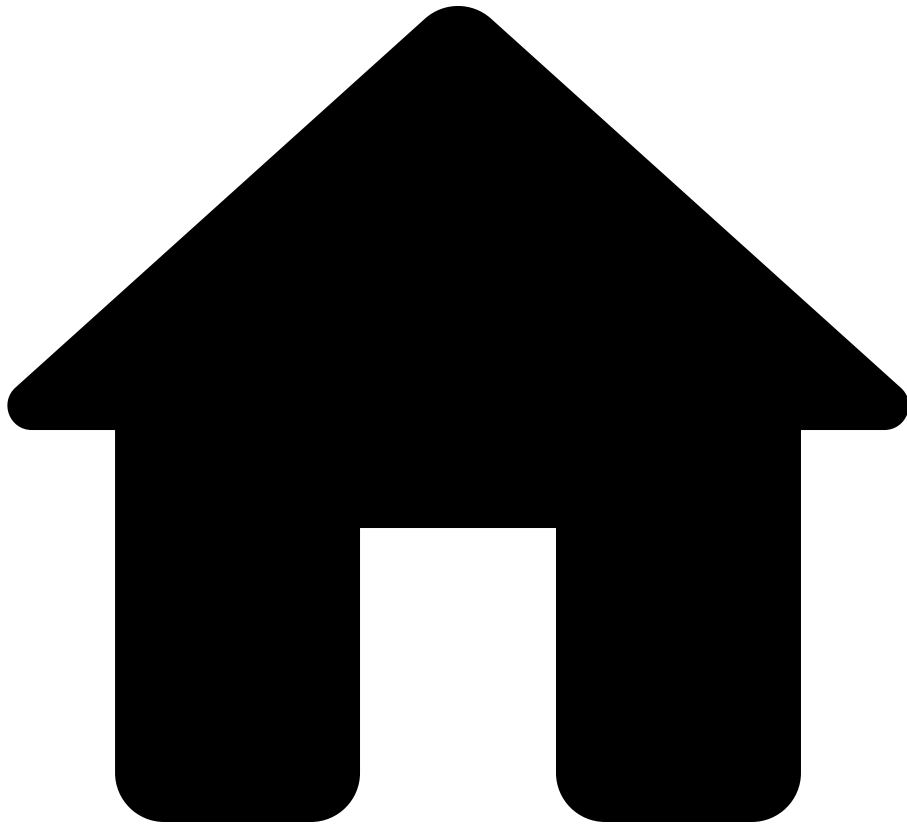
Guides[a](#)

- [Regular Expressions](#) - Comprehensive guide on regex syntax and usage
- [Collections](#) - Working with ordered and unordered collections
- [String](#)
- [Regex](#)

Functions[a](#)

- [hasMatch](#) - Checks whether a string matches the structure specified by a regex or not
- [match](#) - Gets a record of information about text that matches the structure specified by a regex, or otherwise `null`
- [replaceRegexMatches](#) - Replaces all substring matches of a given regex using the provided replacement template

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- replace

On this page

replace

```
replace(text: String, target: String, replacement: String) :  
String
```

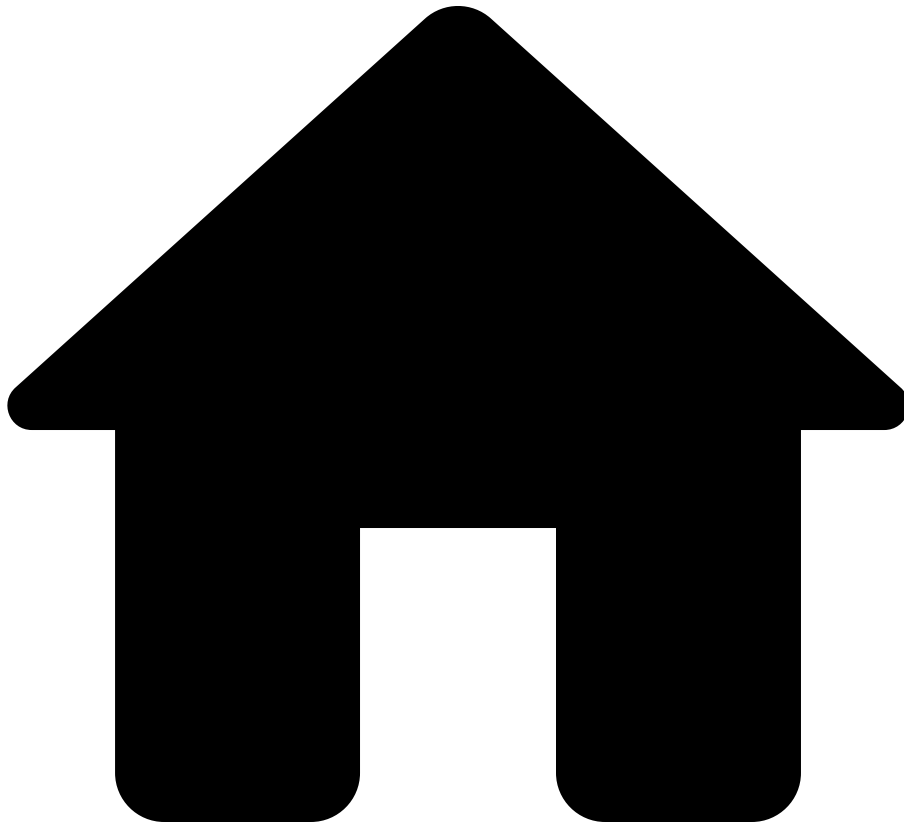
Replaces every occurrence of `target` in `text` with `replacement` . For example, `replace("abc-def", "-", "_")` is `"abc_def"` , `replace("abc-def", "-", "")` is `"abcdef"` and `replace("abab", "a", "cd")` is `"cdbcdb"` .

See also

Types

- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- replaceMatches

On this page

replaceMatches

```
replaceMatches(text: String, glob: String, replacement: String) :  
String
```

Replaces every match of `glob` pattern in `text` with `replacement` . For example, `replaceMatches("abc-123-def", "-*-", "_")` is `"abc_def"` and `replaceMatches("abc-123-def", "-*-", "")` is `"abcdef"` .

Note that in `replaceMatches` wildcards `*` and `?` do NOT match newline characters. For example, in the following example, `afterText` will have value `"another\n"` :

```
text = ""
!! abc
another
!! def

"";

afterText = replaceMatches(text, "!!*\n", "");
```

See also

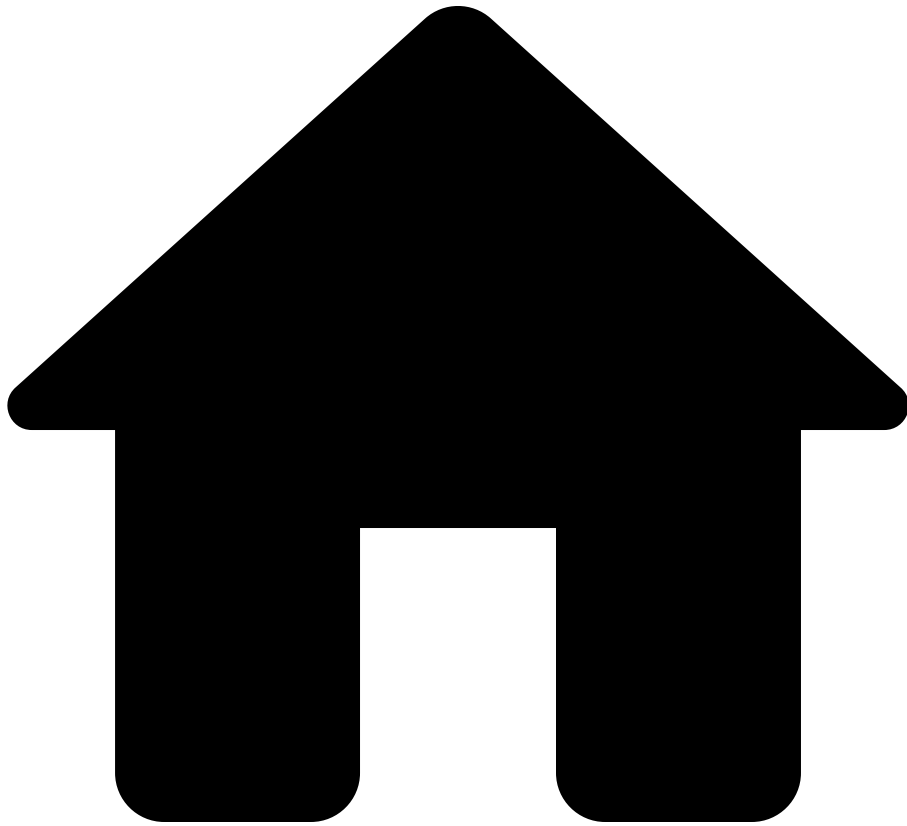
Types

- [String](#)

Functions

- [replace](#)
- [matches](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- replaceRegexMatches

On this page

replaceRegexMatches

```
replaceRegexMatches(text: String, regex: Regex<T>, replacement:  
String) : String
```

Replaces every match of `regex` in `text` with `replacement` .

- `replacement` may reference group captures by *name* such as `${name}` or by *index* such as `$3` (*without braces!*)
- If the index is out of bounds, the query terminates with an error.
 - The index `$0` references the entire matched substring.
- If the dereference does not refer to any named group in `regex` , the query terminates with an error.
 - If the dereference name refers to a named group that occurs multiple locations, then the query terminates with an error advising the use of a positional index reference.
- To embed a literal `$` , use `"\\$"` with *string escapes*.
 - The escape is only necessary when `$` is followed by a digit, but it is always allowed.
 - Only `$` and `\` need escaping, but any character can be escaped.
- References to a non-matched group are allowed and are replaced with `" "` .
 - For example, `replaceRegexMatches("1234", re`(\d)|(\w)` , "a$2")` results in `"aaaa"` .

Examples

| text | regex |
|--|--------------------------------------|
| "hello world" | re`(<word>\w+)` |
| "hello world" | re`(<word>\w+)` |
| "hello world" | re`(<word>\w+)` |
| "abc123def456" | re`(<number:Integer>)` |
| "I have 2 apples and 13.582 bananas" | re`(<count:Float>\d+\.?\d*)` |
| "A1 B2 C3" | re`(<letter:String>)` |
| "12/05/2024 and 20/08/1991 but not 02/01/1974" | re`(<day:Integer>\d{2}/\d{2}/\d{4})` |

| text | regex |
|--|-----------------------|
| "Me: jim@place.com, You: timothy@hilltop.school" | re`(?<local>\w+)@(?< |
| "Contact me at 555-123-4567 or 444-987-6543" | re`(?<area>\d{3})-(?) |
| "I have 10 dollars, and you have 5" | re`(?<money:Integer> |
| "I have 10 dollars, and you have 5" | re`(?<money:Float>\d |
| "I have 10 dollars, and you have 5" | re`(?<money:String>\ |
| "I have 10 dollars, and you have 5" | re`(?<money:String>\ |
| "I have 10 dollars, and you have 5" | re`(?<money:String>\ |
| "I have 10 dollars, and you have 5" | re`(?<money:String>\ |

See also

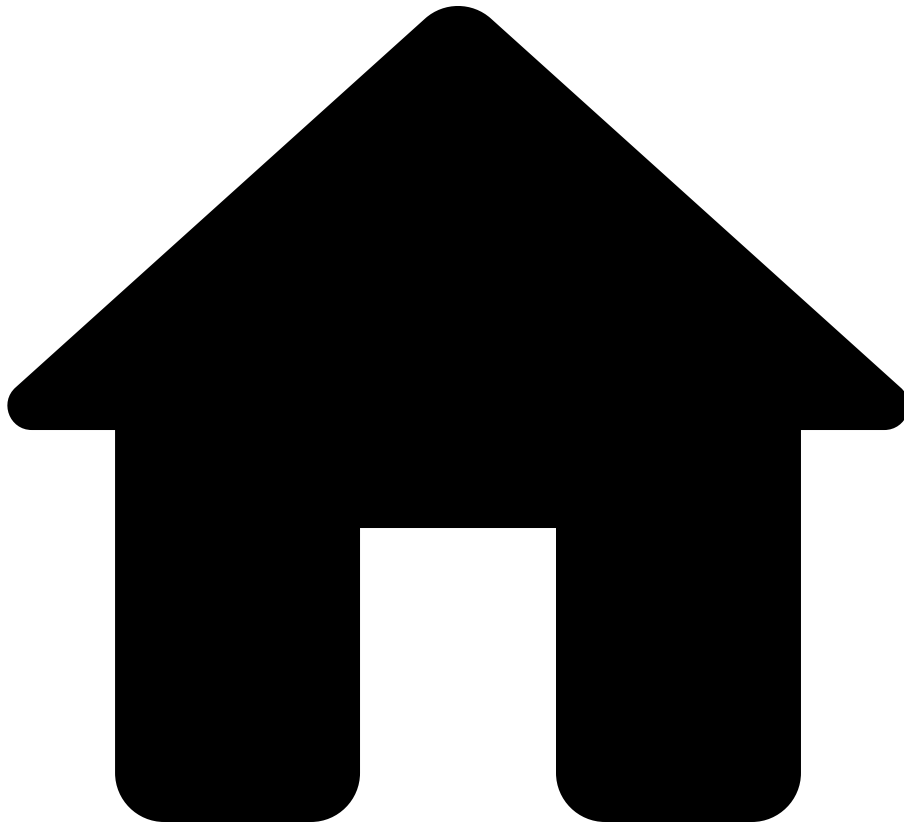
Functions

- [match](#): Gets a record of information about text that matches the structure specified by a regex, or otherwise `null`.
- [regexMatches](#): Gets the list of all substring matches of a given regex.

Types

See [the regular expression guide](#).

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- round

On this page

round

`round(number: Float) : Integer`

Gets an integer from a float by *rounding* to the nearest integer; e.g., `round(2.5)` results in `3`, whereas `round(2.4)` results in `2`.

See also[ââ](#)

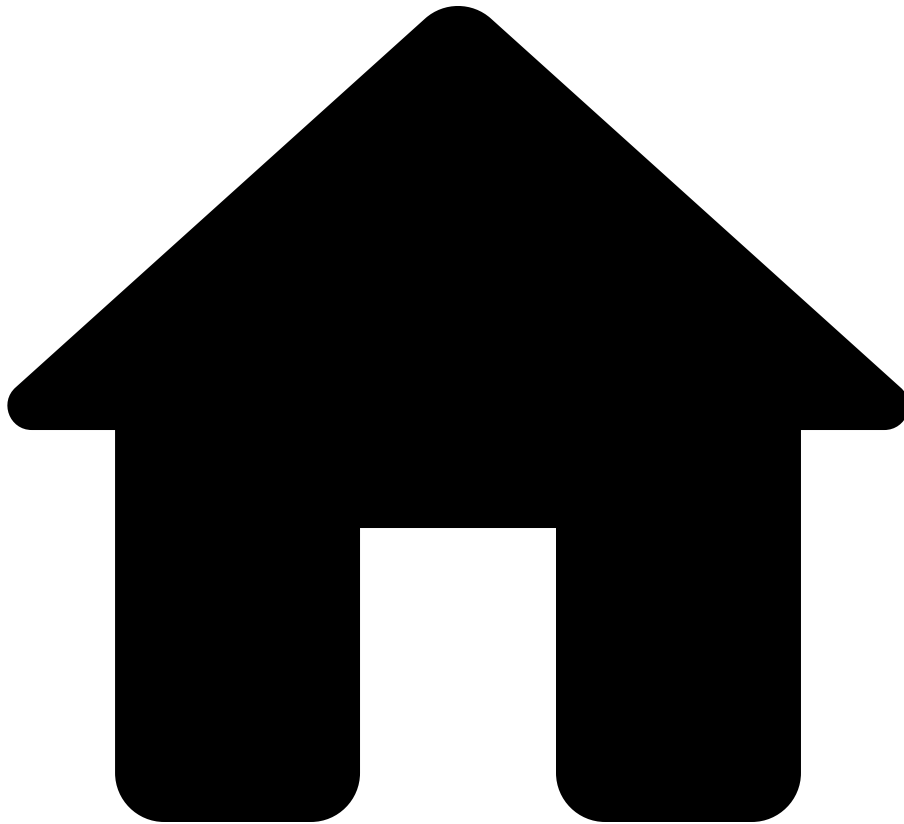
Functions[ââ](#)

- [float](#)
- [int](#)
- [floor](#)
- [ceil](#)
- [toNumber](#)

Types[ââ](#)

- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- seconds

On this page

seconds

```
seconds(value: Integer) : Duration
```

Obtains a `Duration` representing the given number of seconds. For example, `seconds(3)` returns a `Duration` that represents three seconds. The value can be

negative. For example, `seconds (- 2)` returns a `Duration` that represents negative two seconds.

See also[â](#)

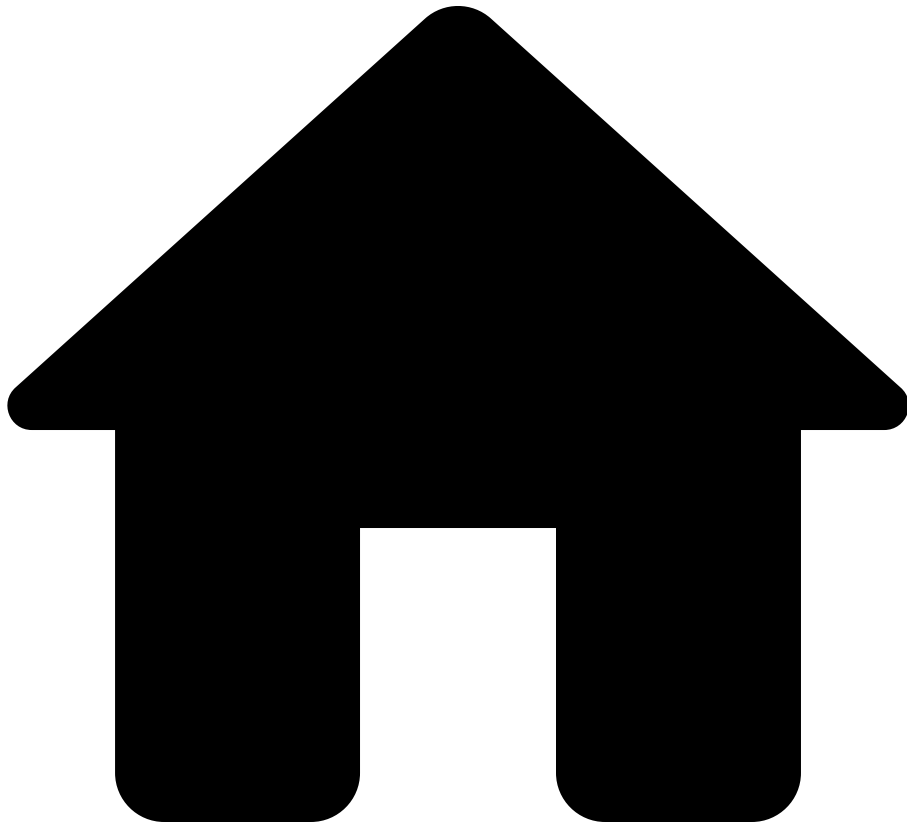
Functions[â](#)

- [date](#)
- [timestamp](#)
- [duration](#)
- [days](#)
- [hours](#)
- [minutes](#)

Types[â](#)

- [Time](#)
- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- sourceConfigText

On this page

sourceConfigText

`sourceConfigText(value: T) : String` for any type `T`.

Returns a `String` containing the lines of configuration linked from the given value. If the value does not link to any lines of configuration, then the returned value is an empty `String`.

Values in NQE sometimes link to lines of configuration to show their source location. This function provides access to such relevant configuration so it can be included in a results export or in an API response to a query execution request.

Here is an example to illustrate the use of `sourceConfigText`. The `name` field of an `AcEntry` typically links to the lines of configuration that define the ACL, so the query

```
foreach device in network.devices
foreach aclEntry in device.aclEntries
select {
  Device: device.name,
  ACL: aclEntry.name,
  Config: sourceConfigText(aclEntry.name)
}
```

is a good use of `sourceConfigText`.

Specifically, `sourceConfigText(aclEntry.name)` could return the `String`

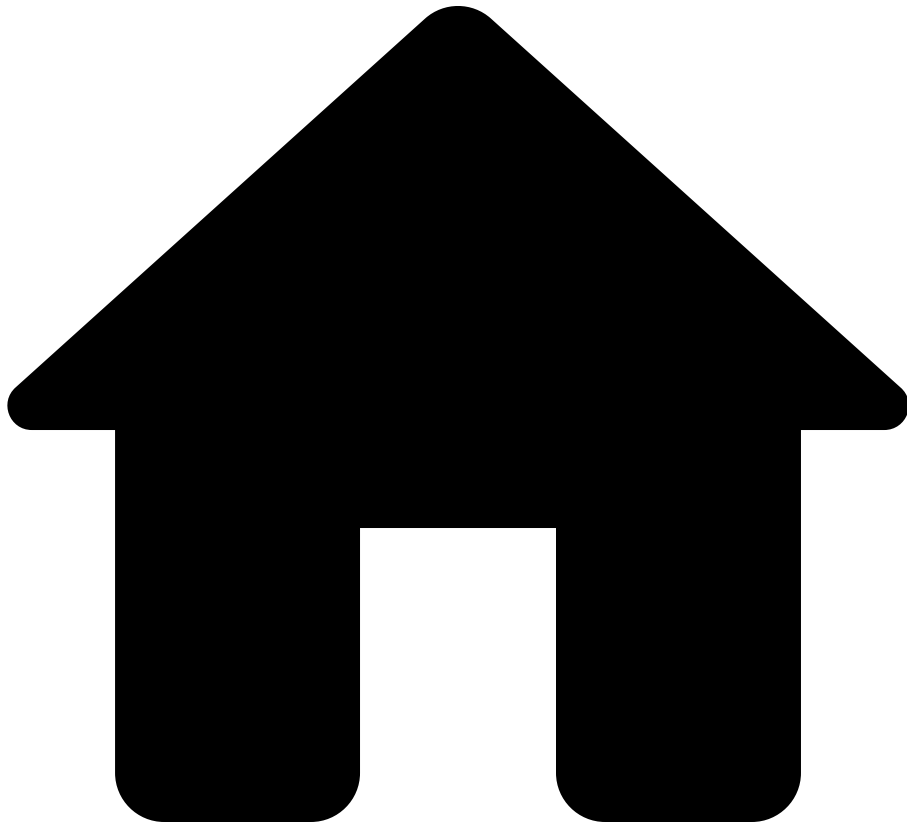
```
<my-device-name,configuration.txt>
access-list MyAcl extended permit tcp host 10.169.24.197 host
10.230.109.245 eq www
```

See also

Types

- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- splitJsonObjects

On this page

splitJsonObjects

```
splitJsonObjects(json: String) : List<String>
```

Splits a string containing multiple JSON objects into an ordered list of individual JSON object strings.

- This function is useful when parsing output that contains multiple JSON objects concatenated together.
- The returned list preserves the order in which the JSON objects appear in the input string.

Example

```
jsonText = ""
{"name": "device1", "status": "up"}
{"name": "device2", "status": "down"}
{"name": "device3", "status": "up"}
"";

foreach objectString in splitJsonObjects(jsonText)
let actualJSON = extractJson[{name: String, status: String}]
(objectString)
let status = actualJSON.status
select actualJSON
order by status ascending
```

Notice that `splitJsonObjects` returns individual objects as `String`s, but if we want them as typed records then we use the [extractJson](#) function.

See also

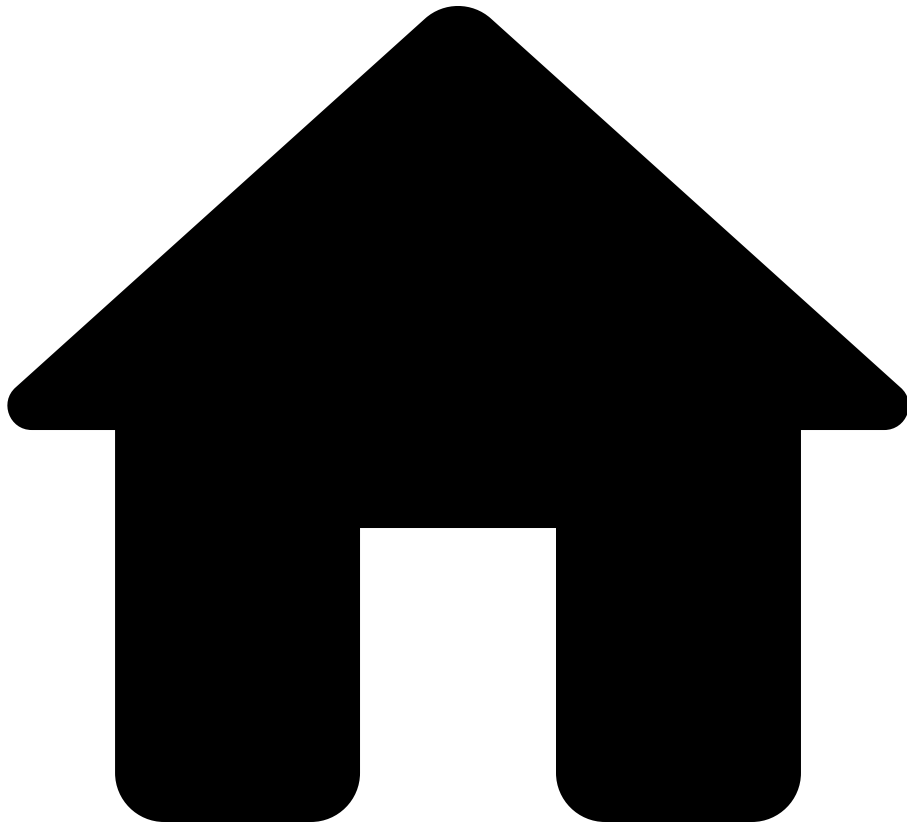
Guides

- [JSON](#) - Working with JSON data
- [Collections](#) - Working with collections
- [String](#)

Functions

- [extractJson](#) - Extract typed data from JSON strings

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- substring

On this page

substring

```
substring(str: String, start: Integer, end: Integer) : String
```

Returns the substring of string `str` starting from index `start` (inclusive) and ending at index `end` (exclusive). For example, `substring("abcdef", 0, 6)` is

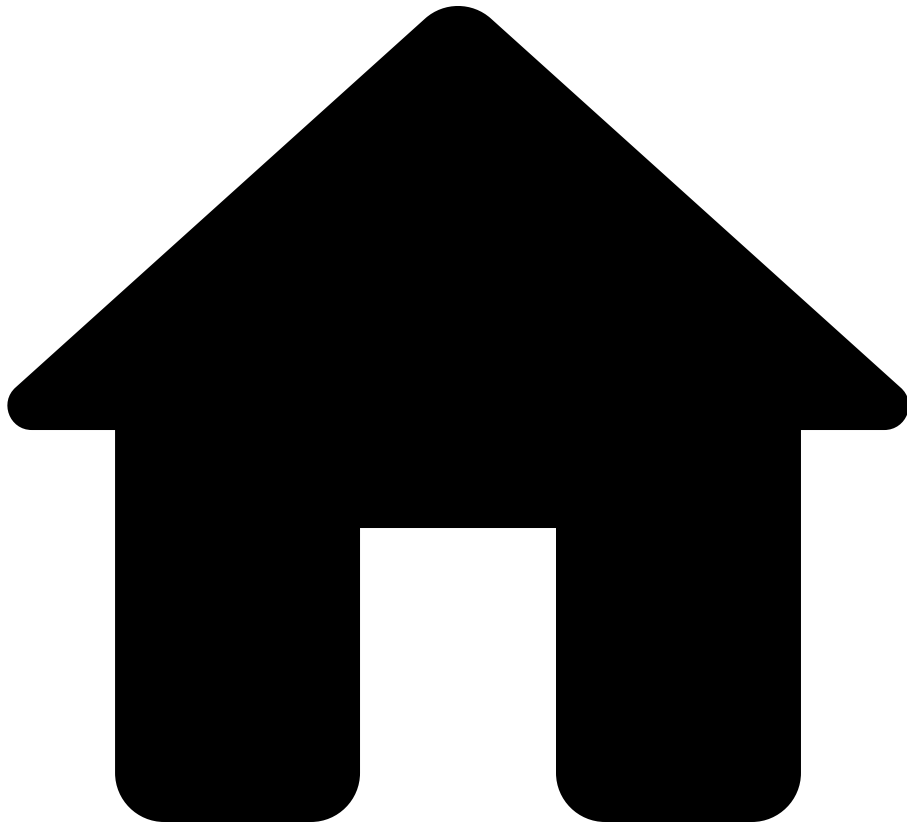
`"abcdef"`, `substring("abcdef", 1, 3)` is `"bc"`, and `substring("abcdef", 1, 1)` is `" "` (the empty string).

See also^{[a](#)[a](#)[a](#)}

Types^{[a](#)[a](#)[a](#)}

- [String](#)
- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- suffix

On this page

suffix

```
suffix(str: String, n: Integer) : String
```

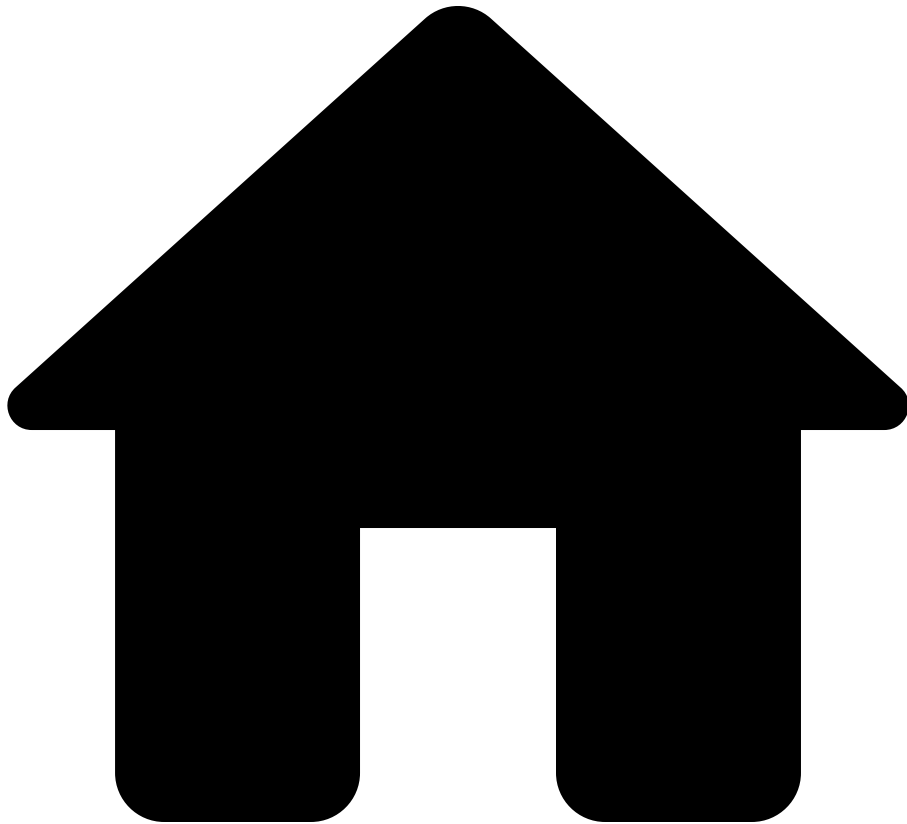
Returns the last `n` characters of string `str`. For example, `suffix("abcdef", 3)` is `"def"`, `suffix("abcdef", 0)` is `" "` (empty string), and `suffix("abcdef", 10)` is `"abcdef"`.

See also [ââ](#)

Types [ââ](#)

- [String](#)
- [Integer](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- sum

On this page

sum

```
sum(numbers: Bag<Integer>) : Integer
sum(numbers: Bag<Float>)   : Float
```

The sum of a *collection of integer numbers* or a *collection of floating-point numbers*.

- For example, on integers, `sum([1, 2, 3])` equals `6`.
- Likewise, on floats, `sum([1.1, 2.1, 3.1])` equals `6.3000000000000001`.
 - Keep in mind that *floating-point arithmetic is generally not precise!*
- You can define a collection in various ways, not just as a list of literal numbers.
 - See [Collections](#) for more detail on working with collections.
 - Importantly, a collection's elements must all be of the same type. As such, `sum([1.1, 2, 3.1])` has a type-error since the list is not well-formed. The fix is to replace *integer* `2` with `2.0` or `2.` to have it be treated as a *float*.
 - You can use `int` and `float` to convert to integers and to floats, respectively.
- Note that this method accepts both `Lists` and `Bags` —since [Lists subtype Bags](#)

See also

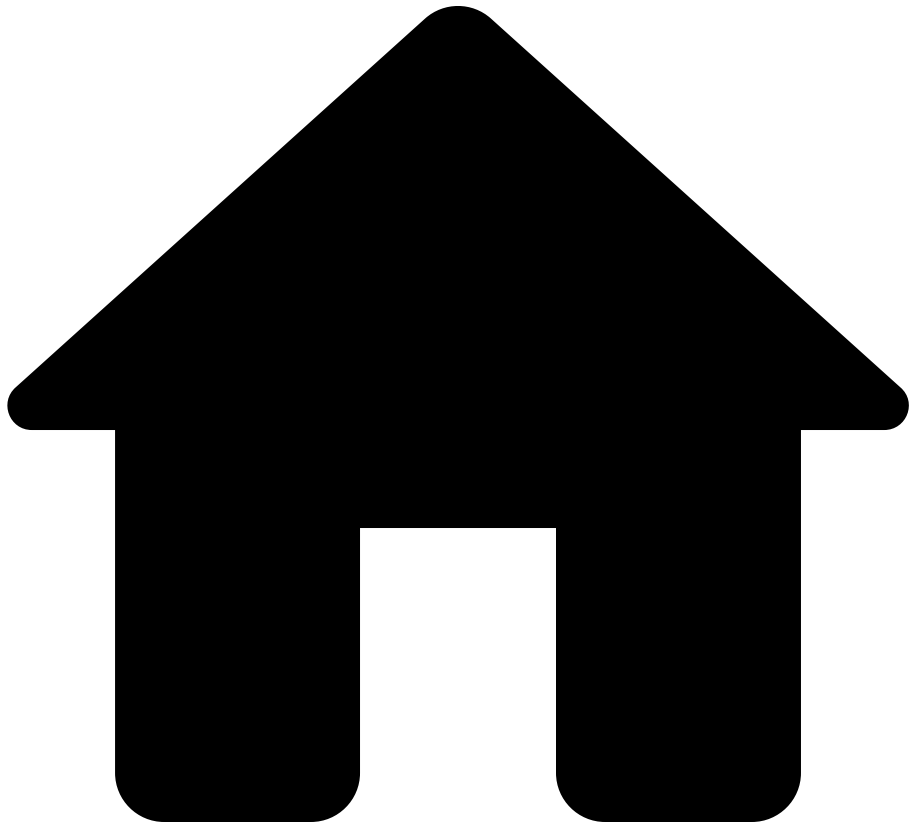
Types

- [Integer](#)
- [Collections](#)

Functions

- [int](#)
- [float](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- the

On this page

the

For any type `T`,

```
the(collection: List<T>) : T?  
the(collection: Bag<T>) : T?
```

Extracts the unique element from a collection, if it has a unique element and returns `null` otherwise.

Examples

Basic uses

```
singleItem = the([42]);           // Returns 42, since the
collection has a unique item
empty      = the(fromTo(1, 0));   // Returns null, since the
collection is empty
multiple   = the([1, 2, 3]);     // Returns null, since the
collection has multiple items
```

Finding a Unique Device

```
// Returns the device if exactly one match exists, null
otherwise
coreRouter = the(
    foreach device in network.devices
    where device.name == "core-router-1"
    select device
);
```

Use Cases

- Extracting a unique result from a filtered collection
- Validating uniqueness constraints
- Safely accessing "the one" element when you expect exactly one

Notes

- `the` is particularly useful when combined with `isPresent` to distinguish between "no results" and "too many results"

- Prefer `the` over manual checks like `length(collection) == 1` for cleaner, more idiomatic code.

That is, replace `if length(myIntegers) == 1 then max(myIntegers) else null:Integer` with just `the(myIntegers)`.

See Also

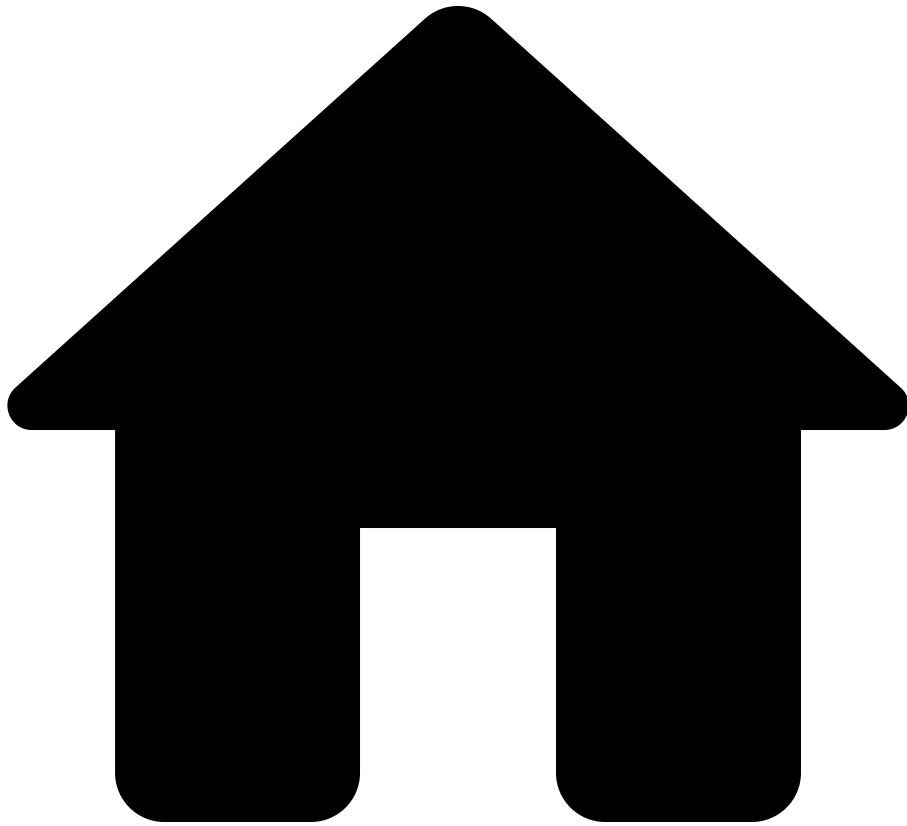
Guides

- [Collections](#) - Comprehensive guide on Lists, Bags, and ordering
- [Missing Values](#) - Working with null values

Functions

- [isPresent](#) - Check if a value is not null
- [length](#) - Get the length of a list
- [limit](#) - Limit the number of elements in a list

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- timestamp

On this page

timestamp

```
timestamp(text: String) : Timestamp
```

Obtains a `Timestamp` from a `String` that represents a date and time in UTC according to the ISO 8601 standard. For example,

`timestamp("2000-01-01T00:00:00Z")` returns a `Timestamp` that represents midnight UTC on January 1, 2000.

This function will raise an error in the following cases:

- if the string is absent;
- if the string deviates from the ISO 8601 standard for a date and time in UTC; and
- if the string represents a date and time that time includes a nonzero amount of nanoseconds, such as the string `"2000-01-01T00:00:00.123Z"`.

```
timestamp(duration: Duration) : Timestamp
```

Obtains a `Timestamp` from a `Duration` by starting with the timestamp for the epoch, which is equal to `timestamp("1970-01-01T00:00:00Z")`, and then adding the duration to it.

This function will raise an error in the following cases:

- if the duration is absent;
- if the duration is less than `-31557014167219200` seconds; and
- if the duration is more than `31556889864403199` seconds.

See also[â](#)

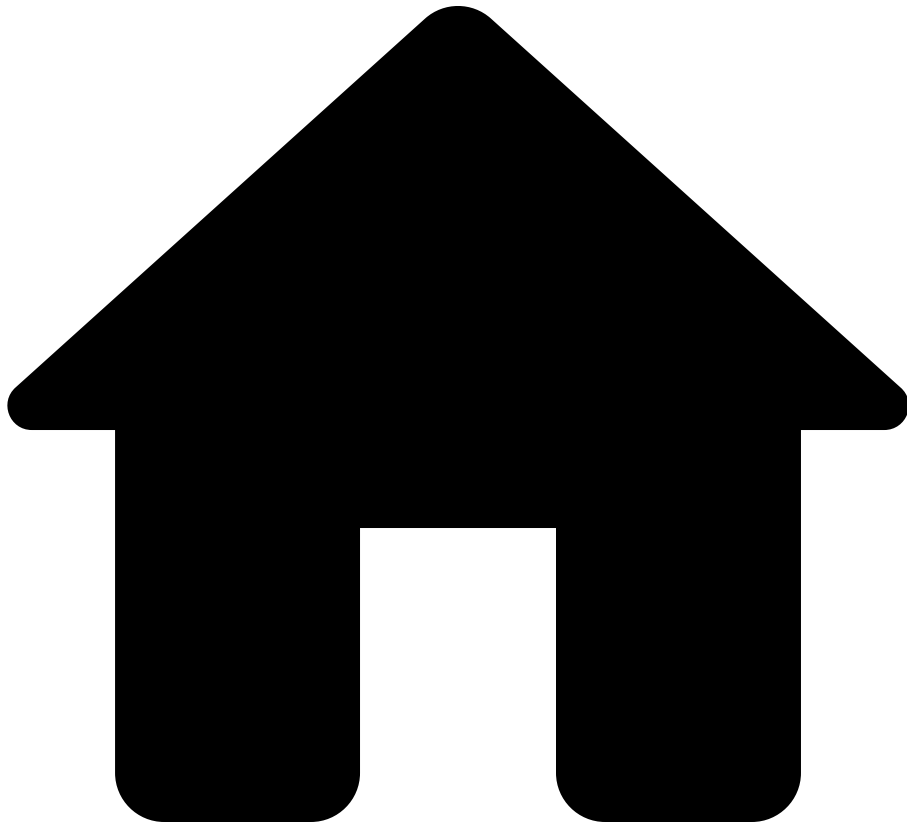
Functions[â](#)

- [date](#)
- [duration](#)
- [days](#)
- [hours](#)
- [minutes](#)
- [seconds](#)

Types[â](#)

- [Time](#)
- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- toLowerCase

On this page

toLowerCase

```
toLowerCase(str: String) : String
```

Changes all alphabetic characters in the string to lowercase. For example,

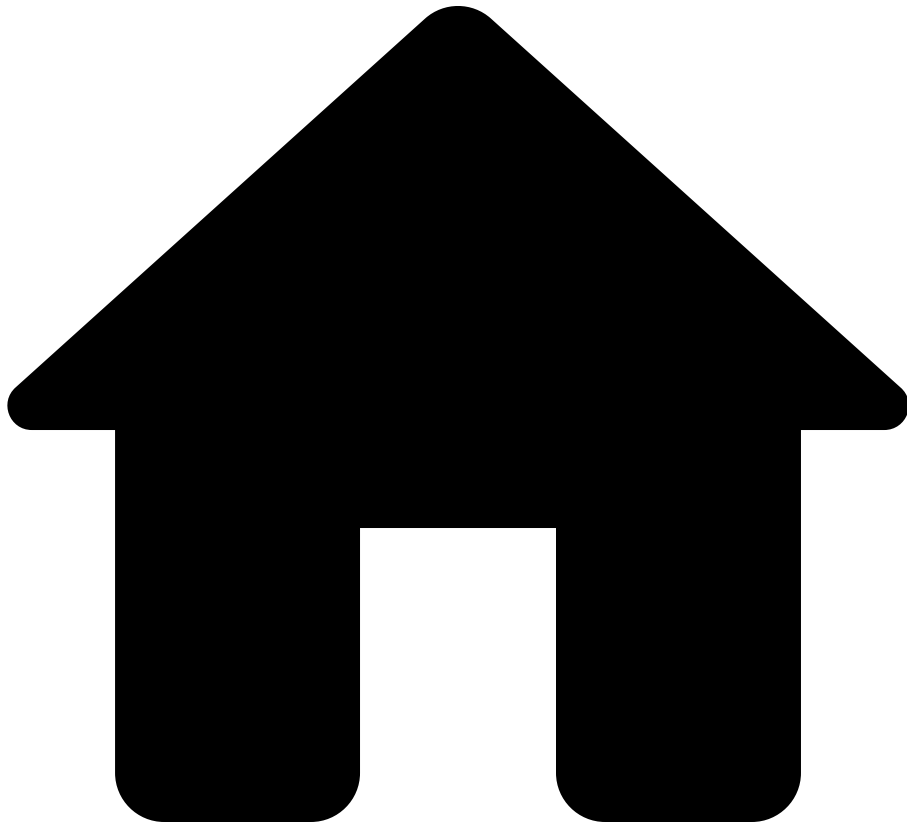
```
toLowerCase("LowerCase") is "lowercase" .
```

See also [a](#)

Types [a](#)

- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- toNumber

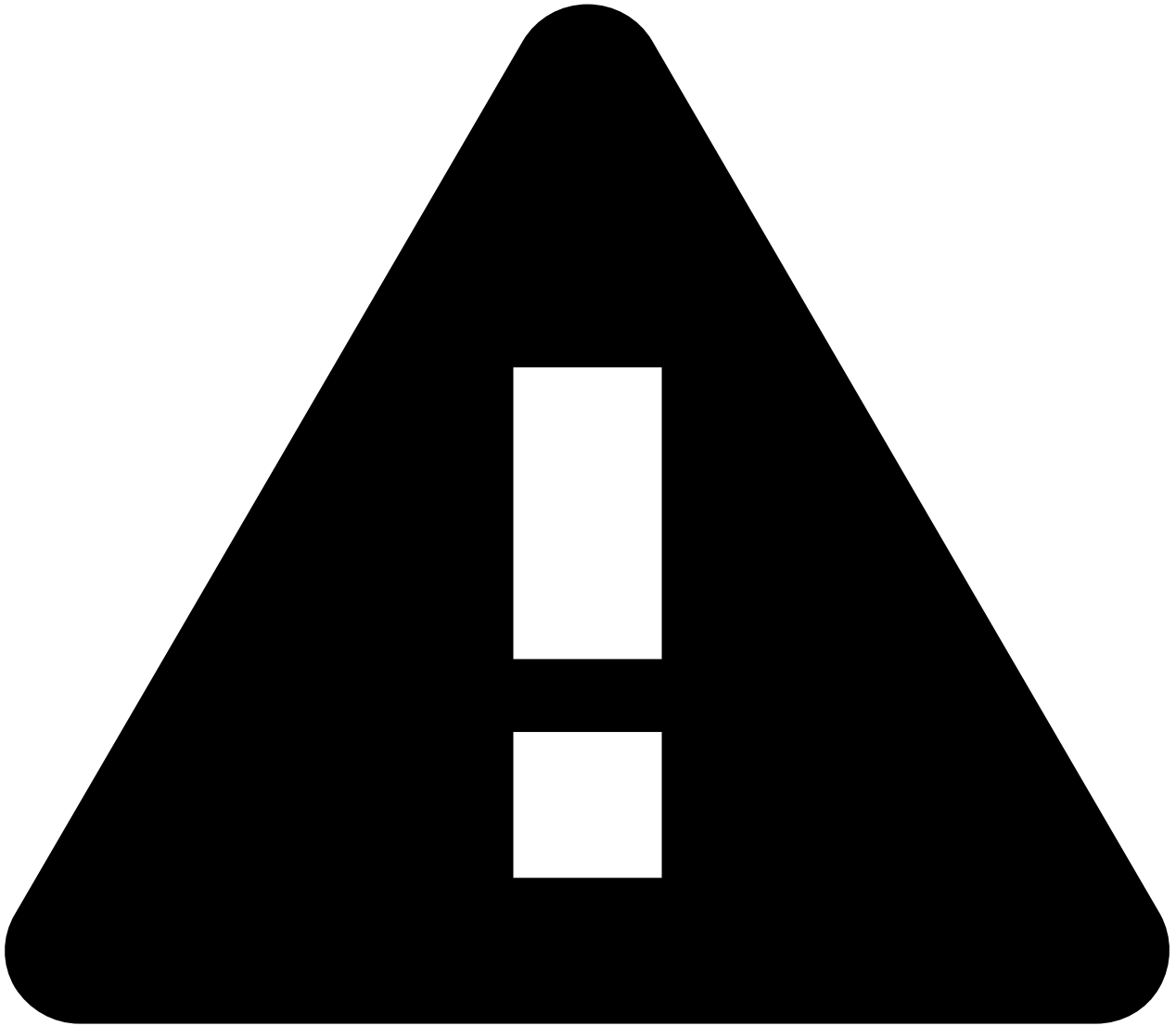
On this page

toNumber

```
toNumber(ipAddr: IPAddress) : Integer
```

Returns the numeric value corresponding to an IPv4 address. For example,

```
toNumber(ipAddress("0.0.1.2")) is 258 .
```



caution

Note that this can only be applied to an IPv4 address.

The [ipv4Address](#) function and the [toNumber](#) are inverses: `toNumber(ip) == num` is true exactly when `ip == ipv4Address(num)` is true, for `ip` an IPv4 address and `num` an integer in `0..232-1`. For example:

- `ipv4Address(toNumber(ipAddress("1.2.3.4"))) == ipAddress("1.2.3.4")`
- `toNumber(ipv4Address(1234)) == 1234`

See also[â€‹](#)

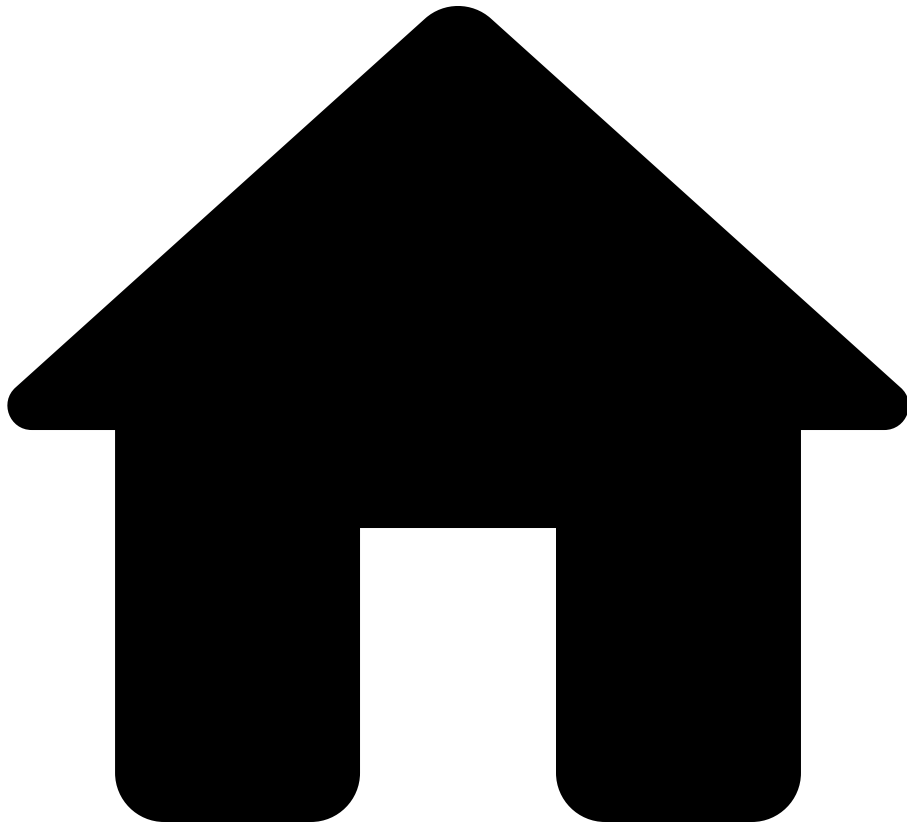
Types[â€‹](#)

- [IpAddress](#)
- [Integer](#)

Functions[â€‹](#)

- [ipAddress](#)
- [ipv4Address](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- toString

On this page

toString

`toString(value: T) : String` for any type `T`.

The `toString` function converts any value to a string. Here are some examples.

```
toString([1, 2, 3]) -> "[1,2,3]"
toString(ipAddress("1.1.1.1")) -> "1.1.1.1"
toString(ipSubnet("1.1.1.1/1")) -> "1.1.1.1/1"
toString(macAddress("11:22:33:44:55:66")) ->
"11:22:33:44:55:66"
toString(SubInterfaceVlan.VLAN\_ID(123)) ->
"SubInterfaceVlan.VLAN\_ID(123)"
toString({ str: "string", bool: true, num: 1}) ->
"{str:string, bool:true, num:1}"
toString(re`(?<name>\w+) (?<age:Integer>\d+)`) -> "re`(?
<name>\w+) (?<age:Integer>\d+)`"
```

Although any value can be converted to a string, using `toString` will raise an error if the resulting string is considered too long, which is longer than 2000 characters. Use `toStringTruncate` to truncate the output instead raising an error.

See also

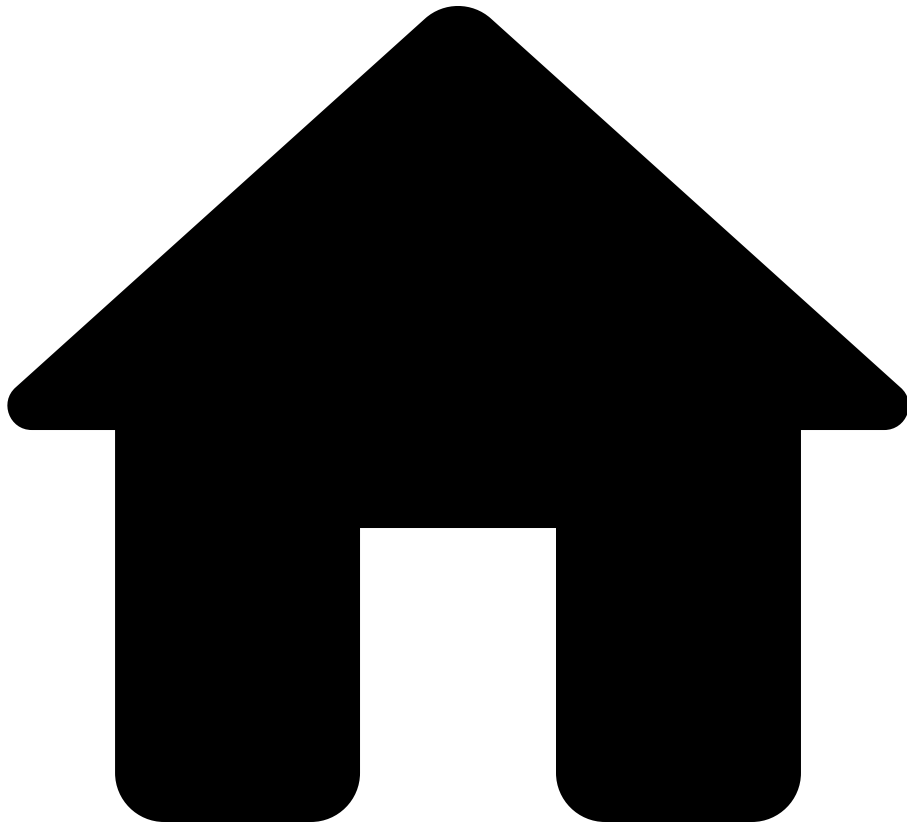
Types

- [String](#)

Functions

- [toStringTruncate](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- toStringTruncate

On this page

toStringTruncate

`toStringTruncate(value: T) : String` for any type `T`.

Equivalent to `toStringTruncate(value, 2000)`.

`toStringTruncate(value: T, maxLength: Integer) : String` for any type `T`.

Truncates the string to a specific number of characters instead of raising an error, then use the function `toStringTruncate`. `toStringTruncate` applies `toString` to its argument but truncates the produced string to the given length. If the resulting string has been truncated, `"<<TRUNCATED>>"` is appended to the resulting string as a suffix.

Here are some examples.

```
toStringTruncate([1, 2, 3]) -> "[1,2,3]" // Truncates to 2000
characters
toStringTruncate([1, 2, 3], 100) -> "[1,2,3]"
toStringTruncate([1, 2, 3], 2) -> "[1<<TRUNCATED>>"
```

See also[â](#)

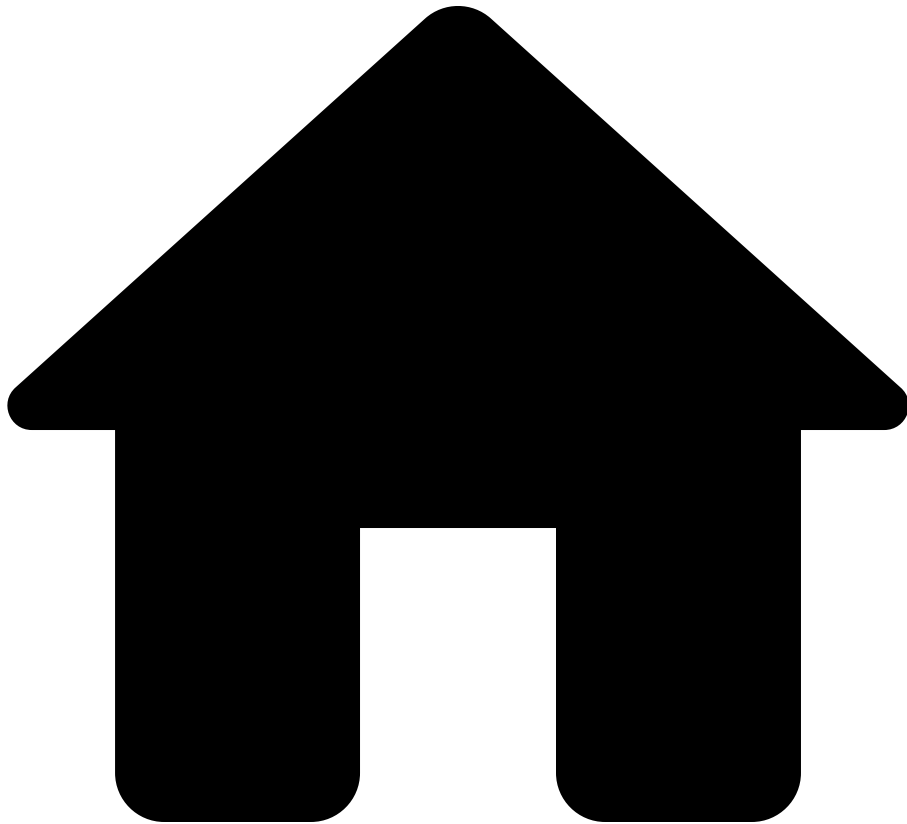
Types[â](#)

- [String](#)
- [Integer](#)

Functions[â](#)

- [toString](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- toUpperCase

On this page

toUpperCase

```
toUpperCase(str: String): String
```

Changes all alphabetic characters in the string to uppercase. For example,

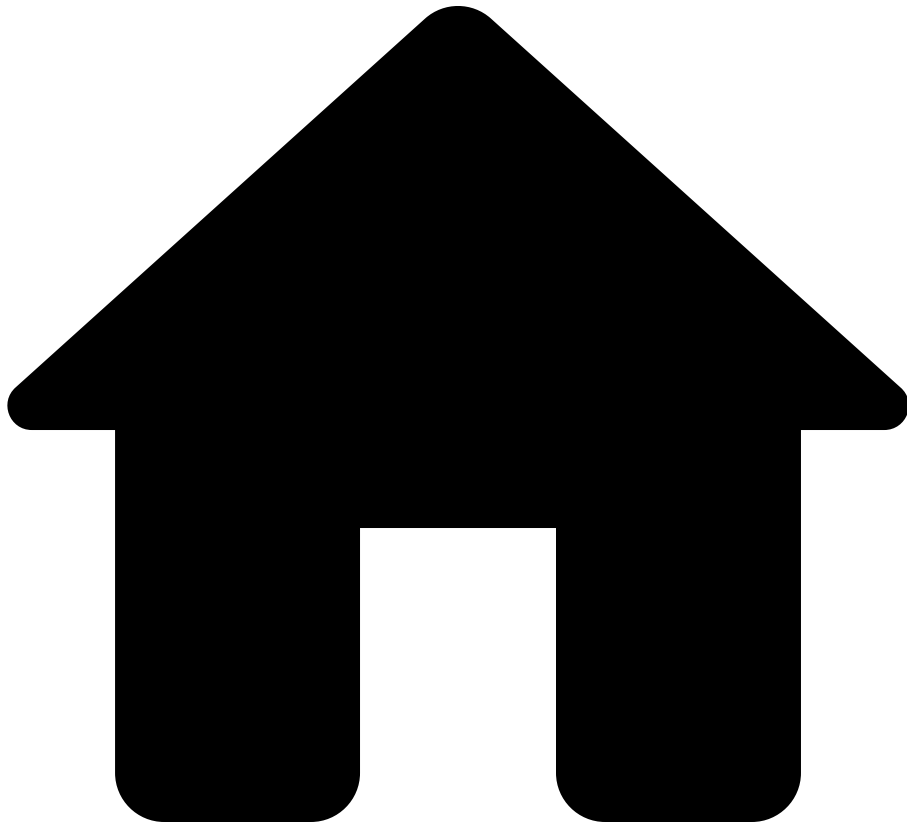
```
toUpperCase("UpperCase") is "UPPERCASE" .
```

See also [a](#)

Types [a](#)

- [String](#)

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- withInfoStatus

withInfoStatus

`withInfoStatus(value: T, infoStatus : InfoStatus) : T`, where `T` is one of the following types:

- String
- Integer
- Float

- Bool
- IpAddress
- IpSubnet
- MacAddress
- Date
- Timestamp
- Duration

Returns a new value that is identical to the provided value, but which is labelled with the given informational status.

The informational status have the following labels :

- OK
- WARNING
- ERROR

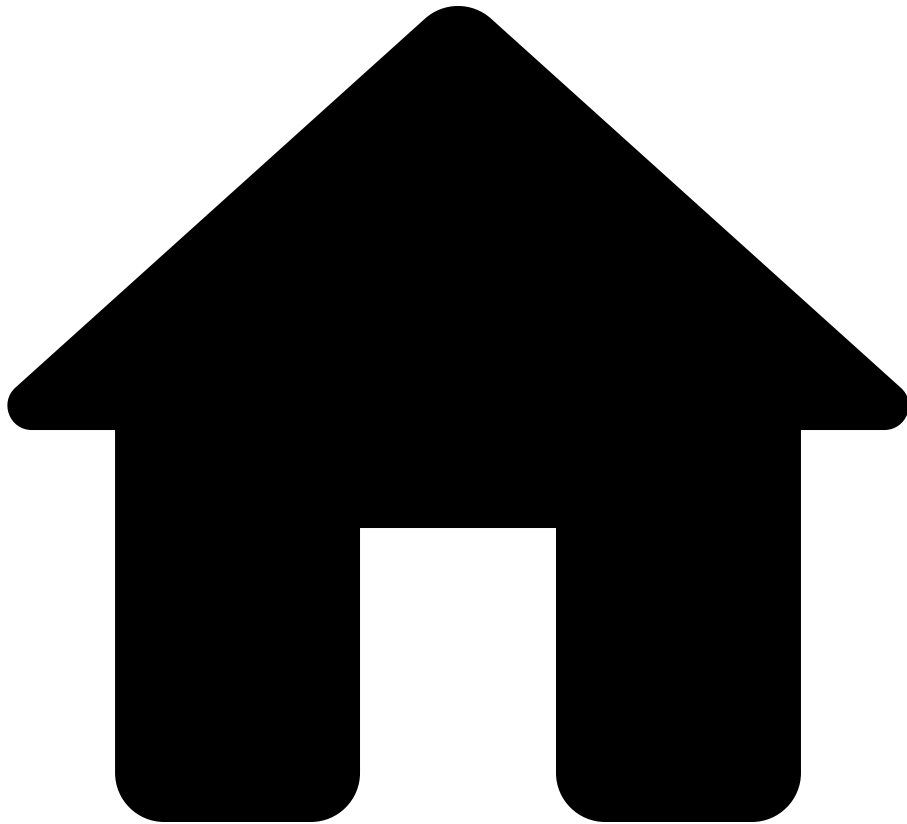
The `withInfoStatus` function is useful to call out certain information in an NQE report. For example, consider the following query, that shows devices and their support dates:

```
foreach device in network.devices
let platform = device.platform
let osSupport = platform.osSupport
where isPresent(osSupport) &&
isPresent(device.snapshotInfo.collectionTime)
let collectionDate = date(device.snapshotInfo.collectionTime)
select {
  Device: device.name,
  Vendor: platform.vendor,
  Model: platform.model,
  OS: platform.os,
  "OS Version": platform.osVersion,
  "Collection Date": collectionDate,
  "End of OS support": osSupport.lastSupportDate
}
```

To highlight the problem when the support date is past the current time, we can conditionally label the support date with an appropriate information label:

```
foreach device in network.devices
let platform = device.platform
let osSupport = platform.osSupport
where isPresent(osSupport) &&
isPresent(device.snapshotInfo.collectionTime)
let collectionDate = date(device.snapshotInfo.collectionTime)
select {
  Device: device.name,
  Vendor: platform.vendor,
  Model: platform.model,
  OS: platform.os,
  "OS Version": platform.osVersion,
  "Collection Date": collectionDate,
  "End of OS support":
withInfoStatus(osSupport.lastSupportDate,
                if
osSupport.lastSupportDate < collectionDate
                then InfoStatus.ERROR
                else InfoStatus.OK)
}
```

.



- Reference
- [NQE Language](#)
- [Standard Library](#)
- withOption

On this page

withOption

The `withOption` functions change the behavior of a block pattern.

```
withOption(pattern: PatternBlocks<T>, option:  
UnmatchedConfigOption) : PatternBlocks<T>
```


`withOption(pattern, option)` returns a new pattern that is identical to `pattern`, but which uses the specified `option` for handling unmatched configuration blocks. For example, to specify that `pattern` should only match if there is no unmatched config lines, use `withOption(pattern, UnmatchedConfigOption.FORBID)`.

Note that block patterns constructed using block pattern literal syntax or via the `blockPattern` function use `UnmatchedConfigOption.ALLOW` option. In other words, by default, block patterns match their subpatterns against configuration lines, allowing for extra lines to be present in the configuration.

```
withOption(pattern: PatternBlocks<T>, option:
LineMatchingOption) : PatternBlocks<T>
```

`withOption(pattern, option)` can be used to obtain a new pattern that is identical to `pattern`, but which uses the specified `option` when matching individual pattern lines to config lines. For example, to specify that each line of `pattern` should only match a config line if it completely matches the line (not just a prefix of the line), use

```
withOption(pattern, LineMatchingOption.COMPLETE)
```

Note that block patterns constructed using block pattern literal syntax or via the `blockPattern` function use `LineMatchingOption.PREFIX` option. In other words, by default, block pattern lines match against lines of configuration as long as the pattern line matches a prefix of the configuration line.

A line in a block pattern literal that ends with `{eof}` will only match configuration lines where every part of the configuration line is matched up with elements in the block pattern line. If the block pattern line does not end with `{eof}` then the block pattern line can match configuration lines which have a prefix that matches the block pattern line. In other words, `pattern1` and `pattern2`, defined below are equivalent:

```
pattern1 = ```
interface {string} {eof}
  switchport access vlan {number} {eof}
```;

pattern2 = withOption(```
interface {string}
```

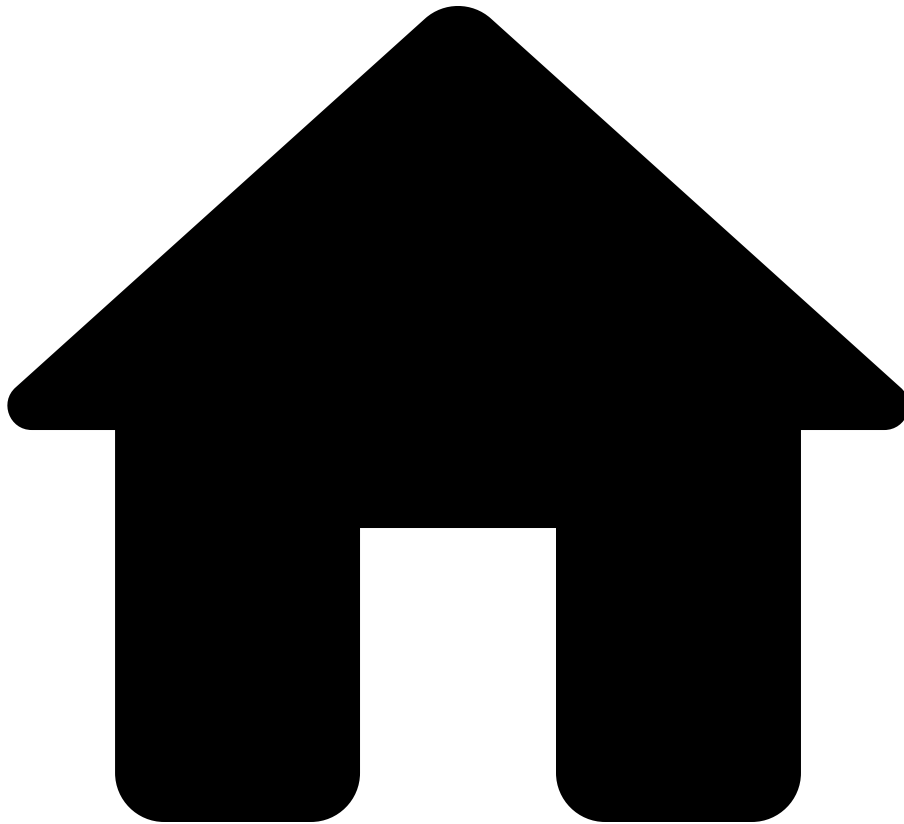
```
switchport access vlan {number}
``, LineMatchingOption.COMPLETE);
```

**See also** [â€‹](#)

**Functions** [â€‹](#)

- [blockPattern](#)

.



• NQE

On this page

## NQE (Network Query Engine)

**An Easier Way to Gather Insights from the Network and Build Custom Verification**[â](#)

Network teams often **need to collect and parse** devices output to gather some insights from the Network, implement specific verification and sanity checks or as part of their network documentation. These scripts collect state and/or configuration from every device in the network and then need to partially parse the gathered text files (typically

text is the only `API` on these devices) to implement the verifications or build documentation. Building and maintaining tools that collect and parse device output across the broad range of devices, OSES, and features found in enterprise networks poses a **significant burden** for Network Teams.

With the introduction of **Network Query Engine (NQE)**, Forward Networks removes this burden by providing **access to normalized structured data** about the network, enabling the network teams to focus on the higher-level aspects of their use cases, which is actually making their networks more resilient, agile and robust without spending time writing collectors and parsers.

Specifically, NQE provides a simple API that exposes information about the network as JSON data, in a fully-parsed form. Moreover, the information is normalized, meaning it is presented uniformly across devices from different vendors. This means that the same sanity, verification and documentation queries and checks can work across the broad fleet.

In summary, Forward NQE provides access to normalized structured data across all the supported vendors in a scalable, easy, maintainable way. In the next section the user will learn how to use this functionality with a few examples of sanity and verifications.

## NQE Interfaces

### REST APIs

To make NQE easy to consume at scale, Forward Enterprise enables users to run queries and get query results through the REST API.

The NQE REST API enables users to build custom verifications based on fully parsed and normalized NQE data. REST APIs require an external system to query and process the data and it is a great option for users with a basic knowledge of APIs and scripting languages like Python.

For more information, please read the [NQE API docs](#) documentation.

### NQE Library

The **NQE Library** provides an integrated development environment (IDE) for working with NQE queries. It includes an advanced editor that provides syntax highlighting, auto-

formatting, query validation, and additional features. Within the NQE Library, users also benefit from the ability to:

- Run queries on various networks or snapshots and download query results as reports for analysis.
- Efficiently save and organize queries into separate directories.
- share queries with other users and set permissions for editing queries.
- Quickly search for specific queries based on name or path.

For more information visit the [NQE Library](#) page.

## **NQE Verifications**

**NQE Verifications** enable users to run and store their custom Verifications entirely in the Forward platform without the need of an external system.

For more information visit the [NQE Verifications](#) page.

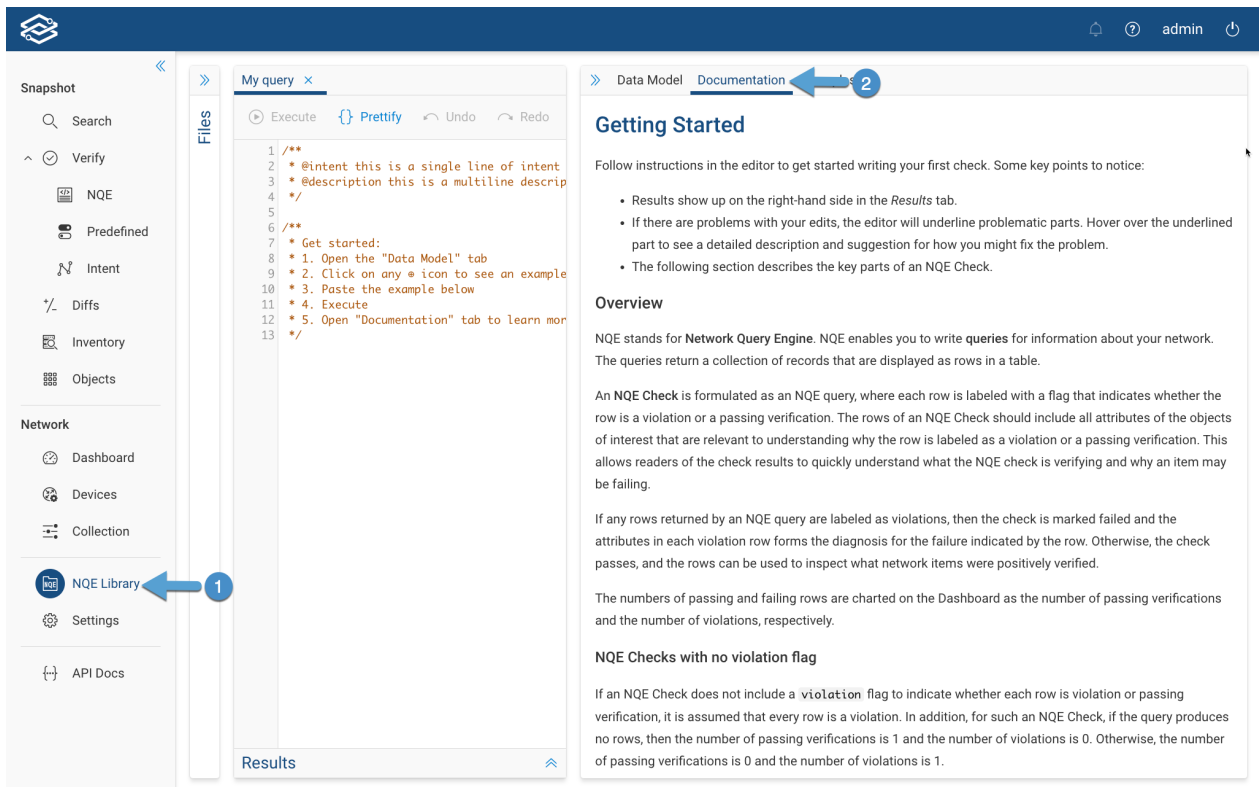
## **NQE Decorators**

**NQE Decorators** enable the use of [NQE Queries](#) to enhance Forward applications by incorporating additional contextual data. As an example, it becomes possible to include information on CVE vulnerabilities detected by a [vulnerability scanner](#) and associate it with a host device card in the Search application.

For more information visit the [NQE Decorators](#) page.

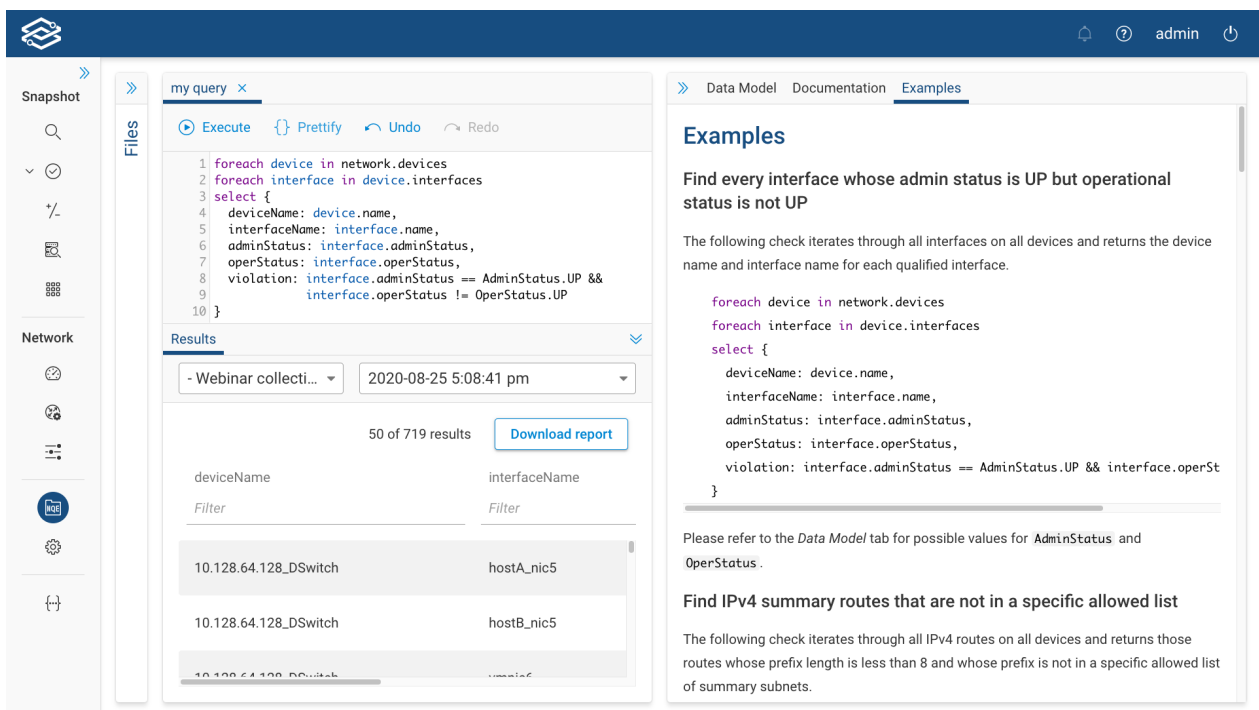
## **NQE Documentation**

You can find additional NQE documentation in the **Documentation** tab in the [NQE Library](#) page.



## Getting Started with NQE

The easiest way to get started with ForwardNQE is to play with some examples.



Or by searching for keywords related to the specific insights you are looking for (e.g. interface, neighbor) in the NQE model:

The screenshot displays the OpenStack DevTools web application. On the left sidebar, there are navigation icons for Snapshot, Files, and Network. The main area is divided into two panes. The left pane shows a code editor with a query about intents and examples. The right pane shows the 'Data Model' tab with a search bar containing 'interface'. Below the search bar, a tree view lists network-related entities like 'network: Network', 'devices: List<Device>', and 'interfaces: List<Iface>'. The bottom section of the right pane provides detailed information about the 'interface' entity, including its name, type, and various values like IF\_SONET, IF\_ETHERNET, etc.

```

1 /**
2 * @intent this is a single line of intent
3 * @description this is a multiline description
4 */
5
6 /**
7 * Get started:
8 * 1. Open the "Data Model" tab
9 * 2. Click on any * icon to see an example
10 * 3. Paste the example below

```

### The query is empty

Start editing the query and click *Execute* to see the results.

**Data Model** Documentation Examples

 interface

- **network:** **Network**
- **devices:** **List<Device>**
  - name:** **string** +
  - + **ha:** **Ha**
  - + **platform:** **Platform**
- **interfaces:** **List<Iface>**
  - name:** **string** +

The name of the **interface**.

- **interfaceType:** **IfaceType** +

The type of the **interface**.

Values:

- IF\_SONET
- SONET/SDH **interface**
- IF\_ETHERNET

Ethernet **interfaces** based on IEEE 802.3 standards, as well as FlexEthernet.

- IF\_TE\_TUNNEL

A TE tunnel **interface**.

- IF\_AGGREGATE

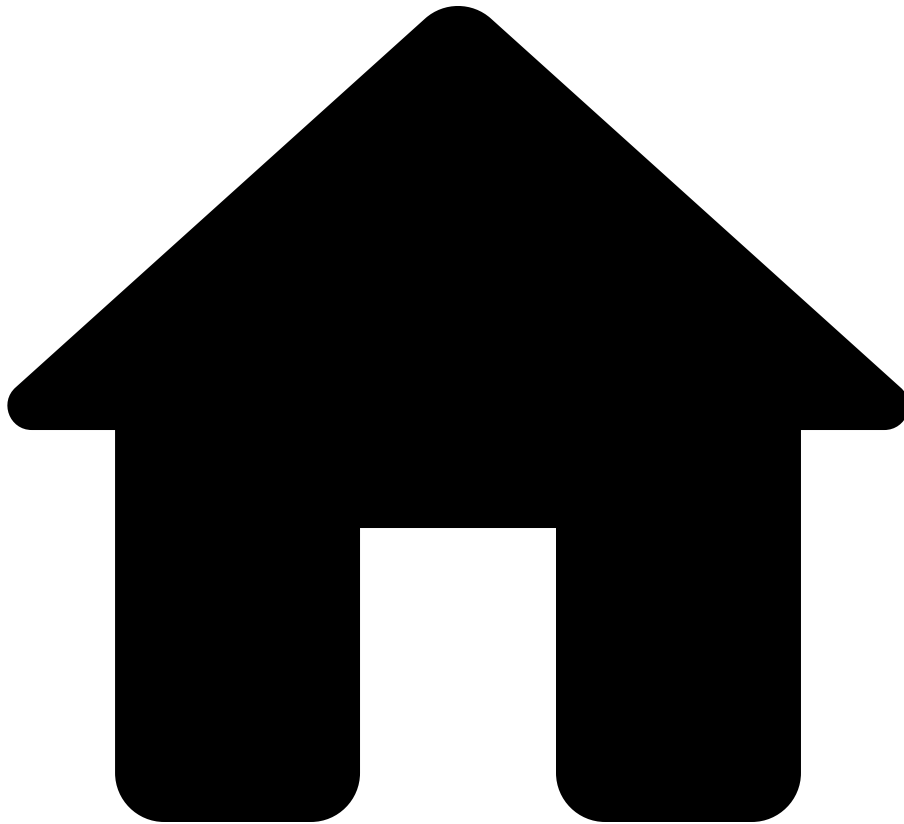
An aggregated, or bonded, **interface** forming a Link Aggregation Group (LAG), or bundle, most often based on the IEEE 802.1AX (or 802.3ad) standard.

- IF\_LOOPBACK

A virtual **interface** designated as a loopback used for various management and operations tasks.

- IF\_TUNNEL\_UNKNOWN

•



• NQE

## NQE

Ask the Network Query Engine for structured, vendor-agnostic network information.

OpenAPI: [YAML](#) | [JSON](#)

---

Network Query Engine (NQE) enables you to query for information about your network. For more information about NQE queries and for help writing queries, see the [NQE tutorial](#) or the Data Model tab of the [Help pane](#) in the NQE Library.

The NQE API allows you to run NQE queries on a network Snapshot. You may also filter and order the results by providing a `QueryOptions` object.

---



### [Run an NQE query](#)

/api/nqe

### [Compare NQE query results](#)

/api/nqe-diffs/:before/:after

### [List all NQE queries](#)

/api/nqe/queries